

UNIVERSIDAD ESAN
FACULTAD DE INGENIERÍA
INGENIERÍA TECNOLOGÍAS DE INFORMACIÓN Y SISTEMAS

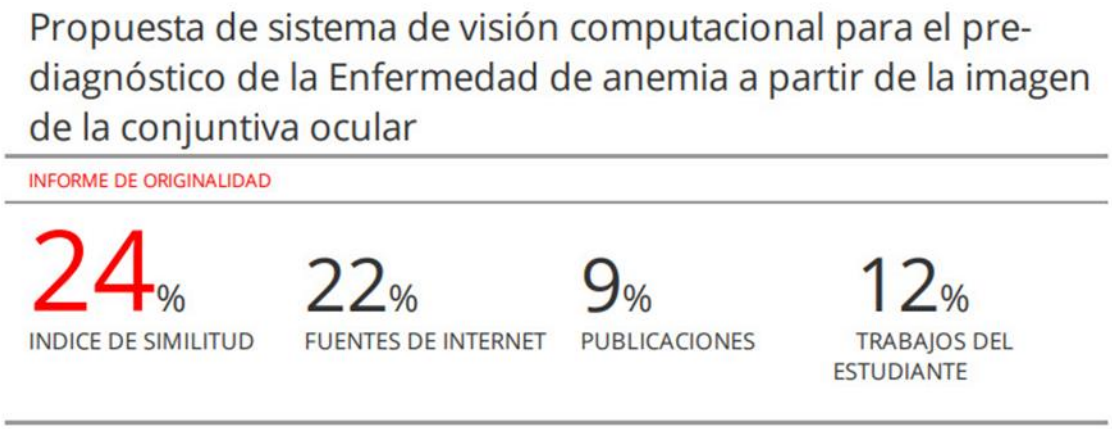
Propuesta de sistema de visión computacional para el pre-diagnóstico de la Enfermedad de anemia a partir de la imagen de la conjuntiva ocular

Alejandro Edward Ildefonso Medina

Asesor: Wilfredo Mamani Ticona

Lima, 3 de julio del 2023

Resultados del informe de similitud



Página de aprobación

Agradecimiento

A mi madre, por brindarme apoyo sin dudar y ser el pilar de mi empeño.

A mis hermanas, por permitirme crecer junto a ellas.

A los profesores que me aconsejaron y estuvieron siempre presentes.

A mi asesor, por ser un guía incansable a lo largo de la investigación.

A mis compañeros, por ayudarme a encontrar soluciones y aportar nuevas ideas.

A mi abuela, por ser una luz constante en todo el proceso.

A la universidad, por proporcionarme un espacio para mi desarrollo.

Dedicatoria

Dedico todo mi trabajo, esfuerzo y progreso a mi familia. A mi madre, para devolverle el esfuerzo y la confianza que depositó en mí desde el principio. A Mikaela y Valentina, para demostrarles que la perseverancia es la clave para nuestro crecimiento. Y a mi abuela, por el cariño y la fe que siempre compartió conmigo.

Índice general

Resultados del informe de similitud 1

Página de aprobación 2

Agradecimiento 3

Dedicatoria 4

Índice general 5

Índice de tablas 10

Índice de figuras 12

Abstract 18

Introducción 19

CAPÍTULO I: PLANTEAMIENTO DEL PROBLEMA 20

1.1. Descripción de la Realidad Problemática 20

1.2. Formulación del Problema 24

1.2.1. Problema General 24

1.2.2. Problemas Específicos 24

1.3. Objetivos de la Investigación 25

1.3.1. Objetivo General 25

1.3.2. Objetivos Específicos 25

1.4. Justificación de la Investigación 25

1.4.1. Teórica 26

1.4.2. Práctica 26

1.4.3. Metodológica 27

1.5. Delimitación del Estudio 30

1.5.1. Delimitación espacial (escenario) 30
1.5.2. Delimitación temporal (periodo de tiempo) 31
1.5.3. Delimitación conceptual (temas, áreas, procesos, etc.) 31
CAPITULO II: MARCO TEÓRICO 33
2.1. Antecedentes 33
2.2. Bases Teóricas 51
2.2.1. Anemia 51
2.2.2. Fisiología Ocular 58
2.2.3. Visión computacional 62
2.2.4. Arquitecturas Avanzadas de Redes Neuronales 83
2.2.5. Aprendizaje Automático y Reconocimiento de Patrones 99
2.2.6. Evaluación de rendimiento: 114
2.2.7. Interpretabilidad de los Modelos de Aprendizaje Automático 115
2.3. Marco Conceptual o Contexto de la Investigación 116
2.4. Hipótesis 121
2.4.1. Hipótesis General 122
2.4.2. Hipótesis Específica 122
2.4.3. Variables e indicadores 122
CAPÍTULO III: METODOLOGÍA 124
3.1. Diseño, tipo y enfoque de Investigación 124
3.1.1. Diseño (No experimental) 124
3.1.2. Tipo – nivel (Aplicada / Explicativo) 124
3.1.3. Enfoque - Cuantitativo 125

3.2. Población y Muestra 125
3.3. Instrumentos de medida 128
3.4. Operacionalización de variables 129
3.5. Técnicas de Recolección de Datos 130
3.6. Técnicas para el procesamiento y análisis de la información 132
3.7. Cronograma de actividades y presupuesto 133
3.8. Metodología para la medición de resultados de la implementación 135
3.9. Metodología de la implementación de la solución 136
3.9.1. Adquisición de la base de datos: 137
3.9.2. Actividades de la fase de Pre procesamiento (Base de datos) 146
3.9.3. Pre procesamiento de la imagen 156
3.9.4. Extracción de características 166
3.9.5. Creación del modelo 176
3.9.6. Desarrollo y evaluación de modelos CNN 184
3.9.7. Evaluación del Modelo Entrenado 193
CAPÍTULO IV: DESARROLLO DEL EXPERIMENTO 196
4.1. Adquisición del dataset 196
4.1.1. Identificación de las Fuentes de Información 196
4.1.2. Anotación de las Imágenes 197
4.1.3. Preprocesamiento de las Imágenes 198
4.1.4. Segmentación de la Imagen 200
4.2. Pre procesamiento Base de datos: 201
4.2.1. Evaluación de la clasificación de imágenes 201

4.2.2. Etiquetado de imágenes 202
4.2.3. Evaluación de las variables predictoras 203
4.2.4. Análisis de la Base de Datos 208
4.3. Pre Procesamiento de la imagen: 211
4.3.1. Organización y exploración de datos 211
4.3.2. Pre procesamiento de imágenes 212
4.3.3. Optimización de imágenes 213
4.3.4. Etiquetado y almacenamiento: 214
4.3.5. Partición de datos 214
4.3.6. Aumento de data 215
4.3.7. División de Datos 218
4.4. Extracción de características 219
4.4.1. Extracción de características con LBPH 219
4.4.2. Extracción de características con HOG 220
4.4.3. Extracción de características con SIFT 222
4.4.4. Extracción de características con SIFT Padding 223
4.4.5. Extracción de características con ORB 224
4.5. Creación del modelo: 226
4.5.1. Codificación de Etiquetas de Clase 226
4.5.2. Creacion del modelo con Tensorflow 226
4.5.3. Creacion del modelo con SVM 228
4.5.4. Creación del modelo con Random Forest 230
4.5.5. Creación del modelo con KNN 231

4.5.6. Creación del modelo con MLP 233
4.6. Desarrollo y evaluación de modelos CNN 234
4.6.1. Creación del Modelo CNN Personalizado 234
4.6.2. CNN con EfficientNet 236
4.6.3. CNN con ResNet 237
4.6.4. CNN con Inception-ResNet 239
4.6.5. Entrenamiento de los Modelos CNN 240
4.6.6. Definición de Parámetros 242
CAPÍTULO V: ANÁLISIS Y DISCUSIÓN DE RESULTADOS 247
5.1. Análisis de los Parámetros Óptimos 247
5.2. Análisis de los Resultados de los modelos CNN 249
5.2.1. Gráficos de Precisión del Modelo en Entrenamiento 249
5.2.2. Gráficos de Pérdida del Modelo en Entrenamiento 251
5.2.3. Gráficos de Precisión del Modelo en Validación 253
5.2.4. Gráficos de Pérdida del Modelo en Validación 255
5.2.5. Gráfico de Precisión de la Prueba del Modelo 257
5.2.6. Gráfico de Pérdida de la Prueba del Modelo 259
5.3. Presentación de los Resultados del Modelo 261
5.4. Implementación de una API con los mejores modelos predictivos 263
CAPÍTULO VI: CONCLUSIONES Y RECOMENDACIONES 265
6.1. Conclusiones 265
6.2. Recomendaciones 266
FUENTES DE INFORMACIÓN 268

Índice de tablas

Tabla 1 Valores normales de hemoglobina en sangre 23

Tabla 2 Análisis de la detección de anemia utilizando el método computarizado 38

Tabla 3 Parámetros iniciales para la aplicación del método DPSO 42

Tabla 4 Fotos de uñas con y sin anemia por género 44

Tabla 5 Resultados de SIFT, SURF (k=10, 50, 100 y 200) y HOG 45

Tabla 6 Parámetros modificados al momento de compilar 46

Tabla 7 Grupo de Geometría Visual 46

Tabla 8 Resultados de las CNN de los modelos de machine learning 47

Tabla 9 Evaluación de métricas de PCNN-15 49

Tabla 10 Población, muestra, unidad de análisis del estudio y Variable y tipo de variable 127

Tabla 11 Descripción de variables 128

Tabla 12 Instrumentos de Medida Utilizados en el Estudio 128

Tabla 13 Operacionalización de variables 129

Tabla 14 Ejemplo de un caso y la información encontrada en la base de datos 131

Tabla 15 Presupuesto 134

Tabla 16 Actividades para la Adquisición de la base de datos 137

Tabla 17 Actividades de la fase de Pre procesamiento (Base de datos) 146

Tabla 18 Tabla de Niveles Normales de Hemoglobina 148

Tabla 19 Pre procesamiento de la imagen 156

Tabla 20 Extracción de características 166

Tabla 21 Creación de modelo 176

Tabla 22 Desarrollo y evaluación de modelos CNN 184

Tabla 23 Evaluación del Modelo Entrenado 193

Tabla 24 Evaluación de la clasificación de imágenes 202

Tabla 25 Precisión de los Modelos de Aprendizaje Automático en el Conjunto de Prueba 211

Tabla 26 Anemia y no anemia 212

Tabla 27 Parámetros para la Red Neuronal Convolucional 242

Tabla 28 Parámetros para la Máquina de Soporte Vectorial (SVM): 243

Tabla 29 Parámetros para el KNN 244

Tabla 30 Parámetros para el Random Forest 245

Tabla 31 Parámetros para el MLP 246

Tabla 32 Resumen de los parámetros óptimos para SVM, KNN, Random Forest, MLP 248

Tabla 33 Precisión de los modelos usando varios extractores de características 262

Tabla 34 Precisión de los modelos de aprendizaje profundo usando diferentes parámetros 262

Índice de figuras

Figura 1 Prevalencia de la anemia en niñas y niños de 6 a 35 meses de edad..... 22

Figura 2 La metodología CRISP-DM..... 30

Figura 3 Diagrama de flujo de la metodología propuesta..... 35

Figura 4 Region de interes..... 36

Figura 5 Conjuntiva ocular anémico (a) y Conjuntiva ocular no anémica (b) 37

Figura 6 Diagrama de la técnica propuesta..... 41

Figura 7 Metodología..... 43

Figura 8 Proceso principal..... 48

Figura 9 Flujo de Detección de Anemia por Appl..... 51

Figura 10 Tipos de Anemia 53

Figura 11 Células Falciformes..... 55

Figura 12 Conjuntiva y el nivel de anemia..... 56

Figura 13 Fisiología Ocular..... 58

Figura 14 Esquema conjuntiva 60

Figura 15 Partes de la conjuntiva ocular..... 61

Figura 16 Ecualización del histograma..... 65

Figura 17 Área de interés conjuntiva ocular..... 69

Figura 18 Aumento de la data..... 70

Figura 19 Extracción de características utilizando Histograma de Gradientes Orientados (HOG) 73

Figura 20 Procesamiento de la cadena de SIFT..... 76

Figura 21 Diagrama de flujo ORB..... 80

Figura 22 Pasos del algoritmo LBPH 81

Figura 23 Pasos CNN 84

Figura 24 Detección de tumores con inteligencia artificial 93

Figura 25 Conexión de Salto 95

Figura 26 Hiperplano de margen máximo y márgenes para un SVM 100

Figura 27 Aplicación de KNN 105

Figura 28 Estructura Random Forest 106

Figura 29 Perceptrón multicapa (MLP)) 109

Figura 30 Estructura de una Red Neuronal Artificial 112

Figura 31 Imagen de la conjuntiva ocular de un paciente 131

Figura 32 Metodología de la implementación 132

Figura 33 Imagen del entorno de labelbox 139

Figura 34 Funcionamiento de la herramienta polygon 140

Figura 35 Curva ROC 155

Figura 36 Página IEEE Explore 197

Figura 37 Funcionamiento de la herramienta polygon 198

Figura 38 Imágenes luego de la selección de la área de interés 199

Figura 39 Área de recorte 200

Figura 40 Imagen recortada 201

Figura 41 Rango de edad y anemia 204

Figura 42 Sexo y anemia 205

Figura 43 Hemoblohina en sangre y anemia 206

Figura 44 Correlación entre edad, nivel de hemoglobina y anemia 207

Figura 45 Distribución de anemia en función de género y edad. 208

Figura 46 Concatenación de datos y selección de variables 209

Figura 47 Código de Implementación del Modelo Secuencial con TensorFlow 210

Figura 48 Optimización de imágenes 214

Figura 49 Particion de datos 215

Figura 50 Generación de la imagen aumentada 216

Figura 51 Etiquetado de las imágenes aumentadas 217

Figura 52 Distribución de etiquetas en la clasificación de imágenes 218

Figura 53 Partición del dataset 219

Figura 54 Extracción de características con LBPH 220

Figura 55 Extracción de características con HOG 221

Figura 56 Extracción de características HOG y generación de histogramas 222

Figura 57 Extracción de características con SIFT 223

Figura 58 Ajuste de dimensiones mediante relleno (padding) 224

Figura 59 Extracción de características con ORB 225

Figura 60 Creación del modelo con Tensorflow 228

Figura 61 Evaluacion del modelo Tensorflow 228

Figura 62 Mejores parámetros encontrados por GridSearch en SVM 229

Figura 63 Creacion del modelo con SVM 229

Figura 64 Creación del modelo con Random Forest 231

Figura 65 Creación del modelo con KNN 232

Figura 64 Mejores parámetros encontrados por GridSearch en KNN 233

Figura 65 Creación del modelo con MLP 233

Figura 68 Mejores parámetros encontrados por GridSearch en KNN 234

Figura 69 Creación del Modelo CNN Personalizado 235

Figura 70 Resultados de entrenamiento y validación de un modelo de clasificación 236

Figura 71 Creación del Modelo CNN usando la arquitectura EfficientNet 237

Figura 72 Creación del Modelo CNN usando la arquitectura ResNet 238

Figura 73 Creación del Modelo CNN usando la arquitectura ResNet 240

Figura 74 Entrenamiento y validación del modelo de aprendizaje profundo 240

Figura 75 Evaluación del modelo y cálculo de métricas de rendimiento 241

Figura 76 Evaluación final del modelo en el conjunto de prueba 241

Figura 77 Gráfico de Precisión del Modelo en Entrenamiento (Inception-ResNet-v2) 249

Figura 78 Gráfico de Precisión del Modelo en Entrenamiento (ResNet) 250

Figura 79 Gráfico de Precisión del Modelo en Entrenamiento (EfficientNet) 250

Figura 78 Gráfico de Precisión del Modelo en Entrenamiento (EfficientNet) 251

Figura 81 Gráfico de Pérdida del Modelo en Entrenamiento (Inception-ResNet-v2) 252

Figura 82 Gráfico de Pérdida del Modelo en Entrenamiento (ResNet) 252

Figura 83 Gráfico de Pérdida del Modelo en Entrenamiento (EfficientNet) 253

Figura 84 Gráfico de Pérdida del Modelo en Entrenamiento (Personalizado) 253

Figura 85 Gráfico de Precisión del Modelo en Validación (Inception-ResNet-v2) 254

Figura 86 Gráfico de Precisión del Modelo en Validación (ResNet) 254

Figura 87 Gráfico de Precisión del Modelo en Validación (EfficientNet) 255

Figura 88 Gráfico de Precisión del Modelo en Validación (Personalizado) 255

Figura 89 Gráfico de Pérdida del Modelo en Validación (Inception-ResNet-v2) 256

Figura 90 Gráfico de Pérdida del Modelo en Validación (ResNet) 256

Figura 91 Gráfico de Pérdida del Modelo en Validación (EfficientNet)..... 257

Figura 92 Gráfico de Pérdida del Modelo en Validación (Personalizado)..... 257

Figura 93 Gráfico de Precisión del Modelo en Prueba (Inception-ResNet-v2)..... 258

Figura 94 Gráfico de Precisión del Modelo en Prueba (ResNet) 258

Figura 95 Gráfico de Precisión del Modelo en Prueba (EfficientNet) 259

Figura 96 Gráfico de Precisión del Modelo en Prueba (Personalizado)..... 259

Figura 97 Gráfico de Pérdida del Modelo en Modelo (Inception-ResNet-v2)..... 260

Figura 98 Gráfico de Pérdida del Modelo en Modelo (ResNet)..... 260

Figura 99 Gráfico de Pérdida del Modelo en Modelo (EfficientNet)..... 261

Figura 100 Gráfico de Pérdida del Modelo en Modelo (Personalizado)..... 261

Figura 101 Diagrama de Flujo de la Interacción de los Scripts y Procesamiento de la API..... 263

Figura 102 Ejecución de la API de Detección de Anemia 264

Resumen

Este estudio abordó el problema significativo de la anemia por deficiencia de hierro en Perú mediante el desarrollo de un sistema de visión computacional para el pre-diagnóstico de la enfermedad. A través de un riguroso proceso metodológico que incluyó la evaluación de datos, el recorte y preprocesamiento de imágenes, el aumento de datos, la extracción de características, la generación de modelos, la evaluación de resultados y la implementación de una API, se investigaron varios modelos de aprendizaje automático y extractores de características. Se encontró que el modelo Inception-ResNet-v2, configurado con 2048 neuronas, 50 épocas y 1500 muestras, demostró ser el más efectivo, alcanzando una precisión del 90.69%% y una pérdida de 0.2788. Además, los modelos KNN, SVM, MLP y RF, con varios extractores de características, también se examinaron, obteniendo los mejores resultados con Random Forest y el extractor de caracterís-ticas HOG, con una precisión del 79%. Esta herramienta computacional tiene el potencial de te-ner un impacto significativo en la salud pública peruana, mejorando la detección temprana y la eficiencia del tratamiento para la anemia.

Palabras clave: Vision computacional, Anemia, Aprendizaje Automático, Exactitud Predictiva, Arquitecturas de redes neuronales convolucionales, Fisologia Ocular, Pre-diagnostico

Abstract

This study addressed the significant problem of iron deficiency anemia in Peru through the development of a computer vision system for the pre-diagnosis of the disease. Through a rigorous methodological process that included data evaluation, image cropping and preprocessing, data augmentation, feature extraction, model generation, result evaluation, and the implementation of an API, various machine learning models and feature extractors were investigated. It was found that the Inception-ResNet-v2 model, configured with 2048 neurons, 50 epochs, and 1500 samples, proved to be the most effective, achieving an accuracy of 90.69% and a loss of 0.2788. In addition, the KNN, SVM, MLP, and RF models, with various feature extractors, were also examined, obtaining the best results with the Random Forest and the HOG feature extractor, with an accuracy of 79%. This computational tool has the potential to have a significant impact on Peruvian public health, improving early detection and treatment efficiency for anemia.

Keywords: Computer Vision, Anemia, Machine Learning, Predictive Accuracy, Convolutional Neural Network Architectures, Ocular Physiology, Pre-diagnosis.

Introducción

La anemia por deficiencia de hierro, caracterizada por un bajo nivel de hemoglobina en la sangre, impacta significativamente en la salud y desarrollo humano, especialmente en áreas rura-les. Esta es una problemática notable en Perú, un país en desarrollo donde a pesar de los esfuer-zos guber-namentales, la anemia afecta al 40% de su población, según datos de 2018. Esta cifra solo dismi-nuyó marginalmente en los años siguientes, evidenciando la magnitud del problema.

Las disparidades geográficas y socioeconómicas, así como la pandemia, han exacerbado el desa-fío, especialmente en provincias lejanas a la capital, donde el diagnóstico y tratamiento son más complejos por la escasez de recursos médicos.

Ante este contexto, la necesidad de soluciones innovadoras es evidente. Esta tesis pro-pone el desarrollo de un sistema de visión computacional para el pre-diagnóstico de la anemia, principal-mente dirigido a la población adulta. Basado en la detección de signos comunes de la enferme-dad, este sistema busca facilitar un pre-diagnóstico temprano, mejorando así las posibi-lidades de un tratamiento efectivo.

Por tanto, este trabajo busca no solo contribuir a la investigación existente sobre la anemia, sino también impactar positivamente en la salud de la población peruana, ofreciendo una herramienta práctica y accesible para el pre-diagnóstico de la anemia.

CAPÍTULO I: PLANTEAMIENTO DEL PROBLEMA

Este capítulo describe diferentes enfoques del problema, objetivos, realidades del problema, razonamiento y representaciones. Esta información ayudará a explicar los detalles de la investigación. Las áreas en las que se enfoca, los problemas que aborda y las metas que quiere lograr.

1.1. Descripción de la Realidad Problemática

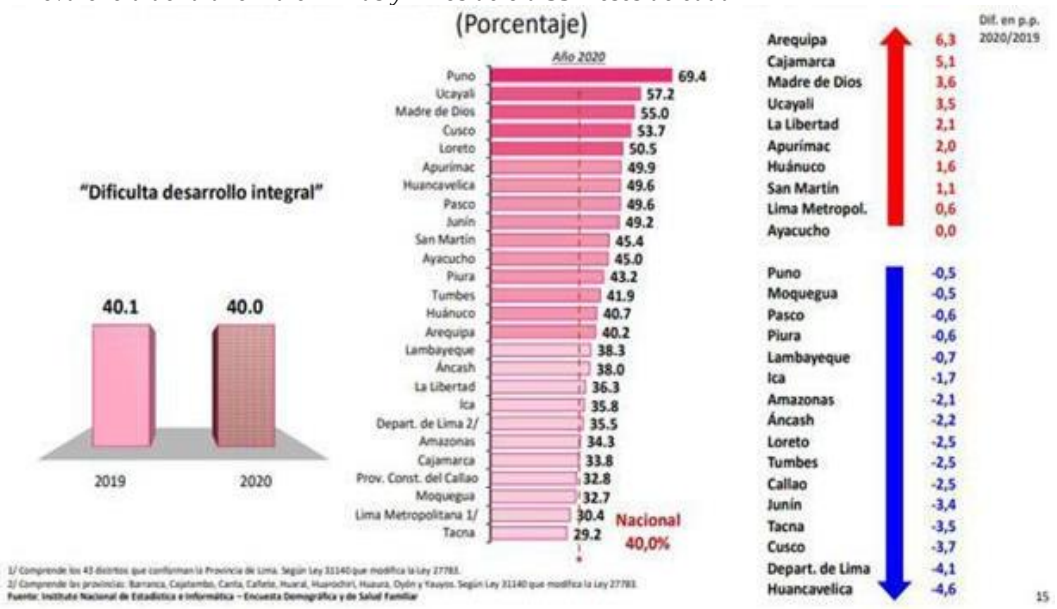
La salud y la felicidad son esenciales en cualquier familia, por lo que es importante que todos los miembros estén sanos y puedan crecer y desarrollarse al máximo, especialmente los niños. Durante la infancia, el cuidado es fundamental para el futuro y la calidad de vida del niño, por eso tiene como objetivo proteger su salud, así como su salud y nutrición. Sin el cuidado adecuado en el hogar, pueden surgir diversos trastornos y enfermedades durante la infancia. (Wallander y Koot, 2016).

La anemia por deficiencia de hierro se caracteriza por un nivel bajo de hemoglobina en la sangre debido a la falta de hierro en el cuerpo (Gordon, 2003). La enfermedad afecta todas las etapas de la vida, pero las mujeres y los niños en edad preescolar son los más susceptibles (OMS, 2017). Cabe señalar que, si un niño está anémico, es probable que tenga retrasos en el desarrollo, lo que puede afectar negativamente el desarrollo cognitivo, psicomotor y del lenguaje del niño. Además, la anemia puede tener efectos a largo plazo en la salud de un niño, posiblemente afectando sus capacidades físicas y mentales cuando sea adulto. En comparación con las personas con una nutrición adecuada, las personas con anemia funcionan peor, ya que una deficiencia reduce la cantidad de oxígeno en la sangre y reduce la energía disponible para aumentar la productividad. (Thompson, Biggs y Pasricha, 2013; Falkingham et al., 2010).

La anemia es un grave problema de salud pública en el Perú y en los últimos diez años se han lanzado muchas campañas y esfuerzos para evitar que este problema siga aumentando (Trenado et al., 2022). la desnutrición infantil ha disminuido en los últimos años; Sin embargo, en 2016, el 13,1% de los niños menores de 5 años presentaba deficiencia de hemoglobina en sangre (MINSA, 2017). Sin embargo, en el 2019 la proporción de niños menores de 5 años con desnutrición crónica fue de 16,3% y de estos el 16,6% se encontraba en riesgo de desnutrición crónica, los datos arrojaron un 31,1% de niños con anemia entre los 12 y 23 meses de edad. edad (INS, 2019).

Actualmente, la anemia sigue siendo un problema grave en el Perú. Según estudios recientes, la prevalencia de anemia en mineros es del 40%, lo que indica una reducción de solo 0,1 punto porcentual respecto de los valores registrados en 2019 (ENDES 2018). Los datos muestran que la tasa de anemia en las áreas rurales (48,4%) es más alta que en las áreas urbanas (36,7%). Si bien la anemia ha disminuido un 3,5% interanual, un gran número de personas siguen afectadas (INEI, 2020).

Figura 1 *Prevalencia de la anemia en niñas y niños de 6 a 35 meses de edad*



Nota. Perú21. (2021). [Gráfico que muestra la prevalencia de anemia en niños peruanos de 6 a 35 meses de edad entre 2015 y 2020]. Recuperado el 23 de mayo de 2023, de https://peru21.pe/lima/peru-prevalencia-de-anemia-en-ninos-de-6-a-35-meses-de-edad-apenas-se-redujo-en-01-en-2020-segun-inei-nndc-noticia/

El Gobierno de Perú está comprometido con la protección de la nutrición infantil y la reducción de la prevalencia de la anemia y la desnutrición crónica entre los niños de todo el país. La anemia infantil y la desnutrición crónica son problemas comunes en todo el mundo, y los gobiernos llevan años realizando campañas de sensibilización y apoyo social para reducir el número de afectados. Según la Organización Mundial de la Salud (OMS, 2015), la anemia tiene impactos tanto microeconómicos como macroeconómicos, ya que puede afectar negativamente el desarrollo cognitivo, la productividad y el nivel de desarrollo humano en países con altos índices de anemia. Por lo tanto, el compromiso del gobierno peruano es vital para mejorar la salud y el bienestar de los niños, así como el desarrollo económico del país. Para obtener un diagnóstico preciso de una persona y determinar si tiene anemia, es necesario realizar pruebas de laboratorio

y análisis de sangre, que determinarán con precisión las características importantes, como el porcentaje de hemoglobina en la sangre, la forma y el volumen de sangre roja (Tvedten, 2022). Para clasificar la condición de un paciente, es necesario verificar si el valor de la concentración de hemoglobina se encuentra dentro del rango normativo, teniendo en cuenta los criterios establecidos por la Organización Mundial de la Salud (OMS). Por ejemplo, los hombres de 12 a 14 años son anémicos con una concentración de hemoglobina inferior a 11,9 gramos por decilitro (g/dL), mientras que los hombres de 15 a 15 años son anémicos si sus niveles de hemoglobina son inferiores a 11,9 gramos por decilitro (g/dL) el nivel en sangre es inferior a 13 g/dl (OMS, 2008).

Tabla 1 *Valores normales de hemoglobina en sangre*

Hemoglobina en:	Valor normal (en g/dl)	Leve	Moderna	Grave
Mujeres	12,1 – 15,1	11 – 11,9	8 – 10,9	Menos de 8
Mujeres embarazadas	11 o superior	10 – 10,9	7 – 9,9	Menos de 7
Hombres	13,8 – 17,2	10 – 12,9	8 – 10,9	Menos de 8
Niños (6 meses- 4 años)	11 o superior	10 – 10,9	7 – 9,9	Menos de 7
Niños (5-11 años)	11,5 o superior	11 – 11,4	8 – 10,9	Menos de 8
Niños (12-14 años)	12 o superior	11 – 11,9	8 – 10,9	Menos de 8

Nota. Extraído de Cuáles son los valores normales de hemoglobina en sangre (2019). https://www.mundodeportivo.com/uncomo/salud/articulo/cuales-son-los-valores-normales-de-hemoglobina-en-sangre-46677.html

En Perú, las provincias ubicadas lejos de la capital carecen de los recursos humanos y materiales necesarios para llevar a cabo una identificación precisa de los pacientes que padecen de anemia. Según el Ministerio de Salud (Andina, 2021), es necesaria una política clara de descentralización; Algunas regiones que no disponen de los recursos humanos que realmente necesitan, y existe una gran disparidad en la distribución de los recursos humanos sanitarios en el país.

Sin embargo, hay diferentes formas de tener indicadores de que una persona puede presentar anemia, la palidez en varias regiones del cuerpo es un indicador crucial para una detección de esta enfermedad, este cambio de color puede presentarse en el lecho ungueal, la conjuntiva, y

los pliegues palmares (Thaver y Baig, 1998). Es importante considerar que en la actualidad el proceso para atender a un paciente fuera de emergencia en un hospital demora mucho por la pandemia, lo que hace que los diagnósticos y la reacción de los involucrados sea tardía. Bajo la idea de poder tener un pre-diagnóstico de anemia a través de los indicadores comunes se plantea la idea de un sistema de visión computacional que se centre en clasificar si el usuario presenta o no signos de padecer esta enfermedad.

1.2. Formulación del Problema

Se ha utilizado una técnica de matriz de contingencia para establecer las preguntas generales y específicas. Las preguntas específicas se centran en la evolución de la anemia en el contexto de Perú. Además, se han formulado las preguntas a partir de los problemas previamente identificados con el fin de encontrar soluciones.

1.2.1. Problema General

¿Es posible desarrollar un sistema de visión por computadora que pueda determinar el pre-diagnóstico de anemia a partir de imágenes de la conjuntiva del ojo?

1.2.2. Problemas Específicos

PE1: ¿Se dispone de bases de datos con imágenes de la conjuntiva ocular de individuos con anemia?

PE2: ¿Qué técnicas de procesamiento de imágenes pueden mejorar la calidad o la cantidad de imágenes en la base de datos?

PE3: ¿Qué técnicas de visión por computadora permiten convertir los descriptores de imagen como el color o la forma en vectores de características?

PE4: ¿Qué medidas se pueden utilizar para evaluar la efectividad de la clasificación de imágenes en la detección de anemia en pacientes?

PE5: ¿Cómo puede el sistema de visión por computadora mejorar el pre-diagnóstico de la anemia utilizando imágenes de la conjuntiva ocular?

1.3. Objetivos de la Investigación

Se ha utilizado una técnica de matriz de contingencia para definir los objetivos generales y objetivos específicos. Además, se han establecido objetivos basados en la identificación previa de problemas, con el objetivo de brindar soluciones.

1.3.1. Objetivo General

Desarrollar e instalar un sistema de procesamiento de imágenes computarizado para categorizar de manera precisa los síntomas preliminares de la anemia utilizando imágenes de la conjuntiva ocular.

1.3.2. Objetivos Específicos

OE1: Recopilar imágenes de la conjuntiva ocular de pacientes, tanto de aquellos que padecen anemia como de aquellos que no la padecen.

OE2: Preparar los datos para su análisis, incluyendo el pre procesamiento, la limpieza y el análisis de datos faltantes y la transformación de variables en estudio.

OE3: Aplicar técnicas de aprendizaje automático, como SVM, RF y RNN, para clasificar las características obtenidas de las imágenes.

OE4: Evaluar los algoritmos propuestos en términos de su ajuste a los datos, su capacidad para estimar el error de manera precisa.

OE5: Utilice métricas para evaluar la eficacia de la clasificación de imágenes y determinar si corresponde al grupo adecuado.

1.4. Justificación de la Investigación

1.4.1. Teórica

El objetivo de este estudio es evaluar la utilidad de las técnicas de visión computacional en el procesamiento de imágenes para identificar características y patrones en las imágenes de individuos con anemia. El enfoque de clasificación propuesto busca proporcionar un pre diagnóstico y alertar sobre la presencia de signos de anemia en estos pacientes. Diferentes investigaciones se han realizado con la premisa de encontrar las características físicas visibles en las fotos de pacientes con anemia, estos estudios demuestran que es posible reconocer ciertos patrones y diferenciar si el usuario presenta alguna anormalidad o si necesita alguna atención a corto plazo. En el Perú, el desarrollo de la tecnología y la implementación de este tipo de estudios ha sido dejado de lado por la falta de incentivos y desconocimiento de las entidades públicas, se busca que este estudio aporte a la implementación de sistemas que faciliten al paciente métodos alternativos de atención y tenga más información accesible sobre su salud.

1.4.2. Práctica

Es importante tomar en cuenta que el resultado obtenido por el paciente luego de que este realice las pruebas del sistema de pre-diagnostico no estará certificado por un médico, y solo podrá aproximarse al diagnóstico preciso que da una prueba de sangre. Si el paciente tiene como resultado que cuenta con las características de anemia y hemoglobina baja tendrá que acercarse a un médico para tener un análisis completo de su estado de salud, el sistema se encargara de detectar las características de alarma y cuáles son las que tienen en común con otros pacientes que tienen anemia.

La investigación podría demostrar que, mediante el uso de un sistema de visión computacional, es posible predecir si una persona padece anemia o no a partir de imágenes

de la conjuntiva ocular. Detectar estas características desde el hogar puede ser crucial para el crecimiento y desarrollo de la persona, ya que, si la anemia se trata a tiempo y se controla regularmente, el paciente no sufrirá consecuencias graves en el futuro.

1.4.3. Metodológica

La introducción de un sistema de pre diagnóstico para la detección temprana de la anemia brinda a la persona la oportunidad de monitorear su salud de manera más efectiva, en caso de los niños, es importante hacer chequeos constantes durante su crecimiento, ya que son la población más propensa a tener esta enfermedad, y la que más resulta afectada, la anemia en un menor de edad resulta en secuelas para toda la vida. El sistema de visión computacional deberá catalogar los resultados, para poder diferenciar si el usuario es tiene o no anemia.

Se utilizará una base de datos denominada EYES-DEFY-ANEMIA que se ha dividido en dos grupos de individuos: aquellos que muestran características particulares en la conjuntiva ocular que suelen presentarse en pacientes con anemia, como una tonalidad de piel notablemente blanquecina, y aquellos que no presentan ninguna de estas irregularidades. Las imágenes utilizadas en este estudio se encuentran en formato "jpg" y han sido capturadas utilizando un teléfono móvil Samsung S6. Estas imágenes serán sometidas a un análisis utilizando técnicas de visión computacional con el fin de detectar patrones anómalos. A partir de este análisis, se realizará un pre diagnóstico de la enfermedad en cuestión.

El CRISP-DM (Cross-Industry Standard Process for Data Mining) es una metodología reconocida y ampliamente utilizada para la planificación y ejecución de proyectos de minería de datos y análisis de negocios. Fue desarrollada en 1996 por un consorcio de

compañías que incluía a SPSS, NCR Corporation y DaimlerChrysler, y ha demostrado ser un marco de trabajo estructurado y flexible.

Esta metodología ofrece orientación a los profesionales en todas las etapas clave del proceso de análisis de datos, asegurando que se aborden adecuadamente y se alcancen los objetivos de negocio establecidos. Desde su creación, CRISP-DM ha sido adoptado en diversas industrias y disciplinas, manteniendo su relevancia a lo largo del tiempo, incluso en el contexto de los avances y nuevas técnicas en el campo del análisis de datos. Se estructura en seis fases fundamentales, las cuales brindan un enfoque sistemático y organizado para abordar los desafíos presentes en un proyecto de minería de datos (Shearer, 2000).

- Comprensión del negocio: En esta etapa, se busca comprender el contexto y los objetivos del proyecto desde una perspectiva empresarial. Se identifican los requerimientos, las limitaciones y los criterios de éxito, y se elabora un plan de proyecto detallado.
- Comprensión de los datos: En esta fase, se recopilan y exploran los datos disponibles. Se realiza un análisis para comprender la calidad de los datos, su estructura, los atributos y las relaciones entre las variables, con el objetivo de obtener una sólida comprensión de los datos a utilizar en el análisis.
- Preparación de los datos: En esta etapa, los datos se limpian, se transforman y se enriquecen para facilitar el modelado y el análisis. Esto puede implicar la corrección de errores, la imputación de valores faltantes, la eliminación de datos atípicos, la creación de variables derivadas y la selección de atributos relevantes.

- Modelado: En esta fase, se seleccionan y aplican técnicas de minería de datos y algoritmos de aprendizaje automático para construir modelos predictivos o descriptivos. Estos modelos se entrenan utilizando los datos preparados y se ajustan para optimizar su rendimiento.
- Evaluación: En esta etapa, se evalúa la calidad y el rendimiento de los modelos construidos en base a los objetivos de negocio y los criterios de éxito establecidos previamente. Se comparan diferentes modelos y enfoques para seleccionar el más adecuado para su implementación.
- Implementación: En esta última fase, se integran los modelos y los conocimientos adquiridos en los sistemas y procesos del negocio. Se monitorea la efectividad de los modelos a lo largo del tiempo y se realizan ajustes y mejoras según sea necesario.

Figura 2 *La metodología CRISP-DM*



Nota. Health Data Miner (s.f.). Recuperado de https://healthdataminer.com/data-mining/crisp-dm-una-metodologia-para-mineria-de-datos-en-salud/

1.5. Delimitación del Estudio

1.5.1. Delimitación espacial (escenario)

Para el presente trabajo de investigación, se utilizará la base de datos EYES-DEFY-ANEMIA, disponible a través del portal IEEE-dataport. Esta base de datos contiene imágenes de pacientes divididas en dos grupos: aquellos que padecen anemia y aquellos que no la padecen. Las imágenes se encuentran en formato "jpg" y han sido tomadas utilizando un teléfono inteligente Samsung S6.

El teléfono móvil estaba equipado con un dispositivo especial diseñado para ampliar las imágenes, estandarizar la iluminación y minimizar la influencia de la luz ambiente durante la captura de las imágenes. Este dispositivo permitió que todas las imágenes se tomaran a una distancia uniforme y bajo condiciones de iluminación consistentes, utilizando luz LED blanca.

El análisis se llevará a cabo siguiendo trabajos previamente realizados en esta área, y se desarrollará un sistema basado en información recopilada de libros, artículos e investigaciones relacionadas. El objetivo principal es diseñar un programa que pueda proporcionar un pre diagnóstico de anemia en pacientes a partir de una imagen de la conjuntiva ocular, lo cual podría facilitar el proceso de diagnóstico y permitir una detección más temprana y precisa de la enfermedad.

1.5.2. Delimitación temporal (periodo de tiempo)

La delimitación temporal de este estudio se basa en el conjunto de datos "EYES-DEFY-ANEMIA", el cual fue actualizado en enero de 2022. Aunque las imágenes de la conjuntiva ocular pueden provenir de diferentes fechas, es esencial considerar la calidad de la imagen y el equipo utilizado para su captura. El rendimiento del programa de pre diagnóstico de anemia dependerá en gran medida de la calidad de las imágenes analizadas. Si el equipo utilizado para obtener las imágenes no está actualizado o si las imágenes son de baja calidad, la precisión del pre diagnóstico podría verse afectada. Por lo tanto, se recomienda enfocar el estudio en imágenes de alta calidad tomadas con equipos actualizados y modernos, principalmente dentro del período que abarca desde enero de 2022 en adelante.

1.5.3. Delimitación conceptual (temas, áreas, procesos, etc.)

Este estudio se centra en el desarrollo de un programa de pre diagnóstico de anemia utilizando análisis de imágenes de la conjuntiva ocular del ojo y herramientas de inteligencia artificial, como la visión computacional. El objetivo principal es clasificar y predecir si una persona padece o no de anemia, basándose en la coloración de la conjuntiva ocular como indicador principal.

Para llevar a cabo este análisis, se explorarán técnicas de procesamiento de imágenes, aprendizaje automático y redes neuronales convolucionales, que permitan identificar y evaluar la coloración de la conjuntiva de manera precisa.

El objetivo de este estudio es brindar una contribución al campo de la medicina al desarrollar soluciones tecnológicas que permitan un acceso temprano al pre diagnóstico de la anemia para pacientes y sus familias. Esto tiene como finalidad mejorar la calidad de vida de los individuos y contribuir a la prevención y control de esta enfermedad.

CAPITULO II: MARCO TEÓRICO

En este capítulo, se presentarán los antecedentes, fundamentos teóricos y conceptos clave que brindan un contexto más completo para el problema estudiado. Esta información es fundamental para comprender los términos y conceptos relevantes, así como para examinar los resultados obtenidos en investigaciones previas con temáticas similares. Se analizarán las conclusiones alcanzadas por diversos autores y se explorará cómo estas conclusiones pueden tener un impacto e influencia en la presente investigación.

2.1. Antecedentes

Con el fin de obtener una comprensión más completa de este trabajo, se presentarán los antecedentes relevantes encontrados en investigaciones previas relacionadas con el tema seleccionado. Esto permitirá tener una visión más clara de cómo se ha abordado el tema en otros casos, los objetivos planteados, las conclusiones alcanzadas y las soluciones propuestas en dichos estudios. Al examinar estos antecedentes, se podrá contextualizar el presente estudio y construir sobre el conocimiento existente en el campo.

A Noninvasive Computerized Technique to Detect Anemia Using Images of Eye Conjunctiva

En el estudio titulado "A Noninvasive Computerized Technique to Detect Anemia Using Images of Eye Conjunctiva" publicado en 2019 por Shubham Bauskara, Prakhar Jaina y Manasi Gyanchandani, los autores enfatizan la importancia global de la anemia, un trastorno sanguíneo causado por la falta de glóbulos rojos sanos o hemoglobina. El objetivo de su investigación es presentar una técnica computarizada no invasiva que utiliza

imágenes de la conjuntiva ocular para detectar la anemia. Esto permitiría un diagnóstico temprano y una intervención oportuna en beneficio de los pacientes.

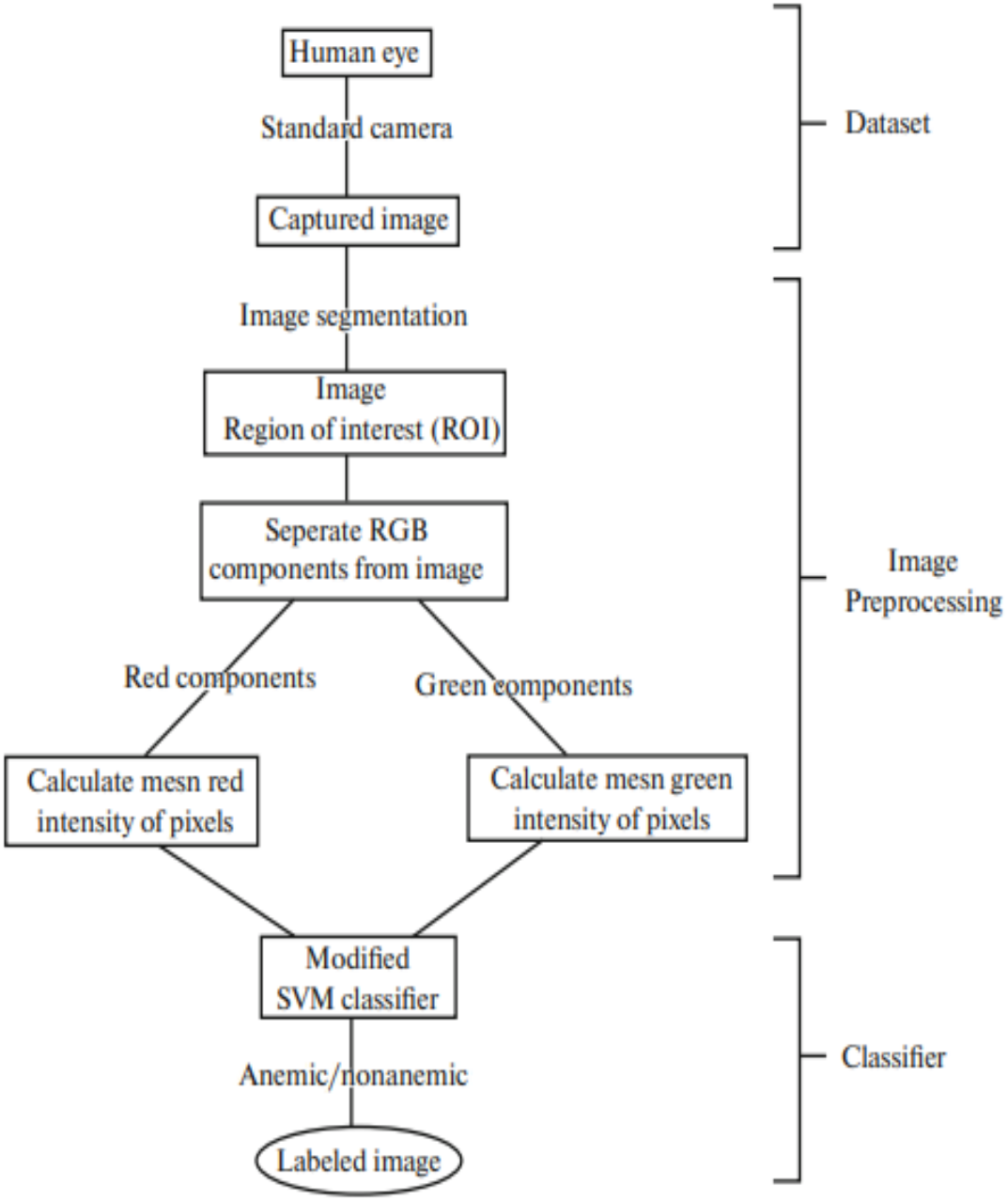
En el año de la investigación de este documento, en 2019, los autores tomaron en cuenta la información proporcionada por la Organización Mundial de la Salud. Según los datos de la organización, la prevalencia mundial de la anemia era del 24,8% de la población mundial, lo que representa aproximadamente dos mil millones de personas (World Health Organization, Geneva, 2008). Los autores destacan los desafíos que enfrentan en su entorno, donde el examen de detección de anemia es tedioso y costoso.

Se toma como premisa que existen técnicas no invasivas que permiten dar un pre-diagnostico si el paciente tiene anemia o no, durante un análisis físico es normal que se examine de forma visual la conjuntiva ocular del paciente y se examine el color de su membrana. Este método para la detección de anemia ha sido útil en muchos casos y puede determinar si se necesita un examen invasivo o si el paciente tiene las características de un paciente con anemia. (A. Kalantri el al,2010)

En este estudio, el proceso comienza con la recopilación de imágenes de la conjuntiva ocular de los pacientes, las cuales luego son analizadas. La adquisición de la base de datos se realizó de manera manual, siguiendo un procedimiento específico. Para capturar las imágenes, se jala suavemente el párpado inferior del ojo utilizando el dedo índice y se toma la fotografía. Es importante destacar que las imágenes se tomaron sin el uso del flash para evitar un exceso de luz.

El dataset utilizado en esta investigación está compuesto por un total de 99 imágenes. De estas, 51 imágenes corresponden a pacientes sanos y 48 imágenes corresponden a pacientes con anemia.

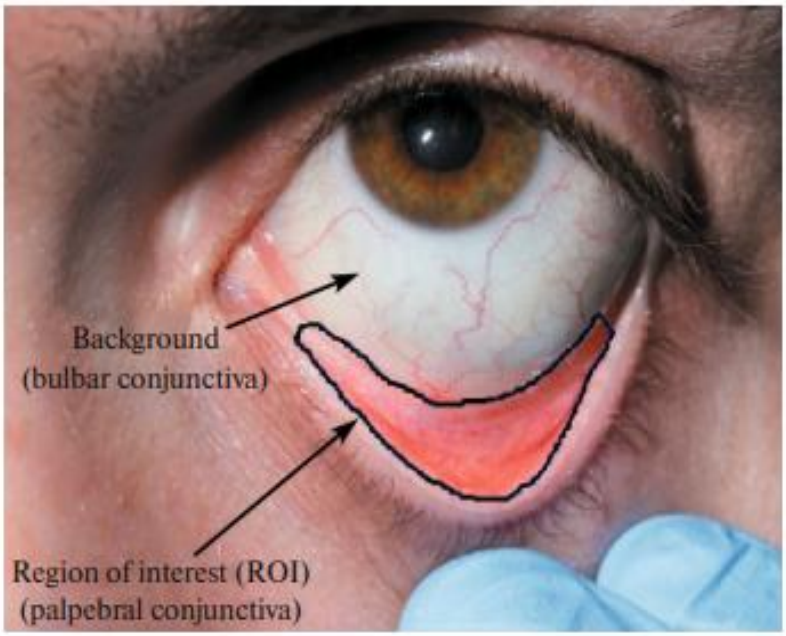
Figura 3 *Diagrama de flujo de la metodología propuesta*



Nota. Bauskar, S., Jain, P., & Gyanchandani, M. (2019). A Noninvasive Computerized Technique to Detect Anemia Using Images of Eye Conjunctiva [Figura 1]. Pattern Recognition and Image Analysis.

Se realiza el pre-procesamiento de las imágenes extrayendo la región de interés (ROI) de la imagen de la conjuntiva ocular del paciente, la cual es solamente la parte de la membrana la cual cambia de coloración dependiendo a su nivel de glóbulos rojos.

Figura 4 *Region de interes*

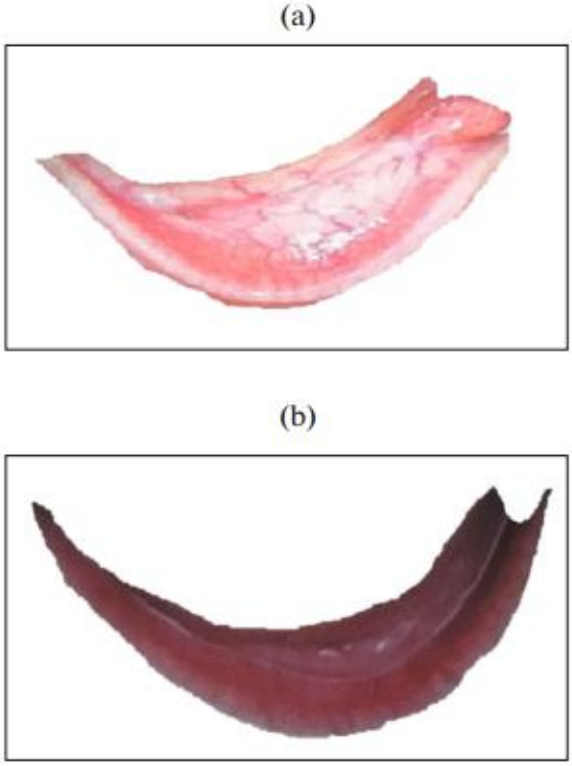


Nota. Bauskar, S., Jain, P., & Gyanchandani, M. (2019). A Noninvasive Computerized Technique to Detect Anemia Using Images of Eye Conjunctiva [Figura 2]. Pattern Recognition and Image Analysis.

Para establecer una relación entre los componentes de la región de interés es importante relacionarla con el nivel de glóbulos rojos que tiene el paciente, en base a los trabajos previos se puede notar que usando el índice de correlación de Pearson, se detectó en varios experimentos que cuando un paciente tienden a tener valores altos de hemoglobina los componentes de la región de interés tienen un color rojo y verde más intenso mientras que en pacientes anémicos no, por lo cual se puede discriminar unos de los otros.

A continuación, para realizar la clasificación de las imágenes se usó el modelo SVM, el cual en base a los parámetros y características de una persona anémica puede ser entrenado para diferenciar las imágenes.

Figura 5 *Conjuntiva ocular anémico (a) y Conjuntiva ocular no anémica (b)*



Nota: Bauskar, S., Jain, P., & Gyanchandani, M. (2019). A Noninvasive Computerized Technique to Detect Anemia Using Images of Eye Conjunctiva [Figura 3]. Pattern Recognition and Image Analysis

El estudio apunta a construir una clasificador para detectar la anemia basado en un método no invasivo que puede proporcionar una estimación más precisa de la concentración de glóbulos rojos analizando la conjuntiva ocular, los resultados de este experimento tuvieron un 93% de precisión usando la técnica SVM , se sigue recomendando que se realice una técnica invasiva para poder confirmar este pre diagnóstico, recomienda y recalca que las imágenes son un punto crucial para que el estudio sea exitoso, ya que en este caso el nivel de brillo ha hecho que algunas imágenes presenten error.

Tabla 2 *Análisis de la detección de anemia utilizando el método computarizado*

#	Region of interest (ROI) extracted from images	Mean intensity of red pixel (µr)	Mean intensity of green pixel (µg)	Blood report hemoglobin level, g/100 mL	Pathologists prediction	Proposed model prediction
1		233.42	206.15	10.3	Anemia	Anemia
2		229.55	205.77	6.8	Anemia	Anemia
3		235.51	214.54	7.8	Anemia	Anemia
4		235.15	190.23	11.8	Anemia	Anemia
5		238.32	205.48	8.2	Anemia	Anemia
6		237.9	200.85	9.2	Anemia	Anemia
7		237.89	199.24	8	Anemia	Anemia
8		229.05	188.53	11.3	Anemia	Anemia

9		212.22	179.72	14.9	NonAnemia	NonAnemia
10		221.59	169.38	15	NonAnemia	NonAnemia
11		202.59	169.38	15.5	NonAnemia	NonAnemia
12		206.97	190.74	15.1	NonAnemia	NonAnemia
13		211.63	183.77	15.8	NonAnemia	NonAnemia
14		216.27	193.59	16	NonAnemia	NonAnemia
15		218.27	183.28	14.7	NonAnemia	NonAnemia
16		206.74	176.8	15.3	NonAnemia	NonAnemia
17		226.25	207.76	11.5	NonAnemia	Anemia
18		213.63	186.6	15.3	NonAnemia	NonAnemia

Nota. Extraído de Bauskar et al, (2019)

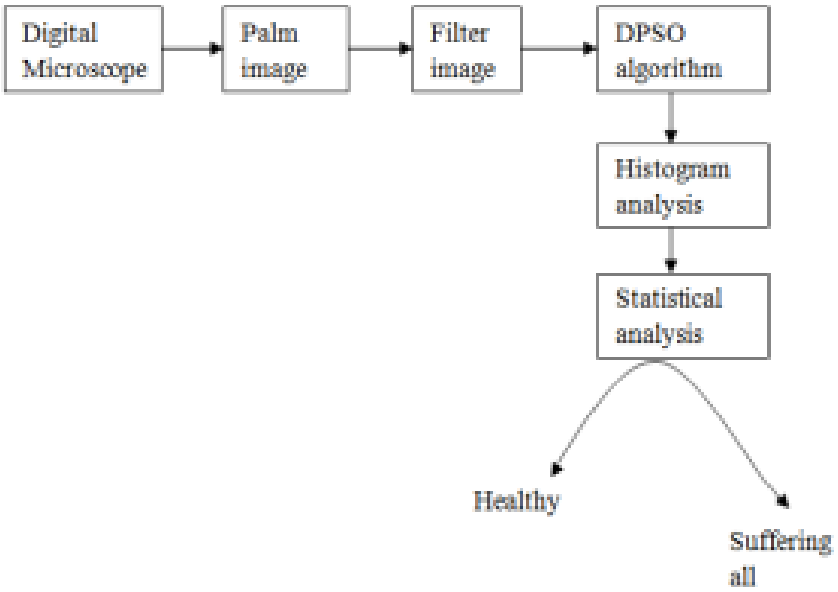
Detection of anemia disease using DPSO Algorithm

Esta investigación fue publicada el año 2019 bajo el título “Detection of anemia disease using dpso algorithm” cuyos autores son S. Dhanasekaran and N. R. Shanker. En la introducción del estudio se detalla que es la anemia, diferentes tipos y cuáles son las características que se muestran cuando un paciente padece esta condición. Por un lado, se habla acerca de cuál es la finalidad de hacer una evaluación a las células en la sangre, que es lo que se busca dentro del análisis y cuáles son los factores que determinar la salud del paciente. Por otro lado, se hace mención a como la tecnología ha ido perfeccionando la precisión de este diagnóstico. La anemia no es lo único que se puede detectar a través de un examen de sangre, la observación sobre las células permite encontrar diferentes condiciones y anomalías, una de ellas es la leucemia).

El estudio resalta que existen hardwares tales como el contador hematológico, el cual tiene como función analizar los principales parámetros: Leucocitos, Eritrocitos/Plaquetas y hemoglobina; así, este contador permite brindar un diagnóstico acertado (Arkalab, 2018). Sin embargo, estos son costosos y necesitan de muestras de sangre, por lo que se crea la necesidad de implementar un sistema simple, rentable y robusto de técnicas de conteo y clasificación de células en la sangre. Para detectar la condición del paciente se hará el recuento de los glóbulos rojos, cuando existe un problema en la sangre las células sanguíneas experimentan cambios morfológicos. La propuesta es dar una técnica computarizada para el conteo de glóbulos rojos, identificación y clasificación utilizando imágenes microscópicas, estas pasaran por métodos de procesamiento para separar las células rojas de los diferentes elementos que se encuentran en la sangre.

El método divide la imagen original en diferentes clases y regionales donde los pixeles que se encuentra en la misma región comparten algunas características en común, No es conveniente trabajar con la imagen entera por lo que se segmenta tomando en cuenta el color, todos los pixeles se representa en la imagen bajo el color rojo, verde y azul. La idea de este método de segmentación permite que el trabajo de análisis sea más sencillo y la carga computacional disminuya. La segmentación de imágenes para este caso también introduce técnicas como métodos basados en histogramas, técnicas de optimización, umbralización, agrupamiento, aumento de data, etc.

Figura 6 *Diagrama de la técnica propuesta*



Nota. Dhanasekaran, S., & Shanker, N. R. (2019). A Noninvasive Computerized Technique to Detect Anemia Using Images of Eye Conjunctiva [Figura 1, "Diagrama de bloques de la técnica propuesta"]. ARPN Journal of Engineering and Applied Sciences.

Los algoritmos PSO se recomiendan para este tipo de proceso, sin embargo, el autor proclama que estos no son efectivos, por lo que se pasara a usar Darwinian Particle Swarm Optimization (DPSO), esta técnica es superior a la recomendada, para este caso cada enjambre (Swarm) funciona por separado como un algoritmo PSO. El algoritmo DPSO hace

que estos enjambres recorran el espacio de la imagen, la partícula pasa por diferentes posiciones hasta encontrar la mejor ubicación o hasta que la velocidad este cerca a 0.

Tabla 3 *Parámetros iniciales para la aplicación del método DPSO*

Serial number	Parameters	DPSO
1	Num of interations	150
2	Population	30
3	P_1	0.8
4	P_2	0.8
5	V_{max}	1.5
6	V_{min}	-1.5
7	X_{max}	255
8	X_{min}	0
9	W	1.2
10	Min population	10
11	Max population	50
12	Num of swarms	4
13	Min swarms	2
14	Max swarms	6
15	Stagnancy	10
16	Fractional corfficient	---

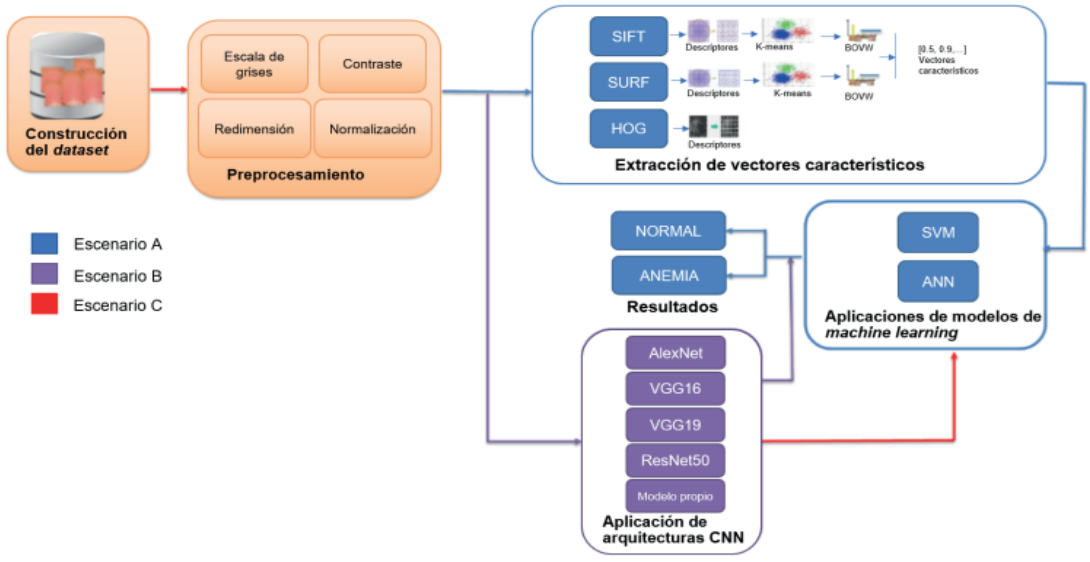
Nota: Extraído de Dhanasekaran and Shanker (2019)

Comparación de técnicas basadas en visión computacional y machine learning para la detección temprana de anemia a partir del análisis de uñas

Publicada en 2020, la investigación titulada "Comparación de técnicas basadas en visión computacional y machine learning para la detección temprana de anemia a partir del

análisis de uñas" fue llevada a cabo por Keico Anavela Heredia-Menor y Wilfredo Mamani-Ticona. El estudio comienza describiendo la prevalencia de la anemia en la población, con especial atención en el contexto peruano. Se destaca la importancia del diagnóstico en niños, adolescentes y mujeres embarazadas, y se subraya que la realización de análisis de sangre en laboratorios puede ser un proceso engorroso e inaccesible en ciertas áreas del país debido a la escasez de personal médico y equipos necesarios. La investigación tiene como objetivo comparar técnicas de visión computacional y machine learning en la detección temprana de anemia mediante el análisis de uñas, proporcionando así al personal médico herramientas adicionales para el diagnóstico de esta enfermedad. Los autores enfatizan que, aunque la anemia puede no ser prominente en sus primeras etapas, puede causar problemas crónicos y un desarrollo inadecuado en el individuo. La metodología se dividió en dos fases principales: la construcción de un dataset y el pre-procesamiento de las cuales se derivan tres escenarios donde se aplica diferentes técnicas de visión computacional y Machine Learning.

Figura 7 *Metodología*



Nota. Extraído de Heredia-Menor, K. A., & Mamani-Ticona, W. (2019). Título del documento [Figura 1, "Metodología de la investigación"]. Actas del III Congreso Internacional de Ingeniería de Sistemas.

En el presente estudio, se creó un conjunto de datos mediante la captura de imágenes de uñas de individuos con y sin anemia. Las fotografías se tomaron a una distancia de 5 cm, siguiendo las directrices propuestas por Mannino et al. (2018) para la toma de imágenes. El conjunto de datos incluye 2048 fotografías de uñas, de las cuales 1004 corresponden a uñas de individuos sin anemia (251 personas) y 1044 a uñas de personas con anemia (261 personas).

Tabla 4 *Fotos de uñas con y sin anemia por género*

Género	Sin anemia	Con anemia
Hombres (5-11años)	30	31
Mujeres (5-11 años)	31	32
Hombres (12-17 años)	29	31
Mujeres (12-17 años)	28	29
Hombres (18-39 años)	32	33
Mujeres (18-39 años)	29	30
Hombres (40-80 años)	35	38
Mujeres (40-80 años)	37	37
Total	251	261

Nota: Extraído de Heredia y Mamani (2021)

Tras la recopilación de datos, se llevó a cabo el pre procesamiento, dividiendo la información en los tres escenarios previamente establecidos. El primer escenario (A) abarca la creación del conjunto de datos y el pre procesamiento de estos, seguido de las etapas de extracción de características y aplicación de modelos de aprendizaje automático. Las imágenes se procesaron utilizando los modelos Scale Invariant Feature Transformation (SIFT), Speeded-Up Robust Features (SURF) e Histogram of Oriented Gradient (HOG). Los descriptores SIFT y SURF identifican puntos característicos de forma diferente y reconocen distintas propiedades de la uña. El descriptor HOG se empleó para detectar información de bordes y texturas en la imagen.

Los resultados del primer escenario indican que el extractor de características SIFT, en conjunto con el algoritmo de Máquinas de Vectores de Soporte (SVM), proporciona los mejores resultados considerando diversas métricas. Esto es importante, ya que se aborda un tema de salud y se busca la mayor confiabilidad posible.

Tabla 5 Resultados de SIFT, SURF (k=10, 50, 100 y 200) y HOG

Descriptores		Sift				Surf				HOG
K		10	50	100	200	10	50	100	200	-
SVM	Accuracy	0.72	0.78	0.82	0.83	0.74	0.78	0.82	0.82	0.76
	Precision	0.75	0.77	0.82	0.83	0.77	0.97	0.82	0.85	0.81
	Recall	0.7	0.7	0.83	0.84	0.71	0.7	0.83	0.8	0.7
	F1 - Score	0.72	0.78	0.82	0.83	0.74	0.78	0.82	0.82	0.79
ANN1	Accuracy	0.74	0.73	0.8	0.8	0.7	0.7	0.8	0.8	0.63
	Precision	0.79	0.82	0.85	0.85	0.73	0.82	0.85	0.85	0.62
	Recall	0.7	0.75	0.72	0.72	0.62	0.75	0.72	0.72	0.63
	F1 - Score	0.73	0.78	0.78	0.78	0.67	0.78	0.78	0.78	0.62
ANN2	Accuracy	0.74	0.79	0.76	0.76	0.7	0.79	0.76	0.76	0.61
	Precision	0.79	0.73	0.77	0.77	0.7	0.73	0.77	0.77	0.62
	Recall	0.73	0.74	0.76	0.76	0.69	0.74	0.76	0.76	0.6
	F1 - Score	0.72	0.72	0.78	0.78	0.71	0.72	0.78	0.78	0.61
ANN3	Accuracy	0.71	0.74	0.8	0.8	0.7	0.72	0.72	0.75	0.61
	Precision	0.72	0.75	0.8	0.8	0.7	0.73	0.74	0.74	0.62
	Recall	0.7	0.74	0.81	0.81	0.69	0.71	0.71	0.71	0.6

Nota: Extraído de Heredia y Mamani (2021)

En el segundo escenario (B), se llevan a cabo las dos primeras fases (creación del conjunto de datos y pre procesamiento) antes de aplicar arquitecturas de redes neuronales convolucionales (CNN). En el marco de esta investigación, se utilizaron las arquitecturas ResNet50, VGG19, VGG16 y AlexNet. Para su implementación, se asignó el 70% del conjunto de datos al entrenamiento con el objetivo de incrementar la cantidad de imágenes y, de esta manera, facilitar el entrenamiento de las CNN. Se emplearon diversos métodos para aumentar el volumen de datos a procesar.

Se asignaron diferentes parámetros a cada modelo con el fin de realizar un análisis más exhaustivo y optimizar el tiempo de ejecución.

Tabla 6 Parámetros modificados al momento de compilar

Modelos	Learning rate	Epochs	Batch_size
AlexNet	1E-07	120	32
VGG16	000,001	120	64
VGG19	0.00001	120	64
ResNet	1E-06	120	32
Modelo pro-pio	0.00001	120	32

Nota: Extraído de Heredia y Mamani (2021)

La investigación determina que el modelo más adecuado para abordar esta problemática pertenece a la familia Visual Geometry Group (VGG), dado que VGG19 logra la mayor precisión y también presenta un rendimiento óptimo en comparación con los demás modelos evaluados.

Tabla 7 *Grupo de Geometría Visual*

Arquitecturas	Accuracy	Precision	Recall	F1-Score	Log Loss
VGG19	0.97	0.97	0.99	0.98	0.05
VGG16	0.96	0.96	0.97	0.96	0.06
AlexNet	0.94	0.95	0.9	0.92	0.18
RestNet50	0.96	0.96	0.98	0.97	0.07
Modelo Propio	0.94	0.94	0.9	0.92	0.25

Nota: Extraído de Heredia y Mamani (2021)

En el tercer escenario (C), se ejecutan inicialmente las dos primeras fases (creación del conjunto de datos y pre procesamiento), seguidas de la aplicación de arquitecturas de redes neuronales convolucionales (CNN) y modelos de aprendizaje automático. Este escenario combina ambas arquitecturas y utiliza el aprendizaje automático como clasificador.

Durante la investigación, se extrajeron los vectores característicos de la penúltima capa de los modelos CNN pre entrenados. Los vectores resultantes después de aplicar los filtros se utilizaron como entradas para los modelos de aprendizaje automático. Al igual que en el proceso previo, se buscó mejorar la calidad de los datos mediante la aplicación de técnicas de aumento de datos al conjunto de datos.

Los resultados mostrados en la tabla correspondiente revelan que VGG19 y VGG16, ambos combinados con modelos SVM, superan a las otras arquitecturas evaluadas. Además, es importante destacar que los modelos de aprendizaje automático obtuvieron un rendimiento superior en comparación con los demás.

Tabla 8 *Resultados de las CNN de los modelos de machine learning*

CNNS	Modelos ML	Accuracy	Precision	Recall	F1-score
VGG19	SVM	0.989	0.98	0.98	0.98
	ANN	0.9589	0.95	0.94	0.94
VGG16	SVM	0.9821	0.98	0.98	0.98
	ANN	0.9581	0.95	0.95	0.95
AlexNet	SVM	0.9789	0.97	0.97	0.97
	ANN	0.948	0.94	0.93	0.93
ResNet50	SVM	0.985	0.98	0.98	0.98
	ANN	0.957	0.95	0.96	0.96

Nota: Extraído de Heredia y Mamani (2021)

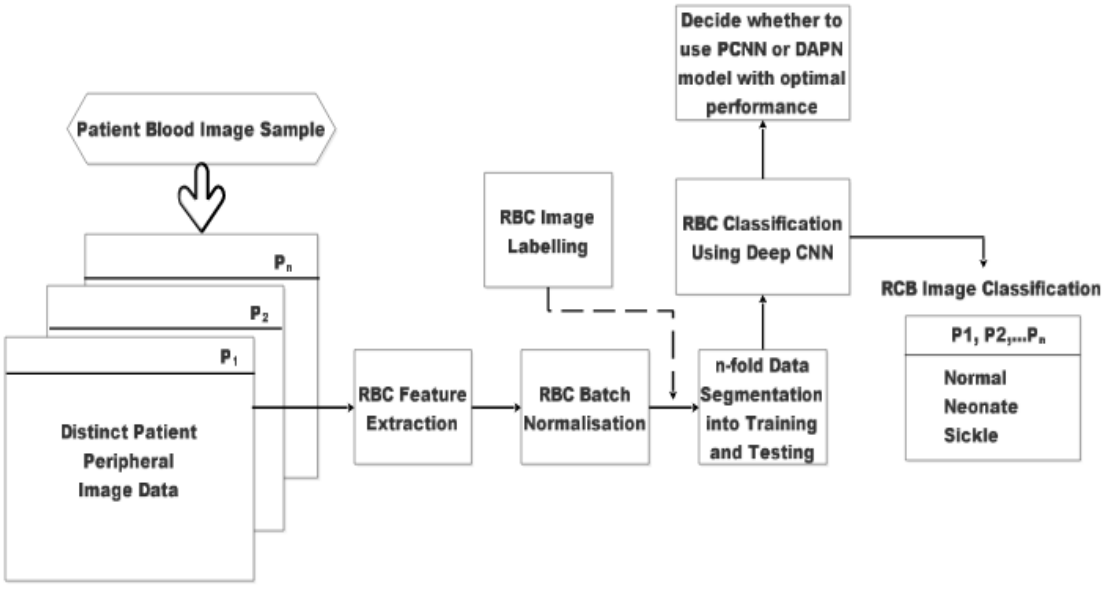
El estudio concluye destacando que los mejores resultados se obtienen utilizando la arquitectura VGG19, tanto en el escenario B como en el C. Sin embargo, los resultados más sobresalientes se encuentran en el escenario C, donde se combinaron las redes neuronales convolucionales (CNN) y los modelos de Máquinas de Vectores de Soporte (SVM), logrando una exactitud de 0,989, una precisión de 0,98, un recall de 0,98 y un puntaje F1 de 0,98. Por lo tanto, las técnicas más eficientes consisten en el uso de CNN en conjunto con la arquitectura VGG19.

Deep Convolutional Neural Network Model for Detection of Sickle Cell Anemia in Peripheral Blood Images

La Anemia drepanocítica es un desorden de las células rojas en la sangre, esta enfermedad reduce la calidad y tiempo de vida del paciente significativamente. Este estudio muestra cómo afecta a las naciones, a diferentes grupos de personas esta condición, también se habla de los costos de las técnicas convencionales de diagnóstico, las cuales tienden a ser costosas, de poca precisión, requieren de médicos expertos y toman tiempo.

El objetivo de esta investigación es encontrar un método que permita la pronta detección y diagnóstico de la Anemia drepanocítica (SCD). Para este trabajo se empleó Plain Convolution Neural Network (PCNN) con 48 capas (DAPN-48), Very Deep Convolutional Networks para el reconocimiento de imágenes grandes con 19 capas (VGG19), y Residual Networks con 50 capas (RESNET-50) para detectar SCD a través de las muestras de imágenes de la sangre periférica.

Figura 8 *Proceso principal*



Nota. Extraído de Gbenga et. al (2022)

El resultado obtenido en la investigación representa como PCNN y DAPN permiten la clasificación RBC con las imágenes de las células normales, células ovaladas y células enfermas, la fusión de el proceso de aumento de data y CNN permite una mejor clasificación y como resultado un nivel de precisión más alto.

Tabla 9 *Evaluación de métricas de PCNN-15*

Algorithms	Sensitivity	Specificity	Balanced Accuracy
Normal	100	99.67	100
Neonate	100	100	99.83
Sickle cell	100	99.83	99.92

Nota: Gbenga et al (2022)

El estudio recomienda trabajar con aumento y PCNN con 15 capas para obtener el mejor performance, para futuros trabajos tomar en cuenta métodos que se integren a la técnica CNN, de misma manera, confirma que este tipo de tecnologías, Deep learning y machine learning son útiles dentro de esta área de la medicina.

Smartphone app for non-invasive detection of anemia using only patient-sourced photos

El origen de este estudio radica en la prevalencia global de la anemia, una condición caracterizada por niveles reducidos de hemoglobina en sangre, que afecta a más de 2 mil millones de individuos en todo el mundo. Los investigadores priorizaron la anemia como el enfoque inicial de sus esfuerzos diagnósticos debido a su impacto significativo en la salud global (Nature Communications, 2023).

La aplicación para teléfonos inteligentes diseñada por los investigadores exhibe una precisión considerable en la detección de la anemia, definida en este contexto como niveles de hemoglobina inferiores a 12,5 g/dL. La aplicación logró una precisión general de ±2,4 g/dL y una sensibilidad del 97% en comparación con los niveles de hemoglobina del re-

cuento completo de sangre (CBC). Estas métricas señalan el potencial de esta aplicación pa-ra funcionar como una herramienta de detección no invasiva para la anemia (Nature Com-munications, 2023).

Además, la aplicación ha sido diseñada con la personalización en mente, especialmen-te para los pacientes que padecen anemia crónica. Con la calibración personalizada, la apli-cación puede lograr una precisión de ±0,92 g/dL de los niveles de hemoglobina del CBC. Esta característica dota a los pacientes con anemia crónica de la capacidad de monitorizar sus niveles de hemoglobina de forma instantánea y remota (Nature Communications, 2023).

La aplicación se basa en algoritmos de análisis de imágenes que se utilizan para examinar fotos de las uñas de los pacientes, que son fáciles de tomar y proporcionan una fuente consistente de imágenes entre diferentes individuos. La palidez de las uñas puede estar asociada con la anemia, y el color en estas áreas se debe principalmente a la hemoglobina en la sangre. Al analizar las fotos de las uñas de los pacientes, la aplicación puede estimar los niveles de hemoglobina y detectar la anemia con un alto grado de precisión (Nature Communications, 2023).

Figura 9 *Flujo de Detección de Anemia por Appl*



Nota. Extraído de Mannino, R.G., Myers, D.R., Tyburski, E.A. et al. Smartphone app for non-invasive detection of anemia using only patient-sourced photos. Nat Commun 9, 4924 (2018). https://doi.org/10.1038/s41467-018-07262-2

Un aspecto distintivo de esta aplicación es su uso de los metadatos de la imagen, una fuente de información considerable que ha sido en gran medida ignorada en otros estudios que utilizan smartphones para el diagnóstico. Al no requerir equipo externo, esta aplicación permite la medición de los niveles de hemoglobina en cualquier momento y lugar, siempre que el usuario tenga su smartphone. Esto facilita la autoevaluación y permite a los usuarios tener un mayor control sobre su atención médica (Nature Communications, 2023).

2.2. Bases Teóricas

2.2.1. Anemia

La anemia ferropénica es una enfermedad hematológica frecuente y multifactorial que se caracteriza por una disminución de la concentración de hemoglobina en sangre de-

bido a la deficiencia de hierro, lo que afecta la producción y función de los eritrocitos (glóbulos rojos) (Camaschella, 2015). La falta de hierro en el organismo resulta en la producción insuficiente de hemoglobina, un componente crítico de los glóbulos rojos, que es necesario para el transporte de oxígeno desde los pulmones hasta los tejidos y órganos del cuerpo.

Varias son las causas subyacentes de esta forma de anemia, con la dieta insuficiente en hierro, las pérdidas crónicas de sangre y la mala absorción del hierro entre las más comunes. En muchos casos, se puede deber a la ingestión insuficiente de hierro en la dieta o a la falta de absorción del hierro por el intestino, como puede ocurrir en algunas enfermedades gastrointestinales. En las mujeres, la pérdida excesiva de sangre durante la menstruación o durante el parto puede llevar a la anemia ferropénica. El tratamiento de la anemia ferropénica a menudo implica la administración de suplementos de hierro y la modificación de la dieta para aumentar la ingesta de alimentos ricos en hierro. Sin embargo, es esencial identificar y tratar la causa subyacente de la anemia para prevenir la recurrencia de la deficiencia de hierro.

Figura 10 *Tipos de Anemia*



Nota. Extraído de https://www.thinglink.com/scene/1370579758693220353

Entre las diferentes causas de la anemia se encuentran:

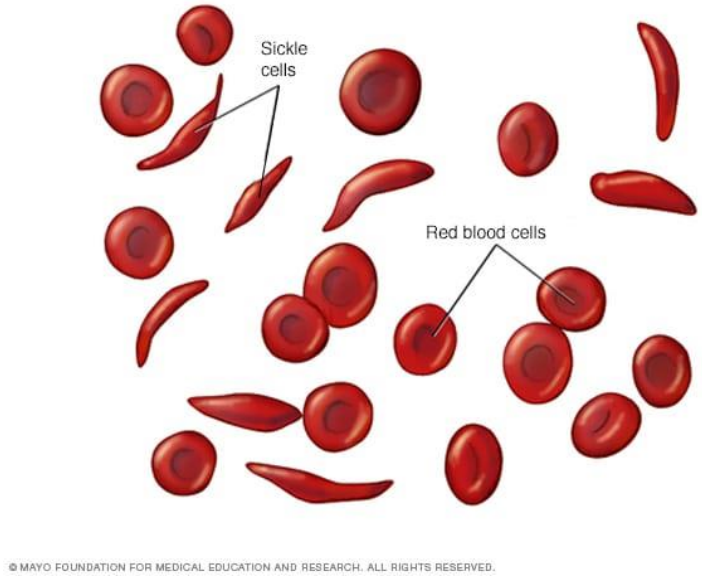
- Deficiencia de Hierro: Esta es la causa más común de anemia en todo el mundo, se produce cuando el cuerpo no tiene suficiente hierro para producir hemoglobina. La deficiencia de hierro puede deberse a la pérdida de sangre, a una dieta baja en hierro o a la incapacidad del cuerpo para absorber adecuadamente el hierro (Camaschella, 2015).
- Deficiencia de Vitamina B12 y Ácido Fólico: La deficiencia de estas dos vitaminas esenciales puede causar anemia megaloblástica, en la que los glóbulos rojos son más grandes y menos eficientes para transportar oxígeno. Este tipo de anemia es común en individuos con una dieta pobre o en aquellos con afecciones que afectan

la absorción de nutrientes, como la enfermedad celíaca o la enfermedad de Crohn (Green et al., 2017).

La anemia puede afectar a personas de todas las edades, géneros y orígenes étnicos. La prevalencia de la anemia varía según la población y puede ser más alta en ciertos grupos demográficos, como mujeres embarazadas, niños y personas mayores (World Health Organization, 2021). Además, la malnutrición y la falta de acceso a la atención médica pueden contribuir a la anemia en ciertas poblaciones (Camaschella, 2015).

El diagnóstico de la anemia generalmente implica pruebas de laboratorio, como un hemograma completo, que mide la cantidad de glóbulos rojos, hemoglobina y otros componentes de la sangre. Dependiendo de los resultados de estas pruebas y de la historia clínica del paciente, se pueden realizar pruebas adicionales para determinar la causa específica de la anemia (Mayo Clinic, 2020). El tratamiento de la anemia dependerá de la causa subyacente y puede incluir suplementos de hierro, vitamina B12 o ácido fólico, medicamentos para tratar enfermedades crónicas, transfusiones de sangre y, en casos raros, trasplantes de médula ósea (Mayo Clinic, 2020).

Figura 11 *Células Falciformes*



Nota. Mayo Clinic. (2022). [Imagen de la anemia de células falciformes]. Recuperado de https://www.mayo-clinic.org/es-es/diseases-conditions/sickle-cell-anemia/symptoms-causes/syc-20355876

La prevención de la anemia puede incluir la adopción de una dieta equilibrada que contenga suficientes nutrientes, como hierro, vitamina B12 y ácido fólico, y la realización de controles médicos regulares para detectar problemas de salud que puedan causar anemia (Mayo Clinic, 2020).

Palidez como signo específico de la anemia:

La conjuntiva ocular es una membrana mucosa que recubre la superficie interna de los párpados y la esclerótica, la porción blanca del ojo. Su apariencia puede variar entre un tono rosado pálido y un rojo profundo, dependiendo de diversos factores, como la vasculatura subyacente y el nivel de oxígeno en la sangre. La coloración de la conjuntiva puede verse afectada por condiciones como la anemia, una enfermedad en la que la capacidad de la sangre para llevar oxígeno está comprometida debido a la disminución de la cantidad de hemoglobina o de glóbulos rojos (RBC) en el cuerpo (Simel & Rennie, 2009).

Una de las señales visuales que se observan en el examen clínico de los pacientes con anemia, en particular con anemia ferropénica, es la palidez de la conjuntiva ocular. Esta condición se manifiesta como un blanqueamiento o pérdida del color rosado normal de la conjuntiva. Esta palidez puede ser un indicador de un nivel bajo de hemoglobina en la sangre, lo que a su vez puede ser un signo de anemia. Según la Organización Mundial de la Salud, se considera anemia cuando los niveles de hemoglobina caen por debajo de 13 g/dL en hombres adultos, 12 g/dL en mujeres no embarazadas y 11 g/dL en mujeres embarazadas (WHO, 2011).

Es importante resaltar que, aunque la palidez de la conjuntiva ocular puede indicar anemia, no es un diagnóstico definitivo de la misma. El diagnóstico de la anemia ferropénica requiere un examen más completo que incluya pruebas de laboratorio, como un conteo sanguíneo completo, que puede proporcionar una medida más precisa de los niveles de hemoglobina y el recuento de glóbulos rojos (Weiss & Goodnough, 2005).

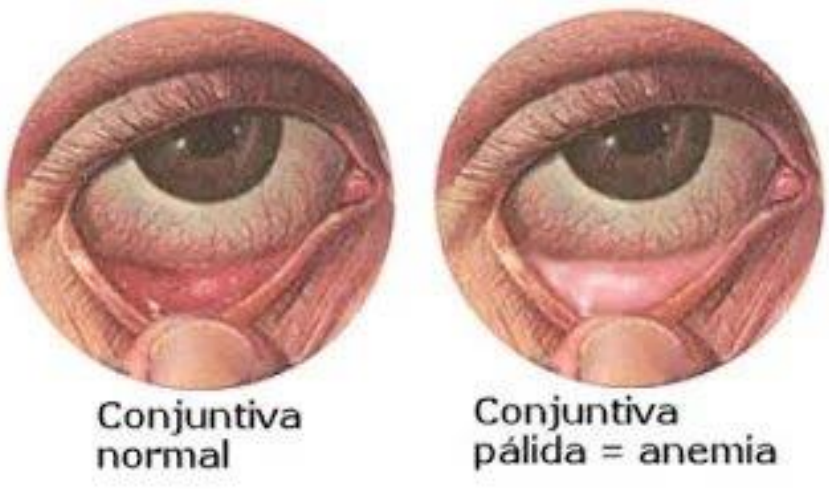


Figura 12 *Conjuntiva y el nivel de anemia*
Nota. Extraído de https://twitter.com/martinfd78/status/1091624116795461632

Diagnóstico de la anemia:

- Hemograma completo (CBC): Hemograma completo (CBC): La prueba de hemograma completo (CBC) es la herramienta de diagnóstico principal para la anemia. Proporciona información detallada sobre los componentes celulares de la sangre, esencialmente los glóbulos rojos, blancos y plaquetas (Zhou et al., 2020). El CBC analiza varios parámetros de los glóbulos rojos, como el recuento de glóbulos rojos, la cantidad de hemoglobina (la proteína que transporta el oxígeno en los glóbulos rojos), el volumen corpuscular medio (VCM, que mide el tamaño promedio de los glóbulos rojos) y la concentración de hemoglobina corpuscular media (CHCM, que mide la cantidad promedio de hemoglobina por glóbulo rojo). La anemia se diagnostica cuando el número de glóbulos rojos y/o la cantidad de hemoglobina es menor de lo normal, lo que reduce la capacidad de la sangre para transportar oxígeno a los tejidos (Bain, 2015).
- Prueba de ferritina sérica: Esta prueba mide la cantidad de ferritina en el suero de la sangre. La ferritina es la principal proteína de almacenamiento de hierro en el cuerpo, y su concentración en el suero se correlaciona con la cantidad total de hierro almacenado en el cuerpo. Cuando las reservas de hierro del cuerpo están bajas, el nivel de ferritina en el suero también disminuye. Esta prueba es particularmente útil para diagnosticar la anemia por deficiencia de hierro, que es el tipo más común de anemia (Hoffman et al., 2018).
- Examen físico: Durante un examen físico, el médico puede buscar signos clínicos de anemia. Estos pueden incluir palidez en la piel y la conjuntiva de los ojos, fragilidad en las uñas, lengua inflamada y/o lisa (glositis), o incluso úlceras en la boca.

Además, los síntomas que el paciente informa, como fatiga, dificultad para respirar, mareos y palpitaciones, pueden indicar anemia (Kumar et al., 2018).

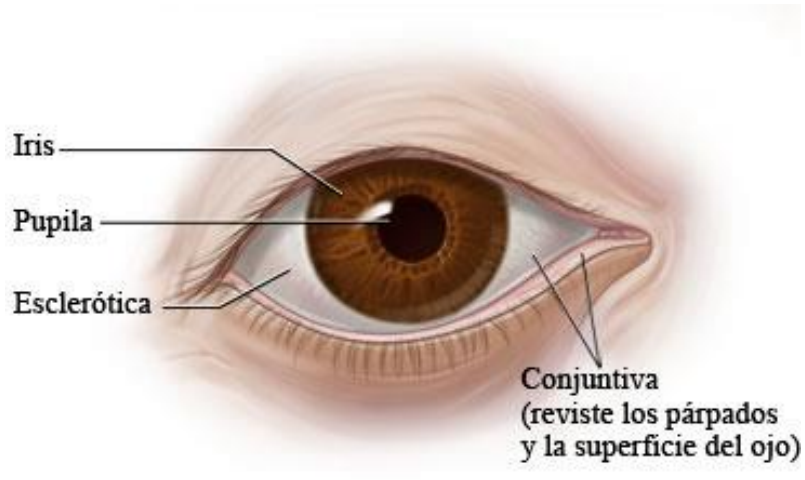
Anatomía, fisiología ocular y anemia:

Para comprender la relación entre la conjuntiva ocular y la anemia, es importante conocer la estructura y función del ojo, así como la función de la conjuntiva en el suministro de oxígeno y nutrientes al tejido ocular.

Conjuntiva ocular: Es una membrana delgada y transparente que recubre la parte frontal del ojo y la parte interna de los párpados. La conjuntiva está compuesta por varias capas, incluida la epitelial, la estromal y la capa más profunda de vasos sanguíneos. Estos vasos sanguíneos son cruciales para proporcionar oxígeno y nutrientes al tejido conjuntival y para eliminar los productos de desecho.

2.2.2. Fisiología Ocular

Figura 13 *Fisiología Ocular*



Nota. Clínica oftalmológica Rahhal. (2021). Imagen [Fotografía]. Recuperado el 17 de mayo de 2023, de https://www.rahhal.com/derrame-ocular-causas-tratamientos/

Circulación sanguínea en la conjuntiva: La circulación sanguínea en la conjuntiva es proporcionada por una red de vasos sanguíneos que irrigan el tejido. La calidad y cantidad de sangre en estos vasos puede verse afectada por la anemia, lo que a su vez puede causar

cambios en la apariencia de la conjuntiva. En casos de anemia, la conjuntiva puede volverse más pálida debido a la disminución de los niveles de hemoglobina y la menor cantidad de glóbulos rojos en los vasos sanguíneos.

Signos de anemia en la conjuntiva ocular: La palidez de la conjuntiva ocular es un signo clínico comúnmente utilizado para evaluar la presencia de anemia. Al examinar la conjuntiva, los médicos pueden estimar la severidad de la anemia y determinar si se requieren pruebas adicionales. Principio del formulario

Conjuntiva palpebral

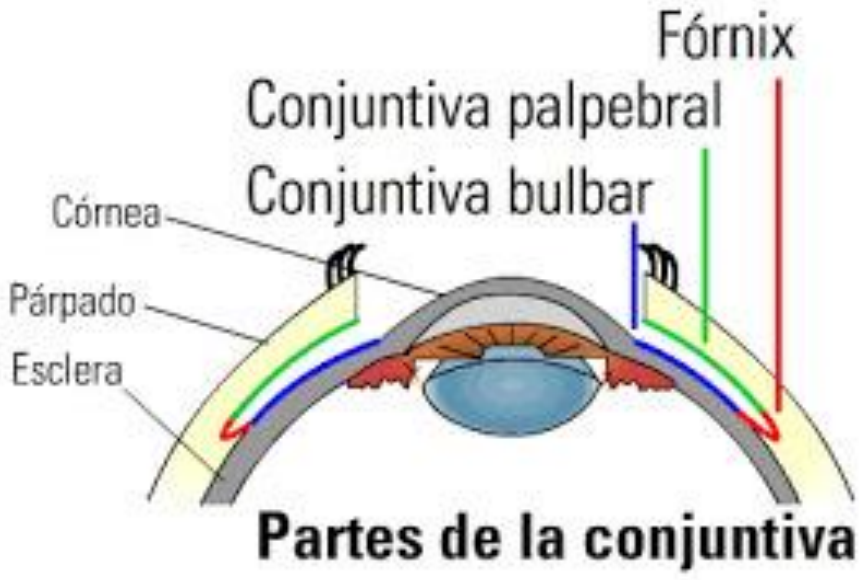
La conjuntiva palpebral es una estructura anatómica vital en el sistema ocular humano, que juega un papel significativo en la protección y el funcionamiento óptimo del ojo. Anatómicamente, la conjuntiva es una membrana delgada y transparente que recubre el interior de los párpados y la superficie blanca del ojo, conocida como esclerótica. Se compone de dos segmentos: la conjuntiva bulbar, que se extiende sobre la esclerótica, y la conjuntiva palpebral, que recubre la superficie interior de los párpados. Estos dos segmentos se encuentran en un área de transición conocida como el fórnix conjuntival (Paulsen et al., 2013).

La conjuntiva palpebral, en particular, es de gran importancia clínica debido a su visibilidad y accesibilidad. Esta fina capa de tejido es extremadamente delgada y contiene numerosas glándulas que producen moco y lágrimas para mantener el ojo lubricado. Esta lubricación es esencial para mantener la córnea, la parte frontal transparente del ojo, húmeda y protegida contra irritantes.

En el contexto de la anemia, la conjuntiva palpebral puede proporcionar indicios visuales importantes sobre la condición del paciente. La anemia, una condición caracterizada

por una disminución en el número de glóbulos rojos o en la cantidad de hemoglobina en la sangre, puede provocar que la conjuntiva palpebral adquiera una apariencia pálida o blanquecina en lugar de su color rosado saludable (Nathan et al., 2003). Este signo clínico puede ser útil para la identificación preliminar de la anemia, especialmente en áreas donde las pruebas de laboratorio pueden no estar fácilmente disponibles. Sin embargo, es importante destacar que, aunque la palidez de la conjuntiva puede indicar la presencia de anemia, no proporciona una medida precisa de la gravedad de la enfermedad.

Figura 14 *Esquema conjuntiva*



Nota. Ribes Escudero, D. (2017). Esquema Conjuntiva [Imagen]. Recuperado de https://drribesescudero.com/superficie-ocular/conjuntivitis-viral/esquema-conjuntiva_web-2/.

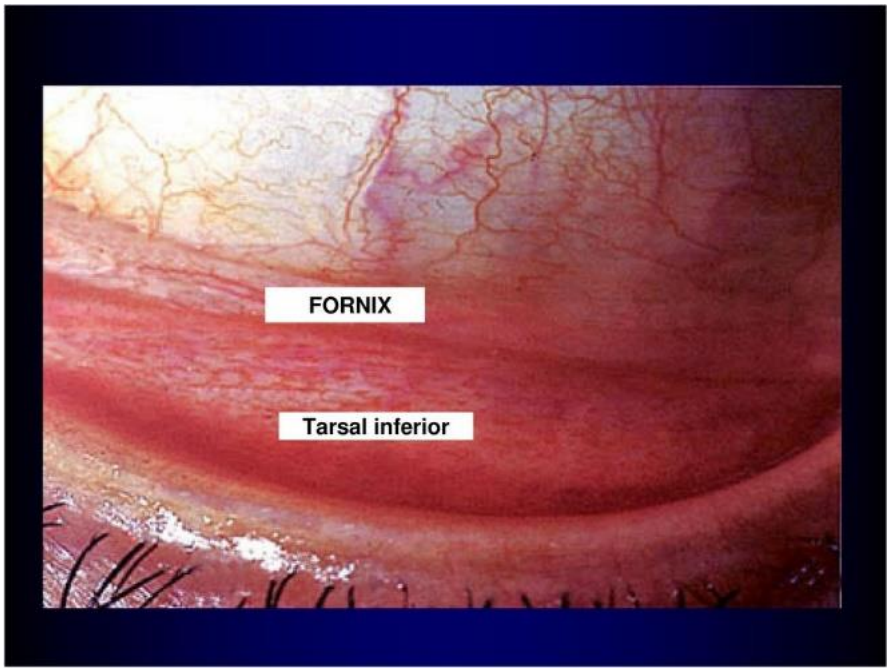
Conjuntiva fórnix

El fórnix conjuntival, o simplemente fórnix, es una estructura importante dentro del sistema ocular humano. Este componente crucial desempeña un papel en el movimiento del ojo y en la distribución de la lágrima sobre la superficie ocular. Anatómicamente, el fórnix conjuntival se encuentra en el punto de transición entre la conjuntiva bulbar y la conjuntiva

palpebral, dentro de la cavidad ocular. Específicamente, este pliegue en forma de U se encuentra en la parte superior e inferior del ojo, donde el párpado se une al globo ocular (Tseng, 1985). Esta disposición anatómica permite que la conjuntiva se mueva libremente con el ojo y los párpados, proporcionando así una amplia gama de movimiento.

El fórnix conjuntival es esencial para el correcto funcionamiento del ojo. Alberga células caliciformes y glándulas que contribuyen a la formación del componente mucoso de la película lagrimal, que es vital para mantener la córnea húmeda y protegida. Además, el fórnix sirve como un reservorio para las lágrimas y el exceso de fluido, permitiendo su distribución uniforme sobre la superficie ocular durante el parpadeo (Thoft & Friend, 1983).

Figura 15 *Partes de la conjuntiva ocular*



Nota. Extraído de https://docplayer.es/5095360-Leccion-6-enfermedades-de-la-conjuntiva-curso-2006-2007.html

2.2.3. Visión computacional

2.2.3.1 Historia

La visión por computadora es un campo de estudio interdisciplinario que se enfoca en cómo las computadoras pueden obtener un entendimiento avanzado a partir de imágenes digitales o videos. Su objetivo, desde una perspectiva de ingeniería, es automatizar tareas que normalmente realiza el sistema visual humano. Esta disciplina se ocupa de extraer, analizar y comprender información útil de una imagen individual o de una secuencia de imágenes. Implica el desarrollo de bases teóricas y algorítmicas para lograr una comprensión visual automática. Como disciplina científica, la visión por computadora investiga la teoría detrás de los sistemas artificiales que extraen información de las imágenes. Los datos de las imágenes pueden presentarse en diversas formas, como secuencias de video, múltiples vistas de cámaras o datos multidimensionales provenientes de escáneres médicos. Como disciplina tecnológica, la visión por computadora busca aplicar estas teorías y modelos en la construcción de sistemas visuales por computadora.

Sin embargo, la visión por computadora representa un desafío debido a la complejidad inherente del mundo visual y a nuestra comprensión limitada sobre el funcionamiento de la visión biológica. A pesar de estos desafíos, se han logrado avances significativos en áreas como el reconocimiento óptico de caracteres y la detección de rostros.

- Primeras etapas: años 1960-1970

La visión computacional, como campo de estudio, comenzó a tomar forma en los años 60 y 70. Durante este tiempo, los investigadores estaban interesados en imitar la visión humana y desarrollar máquinas que pudieran "ver" e interpretar imágenes. Larry Roberts, a menudo citado como uno de los fundadores de la visión computacional, presentó

en 1963 una tesis en el MIT en la que describía objetos tridimensionales a partir de una serie de líneas conectadas. Este trabajo se considera uno de los primeros en el campo de la visión computacional (Roberts, 1963). Posteriormente, en los años 70 la visión computacional continuó evolucionando con el desarrollo de técnicas para detectar bordes y segmentar imágenes. Durante este tiempo, los investigadores también comenzaron a explorar la idea del reconocimiento de objetos, con énfasis en el reconocimiento de formas y la descripción de objetos en términos de sus características o propiedades.

- Desarrollo de algoritmos y técnicas: años 1980-1990

En los años 80 y 90, la visión computacional experimentó un rápido desarrollo con la introducción de algoritmos más sofisticados y técnicas de procesamiento de imágenes. Durante este tiempo, se hicieron avances significativos en la detección de bordes y la segmentación de imágenes, así como en la extracción de características y el reconocimiento de objetos. Los algoritmos de detección de bordes, como el detector de bordes de Canny, y los algoritmos de segmentación de imágenes, como el algoritmo de Watershed, fueron desarrollados durante este tiempo (Canny, 1986; Beucher & Meyer, 1993).

- Revolución de la inteligencia artificial: años 2000-presente

A principios de los años 2000, la visión computacional experimentó una revolución con la introducción de la inteligencia artificial y, en particular, del aprendizaje profundo. El aprendizaje profundo proporciona un marco para el entrenamiento de modelos que pueden aprender características relevantes de los datos de entrada, eliminando la necesidad de características diseñadas manualmente. En 2012, la red neuronal convolucional (CNN) de Alex Krizhevsky, Ilya Sutskever y Geoffrey Hinton, conocida como AlexNet, ganó el desafío de clasificación de ImageNet por un amplio margen, marcando el comienzo de la

era del aprendizaje profundo en la visión computacional (Krizhevsky, Sutskever & Hinton, 2012). Desde entonces, las CNN y otras arquitecturas de aprendizaje profundo se han convertido en la piedra angular de la mayoría de las aplicaciones de visión computacional modernas.

2.2.3.2. Técnicas de Procesamiento de Imágenes

El procesamiento de imágenes es un campo dentro de la informática y la matemática aplicada que se ocupa del análisis y manipulación de imágenes digitales. En la visión por computadora, es un paso esencial que prepara las imágenes para su análisis posterior y su utilización en diversas aplicaciones. A continuación, se detallan algunas de las técnicas más comúnmente utilizadas.

- Mejora de la imagen

La mejora de la imagen es un componente crucial en el procesamiento de imágenes y en particular en la visión computacional, donde se emplea para aumentar la calidad y la claridad de las imágenes para su posterior análisis y procesamiento (Gonzalez & Woods, 2017). La mejora de la imagen se puede realizar utilizando varias técnicas, cada una de las cuales se centra en mejorar ciertos aspectos de la imagen. Entre las técnicas comunes de mejora de la imagen se incluyen la mejora del contraste, la reducción de ruido, la afiladura (sharpening) y la ecualización del histograma.

La mejora del contraste implica modificar los valores de intensidad de los píxeles en la imagen para hacer que las regiones de interés sean más distinguibles. En el caso de imágenes de la conjuntiva ocular, por ejemplo, el aumento del contraste podría ayudar a resaltar las diferencias de color entre la conjuntiva y el resto del ojo, lo que facilitaría la detección de la palidez asociada a la anemia (Rajagopal & Thakur, 2017).

La reducción de ruido es otra técnica de mejora de la imagen, que tiene como objetivo minimizar el impacto del ruido en la imagen. El ruido puede surgir debido a varios factores, como las condiciones de iluminación y la calidad de la cámara, y puede dificultar el análisis posterior de la imagen. Los filtros de media, mediana y gaussiano son ejemplos de técnicas utilizadas para la reducción de ruido (Gonzalez & Woods, 2017).

La afiladura de la imagen es una técnica que se utiliza para realzar los bordes y otras características de alta frecuencia en la imagen. Esto se hace generalmente utilizando operadores de convolución como el filtro de Laplace o el filtro de Sobel (Gonzalez & Woods, 2017).

Por último, la ecualización del histograma es una técnica que se utiliza para mejorar el contraste de la imagen ajustando la distribución de intensidad de los píxeles en la imagen. Esto puede ser particularmente útil cuando la imagen tiene un contraste pobre debido a una iluminación insuficiente o excesiva (Gonzalez & Woods, 2017).

Figura 16 *Ecualización del histograma*



Nota. Extraído de https://www.unocero.com/entretenimiento/programacion-ludica-ecualizacion-del-histograma-de-una-imagen/

- Normalización de la imagen

La normalización de la imagen es un proceso esencial en el procesamiento de imágenes, que tiene como objetivo ajustar la intensidad de los píxeles de una imagen para mejorar la interpretación de los datos y proporcionar una base consistente para la comparación y el análisis de las imágenes (Reyes-Aldasoro, 2016). La normalización de la imagen implica generalmente ajustar los valores de los píxeles en una imagen para que caigan dentro de un rango específico, como 0-255 para imágenes de 8 bits o 0-1 para imágenes de punto flotante. Esto se puede hacer de diversas maneras, incluyendo la escala min-max, donde el valor del píxel más pequeño se ajusta a 0 y el valor del píxel más grande a 255 (o 1), y todos los demás valores de píxeles se escalan en consecuencia. Otra técnica común es la normalización Z-score, donde los valores de los píxeles se escalan para que tengan una media de 0 y una desviación estándar de 1 (Reyes-Aldasoro, 2016).

Además de proporcionar una base consistente para el análisis de la imagen, la normalización de la imagen también puede ayudar a resaltar características de interés y reducir la variabilidad causada por factores como la iluminación, la calibración de la cámara y otros factores externos. En el contexto de la visión computacional aplicada a la detección de anemia, por ejemplo, la normalización de la imagen podría utilizarse para ajustar las imágenes de la conjuntiva ocular tomadas bajo diferentes condiciones de iluminación, lo que permitiría una comparación más precisa y una detección más precisa de la anemia (Zhang et al., 2020).

- Corrección de iluminación

La corrección de iluminación es una técnica crucial en el procesamiento de imágenes y la visión computacional que trata de mitigar las variaciones en la iluminación en una imagen (Reyes-Aldasoro, 2016). Estas variaciones pueden ser el resultado de una serie de factores, incluyendo la iluminación ambiental, la calibración de la cámara, y la reflectancia de los objetos en la imagen.

Las técnicas de corrección de iluminación son diversas y dependen de la naturaleza específica de la variabilidad de la iluminación en la imagen. Algunas técnicas comunes incluyen la ecualización del histograma, que intenta redistribuir la intensidad de los píxeles en la imagen de tal manera que el histograma de intensidad de la imagen sea aproximadamente plano. Esto puede ayudar a aumentar el contraste en la imagen y a hacer resaltar las características que pueden haber sido oscurecidas por la iluminación deficiente (Pizer et al., 1987). Otra técnica común es la corrección de fondo o la eliminación de sombras, que trata de eliminar la variabilidad en la iluminación de fondo que puede ser el resultado de sombras o reflexiones. Esto a menudo implica calcular un modelo de la iluminación de fondo y luego restarlo de la imagen original (Guo et al., 2017).

En el contexto de la visión computacional para la detección de anemia, la corrección de iluminación podría ser esencial para asegurar que las imágenes de la conjuntiva ocular se puedan comparar de manera justa, independientemente de las condiciones de iluminación bajo las cuales se tomaron las imágenes.

- ROI

La Región de Interés (ROI) es un componente clave en el análisis y procesamiento de imágenes. Dentro del marco de una imagen completa, la ROI es un subconjunto específico

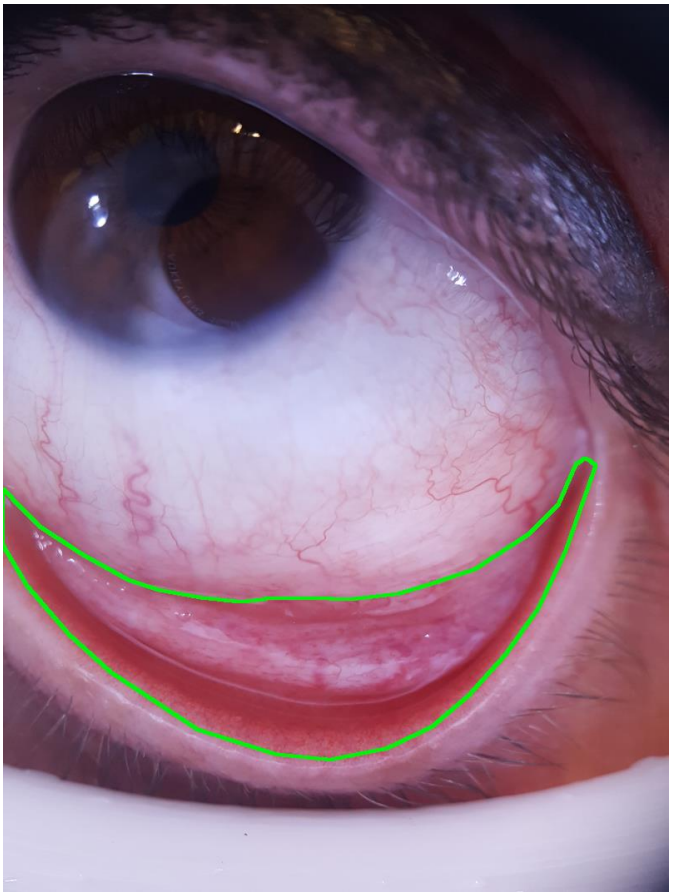
que se ha seleccionado para un análisis más detallado y riguroso (Szeliski, 2010). Dentro de este contexto, la selección de la ROI es una tarea no trivial y puede realizarse a través de varias técnicas, incluyendo la selección manual, el umbral y los métodos basados en el aprendizaje automático. La elección de la técnica depende del contexto de aplicación, la naturaleza de la imagen y los recursos disponibles (Gonzalez & Woods, 2007).

El método de selección manual de la ROI se basa en la intervención humana para seleccionar y delimitar la región de interés. Aunque este método puede proporcionar resultados precisos, es laborioso y puede llevar a variaciones debido a la subjetividad humana.

El método de umbral es un enfoque más automatizado para la selección de la ROI. Este método utiliza umbrales en la intensidad de los píxeles o en las características extraídas de la imagen para segmentar la región de interés. El método de umbral puede ser muy efectivo si la región de interés tiene una intensidad o características significativamente diferentes del fondo. Sin embargo, la elección de un umbral apropiado puede ser desafiante en imágenes con ruido o variaciones de intensidad (Pal & Pal, 1993).

Así, los métodos basados en el aprendizaje automático han surgido como un enfoque prometedor para la selección automática de la ROI. Estos métodos pueden aprender de los datos y adaptarse a las variaciones en la imagen. Los métodos basados en el aprendizaje automático pueden ser particularmente útiles en aplicaciones donde la variación intra e interclase de las regiones de interés es alta, y donde se dispone de suficientes datos de entrenamiento (LeCun, Bengio, & Hinton, 2015). Para concluir, la selección de la ROI es presentada como un paso esencial en el procesamiento y análisis de imágenes, además, puede impactar significativamente en el rendimiento de los algoritmos de análisis de imágenes.

Figura 17 *Área de interés conjuntiva ocular*



Nota. https://ieee-dataport.org/documents/eyes-defy-anemia

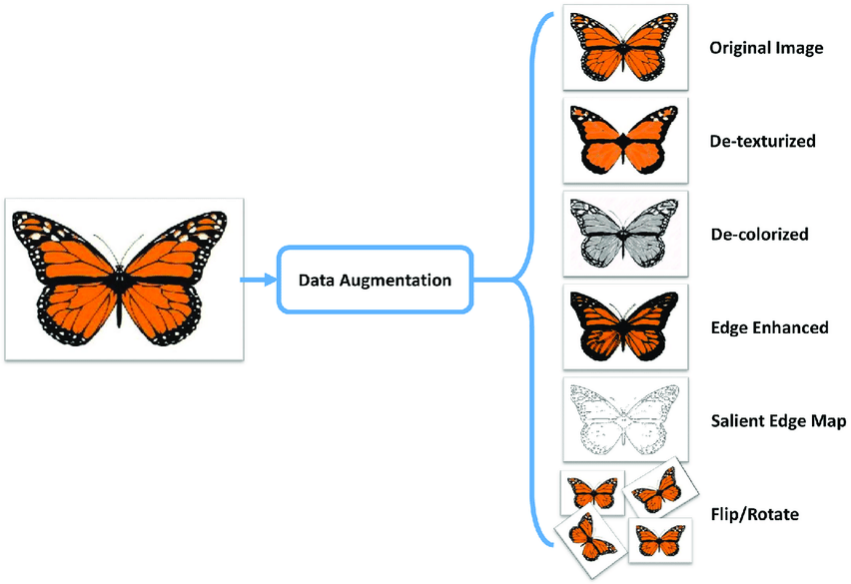
- Aumento de Datos en el Procesamiento de Imágenes

El aumento de datos es una táctica imprescindible en el dominio del aprendizaje automático y profundo que busca optimizar la precisión y la robustez de los modelos, con especial aplicación en el procesamiento de imágenes. Esta técnica implica generar instancias de datos adicionales a partir de los conjuntos de datos originales, aplicando una serie de transformaciones o perturbaciones, como rotación, escala, traslación, inversión, ruido y recorte.

Cada una de estas modificaciones tiene como objetivo incrementar la capacidad del modelo para reconocer los objetos de interés en las imágenes, independientemente de variaciones en su orientación, tamaño, posición, reflexión, perturbaciones aleatorias y visibilidad parcial.

El aumento de datos representa una estrategia eficaz para incrementar la capacidad de los modelos de aprendizaje para generalizar a partir de los datos de entrenamiento a los datos no observados previamente. No obstante, es esencial tener en cuenta que la efectividad de estas técnicas de aumento de datos puede variar dependiendo de la tarea específica y del conjunto de datos en uso. Por tanto, la elección de las técnicas de aumento más adecuadas demanda un entendimiento especializado de la tarea y los datos a mano.

Figura 18 *Aumento de la data*



Nota. Extraído de https://medium.com/secure-and-private-ai-writing-challenge/data-augmentation-increases-accuracy-of-your-model-but-how-aa1913468722

- Rotación

La rotación es una técnica común de aumento de datos que implica girar la imagen original en el espacio de la imagen. Al rotar las imágenes, se puede aumentar artificial-

mente el tamaño de nuestro conjunto de datos y al mismo tiempo permitir que nuestro modelo de aprendizaje automático aprenda características más generalizadas. Por ejemplo, una imagen de un gato girada 45 grados sigue siendo un gato. Por lo tanto, la rotación puede ayudar al modelo a ser invariante a la orientación espacial.

- Escala

El escalado es otra técnica de aumento de datos que implica cambiar el tamaño de la imagen. El escalado puede realizarse de dos maneras: escalado hacia arriba (aumento del tamaño de la imagen) o escalado hacia abajo (disminución del tamaño de la imagen). Al igual que con la rotación, el escalado ayuda a aumentar el tamaño del conjunto de datos y a fomentar la invariancia de escala en el modelo.

- Traslación

La traslación es una técnica que implica mover la imagen en el espacio. Por ejemplo, una imagen puede ser desplazada hacia la derecha, la izquierda, arriba o abajo. Al igual que las técnicas anteriores, la traslación puede ayudar a aumentar la cantidad de datos disponibles y a enseñar al modelo a ser invariante a la posición de las características en la imagen.

- Inversión

La inversión, también conocida como volteo, es una técnica que implica reflejar la imagen a lo largo de un eje. Esta técnica puede ser especialmente útil en contextos donde la orientación de la imagen no es crítica para la clasificación. Al igual que las otras técnicas, el volteo puede ayudar a aumentar el tamaño del conjunto de datos y a fomentar la invariancia de la orientación en el modelo.

- Ruido

La adición de ruido es una técnica que implica agregar pequeñas perturbaciones aleatorias a la imagen. Esto puede ayudar a mejorar la robustez del modelo al evitar el sobreajuste a los detalles específicos de las imágenes del conjunto de entrenamiento.

- Recorte

El recorte, o cropping, es una técnica que implica recortar una sección de la imagen original. Este enfoque puede ayudar a aumentar la cantidad de datos y a enseñar al modelo a reconocer los objetos de interés incluso cuando solo están parcialmente visibles.

2.2.3.3 Extracción y descriptores de características

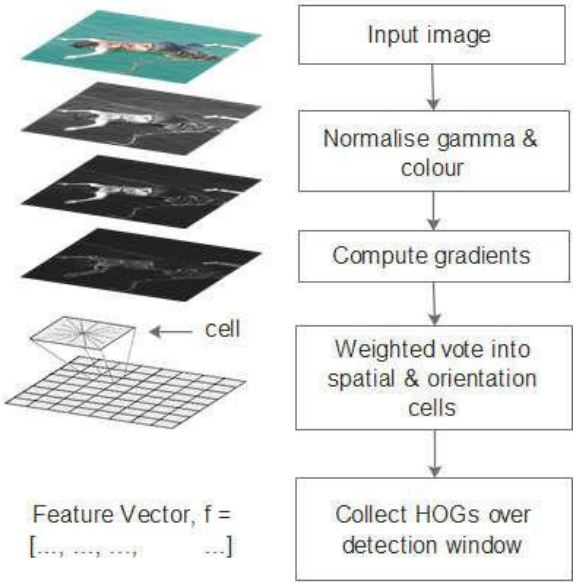
La extracción de características es un paso fundamental en cualquier sistema de visión por computadora. El objetivo principal de este proceso es transformar los datos de entrada en un conjunto reducido de características que sean relevantes para la tarea en cuestión. Esta transformación implica la eliminación de información redundante de los datos y la concentración en aspectos específicos que son importantes para distinguir diferentes categorías de imágenes (Szeliski, 2010). Las características se extraen con el fin de representar la información de la imagen de una forma que simplifique el proceso de análisis posterior. Por lo tanto, la calidad de estas características extraídas puede influir directamente en el rendimiento del sistema de visión por computadora.

Los descriptores de características son representaciones cuantitativas y cualitativas de aspectos específicos de una imagen. Por ejemplo, pueden referirse a elementos como el color, la forma, la textura, la orientación o la ubicación de ciertos objetos o regiones de interés dentro de la imagen. Existen diversos tipos de descriptores de características, algunos de los más utilizados en la visión por computadora incluyen:

- Histograma de Gradientes Orientados (HOG)

El Histograma de Gradientes Orientados (HOG) es una técnica de extracción de características popularmente usada en visión por computadora y aprendizaje automático para la detección de objetos. Fue presentado por Dalal y Triggs en su trabajo del 2005 en el que propusieron usarlo para la detección de peatones (Dalal & Triggs, 2005).

Figura 19 *Extracción de características utilizando Histograma de Gradientes Orientados (HOG)*



Nota. Extraído de https://www.researchgate.net/figure/Feature-extraction-using-Histogram-of-Oriented-Gradient-HOG_fig5_322478509

La técnica de Histograma de Gradientes Orientados (HOG) constituye un método integral en visión por computadora que evalúa distribuciones normalizadas de gradientes de imágenes locales. El gradiente de una imagen, un cambio en la intensidad de los píxeles, es frecuentemente utilizado para detectar bordes y características pertinentes. Los gradientes son obtenidos mediante derivadas parciales en las direcciones x (horizontal) y y (vertical), generando un vector bidimensional por píxel. Debido a que este proceso es susceptible al ruido, comúnmente se aplica un filtro de suavizado, como el filtro Gaussiano, previo al cálculo del gradiente.

Para la implementación de los descriptores HOG, la imagen es segmentada en pequeñas regiones espaciales denominadas celdas, donde cada celda normalmente contiene un número fijo de píxeles (por ejemplo, 8x8). Para cada celda, se construye un histograma de gradientes orientados, basándose en los gradientes de la imagen dentro de la misma. Estos histogramas son posteriormente normalizados y amalgamados para generar el descriptor de características global de la imagen.

La normalización, un paso crucial en el procesamiento de imágenes y la visión computacional, tiene como objetivo ajustar los valores de intensidad de los píxeles a una escala común. En este contexto, este ajuste frecuentemente implica la transformación de los valores de intensidad de los píxeles (que pueden oscilar entre 0 y 255 para imágenes en escala de grises) a un rango predefinido, como [0,1] o [-1,1]. La normalización tiene relevancia particular en la mejora de la eficacia y convergencia de diversos algoritmos de aprendizaje automático y visión computacional.

Las características HOG representan la distribución de las direcciones de los gradientes en una imagen o en segmentos específicos de ella. Tras el cálculo de los histogramas, estos son normalizados en bloques más grandes, normalmente de 2x2 o 3x3 celdas, para aumentar la precisión del detector. Estos bloques pueden solaparse para incrementar la robustez de las características HOG a las variaciones de posición. Finalmente, todos los histogramas normalizados se concatenan en un solo vector de características.

A pesar de su aparente simplicidad, las características HOG han demostrado ser extremadamente poderosas al capturar formas y estructuras locales en imágenes, siendo especialmente efectivas en tareas que involucran la detección de formas y estructuras, ya que son sensibles a los contornos de los objetos, pero inmunes a variaciones de textura.

El gradiente de una imagen I en un píxel (x,y) se calcula generalmente usando las siguientes fórmulas:

Fórmula 1: "Cálculo de gradiente en imágnes digitales" o "Operador de Sobel"

$$Gx = I(x+1, y) - I(x-1, y)$$

$$Gy = I(x, y+1) - I(x, y-1)$$

Donde Gx y Gy son los gradientes de la imagen en las direcciones x e y respectivamente. Luego, la magnitud y la orientación del gradiente se calculan como:

Fórmula 2: Cálculo de Magnitud y Dirección del Gradiente

$$|G| = sqrt(Gx\text{^}2 + Gy\text{^}2)$$

$$\theta = atan2(Gy, Gx)$$

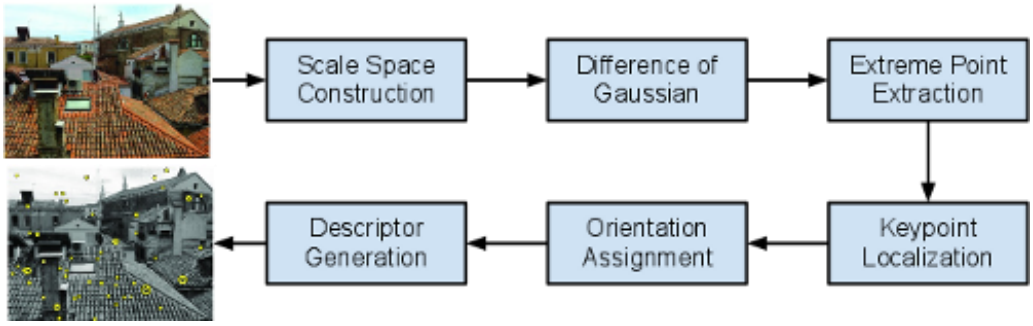
Aquí, |G| es la magnitud del gradiente y θ es la orientación.

- Descriptor de Bordes de Escala Invariante (SIFT)

El Descriptor de Bordes de Escala Invariante (SIFT) es un algoritmo en visión por computadora para extraer y describir características locales en imágenes. Fue desarrollado por David Lowe en 1999 y ha sido utilizado en una variedad de aplicaciones, como reconocimiento de objetos, emparejamiento de imágenes, panoramización y modelado 3D (Lowe, 2004).

La característica clave del SIFT es su resistencia a los cambios en la escala, orientación y ubicación de las características en una imagen. Además, proporciona cierta resistencia a los cambios en la iluminación y al ruido.

Figura 20 *Procesamiento de la cadena de SIFT*



Nota. Extraído de https://sites.google.com/site/5kk73gpu2011/assignments/sift

El algoritmo Scale-Invariant Feature Transform (SIFT), desarrollado por Lowe en 2004, se realiza a través de varios pasos y busca identificar ubicaciones únicas en una imagen que se mantienen constantes a través de diferentes vistas de la misma escena u objeto. Estos puntos de interés o puntos clave, corresponden a áreas con textura rica, bordes o esquinas, y su detección se realiza mediante la aplicación de una serie de filtros gaussianos a diferentes escalas. El proceso busca ubicaciones que sean máximas o mínimas locales del Laplaciano del Gaussiano de la imagen, utilizando para ello una pirámide de imágenes a diferentes escalas. Así, no sólo se considera la ubicación espacial del punto clave, sino también la escala en la que se detectó la característica, permitiendo invarianza a la escala.

Posteriormente, se asigna una orientación a cada uno de los puntos clave detectados. Este paso es esencial para garantizar la invarianza a la rotación. Para asignar la orientación, se calcula un histograma de gradientes locales en la región alrededor del punto clave, identificándose la dirección del gradiente que posee la mayor energía. El proceso se realiza para cada pixel en una vecindad del punto clave y se añade a un histograma de 36 bins (cada uno cubriendo 10 grados). Así, el resultado es un conjunto de puntos clave asociados a una ubicación, escala y orientación.

Una vez se han detectado los puntos clave y asignado escalas y orientaciones, se procede a extraer un descriptor para cada punto clave. Este descriptor codifica información sobre la apariencia local de la imagen en la vecindad del punto clave y está diseñado para ser robusto a variaciones en la imagen. Para la extracción del descriptor SIFT, se utiliza el histograma de gradientes orientados en una región alrededor del punto clave. Esta región se divide en una cuadrícula de subregiones (usualmente 4x4) y en cada subregión se calcula un histograma de gradientes orientados. Los gradientes se ponderan mediante una ventana gaussiana centrada en el punto clave, dando mayor importancia a los gradientes más cercanos al punto clave. Los histogramas de gradientes locales se concatenan para formar el descriptor final.

El descriptor SIFT resultante es un vector de alta dimensionalidad (generalmente 128 dimensiones), que proporciona una representación distintiva y robusta para cada punto clave. Aunque este proceso resulta muy poderoso para la extracción de características, también puede resultar computacionalmente costoso en comparación con otros métodos, como HOG (Histogram of Oriented Gradients) o ORB (Oriented FAST and Rotated BRIEF).

Fórmula 3*:* Convolución de una imagen con un filtro gaussiano para la creación de una escala de espacio

$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$$

- 'L(x, y, σ)' es la versión suavizada de tu imagen en una posición específica (x, y) y a una cierta escala σ.
- 'G(x, y, σ)' es un filtro que ayuda a suavizar la imagen. Este filtro se llama filtro Gaussiano.
- 'I(x, y)' representa la imagen original en una posición (x, y).

- ‘*’ indica la operación que se realiza entre el filtro Gaussiano y la imagen original para obtener la imagen suavizada.

Fórmula 4: La construcción del espacio de diferencias de escala, o DoG (Difference of Gaussian)

$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma)$$

- D(x, y, σ) representa la Diferencia de Gaussianas en una posición (x, y) y a una escala σ. Esta es la imagen que se obtiene después de restar dos imágenes Gaussianas adyacentes.
- L(x, y, kσ) es la versión suavizada de la imagen original a una escala mayor, que es kσ. k es un factor constante que determina cuánto se incrementa la escala.
- L(x, y, σ) es la versión suavizada de la imagen original a la escala σ.
- L(x, y, kσ) - L(x, y, σ) es la operación que se realiza para restar la imagen suavizada a la escala σ de la imagen suavizada a la escala mayor kσ.

Padding en SIFT: La inclusión del 'padding' o relleno en el algoritmo SIFT (Scale-Invariant Feature Transform) ofrece ventajas significativas para el análisis de los puntos de interés, o 'keypoints', ubicados cerca de los bordes de la imagen. En la práctica del procesamiento de imágenes, el 'padding' se refiere al proceso de agregar datos extra en los márgenes de una imagen. La idea es facilitar la ejecución de ciertos algoritmos que podrían ser limitados en los bordes de la imagen debido a la ausencia de vecinos adyacentes (Gonzalez & Woods, 2008).

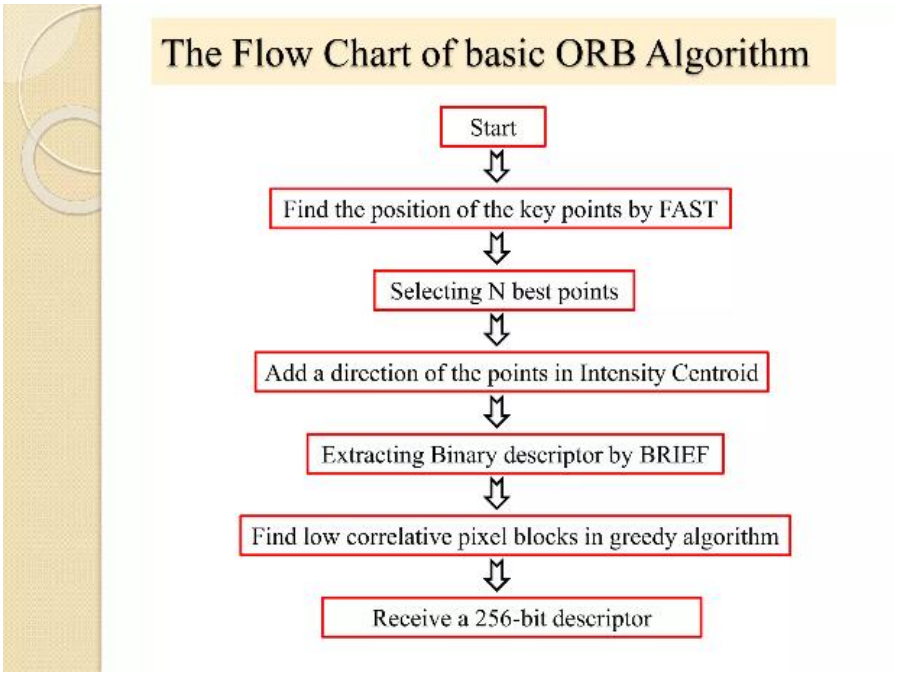
En el contexto de SIFT, el 'padding' puede tener un impacto significativo en la etapa de creación de descriptores para cada 'keypoint'. Dado que SIFT genera estos descriptores basándose en la magnitud y orientación del gradiente en una región local alrededor de cada punto de interés, los 'keypoints' situados cerca de los bordes de la imagen podrían no contar con suficientes vecinos para una descripción completa (Lowe, 2004). Esto puede resultar en una representación inexacta de las áreas de borde en los datos de salida de SIFT.

La introducción de 'padding' permite la generación de descriptores más robustos y completos para estos 'keypoints' de borde, lo que lleva a un aumento en la efectividad global del algoritmo SIFT. Sin embargo, es crucial considerar que un 'padding' inadecuado puede introducir artefactos y sesgar los resultados si no refleja de manera realista las características de la imagen más allá de sus bordes (Szeliski, 2010).

- Descriptor ORB (Oriented FAST and Rotated BRIEF)

El algoritmo ORB (Oriented FAST and Rotated BRIEF) se basa en una serie de pasos que permiten la detección y descripción eficiente de características clave en las imágenes. Este procedimiento se inicia con la detección de los puntos clave utilizando el algoritmo FAST (Features from Accelerated Segment Test), famoso por su velocidad y eficiencia. Los puntos clave, también conocidos como puntos de interés, son elementos en una imagen que poseen características únicas, como bordes o esquinas, o zonas de variación intensa de la intensidad de los píxeles. El algoritmo FAST selecciona un píxel en el centro de una ventana circular y lo compara con los píxeles en la circunferencia. Si un número suficientemente grande de estos píxeles periféricos son considerablemente más brillantes o más oscuros que el central, este último se considera un punto clave.

Figura 21 *Diagrama de flujo ORB*



Nota. Extraído de https://www.slideshare.net/NitinMauryaKashipur/orb-feature-by-nitin

Sin embargo, el algoritmo FAST no provee orientación a los puntos clave detectados, es decir, no es invariante a la rotación. Para solucionar esto, ORB combina FAST con un método para asignar una orientación a cada punto clave utilizando momentos de intensidad de imagen. Los momentos, que son medidas estadísticas de la distribución de las intensidades de los píxeles en un área dada de la imagen, se calculan en relación con el centro del punto clave. Tras calcular estos momentos, se determina la orientación del punto clave utilizando el ángulo de la línea que pasa por el origen y el punto definido por los dos momentos.

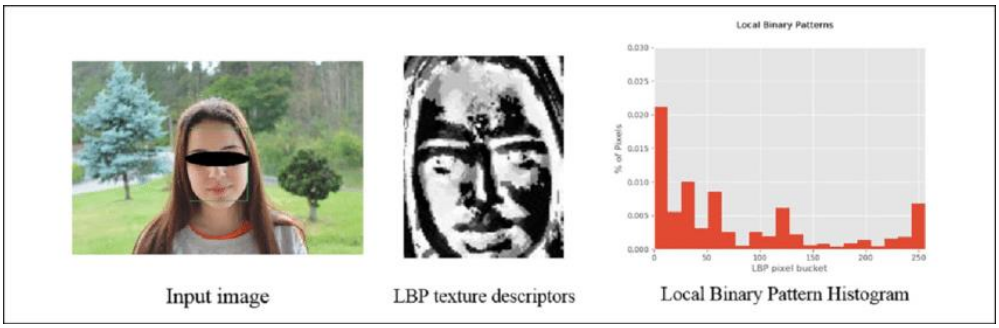
Después de haber detectado y asignado orientaciones a los puntos clave, se procede a describirlos usando una versión modificada del descriptor BRIEF (Binary Robust Independent Elementary Features). Este proceso convierte los puntos clave en descriptores, vectores que contienen información sobre la apariencia local de la imagen en la vecindad del

punto clave. En ORB, los pares de píxeles se seleccionan para proporcionar la máxima varianza y tener una distribución equilibrada de ceros y unos, lo que los diferencia del BRIEF original. Además, ORB aplica una rotación basada en la orientación del punto clave a los pares de píxeles antes de generar el descriptor, lo que asegura que estos descriptores sean invariantes a la rotación.

- LBPH (Local Binary Patterns Histogram)

El método del Histograma de Patrones Binarios Locales (Local Binary Patterns Histogram, LBPH) es una extensión del operador LBP que ha ganado relevancia debido a su simplicidad y eficacia en la caracterización de texturas en imágenes. El LBPH se basa en el concepto de que los patrones de la textura local son relevantes para la descripción de la apariencia de un objeto (Ojala, Pietikainen & Harwood, 1996).

Figura 22 *Pasos del algoritmo LBPH*



Nota. Extraído de https://www.researchgate.net/figure/Steps-of-LBPH-algorithm-LBPH-local-binary-patterns-histogram_fig7_342812992

Proceso de LBPH

Tras la división, el procedimiento avanza con el cálculo del Patrón Binario Local (LBP) para cada píxel en las subregiones definidas. El LBP es un descriptor de textura local que funciona mediante la comparación de cada píxel con sus vecinos. En este proceso, el valor del píxel central se contrasta con los de los píxeles vecinos, asignándose un valor de 1 si el píxel vecino tiene un valor mayor o igual al del central, o un 0 en caso contrario. Este

patrón binario resultante se convierte a un valor decimal que reemplaza al valor original del píxel.

A continuación, el procedimiento entra en su tercera fase, donde se crean histogramas de los valores de LBP en cada subregión. Estos histogramas describen la distribución de los patrones de textura y se normalizan para reducir los efectos de las variaciones de iluminación. Este paso es crucial para capturar la estructura de la textura local en cada subregión.

Finalmente, el proceso LBPH culmina con la concatenación de los histogramas de todas las subregiones, formando así un solo vector de características. Este vector de características describe la textura de la imagen original y captura la estructura espacial de la textura de la imagen. Por tanto, la fórmula de LBPH se enfoca en el cálculo del Patrón Binario Local, generando un número decimal a partir de un patrón binario de 8 bits, que se deriva de la comparación de los valores de los píxeles vecinos con el del píxel central.

Dado un píxel central con intensidad de gris gc y ocho píxeles vecinos P0, P1, P2, ..., P7, la fórmula para calcular el LBP es la siguiente:

Fórmula 5: Fórmula de LBPH

$$LBP = \sum P(i) * 2\hat{}i, (i = 0, 1, 2, \ldots, 7)$$

- LBP: es el Patrón Binario Local, que es un número decimal que representa un patrón binario de 8 bits.
- ∑: es el operador de sumatoria que suma los productos de P(i) y 2^i para i desde 0 hasta 7.

- P(i): es una función que se vuelve 1 si el píxel vecino i es mayor o igual que el píxel central, y 0 si no lo es.
- 2^i: eleva 2 al poder de la posición del píxel vecino i.
- i: es el índice que recorre los 8 píxeles vecinos del píxel central (Ojala et al., 2002)

Donde:

$$P(i) = 1, si\ P(i) \geq gc;$$
$$P(i) = 0, si\ P(i) < gc.$$

La función P(i) aquí es la función de umbralización que compara el valor del píxel vecino P(i) con el valor del píxel central gc. Si el valor del píxel vecino es mayor o igual que el valor del píxel central, se le asigna un valor de 1; de lo contrario, se le asigna un 0.

Esta fórmula genera un patrón binario de 8 bits para cada píxel de la imagen, y este patrón se convierte a un número decimal para obtener el valor de LBP del píxel central. Este valor reemplaza al valor de gris original del píxel central en la imagen (Ojala et al., 2002).

2.2.4. Arquitecturas Avanzadas de Redes Neuronales

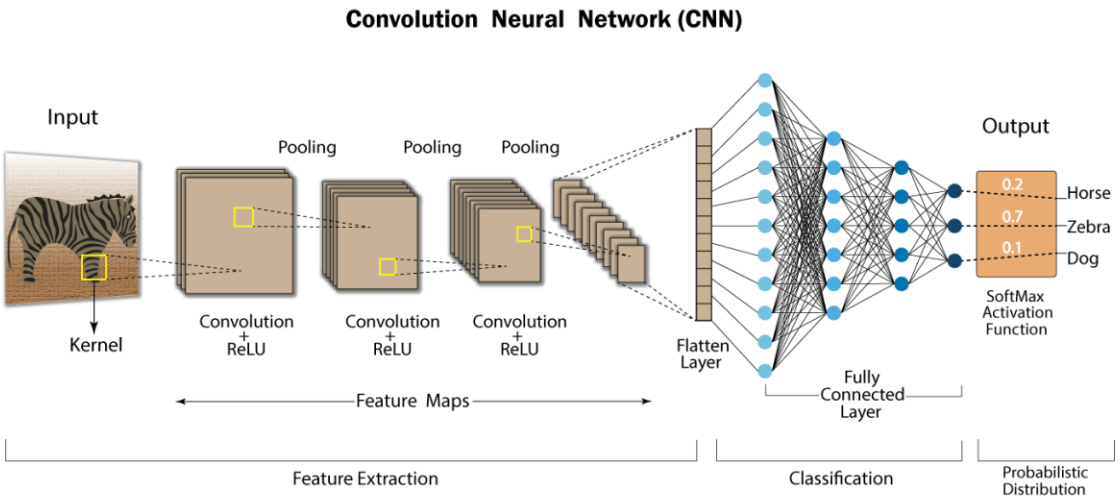
Las redes neuronales artificiales (ANN) son modelos de aprendizaje automático inspirados en las redes neuronales biológicas presentes en los cerebros animales. Estos modelos son capaces de aprender a través de la experiencia y son extremadamente eficaces para tareas de clasificación y regresión. Sin embargo, existen varias arquitecturas avanzadas de redes neuronales que se han diseñado específicamente para manejar tareas más complejas y desafiantes, como el procesamiento de imágenes y el análisis de secuencias. Entre estas,

destacan las Redes Neuronales Convolucionales (CNN) y las Redes Neuronales Residuales (ResNet).

2.2.4.1 Redes Neuronales Convolucionales (CNN)

Las Redes Neuronales Convolucionales (CNN, por sus siglas en inglés) representan una categoría de Redes Neuronales Artificiales (ANN) diseñada especialmente para procesar datos con una topología de malla en forma de cuadrícula, como es el caso de las imágenes (LeCun, Bottou, Bengio & Haffner, 1998). Estas redes han revolucionado el campo del aprendizaje profundo, especialmente en áreas como el reconocimiento de imágenes y la visión por computadora.

Figura 23 *Pasos CNN*



Nota. Extraído de https://developersbreach.com/convolution-neural-network-deep-learning/

2.2.4.2 Arquitectura de la CNN

La arquitectura de una CNN se compone de varias capas con distintas funciones. Las capas básicas de una CNN son la capa de convolución, la capa de agrupación o pooling, y la capa completamente conectada.

- Capa de Convolución**:**

La capa de convolución, como su nombre indica, aplica la operación matemática de convolución a los datos de entrada. Esta operación es esencial en el procesamiento de imágenes, ya que puede utilizarse para extraer características útiles como bordes, texturas y formas, que son de vital importancia para el reconocimiento y la clasificación de imágenes (Lecun et al., 1998). La convolución se realiza utilizando un conjunto de filtros o kernels, que son pequeñas matrices de pesos que se aplican a la imagen de entrada. Cada filtro se desliza o convoluciona a lo largo y ancho de la imagen de entrada, realizando un producto punto entre los pesos del filtro y los píxeles de la imagen en su campo receptivo correspondiente. El campo receptivo se refiere a la región local de la imagen que se está procesando en un momento dado.

Por lo tanto, durante la convolución, cada filtro se multiplica de manera escalada por una pequeña región de la imagen de entrada (el campo receptivo), y el resultado es una suma ponderada de los valores de los píxeles. Esta suma ponderada se registra en un mapa de características correspondiente a ese filtro. Como resultado, cada filtro produce un mapa de características diferente, representando diferentes características de la imagen.

La cantidad de filtros en una capa de convolución determina la profundidad del mapa de características de salida. Si se utilizan más filtros, se pueden detectar más características diferentes, lo que puede aumentar la capacidad de la red para reconocer una variedad de patrones en las imágenes de entrada. El objetivo de la capa de convolución es aprender los pesos de los filtros de tal manera que sean capaces de extraer las características más útiles de los datos de entrada. Durante el entrenamiento de la red, estos pesos se ajustan mediante retro propagación y descenso de gradiente para minimizar el error de clasificación de la red.

- Capa de Pooling**:**

Las Capas de Pooling, también conocidas como capas de sub muestreo o agrupación, juegan un papel vital en la estructura de las Redes Neuronales Convolucionales (CNN). Estas capas se encargan de reducir la dimensión espacial de la entrada, esto es, el ancho y el alto, mientras mantienen la profundidad de la misma intacta. La reducción de la dimensionalidad facilita la computación en las capas posteriores, minimiza el riesgo de sobreajuste y permite al modelo ser invariantes a pequeñas transformaciones, distorsiones y traslaciones en la imagen de entrada (LeCun et al., 1998).

Max Pooling es una técnica de operación de pooling que se utiliza en la construcción de redes neuronales convolucionales. Esta técnica tiene un papel muy importante en la reducción de la dimensionalidad espacial de los mapas de características resultantes después de la convolución (Boureau, Ponce, & LeCun, 2010). La idea principal detrás del Max Pooling es extraer las características más sobresalientes (el valor máximo) de una región específica del mapa de características. Al hacer esto, el modelo se vuelve más robusto a las pequeñas variaciones y traslaciones de la imagen, lo que ayuda a que el modelo generalice mejor y a que no se sobreajuste a los datos de entrenamiento.

En la operación de Max Pooling, se define una ventana de tamaño n × n que se desliza sobre el mapa de características. En cada posición de la ventana, se toma el valor máximo de los valores dentro de la ventana y se coloca en la matriz de salida. En términos matemáticos, si se denota A como el mapa de características de entrada y a B como el mapa de características de salida, la operación de Max Pooling se puede representar como:

$$B[i][j] = max(A[i:i+n][j:j+n])$$

Donde (i,j) representa la posición actual de la ventana en el mapa de características de entrada A, y (i+n, j+n) representa la posición final de la ventana en A. n es el tamaño de la ventana de Max Pooling.

Por otro lado, Average Pooling es otra técnica fundamental utilizada en las redes neuronales convolucionales para la reducción de la dimensionalidad de los mapas de características. En lugar de seleccionar el valor máximo de una región particular del mapa de características (como se hace en el Max Pooling), el Average Pooling calcula el valor promedio de los elementos dentro de la ventana de pooling (Scherer, Müller, & Behnke, 2010).

La operación de Average Pooling es relativamente directa: se define una ventana de tamaño n × n que se desliza sobre el mapa de características. Para cada posición de la ventana, se calcula el valor promedio de los valores dentro de la ventana y se coloca en la matriz de salida. Este promedio es a menudo un promedio no ponderado, aunque en algunos casos se puede utilizar un promedio ponderado. Matemáticamente, si se denota A como el mapa de características de entrada y B como el mapa de características de salida, la operación de Average Pooling se puede representar como:

Fórmula 6: Average Pooling

$$B[i][j] = sum(A[i:i+n][j:j+n]) \,/\, (n * n)$$

Donde (i,j) representa la posición actual de la ventana en el mapa de características de entrada A, y (i+n, j+n) representa la posición final de la ventana en A. n es el tamaño de la ventana de Average Pooling.

La operación de Average Pooling puede ser vista como una forma de suavizar la representación de las características. Esta técnica puede ser útil en algunos contextos en los

que todas las características son igualmente importantes dentro de una cierta región de la imagen.

- Capa Completamente Conectada:

La Capa Totalmente Conectada (Fully Connected Layer, FC) es un elemento crucial dentro de las Redes Neuronales Convolucionales (Convolutional Neural Networks, CNN). Este componente funciona en esencia como un traductor que interpreta y utiliza la información extraída por las capas convolucionales previas para realizar la tarea final de la red, ya sea una clasificación, una regresión, entre otras (LeCun et al., 2015).

Esta capa está compuesta por una serie de neuronas que están vinculadas a cada una de las neuronas en las capas anteriores y posteriores, de ahí el término "totalmente conectada". Por consiguiente, cada neurona en una FC recibe información de todas las neuronas de la capa anterior y a su vez, transmite su salida a todas las neuronas de la capa siguiente (Bishop, 2006).

El propósito central de la capa completamente conectada es utilizar las representaciones de alto nivel generadas por las capas convolucionales y de agrupamiento para realizar la tarea final de la CNN. Para ilustrarlo, en una tarea de clasificación, las capas FC pueden considerarse como un clasificador multinomial que toma las características extraídas por las capas anteriores y genera una distribución de probabilidad sobre las clases objetivo.

Las FC pueden aprender patrones complejos y no lineales debido a sus múltiples capas y alta capacidad de aprendizaje. Estos patrones son aprendidos durante la fase de entrenamiento, donde los parámetros de la red (pesos y sesgos) son ajustados para minimizar la

diferencia entre las salidas de la red y las etiquetas verdaderas. Este proceso de optimización se realiza a través de técnicas como el descenso del gradiente estocástico (Bottou, 2010).

Es importante resaltar que, aunque las capas FC se utilizan comúnmente al final de las CNNs, estas también pueden encontrarse en otras arquitecturas de redes neuronales. Además, las FC no son exclusivas para tareas de clasificación, sino que también pueden usarse para tareas de regresión o incluso en la generación de imágenes en las GANs (Goodfellow et al., 2014).

- Estructura de la capa completamente conectada: La estructura de la capa completamente conectada (también conocida como capa densa o capa de perceptrón) en una Red Neuronal Convolucional (CNN) tiene una configuración fundamental que permite a la red procesar la información que recibe y tomar decisiones finales con base en dicha información. Las capas completamente conectadas se llaman así debido a que cada una de sus neuronas está conectada a todas las neuronas de la capa anterior. Estas conexiones no son compartidas, lo que diferencia a las capas completamente conectadas de las capas convolucionales. En términos matemáticos, las capas completamente conectadas implementan una función lineal de la forma $y = Wx + b$, donde x es la entrada, W son los pesos de las conexiones, b es el sesgo y y es la salida de la capa. Las dimensiones de W y b dependen del número de neuronas en la capa actual y en la capa anterior.

Después de aplicar esta función lineal, cada neurona en la capa completamente conectada aplica una función de activación a su entrada para generar su salida. Las funciones de activación introducen no linealidades en el modelo, permitiendo que la red pueda aprender y modelar relaciones más complejas entre las entradas y las salidas. Algunas funciones

de activación comúnmente utilizadas en las capas completamente conectadas son la Unidad Lineal Rectificada (ReLU), la función sigmoide y la función tangente hiperbólica (tanh). Es importante señalar que las capas completamente conectadas suelen ser las últimas capas de una CNN. Su objetivo es tomar las características de alto nivel extraídas por las capas anteriores y usarlas para realizar la tarea final, como una clasificación o una regresión.

- Tipos de Función de activación:

a. Función Sigmoide (Sigma):

La función sigmoide es una de las funciones de activación más utilizadas en las redes neuronales. Tiene una forma de "S" suave y convierte los números de entrada en valores entre 0 y 1. Esto la hace útil para las tareas donde sean necesarias probabilidades como salida, como en la clasificación binaria. Sin embargo, la función sigmoide sufre de desvanecimiento de gradiente, lo que significa que las actualizaciones de los pesos pueden ser muy pequeñas y la red puede aprender muy lentamente o dejar de aprender si las entradas son muy altas o muy bajas.

b. Función Tangente Hiperbólica (Tanh):

La función tangente hiperbólica, o tanh, es otra función de activación utilizada en las redes neuronales. Es similar a la función sigmoide, pero su salida varía de -1 a 1. Esto significa que la función tanh es cero centrada, lo que a veces puede facilitar el aprendizaje de la red neuronal. Sin embargo, al igual que la función sigmoide, tanh también puede sufrir de desvanecimiento de gradiente.

c. Función Rectificada de Unidad Lineal (ReLU):

La función rectificada de unidad lineal, o ReLU, es actualmente una de las funciones de activación más utilizadas en las redes neuronales, especialmente en las redes

neuronales convolucionales (CNNs). ReLU es una función no lineal que da como salida el valor de entrada si es positivo, y cero en caso contrario. Esto la hace computacionalmente eficiente y ayuda a mitigar el problema del desvanecimiento de gradiente. Sin embargo, ReLU puede sufrir de "neuronas muertas", es decir, neuronas que solo producen ceros y, por lo tanto, no aprenden durante el entrenamiento. Para abordar este problema, se han propuesto variantes de ReLU, como Leaky ReLU y Parametric ReLU.

- Función de la capa completamente conectada: La función de la capa completamente conectada en una Red Neuronal Convolucional (CNN) es clave para el procesamiento y la clasificación de la información que ha sido codificada y transformada por las capas convolucionales y de pooling anteriores. Como su nombre lo indica, cada neurona en una capa completamente conectada está conectada a todas las neuronas de la capa anterior. Esto permite que las neuronas en estas capas puedan combinar las características de manera global para realizar la clasificación.

La capa completamente conectada opera sobre la premisa fundamental de que cada característica de entrada puede interactuar con todas las demás. En otras palabras, en lugar de considerar las características locales, que es el enfoque de las capas convolucionales y de pooling, las capas completamente conectadas permiten considerar todas las interacciones entre características.

Dentro de una capa completamente conectada, la operación principal es una combinación lineal de las entradas, seguida de una función de activación. La función de activación introduce la no linealidad en la red neuronal, permitiendo a la red aprender de datos más complejos y proporcionando una mayor flexibilidad para el modelo.

En términos matemáticos, la operación de una capa completamente conectada puede describirse como sigue. Por ejemplo, si se tiene una capa completamente conectada con entrada x e y w y b son los pesos y los sesgos de la capa, respectivamente, entonces, la salida de la capa, z, se calcula como:

Fórmula 7: Función lineal

$$z = wx + b$$

Donde w es la matriz de pesos, x es el vector de entrada, y b es el vector de sesgo. El producto wx es una operación de producto punto entre el vector de pesos y el vector de entrada. Luego se suma el vector de sesgo para dar el vector de salida bruta, z.

Después de calcular z, se aplica la función de activación, denotada por f. El resultado final, y, es:

Fórmula 8: Función de activación

$$y = f(z)$$

Donde f(z) es la función de activación aplicada elemento a elemento a z. Las funciones de activación comunes incluyen la función sigmoidal, la tangente hiperbólica (tanh), y la función de activación de rectificación lineal unitaria (ReLU).

2.2.4.3 Importancia y aplicaciones prácticas

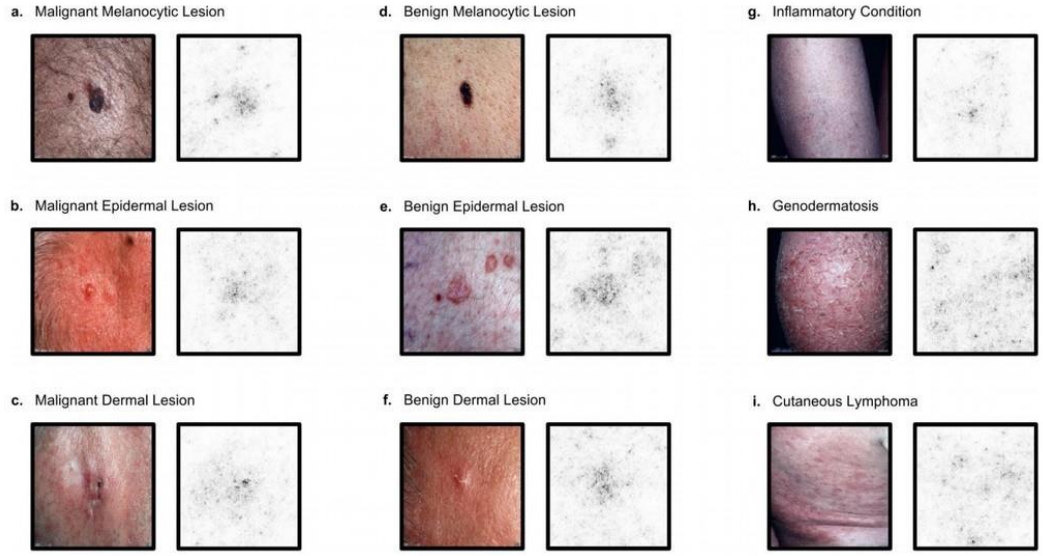
Las Redes Neuronales Convolucionales (CNN) han encontrado aplicaciones prácticas en una variedad de contextos, destacándose por su relevancia y efectividad

en el campo médico. En particular, dentro del ámbito médico, las CNN se han convertido en herramientas poderosas para procesar y analizar imágenes médicas, lo que es crucial en diversas ramas de la medicina y la investigación médica.

Las CNN son especialmente útiles para el diagnóstico de imágenes médicas, lo que incluye modalidades como la resonancia magnética (MRI), la tomografía computarizada (CT), la mamografía, y más recientemente, la fotografía de la conjuntiva ocular. Los algoritmos basados en CNN son capaces de detectar y clasificar anormalidades en estas imágenes con una precisión comparable, y en algunos casos superior, a la de los médicos humanos (Litjens et al., 2017).

Por ejemplo, en la detección de cáncer de mama en mamografías, las CNN han demostrado una precisión diagnóstica excepcionalmente alta. Al analizar y aprender de grandes conjuntos de datos, estas redes pueden identificar patrones sutiles en las imágenes que pueden ser difíciles de detectar para los humanos.

Figura 24 *Detección de tumores con inteligencia artificial*



Nota. Extraído de https://www.bloglenovo.es/tumor-inteligencia-artificial/

2.2.4.4 Redes Neuronales Residuales (ResNet)

Las Redes Neuronales Residuales, o ResNets, son una variante particularmente relevante de las arquitecturas de redes neuronales convolucionales que fueron presentadas por primera vez por Kaiming He, Xiangyu Zhang, Shaoqing Ren y Jian Sun en su trabajo titulado "Deep Residual Learning for Image Recognition" publicado en 2015 (He et al., 2015). Esta nueva arquitectura introdujo un concepto esencialmente revolucionario en el ámbito de las redes neuronales convolucionales, es decir, los "bloques residuales" o "conexiones residuales".

El objetivo fundamental de las ResNets es abordar un problema común en la formación de redes neuronales profundas denominado degradación de la red. En un sentido amplio, la degradación de la red se refiere a la tendencia inexplicada de las redes neuronales profundas a tener una disminución en la precisión del entrenamiento a medida que la profundidad de la red aumenta, a pesar de la ausencia de un sobreajuste claro (He et al., 2015). Este fenómeno contradice la expectativa teórica de que una red más profunda debería ser capaz de rendir al menos tan bien como su contraparte más superficial.

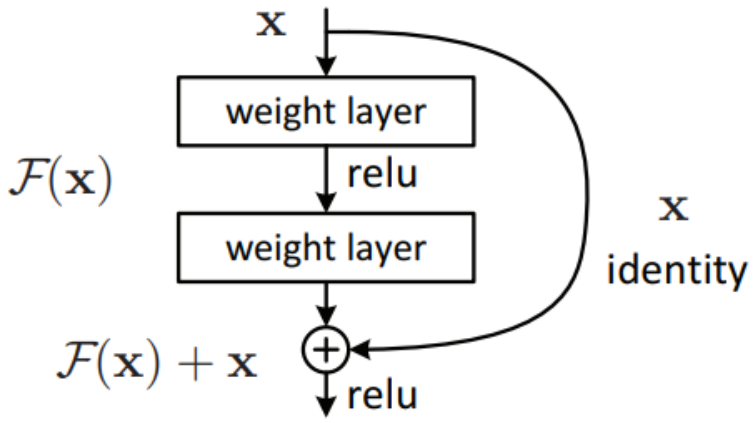
Un bloque residual es una estructura específica dentro de la red neuronal en la que se permite que las entradas de algunas capas se "salten" o se transmitan directamente a capas más avanzadas. Esta conexión se logra a través de una "conexión de salto" o "atajo", que es una conexión directa que salta una o más capas intermedias. Matemáticamente, si se denota la entrada a un bloque residual como x, la operación realizada por un bloque residual se puede describir de la siguiente manera:

Fórmula 9: Conexión de salto" o "skip connection"

$$y = F(x, \{Wi\}) + x$$

- **y** es la salida de la capa de la red neuronal.
- **F(x, {Wi})** es una función de la entrada **x** y un conjunto de pesos **{Wi}**. En el contexto de una red neuronal, esta función podría implicar varias capas de la red neuronal, cada una con su propia función de activación y conjunto de pesos.
- **x** es la entrada original a la capa.
- **F(x, {Wi}) + x** indica que la salida de la función **F(x, {Wi})** se suma a la entrada original **x**. Esto es lo que se llama una "conexión de salto", ya que la entrada **x** "salta" por encima de algunas capas para ser sumada a la salida.

Figura 25 *Conexión de Salto*



Nota. Extraído de https://www.geeksforgeeks.org/residual-networks-resnet-deep-learning/

Aquí, F(x, {Wi}) representa las operaciones de la red (como convoluciones, aplicaciones de la función ReLU, etc.) realizadas dentro del bloque residual, y {Wi} representa los pesos asociados a estas operaciones. Luego, la salida del bloque, y, se calcula sumando la salida F(x, {Wi}) y la entrada original x.

Esta suma se realiza mediante una operación de suma por elementos, que suma simplemente los valores en los mapas de características correspondientes de F(x, {Wi}) y x. Es crucial tener en cuenta que para que esta suma por elementos sea posible, tanto F(x, {Wi}) como x deben tener las mismas dimensiones.

2.2.4.5 La arquitectura EfficientNet

Las arquitecturas de Redes Neuronales Convolucionales (CNNs, por sus siglas en inglés) han experimentado un rápido desarrollo en la última década. En su intento por mejorar la precisión del modelo, las investigaciones han seguido una tendencia de escalar las redes, bien sea en profundidad, ancho o resolución de imagen. Sin embargo, este enfoque a menudo conduce a una mayor complejidad y costos computacionales.

A la luz de estos desafíos, se desarrolló una arquitectura innovadora llamada EfficientNet (Tan & Le, 2019). Basándose en el estudio de escalado de modelos, los investigadores de Google AI han creado un método que equilibra todos los aspectos del escalado de una red (profundidad, ancho y resolución), llamado escalado compuesto.

- Principios Básicos de EfficientNet

  La principal innovación de EfficientNet radica en su estrategia de escalado compuesto. En lugar de aumentar la profundidad, el ancho o la resolución de forma aislada, EfficientNet escala los tres factores simultáneamente, permitiendo así que el modelo se beneficie de cada factor de escalado sin incurrir en costos computacionales desmedidos.

  Además, EfficientNet comienza su escalado a partir de un modelo base eficiente que se obtiene mediante la búsqueda automática de arquitecturas neuronales

(NAS, por sus siglas en inglés), una estrategia que permite hallar una arquitectura neural óptima mediante la exploración del espacio de arquitectura de modelos.

- Desempeño de EfficientNet

  EfficientNet ha demostrado un desempeño superior en comparación con las arquitecturas de CNN tradicionales. En pruebas de ImageNet, EfficientNet ha logrado un rendimiento superior, tanto en precisión como en eficiencia, en comparación con otras arquitecturas líderes como ResNet y DenseNet. Aún cuando es un modelo más pequeño y eficiente, EfficientNet es capaz de alcanzar, e incluso superar, la precisión de los modelos más grandes y complejos. Esto se debe a su estrategia de escalado compuesto, que permite un uso más eficiente de los recursos computacionales, mejorando la eficiencia sin sacrificar la precisión.

- Aplicaciones y Adaptaciones de EfficientNet

  EfficientNet ha encontrado aplicaciones en una serie de dominios. Gracias a su eficiencia y alta precisión, ha sido adaptado para su uso en la clasificación de imágenes médicas, el análisis de videos, y otras tareas de visión por computadora.

2.2.4.6 La Arquitectura Inception-ResNet

Las Redes Neuronales Convolucionales (CNNs, por sus siglas en inglés) han demostrado una eficacia extraordinaria en tareas de clasificación de imágenes, pero el aumento de la precisión ha venido acompañado de una creciente complejidad y requisitos de recursos computacionales. A continuación, se discute la arquitectura Inception-ResNet, que se propone como una solución a estos desafíos.

- Fundamentos de Inception-ResNet

La arquitectura Inception-ResNet combina dos arquitecturas de red neuronal potentes y comprobadas: Inception y ResNet (Szegedy, Ioffe, Vanhoucke, & Alemi, 2017). La arquitectura Inception, conocida por su eficiencia en términos de recursos computacionales, utiliza módulos de "inception" que permiten a la red hacer su propia elección sobre qué funciones son útiles para aprender. ResNet, por otro lado, presenta un concepto innovador llamado "conexiones residuales" que facilita el entrenamiento de redes más profundas. Inception-ResNet combina estos dos enfoques de una manera que permite a la red beneficiarse tanto de la eficiencia de Inception como de la profundidad de ResNet.

- Desempeño de Inception-ResNet

La combinación de los principios de Inception y ResNet resulta en una arquitectura de red neuronal que es tanto eficiente en términos de recursos como capaz de aprender características de alto nivel a través de su profundidad. En comparación con las arquitecturas de CNN más tradicionales, Inception-ResNet ha demostrado tener un desempeño superior en varios conjuntos de datos de referencia, como ImageNet. Su rendimiento en términos de precisión es comparable al de las arquitecturas más grandes y complejas, pero con una significativa reducción en los requisitos de recursos computacionales.

- Aplicaciones y Adaptaciones de Inception-ResNet

Las aplicaciones de Inception-ResNet son diversas y abarcan numerosos dominios. Ha encontrado uso en la clasificación de imágenes médicas, la detec-

ción de objetos, y otras áreas de la visión por computadora. Su eficiencia en términos de recursos también la hace adecuada para aplicaciones en dispositivos con recursos computacionales limitados.

En resumen, Inception-ResNet ofrece una combinación convincente de eficiencia y profundidad, lo que permite un rendimiento impresionante en términos de precisión con una reducción en los recursos computacionales necesarios.

2.2.5. Aprendizaje Automático y Reconocimiento de Patrones

El aprendizaje automático y el reconocimiento de patrones constituyen una base crítica para numerosas aplicaciones tecnológicas en la era moderna. El campo del aprendizaje automático, una rama especializada de la inteligencia artificial, se centra en la construcción de sistemas que puedan aprender de los datos sin tener que ser explícitamente programados para ello (Samuel, 1959). Por otro lado, el reconocimiento de patrones, una disciplina en la intersección del aprendizaje automático y las estadísticas, está orientada hacia la clasificación y descripción de observaciones (Duda, Hart & Stork, 2001).

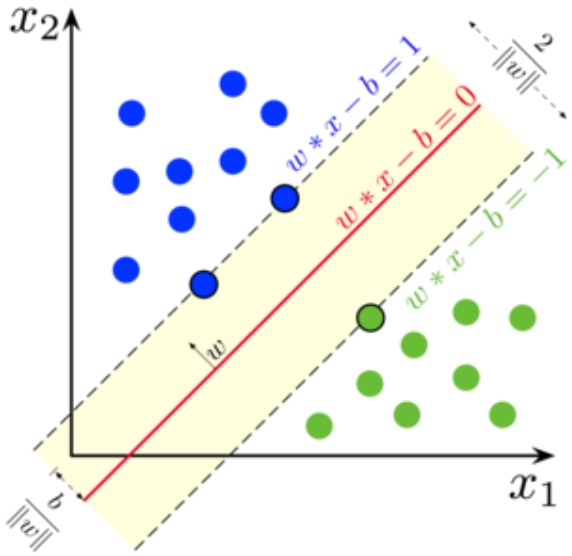
El reconocimiento de patrones tiene como objetivo identificar la regularidad y/o las dependencias en los datos de entrada para tomar decisiones informadas. Esto se logra mediante el uso de diversos algoritmos y técnicas, incluyendo, pero no limitándose a, las Máquinas de Vectores de Soporte (SVM), los K Vecinos más Cercanos (KNN), los Bosques Aleatorios (Random Forest) y las Redes de Perceptrón Multicapa (MLP).

2.2.5.1 Máquinas de Vectores de Soporte (SVM)

Las Máquinas de Vectores de Soporte (SVM, por sus siglas en inglés) son una familia de algoritmos de aprendizaje supervisado asociados con la clasificación y la regresión (Vapnik, 1995). Este modelo es capaz de capturar relaciones complejas en los conjuntos de

datos mediante la transformación de los datos en un espacio de mayor dimensión y la búsqueda de un hiperplano que maximice la distancia (margen) entre las categorías.

Figura 26 *Hiperplano de margen máximo y márgenes para un SVM*



Nota. Extraído de https://en.wikipedia.org/wiki/Support_vector_machine

Matemáticamente, las SVM intentan encontrar un hiperplano óptimo que separe las categorías. En un espacio bidimensional, un hiperplano es una línea recta definida por la ecuación:

Fórmula 10: Forma general de una recta en un sistema de coordenadas bidimensional

$$w.x + b = 0$$

Donde:

- w es un vector de pesos o coeficientes que determinan la dirección y pendiente de la recta.
- x es un vector de variables de entrada (en este caso, una variable en un espacio bidimensional).

- b es un término de sesgo (bias) que desplaza la recta hacia arriba o hacia abajo.

El objetivo de la SVM es maximizar el margen entre las categorías. El margen se define como la distancia más corta entre la línea y el punto de datos más cercano de cada categoría. La maximización del margen es un problema de optimización cuadrática que puede resolverse mediante técnicas de programación cuadrática.

La implementación de las SVM se realiza generalmente en tres etapas principales:

1. Pre procesamiento de datos en SVM: En el pre procesamiento de datos para SVM, se lleva a cabo la limpieza y transformación de los datos para eliminar el ruido y hacer que los datos sean más adecuados para el modelo de SVM. Esta etapa implica varios pasos, incluyendo la normalización, la eliminación de valores atípicos y la codificación de características categóricas (Hsu, Chang & Lin, 2003). La normalización de los datos a una escala específica (por ejemplo, entre -1 y 1) es una práctica común en SVM, ya que puede mejorar la eficiencia del algoritmo y evitar que las características con escalas de valores más grandes dominen el modelo (Scholkopf & Smola, 2001).
2. Entrenamiento del modelo SVM: El entrenamiento del modelo SVM implica encontrar el hiperplano óptimo que maximiza el margen entre las clases en el conjunto de datos de entrenamiento. La selección del kernel también es un componente crítico en esta etapa. Los diferentes kernels pueden transformar los datos a diferentes espacios dimensionales, lo que puede tener un impacto significativo en el rendimiento del modelo SVM. El proceso

de entrenamiento del modelo también puede implicar la optimización de parámetros, como el parámetro de coste C y el parámetro de kernel γ, a través de métodos como la validación cruzada y la búsqueda de cuadrícula (Hsu, Chang & Lin, 2003).

3. Predicción con el modelo SVM: Una vez que el modelo SVM se ha entrenado con un conjunto de datos, se puede utilizar para hacer predicciones en nuevos datos. La predicción implica mapear los datos de entrada en el mismo espacio de características que el modelo entrenado y luego determinar a qué lado del hiperplano caen los datos. Si los datos caen de un lado del hiperplano, se clasifican como una clase, y si caen del otro lado, se clasifican como la otra clase (Scholkopf & Smola, 2001).

2.2.5.2. K Vecinos más Cercanos (KNN)

K Vecinos más Cercanos (KNN, por sus siglas en inglés) es un método de aprendizaje supervisado que se utiliza tanto para la clasificación como para la regresión (Cover & Hart, 1967). El algoritmo KNN se basa en la idea de que los objetos similares se encuentran en proximidad cercana.

KNN opera calculando la distancia entre una muestra de prueba y todas las muestras de entrenamiento. La distancia puede ser medida usando varias métricas, pero la distancia euclidiana es la más comúnmente utilizada:

Fórmula 11: Fórmula de la distancia euclidiana

$$dist(x, y) = sqrt(sum((x - y)\text{^}2))$$

- x, y: Dos puntos en un espacio n-dimensional, donde cada elemento representa una coordenada.
- x - y: La diferencia entre las coordenadas correspondientes de los dos puntos.
- (x - y)^2: La diferencia cuadrada, eliminando cualquier signo negativo de la resta.
- sum((x - y)^2): La suma de todas las diferencias cuadradas.

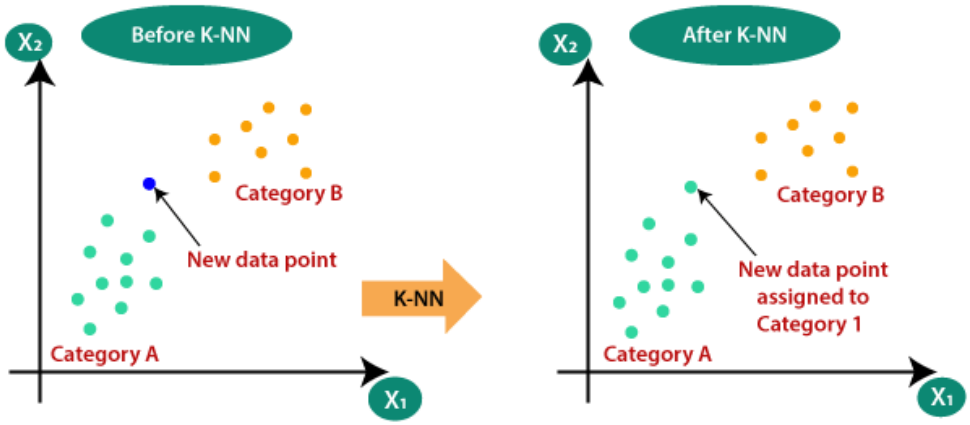
Una vez que las distancias son calculadas, las k muestras de entrenamiento más cercanas (es decir, con las distancias más pequeñas) son seleccionadas. En un problema de clasificación, la etiqueta de la muestra de prueba es predicha basándose en la etiqueta más común entre los k vecinos más cercanos.

- Selección del parámetro 'k': La elección del número de vecinos k es un aspecto crucial en KNN. k es un hiperparámetro del algoritmo KNN. Un k pequeño puede hacer que el algoritmo sea sensible al ruido y a las anomalías, mientras que un k grande puede suavizar demasiado las fronteras de decisión. La elección del k adecuado es un equilibrio entre la eliminación del ruido y la preservación de las estructuras de datos relevantes. En la práctica, la selección de k se realiza a menudo a través de la validación cruzada, que es un proceso que permite determinar cuál es el valor de k que proporciona la mejor precisión de predicción en los datos de prueba.
- Proceso de implementación de KNN: La implementación de KNN puede ser resumida en tres etapas principales:

1. Pre procesamiento: Esta es la primera fase en la implementación de KNN y comprende diversas tareas relacionadas con la preparación y la manipulación de los datos antes del entrenamiento del modelo. Las tareas comunes incluyen la limpieza de datos (eliminar datos no válidos o corregir datos con errores), la normalización de datos (ajustar los datos a una escala común para evitar que ciertos atributos dominen otros), y la codificación de datos categóricos en una forma que pueda ser procesada por el modelo (Aggarwal, 2018).
2. Entrenamiento del modelo: A diferencia de otros algoritmos de aprendizaje automático, el algoritmo KNN no tiene una fase de "entrenamiento" per se. Más bien, el modelo simplemente almacena los datos de entrenamiento. Cuando se le presenta una nueva instancia para clasificar o predecir, el algoritmo KNN busca en los datos almacenados para encontrar los K vecinos más cercanos a la nueva instancia. Este enfoque se denomina aprendizaje basado en instancias o aprendizaje perezoso (Aha, Kibler, & Albert, 1991).
3. Predicción: La fase de predicción en KNN implica determinar la salida para una nueva instancia basándose en sus K vecinos más cercanos. Para un problema de clasificación, la clase de la nueva instancia se determina a partir de la clase más común entre sus K vecinos más cercanos. En el caso de una tarea de regresión, el valor objetivo se calcula a partir del promedio (o alguna otra medida agregada) de los valores objetivo de los K vecinos más cercanos. La función de distancia utilizada para determinar los vecinos más cercanos puede variar; la distancia euclidiana es una opción común, aunque

hay otras, como la distancia de Manhattan o la distancia de Minkowski (Cover & Hart, 1967).

Figura 27 *Aplicación de KNN*



Nota. Extraído de https://www.javatpoint.com/k-nearest-neighbor-algorithm-for-machine-learning

2.2.5.3 Bosques Aleatorios (Random Forest)

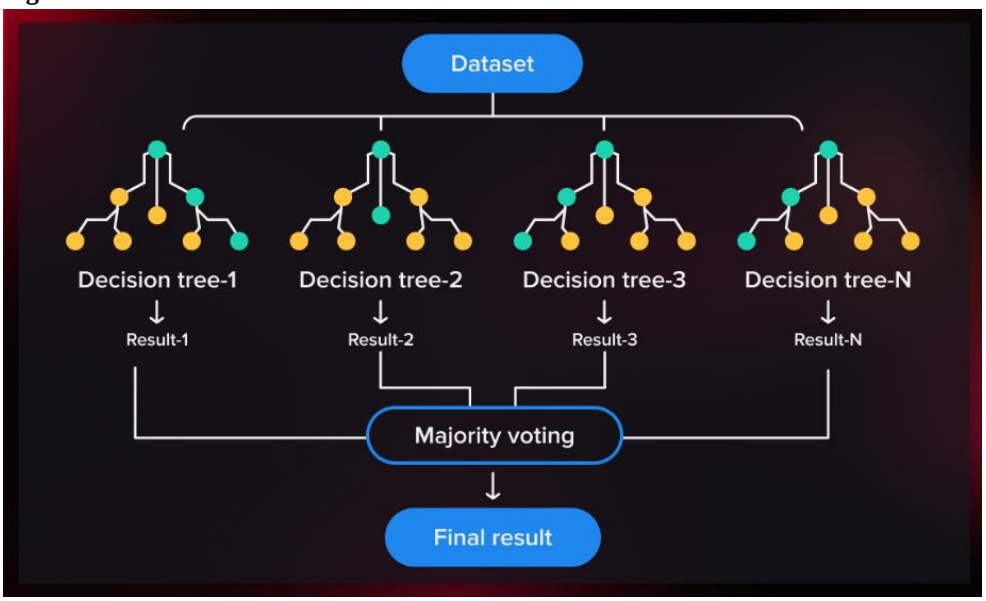
Los Bosques Aleatorios (Random Forest, RF) son un método de aprendizaje supervisado propuesto por Leo Breiman en 2001. Es un algoritmo de ensamble que combina múltiples árboles de decisión para obtener un rendimiento de predicción superior (Breiman, 2001).

- Mecánica del algoritmo Random Forest

  El algoritmo RF crea un conjunto de árboles de decisión independientes que se entrenan con diferentes subconjuntos de datos. El subconjunto de datos se selecciona utilizando un método llamado 'bootstrap', que implica muestrear los datos con reemplazo (Efron & Tibshirani, 1993). Cada árbol se entrena utilizando un subconjunto de características seleccionadas al azar en cada división, lo que proporciona una mayor diversidad entre los árboles.

La predicción de un RF se obtiene agregando las predicciones de todos los árboles de decisión individuales. En un problema de clasificación, se selecciona la clase que recibe la mayoría de los votos (Biau, 2012).

Figura 28 *Estructura Random Forest*



Nota. Extraído de https://serokell.io/blog/random-forest-classification

- Hiperparámetros principales de Random Forest

  Los hiperparámetros más importantes en un RF son el número de árboles (n_estimators) y el número de características a considerar en cada división (max_features). Un número más alto de árboles reduce la varianza del modelo, pero puede aumentar el tiempo de cómputo. Por otro lado, el número de características consideradas en cada división controla el equilibrio entre la varianza y el sesgo del modelo.

- Proceso de implementación de Random Forest

  El proceso de implementación de RF se puede resumir en tres etapas principales:

1. Preparación de los datos: Esta fase implica la preparación de los datos para el análisis con el algoritmo de Bosques Aleatorios (Random Forest). Los datos pueden necesitar limpieza (por ejemplo, la eliminación de valores atípicos, la imputación de valores perdidos, la corrección de errores), la transformación (por ejemplo, la normalización para ajustar los datos a una escala común, la codificación de variables categóricas en un formato numérico), y la selección de características (para reducir la dimensionalidad y eliminar características irrelevantes o redundantes). El pre procesamiento es una etapa crítica que puede tener un impacto significativo en el rendimiento del modelo de Bosques Aleatorios (Liaw & Wiener, 2002).
2. Entrenamiento del modelo: El algoritmo de Bosques Aleatorios entrena un conjunto de árboles de decisión, cada uno de los cuales se entrena en un subconjunto de los datos de entrenamiento seleccionado al azar. Cada árbol se construye de manera recursiva seleccionando en cada nodo una característica y un umbral que mejor dividen el conjunto de datos según una medida de pureza (por ejemplo, la impureza de Gini o la entropía). Una característica importante del algoritmo de Bosques Aleatorios es que en cada nodo del árbol, la característica y el umbral se seleccionan a partir de un subconjunto de características seleccionadas al azar, en lugar de todas las características. Esta aleatorización adicional ayuda a descorrelacionar los árboles y mejora la robustez y la capacidad de generalización del modelo (Breiman, 2001).

3. Predicción: Para realizar una predicción en un modelo de Bosques Aleatorios, la entrada se pasa a través de cada uno de los árboles en el bosque, y cada árbol da una predicción independiente. En el caso de un problema de clasificación, la clase que se predice con más frecuencia entre todos los árboles se selecciona como la predicción final. En el caso de un problema de regresión, el valor de salida final se calcula como la media (o alguna otra medida agregada) de las predicciones de todos los árboles (Breiman, 2001).

2.2.5.4 Redes de Perceptrón Multicapa (MLP)

Las Redes de Perceptrón Multicapa (Multi-Layer Perceptron, MLP), como una especie de red neuronal artificial feedforward, han emergido como uno de los pilares fundamentales en el campo de la Inteligencia Artificial, proporcionando soluciones en innumerables problemas de clasificación y regresión (Hagan et al., 1996). Este tipo de red neuronal es una extensión del Perceptrón Simple y utiliza una estructura de múltiples capas para modelar y resolver problemas no lineales complejos (Rosenblatt, 1958).

- Arquitectura Intrínseca de las MLP

  El diseño de una MLP consiste en una disposición de capas de neuronas o nodos en un sistema de alimentación hacia adelante. Estas capas se dividen en tres categorías: capa de entrada, capas ocultas y capa de salida.

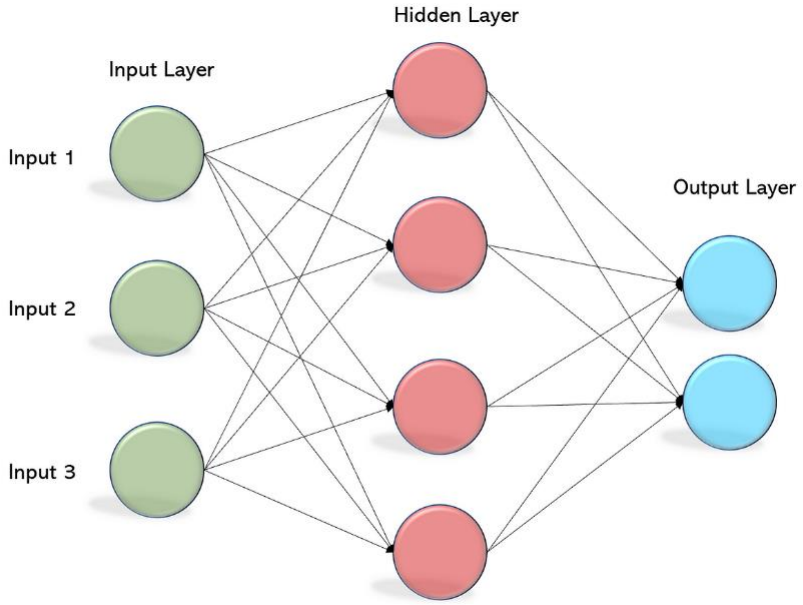
  La arquitectura del MLP incluye conexiones entre las neuronas de las diferentes capas, siendo estos enlaces unidireccionales, es decir, las conexiones fluyen desde la capa de entrada hacia la capa de salida. Esta configuración evita que existan conexiones entre neuronas en una misma

capa, permitiendo que la información siga una trayectoria sin conflictos (Hornik et al., 1989).

Cada neurona en una MLP toma una entrada ponderada, aplica una función de activación y pasa la salida a la siguiente capa. Las funciones de activación, que son no lineales, permiten que el MLP modele relaciones no lineales. Estas funciones pueden variar desde la función sigmoide hasta la función de ReLU, que es ampliamente utilizada en la actualidad.

2.2.5.5 Multi layer Perceptron (MLP)

Figura 29 *Perceptrón multicapa (MLP))*



Nota. Extraído de https://becominghuman.ai/multi-layer-perceptron-mlp-models-on-real-world-banking-data-f6dd3d7e998f?gi=9a72138df082

- Proceso de implementación de MLP

1. Preprocesamiento: El preprocesamiento es una etapa esencial en el uso de MLP, al igual que en otros algoritmos de aprendizaje automático. Incluye la limpieza de los datos, la eliminación de valores atípicos, la imputación de valores perdidos, la corrección de errores y la transformación de los datos

para su análisis. A menudo, esto también puede implicar la normalización de los datos para garantizar que todas las características tengan una escala común y el codificado one-hot de las variables categóricas. Además, puede ser útil realizar una selección de características para reducir la dimensionalidad de los datos y eliminar características irrelevantes o redundantes (Kriesel, 2007).

2. Entrenamiento del modelo: Durante el entrenamiento, un MLP aprenderá a mapear un conjunto de entradas a sus correspondientes salidas a través de un proceso iterativo. Utilizando un algoritmo de retropropagación y un método de optimización (como el descenso del gradiente estocástico), la red actualizará iterativamente los pesos y sesgos de la red para minimizar la función de coste, que es una medida del error entre las predicciones del modelo y los verdaderos valores de salida. Este proceso continuará hasta que se alcance un número determinado de iteraciones, o hasta que el error del modelo en los datos de validación deje de mejorar (Rojas, 1996).
3. Predicción: Una vez entrenado, un MLP puede utilizarse para hacer predicciones sobre nuevos datos. Para una nueva entrada, la red realizará una serie de cálculos basados en los pesos y sesgos aprendidos durante el entrenamiento, y proporcionará una salida que puede interpretarse como la predicción del modelo. En un problema de clasificación, la salida del MLP suele ser un vector de probabilidades que indican la probabilidad de que la entrada pertenezca a cada clase posible (Rojas, 1996).

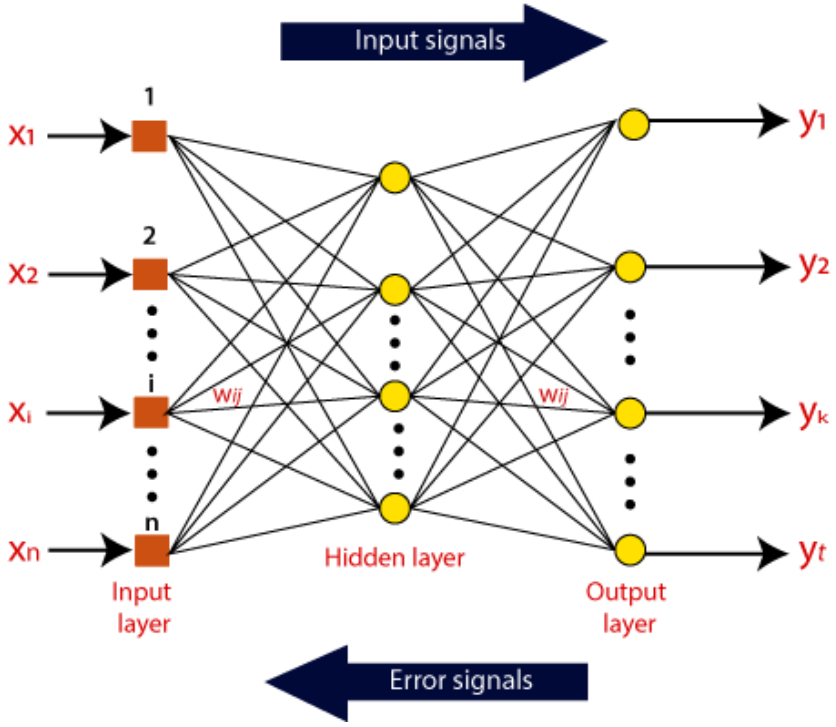
2.2.5.6 Redes Neuronales Artificiales (ANN)

Las Redes Neuronales Artificiales (ANN, por sus siglas en inglés) son modelos computacionales inspirados en el cerebro humano que se utilizan ampliamente en el campo de la inteligencia artificial para la resolución de problemas de aprendizaje supervisado y no supervisado. Estas redes se componen de nodos interconectados, llamados "neuronas" o "unidades", que simulan las neuronas biológicas del cerebro humano (Haykin, 2009).

Las ANN constan de una capa de entrada, una o más capas ocultas y una capa de salida. La capa de entrada recibe las señales de entrada, las capas ocultas procesan las señales y la capa de salida genera las señales de salida. Cada neurona en una capa está conectada a todas las neuronas en la capa siguiente, con cada conexión ponderada por un valor de peso, que determina la importancia relativa de esa entrada para el cálculo realizado por la neurona (Goodfellow, Bengio, & Courville, 2016).

Las ANN aprenden ajustando iterativamente los pesos de las conexiones en respuesta a los datos de entrada. El aprendizaje se realiza mediante un proceso de optimización que busca minimizar una función de pérdida, que mide la discrepancia entre las predicciones de la red y los verdaderos valores de salida. El algoritmo de retropropagación es comúnmente utilizado para realizar este ajuste de los pesos, y suele ser combinado con un método de optimización como el descenso del gradiente estocástico (Rumelhart, Hinton, & Williams, 1986).

Figura 30 *Estructura de una Red Neuronal Artificial*



Nota. Extraído de https://www.javatpoint.com/artificial-neural-network

Las ANN han demostrado ser herramientas poderosas para una amplia gama de tareas, incluyendo el reconocimiento de patrones, la clasificación, la regresión, el procesamiento del lenguaje natural, y la visión por computadora. Debido a su capacidad para aprender representaciones complejas y no lineales, las ANN son especialmente efectivas cuando se trata de grandes conjuntos de datos de alta dimensión (LeCun, Bengio, & Hinton, 2015).

- **Etapas de Procesamiento de Datos ANN:**

1. Preprocesamiento de datos: Antes de que los datos puedan ser alimentados a una ANN para el entrenamiento, deben ser preprocesados. Esto incluye la limpieza de datos (manipulación de valores perdidos o erróneos), la normalización de datos (reescalado de valores para que caigan dentro de un rango

predefinido), la codificación de variables categóricas (transformación de variables categóricas en una forma que la ANN pueda entender), y la división de los datos en conjuntos de entrenamiento y prueba (para evaluar el rendimiento de la ANN). En el preprocesamiento también se puede incluir la selección de características para reducir la dimensión de los datos y aumentar la eficiencia computacional (Kotsiantis, Kanellopoulos, & Pintelas, 2006).

2. Entrenamiento del modelo: Esta etapa implica el ajuste iterativo de los pesos de las conexiones de la red en respuesta a los datos de entrada, utilizando un algoritmo de aprendizaje, como la retropropagación. Este proceso se realiza normalmente con el objetivo de minimizar una función de pérdida, que mide la discrepancia entre las salidas producidas por la ANN y las salidas deseadas (objetivos). El proceso de entrenamiento también puede incluir técnicas de regularización para prevenir el sobreajuste, es decir, para evitar que la red aprenda el "ruido" en los datos de entrenamiento a costa de su capacidad para generalizar a nuevos datos (Goodfellow, Bengio, & Courville, 2016).
3. Predicción: Una vez que la ANN ha sido entrenada, puede ser usada para hacer predicciones en base a nuevos datos de entrada. En la práctica, esto implica alimentar los datos de entrada a la ANN y propagar las señales a través de la red (utilizando los pesos de las conexiones aprendidas durante el entrenamiento) para producir salidas. En una tarea de clasificación, por ejemplo, la salida de la ANN podría ser interpretada como las probabilidades de que la entrada pertenezca a cada una de las clases posibles (Haykin, 2009).

2.2.6. Evaluación de rendimiento:

Al evaluar el rendimiento de un modelo de aprendizaje automático, se deben tener en cuenta múltiples aspectos. Sin embargo, hay tres factores que son particularmente cruciales debido a su impacto directo en la utilidad y aplicabilidad del modelo: la precisión predictiva, la robustez y la eficiencia computacional.

- Precisión Predictiva: Este es un indicador clave del rendimiento de un modelo y se refiere a qué tan bien el modelo predice la variable objetivo en datos no vistos. Para medir la precisión predictiva, se emplean diversas métricas, como el error cuadrático medio (MSE) en regresión y la precisión, el recall y el F1-score en clasificación (Hastie, Tibshirani, & Friedman, 2009).
- Robustez: Un modelo robusto es aquel que mantiene su rendimiento frente a variaciones pequeñas en los datos de entrada o en presencia de datos ruidosos o atípicos. Los modelos robustos son menos propensos a ser afectados negativamente por los cambios en los datos de entrenamiento y, por tanto, suelen ser más confiables en la práctica (Chen, Li, Ribeiro, Schmidt, & Liang, 2018).
- Eficiencia Computacional: Se refiere a la cantidad de recursos computacionales que el modelo requiere para entrenar y realizar predicciones. En entornos de producción donde los recursos pueden ser limitados, la eficiencia computacional es un factor crítico que puede afectar la viabilidad del uso del modelo. Elementos a considerar incluyen el tiempo requerido para entrenar el modelo y realizar predicciones, y la cantidad de memoria necesaria (Bottou, 2010).

2.2.7. Interpretabilidad de los Modelos de Aprendizaje Automático

Los modelos de aprendizaje automático, particularmente los modelos de aprendizaje profundo, se han vuelto extremadamente exitosos en una amplia variedad de tareas, desde la clasificación de imágenes hasta el análisis de texto. Sin embargo, estos modelos son a menudo criticados por su falta de interpretabilidad; es decir, es difícil entender por qué y cómo están tomando las decisiones que toman. Esta opacidad puede ser problemática en situaciones donde es importante entender las razones detrás de las predicciones del modelo, por ejemplo, en medicina o en decisiones de crédito. En este contexto, los mapas de calor de activación representan una valiosa herramienta para entender mejor estos modelos.

Los mapas de calor de activación, también conocidos como mapas de atención o mapas de características, son una técnica para visualizar las regiones de los datos de entrada que fueron más importantes para la decisión del modelo. En la visión por computadora, por ejemplo, un mapa de calor de activación sobre una imagen puede resaltar las partes de la imagen que contribuyeron más a la predicción del modelo. Esta es una forma de "interpretación local", ya que muestra cuál es la contribución de cada parte de la entrada para la predicción final.

Los mapas de calor de activación se calculan utilizando las activaciones de la red en ciertas capas. En una red neuronal convolucional, por ejemplo, se podría calcular un mapa de calor de activación sumando las activaciones en el espacio de los canales de una capa particular, y luego haciendo un upsample de este mapa para tener el mismo tamaño que la imagen de entrada. Este mapa de calor puede entonces ser superpuesto en la imagen de entrada para visualizar las regiones de mayor activación.

2.3. Marco Conceptual o Contexto de la Investigación

Visión Computacional

La visión computacional es una rama de la inteligencia artificial que interpreta y comprende el mundo visual a través de imágenes y videos usando algoritmos y técnicas de procesamiento de imágenes (Szeliski, 2010).

Conjuntiva Ocular

La conjuntiva ocular es una membrana que cubre la parte frontal del ojo y el interior de los párpados, cuyos cambios de color pueden indicar condiciones de salud como la anemia (Wright et al., 2016).

Anemia

La anemia es una afección médica caracterizada por la falta de suficientes glóbulos rojos o hemoglobina en la sangre, a menudo indicada por la palidez de la conjuntiva ocular (World Health Organization, 2011).

Segmentación de Imágenes

La segmentación de imágenes es una técnica de visión computacional que divide una imagen en partes o regiones correspondientes a diferentes objetos o características (Pham et al., 2018).

Análisis de Color

El análisis de color es una técnica en la visión computacional que extrae e interpreta información de color en imágenes, útil para evaluar la palidez de la conjuntiva ocular en el diagnóstico de anemia (Gijsenij et al., 2011).

Procesamiento de Imágenes

El procesamiento de imágenes es una técnica que mejora la calidad de las imágenes y extrae información útil de ellas. Es fundamental en la visión computacional para preparar y optimizar las imágenes antes de su análisis (Gonzalez & Woods, 2017).

Algoritmos de Aprendizaje Automático

Los algoritmos de aprendizaje automático son una categoría de algoritmos que permiten a las computadoras aprender de los datos y realizar predicciones o decisiones sin ser explícitamente programadas para hacerlo. Son clave en la visión computacional para el reconocimiento de patrones y el análisis de imágenes (Russell & Norvig, 2016).

Diagnóstico Médico

El diagnóstico médico es el proceso de identificar una enfermedad o condición en un paciente a partir de sus síntomas o resultados de pruebas. En el contexto de la anemia, esto puede implicar la interpretación de los cambios de color en la conjuntiva ocular (Laposata, 2011).

Aprendizaje Profundo

El aprendizaje profundo es una subcategoría de aprendizaje automático que utiliza redes neuronales con múltiples capas (denominadas redes neuronales profundas) para modelar y entender datos complejos, como imágenes médicas (Goodfellow, Bengio & Courville, 2016).

Conjunto de Datos

Un conjunto de datos es una colección de datos relacionados utilizados para el análisis y la formación de algoritmos de aprendizaje automático. En este caso, un conjunto de

datos podría consistir en imágenes de la conjuntiva ocular de personas con y sin anemia (Harrington, 2012).

Clasificación de Imágenes

La clasificación de imágenes es una tarea de aprendizaje supervisado que asigna una etiqueta a una imagen completa basándose en su contenido visual. Este proceso puede ser usado en la detección de anemia, clasificando las imágenes de la conjuntiva ocular como "anémicas" o "no anémicas" (Krizhevsky, Sutskever & Hinton, 2012).

Extracción de Características

La extracción de características es el proceso de identificar y extraer información relevante de los datos de entrada (como las imágenes) para su posterior análisis. En el caso de la detección de anemia, las características podrían incluir el color, la textura y el tamaño de la conjuntiva ocular (Mirmehdi, Xie & Suri, 2011).

Conjuntos de Entrenamiento y Prueba

Los conjuntos de entrenamiento y prueba son divisiones de un conjunto de datos. El conjunto de entrenamiento se utiliza para entrenar el modelo de aprendizaje automático, mientras que el conjunto de prueba se utiliza para evaluar su rendimiento y generalización en datos no vistos anteriormente (Harrington, 2012).

Evaluación de Modelos

La evaluación de modelos es el proceso de medir el rendimiento y la efectividad de un modelo de aprendizaje automático. Los métodos de evaluación pueden incluir métricas como precisión, sensibilidad, especificidad y el área bajo la curva ROC (Fawcett, 2006).

Segmentación de Imágenes

La segmentación de imágenes divide una imagen en partes o regiones que comparten atributos similares para su análisis detallado (Gonzalez & Woods, 2017).

Overfitting

El overfitting ocurre cuando un modelo se ajusta demasiado a los datos de entrenamiento y tiene un mal rendimiento en datos nuevos (Hawkins, 2004).

Aumento de Datos

El aumento de datos es una técnica para incrementar la cantidad de datos de entrenamiento al crear versiones modificadas de los datos existentes (Perez & Wang, 2017).

ROI (Region Of Interest)

ROI se refiere a las áreas de una imagen que son especialmente relevantes para una tarea específica (Gonzalez & Woods, 2017).

HOG (Histogram of Oriented Gradients)

HOG es una técnica de extracción de características que se utiliza comúnmente para la detección de objetos en imágenes (Dalal & Triggs, 2005).

SIFT (Scale-Invariant Feature Transform)

SIFT es un método para detectar y describir características locales en imágenes que son invariantes a la escala y la orientación (Lowe, 2004).

ORB (Oriented FAST and Rotated BRIEF)

ORB es un método eficiente para la extracción de características y la correspondencia de características en imágenes (Rublee et al., 2011).

LBP (Local Binary Patterns)

LBP es un método simple pero eficaz para la extracción de características de textura en imágenes (Ojala, Pietikainen & Maenpaa, 2002).

CNN (Convolutional Neural Network)

Una CNN es un tipo de red neuronal profunda especialmente diseñada para el procesamiento de imágenes (Krizhevsky, Sutskever & Hinton, 2012).

Capa

En el contexto de las redes neuronales, una capa es un conjunto de neuronas que procesan los datos en paralelo y transfieren la salida a la siguiente capa (Goodfellow, Bengio & Courville, 2016).

Perceptrón

El perceptrón es una de las arquitecturas más simples de red neuronal, con una sola capa de neuronas de salida (Rosenblatt, 1958).

SVM (Support Vector Machines)

Las SVM son modelos de aprendizaje supervisado que se utilizan para la clasificación y la regresión (Cortes & Vapnik, 1995).

KNN (K-Nearest Neighbors)

KNN es un método de aprendizaje supervisado que clasifica los objetos basándose en los k objetos más cercanos en el espacio de características (Cover & Hart, 1967).

RF (Random Forest)

RF es un método de aprendizaje supervisado que utiliza un conjunto de árboles de decisión para la clasificación o regresión (Breiman, 2001).

MLP (Multilayer Perceptron)

MLP es una red neuronal artificial que consta de múltiples capas de neuronas, y se utiliza comúnmente para la clasificación y regresión (Rumelhart, Hinton & Williams, 1986).

ANN (Artificial Neural Network)

Las ANN son modelos computacionales que se inspiran en las redes neuronales biológicas y se utilizan para tareas como la clasificación, la regresión y la extracción de características (Goodfellow, Bengio & Courville, 2016)

EfficientNet:

Es una arquitectura de red neuronal convolucional que utiliza la escalación compuesta para ajustar la profundidad, la anchura y la resolución de la red, logrando un rendimiento superior con menos parámetros y menor costo computacional (Tan & Le, 2019).

ResNet (Residual Network):

Es una red neuronal convolucional que utiliza conexiones de salto o "shortcut connections" para facilitar el flujo de la señal a través de la red, lo que permite construir redes mucho más profundas sin sufrir el problema del desvanecimiento del gradiente (He et al., 2016).

Inception-ResNet:

Es una arquitectura de red neuronal que combina elementos de las arquitecturas Inception y ResNet. Utiliza los módulos Inception para realizar operaciones convolucionales en múltiples escalas, y las conexiones de salto de ResNet para facilitar el flujo de la señal (Szegedy et al., 2017).

2.4. Hipótesis

Para la identificación de la hipótesis general y de las hipótesis específicas se determinó de acuerdo a los problemas y objetivos identificados previamente. Asimismo, estas hipótesis se han moldeado en base a el contexto que se encuentre el avance de la anemia en el Perú.

2.4.1. Hipótesis General

HG_1: Mediante la implementación de un modelo de visión computacional se logra determinar el pre-diagnóstico de la enfermedad de anemia a partir de la conjuntiva ocular.

HG_0: Mediante la implementación de un modelo de visión computacional no se logra determinar el pre-diagnóstico de la enfermedad de anemia a partir de la conjuntiva ocular.

2.4.2. Hipótesis Específica

HE1: Existen datos suficientes para ser modelados usando los algoritmos de machine learning.

HE2: El conjunto de datos es apto para ser modelado y entrenado usando algoritmo de aprendizaje no supervisado.

HE3: Los algoritmos de aprendizaje Random Forest, SVM, RNN, son modelados y entrenados satisfactoriamente.

HE4: El algoritmo de Random Forest presenta mayor bondad de ajuste a los datos, menor error de estimación y tiempo de procesamiento en comparación a los modelos de aprendizaje no superviado SVM, RNN.

2.4.3. Variables e indicadores

Para la identificación de la hipótesis general y de las hipótesis específicas se determinó de acuerdo a los problemas y objetivos identificados previamente. Asimismo, estas hipótesis se han moldeado en base a el contexto que se encuentre el avance de la anemia en el Perú.

Las variables independientes en esta investigación se definen como "Imagen de la conjuntiva ocular" y "Técnicas de Machine Learning".

Bajo el término "Imagen de la conjuntiva ocular", se contempla la fotografía del ojo humano que será objeto de análisis para la detección de la anemia. Esta variable es medible mediante tres indicadores: la resolución de la imagen, cuantificada en la cantidad total de píxeles; el procesamiento de la imagen, evaluado a partir del nivel de nitidez, contraste o ruido presente en la misma; y finalmente, las características extraídas de la imagen, que se miden en función del número y tipo de atributos recabados de la imagen.

Por otro lado, las "Técnicas de Machine Learning" hacen referencia a los algoritmos aplicados para la interpretación y análisis de los datos extraídos de las imágenes y la consiguiente elaboración de predicciones. Los indicadores de esta variable engloban el modelo de machine learning seleccionado y las métricas de desempeño del modelo, incluyendo la precisión (accuracy), la exhaustividad (recall) y la puntuación F1 (F1-score).

La variable dependiente se identifica como el "PreDiagnóstico de anemia". Este resultado, que se busca prever utilizando las variables independientes, se mide por medio de un único indicador: el porcentaje de diagnósticos correctos, que se calcula tomando el número de diagnósticos correctos sobre el número total de diagnósticos, y multiplicándolo por 100.

CAPÍTULO III: METODOLOGÍA

Para el siguiente capítulo se plantear un plan de acción sobre cómo se desarrollará el proyecto, desde los detalles de la adquisición de la base de datos que van a hacer usadas hasta la aplicación de los modelos de predicción. Es importante considerar que esta investigación se centra en entregar un algoritmo que este centrado en la predicción de la enfermedad.

3.1. Diseño, tipo y enfoque de Investigación

3.1.1. Diseño (No experimental)

Un diseño no experimental se puede definir como un estudio realizado sin manipulación intencional de variables. Esto quiere decir que son estudios que no modifican intencionalmente las variables independientes para ver su efecto sobre otras variables. Lo que se hace en la investigación no empírica es la observación o medición de fenómenos y variables que ocurren en el medio natural con fines de análisis. (Hernández y Mendoza, 2018, p. 174).

El trabajo a realizar para este proyecto es de tipo no experimental, consiste en diseñar un sistema de visión por computadora para pre diagnosticar anemia a partir de una fotografía que muestra la conjuntiva palmar del ojo.

3.1.2. Tipo – nivel (Aplicada / Explicativo)

Este estudio se ubica dentro del tipo de investigación aplicada, puesto que se fundamenta en los hallazgos de la investigación básica con el propósito de abordar y resolver problemas concretos de la sociedad. La veracidad de este enunciado radica en el hecho de que los problemas sociales reales se originan a partir de los resultados obtenidos en la investigación básica (Ñaupas, Valdivia, Palacios & Romero, 2018, p. 136).

De igual manera, en función del nivel de explicación requerido, este tipo de investigación no se limita a la mera descripción de fenómenos, conceptos o variables, o a la determinación de relaciones entre estos. Por el contrario, se propone el desafío de indagar las causas que dan origen a distintos hechos y fenómenos (ya sean naturales, sociales, psicológicos, sanitarios, etc.). En consecuencia, su objetivo se centra en dilucidar por qué un fenómeno sucede, en qué circunstancias se presenta, o cuál es la razón por la que dos o más variables se encuentran interrelacionadas (Hernández y Mendoza, 2018, p. 113).

3.1.3. Enfoque - Cuantitativo

Esta investigación es de enfoque cuantifico por que se alinean a los objetivos planteados a la investigación. Los métodos cuantitativos utilizan la recopilación y el análisis de datos para responder preguntas de investigación y probar hipótesis prefabricadas, y se basan en la medición de variables y herramientas de investigación (Ñaupas, Valdivia, Palacios y Romero, 2018, página 140).

3.2. Población y Muestra

Población:

La población puede definirse como el número total de unidades de estudio que contienen las características necesarias para ser consideradas como tales. Estas unidades pueden ser personas, objetos o eventos que presenten características necesarias para una investigación (Ñaupas, Valdivia, Palacios, & Romero, 2018, pp. 334). La población para el estudio son el total de imágenes de pacientes con problemas oculares que fueron atendidos por la clínica durante el estudio. La unidad de análisis es cada imagen del ojo del paciente con la conjuntiva palpebral del ojo

Muestra:

Una muestra representativa es un subconjunto de una población o universo, seleccionado por varios métodos, de modo que el grupo seleccionado sea adecuadamente representativo de toda la población (Silvestre & Huamán, 2019, p. 310). Para este caso se toma como muestra los 218 imágenes tomando el ojo del paciente mostrando la parte inferior, están acompañadas por recortes de la conjuntiva palpebral del ojo (fornix, palpebral, fornix y palpebral) y documentación donde se tiene el nivel de hemoglobina en sangre del paciente, su edad y su sexo, dicha muestra fue extraída del repositorio de datos EYES-DEFY-ANEMIA dentro la página de IEEEDataPort actualizada hasta el 31 de enero del año 2022, estas se recolectaron de un grupo de pacientes de la India e Italia.

Para el presente trabajo de investigación estas imágenes pasaran por un proceso de llamado Aumento de datos, el objetivo es mejorar el volumen del dataset, para que el modelo tenga un rendimiento aceptable y los resultados sean más precisos. Inicialmente, se hará un estudio de la data encontrada en la documentación para la validación del dataset, luego de esto la muestra pasará por un modelo de machine learning, separando los datos en enteramiento (70%), test (20%) y prueba (10%), dependiendo al resultado se tomará una decisión con que algoritmo tiene se tiene una mejor precisión.

Tabla 10 *Población, muestra, unidad de análisis del estudio y Variable y tipo de variable*

Dimensión	Definición	Estudio
Población	Totalidad de casos que comparten características de interés para el estudio (Ñaupas et al., 2018).	En este caso, la población consiste en todas las imágenes de pacientes con problemas oculares atendidos por la clínica.
Muestra	Subgrupo representativo seleccionado de la población (Silvestre & Huamán, 2019).	La muestra se compone de 218 imágenes oculares de pacientes, con datos relacionados como el nivel de hemoglobina en sangre, la edad y el sexo, obtenidas del repositorio de datos EYES-DEFY-ANEMIA en IEEE-DataPort.
Unidad de análisis	Elemento individual del que se recoge información en un estudio (Kerlinger, 1986).	La unidad de análisis aquí es cada imagen individual de un ojo de paciente que muestra la conjuntiva palpebral.
Variable y tipo de variable	Atributo que puede variar entre los elementos de estudio (Kothari, 2004).	La "imagen de la conjuntiva ocular" es una variable independiente en el estudio.

Nota. Elaboración propia

Tabla 11 *Descripción de variables*

DEFINICIÓN DE VARIABLES			
VARIABLE Y DEFINICIÓN	Variable	Definición conceptual	Valores
Y: Respuesta	Anemia	Indica si el paciente posee o no la enfermedad de anemia	0=Ausencia 1= Presencia
X1: Explicativa	Genero	Género del paciente entre masculino y femenino	0= Mujer 1= Hombre
X2: Explicativa	Edad	La edad del paciente al momento de realizar la prueba	65+
X3: Explicativa	Hemoglobina	Índice que mide el nivel de glóbulos rojo en sangre	10-15
X4: Explicativa	Nacionalidad	De donde proviene el paciente	0= Italia 1 = India

Nota. Extraido de IEEDataPort dataset EYES-DEFI-ANEMIA

3.3. Instrumentos de medida

Tabla 12 *Instrumentos de Medida Utilizados en el Estudio*

Instrumento de Medida	Descripción
Cámara o dispositivo de captura de imágenes	Utilizado para tomar fotografías de alta resolución de la conjuntiva ocular, proporcionando la materia prima para el análisis.
Software de procesamiento de imágenes	Sirve para mejorar la calidad de las imágenes, eliminar ruido y extraer las características relevantes para el análisis de la anemia.
Algoritmo de Machine Learning	Esencial para analizar las características extraídas de las imágenes y realizar la predicción de la presencia o ausencia de anemia.
Herramientas de evaluación de modelos	Necesarias para medir el rendimiento del algoritmo de Machine Learning en términos de precisión, exhaustividad, puntuación F1 y exactitud.
Datos de pacientes	Son fundamentales para comparar los resultados del sistema propuesto con diagnósticos clínicos confirmados de anemia. Estos podrían incluir datos demográficos, historiales médicos y resultados de exámenes de sangre, entre otros.

Nota. Elaboración propia

3.4. Operacionalización de variables

Tabla 13 *Operacionalización de variables*

	Definición (Referencia)	Indicador	Fórmula de Indicador
Independiente			
Imagen de la conjuntiva ocular	Según Rafael et al. (2020), la conjuntiva ocular es una parte del ojo que puede indicar varios estados de salud, incluyendo la anemia.	Resolución de la imagen	Cantidad de píxeles en la imagen
		Procesamiento de la imagen	Nivel de nitidez, contraste o ruido de la imagen
		Características extraídas	Número y tipo de características extraídas de la imagen
Técnicas de Machine Learning	De acuerdo con Alpaydin (2020), el aprendizaje automático implica el uso de algoritmos para descubrir patrones en datos y hacer predicciones o decisiones sin ser explícitamente programados para realizar la tarea.	Modelo de clasificación	Tipo de modelo de machine learning utilizado
		Accuracy	$TP + TN/TP + FP + FN + T$N
		Precision del Modelo	TP / ($TP + FP$)
		Recall del Modelo	TP / ($TP + FN$)
		F1-score del Modelo	2 ∗ (Precision ∗ Recall) / (Precision + Recall)
Dependiente			
PreDiagnóstico de anemia	Según Ong et al. (2019), el diagnóstico de anemia se basa en la medición de los niveles de hemoglobina y/o hematocrito en la sangre del paciente. En este estudio, se propone usar imágenes de la conjuntiva ocular para predecir la anemia.	Porcentaje de diagnósticos correctos	(Número de diagnósticos correctos / Número total de diagnósticos) * 100

Nota. Elaboración propia

Donde:

- TP (Verdaderos Positivos): Predicciones correctas de anemia (Berrar, 2019).
- FP (Falsos Positivos): Diagnósticos erróneos de anemia (Berrar, 2019).
- FN (Falsos Negativos): Casos de anemia no detectados (Berrar, 2019).

- TN (Verdaderos Negativos): Predicciones acertadas de ausencia de anemia (Berrar, 2019).

3.5. Técnicas de Recolección de Datos

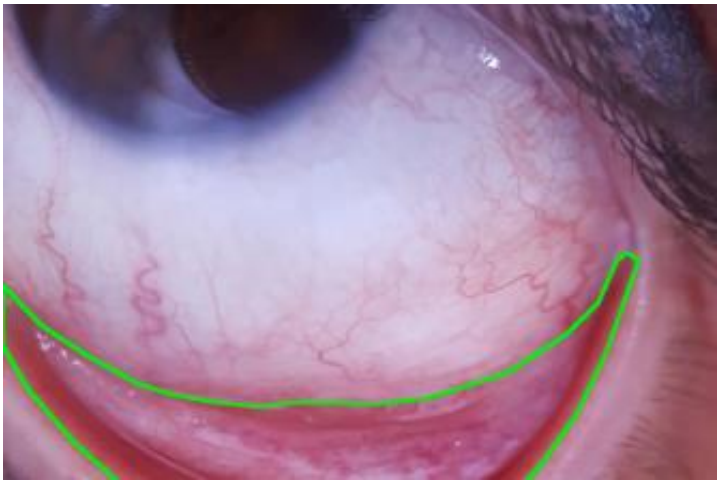
La recolección de datos para este estudio se realizó a través de un conjunto de datos titulado "Eyes Defy Anemia" publicado en IEEE DataPort por un equipo de autores liderado por Giovanni Dimauro de la Università degli Studi di Bari, Italia, y Rosalia Maglietta del Italian National Council of Research (CNR) - STIIMA, junto con Thulasi Bai y Sivachandar Kasiviswanathan del Electronics and Communication Engineering Department, KCG College of Technology, Chennai, India (Dimauro et al., 2022).

Este conjunto de datos incluye 218 imágenes de conjuntivas oculares, que representan una muestra de pacientes de Italia e India. Las imágenes se capturaron con un smartphone Samsung S6 equipado con un dispositivo especial para estandarizar la iluminación, eliminar la influencia de la luz ambiental y magnificar las imágenes. Además, cada imagen del conjunto de datos viene acompañada de elementos segmentados (conjuntiva palpebral, forniceal y palpebral + forniceal), así como información esencial, como el valor de Hb medido en el laboratorio, la edad y el sexo del paciente (Dimauro et al., 2022).

El conjunto de datos se divide en dos carpetas, una para los pacientes italianos y otra para los pacientes indios. En la carpeta de Italia, hay 123 carpetas y un archivo Italy.xlsx. Cada carpeta numerada contiene la imagen original del ojo y las imágenes de las conjuntivas segmentadas a mano. Similarmente, la carpeta de India contiene 95 carpetas y un archivo India.xlsx con una estructura de archivo similar (Dimauro et al., 2022).

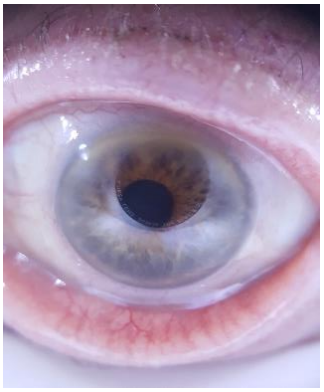
Las imágenes recopiladas permiten estudiar los algoritmos de segmentación de conjuntivas o partes expuestas de la esclerótica y el iris, además de ser útiles para correlacionar directamente la palidez con el valor de Hb y evaluar el rendimiento de los algoritmos de segmentación (Dimauro et al., 2022).

Figura 31 *Imagen de la conjuntiva ocular de un paciente*



Nota. Extraído de https://ieee-dataport.org/documents/eyes-defy-anemia

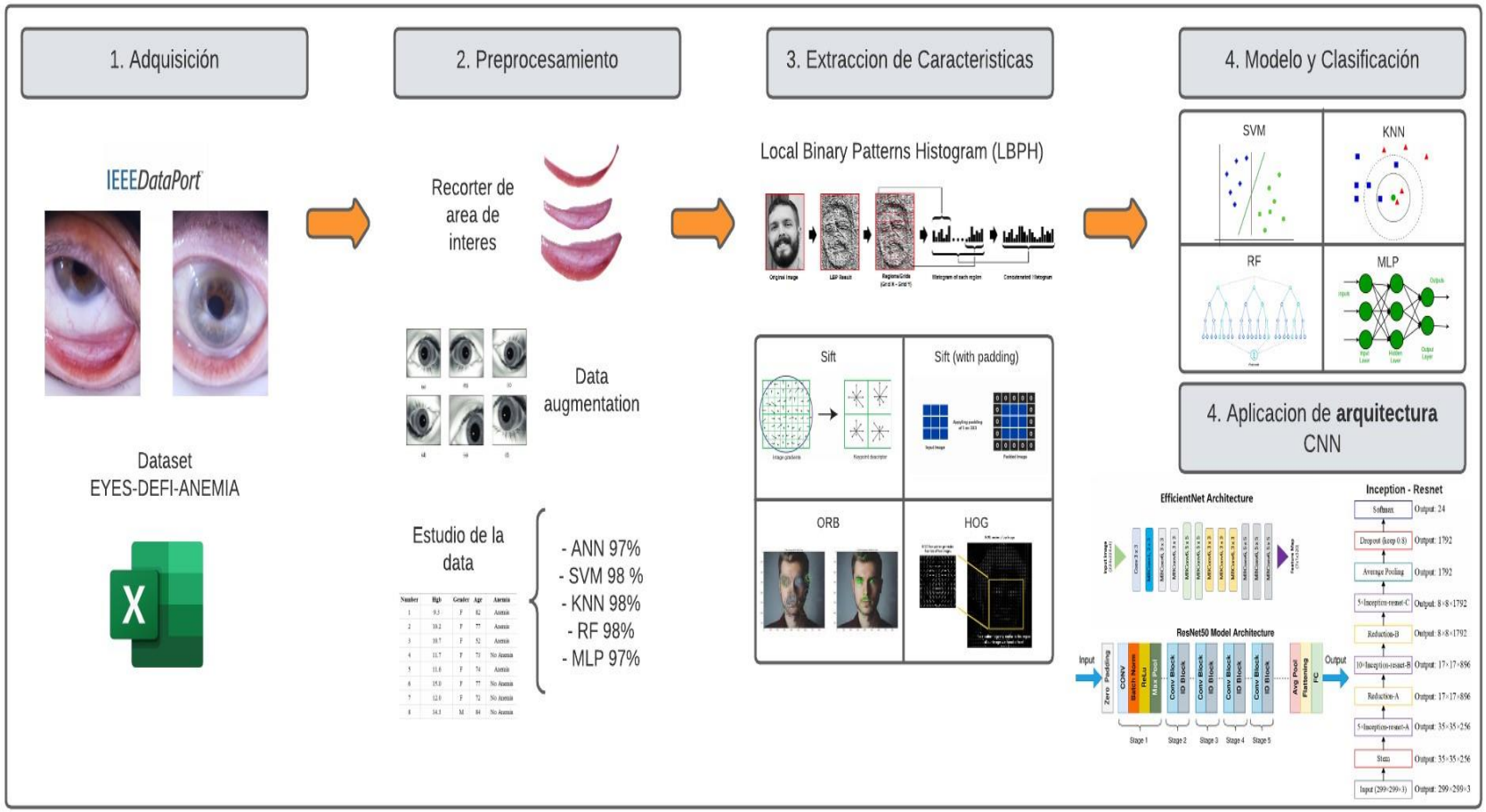
Tabla 14 *Ejemplo de un caso y la información encontrada en la base de datos*

Paciente 1 Grupo Italy	
	**Hgb** 9.3
	Gender F
	Age 82

Nota. Elaboración propia

3.6. Técnicas para el procesamiento y análisis de la información

Figura 32 *Metodología de la implementación*



Nota. Elaboración propia

3.7. Cronograma de actividades y presupuesto

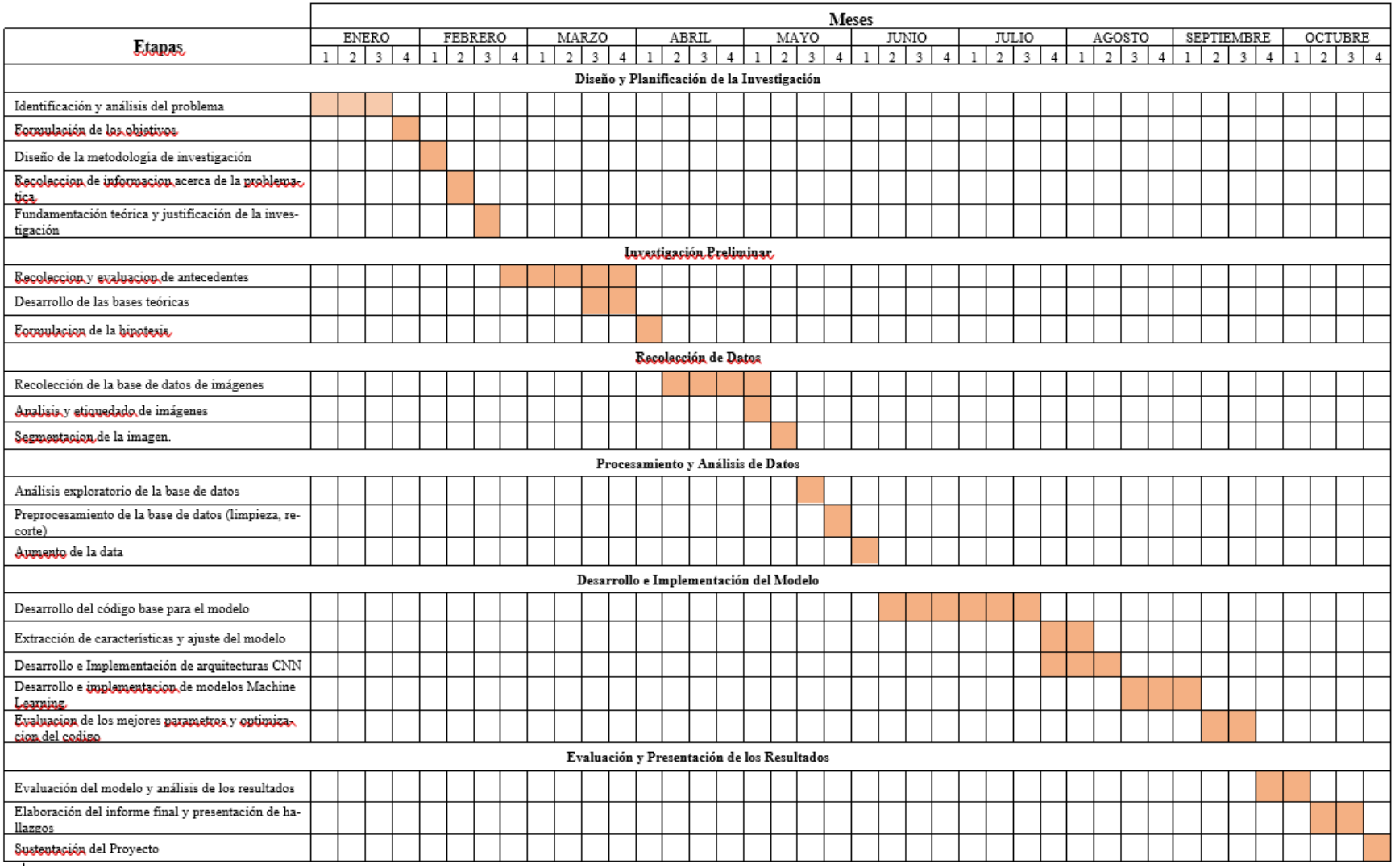


Tabla 15 *Presupuesto*

Recurso	Descripción	Costo (S/.)
Laptop	Laptop de alto rendimiento para procesar datos e imágenes (Intel Core i7, 16GB RAM)	Propio
Software de Desarrollo	Python, bibliotecas específicas de visión computacional y Machine Learning (OpenCV, TensorFlow, Keras, etc)	-
Base de datos de imágenes	Adquisición de la base de datos a través de la página IEEEDataPort	S/. 154 (40 USD)
Espacio de almacenamiento	Uso de Google Drive para almacenamiento de datos	-
Gastos de Investigación	Incluye el tiempo dedicado al estudio y desarrollo de la investigación	S/. 5000
Asesoría	Ayuda para la revisión del estilo del documento y guía en el desarrollo del código	S/. 300
Acceso a literatura científica	Acceso gratuito a través de la universidad o contacto con autores	-
Electricidad e internet	Costos operativos de servicios para el desarrollo de la investigación (estimados)	S/. 200
Consulta a Hematólogo	Revisión de la base de datos y verificación del trabajo	S/. 100
TOTAL	-	**S/. 5745**

Nota. Elaboración propia

3.8. Metodología para la medición de resultados de la implementación

La evaluación de los resultados de un sistema de aprendizaje automático es un paso crucial que garantiza que el modelo es eficaz en su aplicación específica. Esta evaluación requiere el uso de varias métricas que se emplean para analizar la precisión y la fiabilidad de las predicciones del modelo. Siguiendo el enfoque de Harrington (2012), se describen varias métricas que se pueden utilizar para medir los resultados de la implementación de un sistema de aprendizaje automático.

Las métricas a continuación se derivan de la interpretación de la matriz de confusión, un cuadro que resume el desempeño del modelo comparando las predicciones del modelo con los verdaderos valores conocidos. Las métricas se calculan a partir de los valores de Verdadero Positivo (TP), Falso Positivo (FP), Verdadero Negativo (TN) y Falso Negativo (FN) presentes en la matriz.

Accuracy (Precisión Global): Esta métrica proporciona una relación general de las predicciones correctas hechas por el modelo, incluyendo tanto las predicciones positivas como las negativas. Se calcula como:

$$Accuracy = (TP + TN) / (TP + FP + FN + TN)$$

No obstante, la accuracy no es una métrica adecuada cuando se trabaja con datos desbalanceados, ya que puede llevar a interpretaciones erróneas.

Precision (Precisión): Esta métrica representa la proporción de predicciones positivas que fueron clasificadas correctamente por el modelo. Se calcula como:

$$Precision = TP / (TP + FP)$$

Una alta precision indica una menor cantidad de falsos positivos.

Recall (Exhaustividad): Esta métrica indica la proporción de instancias positivas reales que el modelo clasificó correctamente. Se calcula como:

$$Recall = TP / (TP + FN)$$

Un alto recall sugiere que el modelo es capaz de identificar la mayoría de los resultados positivos en el conjunto de datos.

F1-Score: Esta es una medida que busca encontrar un equilibrio entre Precision y Recall. Es el promedio armónico de la precisión y la exhaustividad, y se utiliza a menudo cuando se manejan datos desbalanceados para proporcionar una evaluación más completa del rendimiento del modelo. Se calcula como:

$$F1 - Score = 2 * (Precision * Recall) / (Precision + Recall)$$

Estas métricas proporcionan una manera robusta y completa de evaluar el rendimiento de un modelo, y cada una tiene sus propias fortalezas y debilidades que deben tenerse en cuenta al interpretar los resultados.

3.9. Metodología de la implementación de la solución

Para el presente trabajo se dividirá en diferentes pasos y tareas con la finalidad de poder procesar, analizar y catalogar la data obtenida, La visión artificial permite procesar imágenes obtener diferentes características para poder así entenderlas, darles una tarea y un valor (Culjak et al., 2012).

3.9.1. Adquisición de la base de datos:

Para la ejecución de este estudio, se ha obtenido un conjunto de datos de vital importancia a través de IEEEDataPort, el cual se encuentra organizado por carpetas correspondientes a cada paciente. A continuación, se presentará una tabla detallando las acciones y procedimientos realizados con este dataset en el marco de nuestra investigación.

Tabla 16 *Actividades para la Adquisición de la base de datos*

Fases	Descripción	Procedimientos
Identificación de las Fuentes de Información	Selección de repositorios de datos que alberguen imágenes claras y de alta calidad de la conjuntiva ocular.	Estudiar el papel específico de la conjuntiva ocular en la identificación de la anemia.
		Investigar bases de datos que posean imágenes de alta calidad de la conjuntiva ocular.
		Determinar si las bases de datos contienen información complementaria sobre los sujetos de las imágenes.
		Descargar el conjunto de datos del repositorio seleccionado.
		Segmentar las imágenes en dos categorías: sujetos con anemia y sujetos sin anemia.
Anotación de las Imágenes	Etiquetar y delimitar el área de interés dentro de cada imagen ocular para futuros análisis.	Importar el conjunto de datos a la plataforma LabelBox.
		Transferir las imágenes seleccionadas al área de etiquetado
		Anotar la región de interés en las imágenes seleccionadas
		Descargar las imágenes anotadas en formato ndjson para el procesamiento posterior.
Preprocesamiento de las Imágenes	Utilizar un entorno de programación Python en Jupyter para transformar las anotaciones en coordenadas para el recorte de la imagen.	Importar el archivo ndJson al entorno de programación.
		Analizar la estructura del archivo y localizar las anotaciones.
		Transformar las coordenadas anotadas en un array de NumPy.
		Dibujar el polígono correspondiente a la región de interés en cada imagen.
Segmentación de la Imagen		Generar una máscara para el polígono en cada imagen.

	Aislar la región de interés de cada imagen creando una imagen derivada para el análisis y eliminar el fondo oscuro de la imagen resultante	Aplicar la máscara para segmentar la imagen. Guardar la imagen segmentada para su análisis posterior. Aplicar un algoritmo para eliminar el fondo oscuro de cada imagen.
Validación de la base de datos con hematólogo	Verificación de la precisión del etiquetado por un profesional de la salud especializado en Anemia	Establecer comunicación con un hematólogo Solicitar la validación del proceso de etiquetado Realizar ajustes necesarios con base a la validación del profesional

Nota. Elaboración propia

- **Actividad 1: Identificación de las Fuentes de Información:**

Esta fase se centra en localizar e identificar bases de datos adecuadas que contengan imágenes de alta calidad de la conjuntiva ocular. Se debe prestar atención a las imágenes en color, ya que los matices en el color de la conjuntiva pueden indicar la presencia de anemia. Para garantizar una formación y validación efectiva del sistema de visión por computadora, es esencial que las bases de datos contengan una cantidad considerable de imágenes de individuos anémicos y no anémicos (Sengur et al., 2017).

Además, es fundamental contar con información adicional relevante como edad, género, hemoglobina, la confirmación del estado de anemia, etc. Esta información contextual mejora la precisión del sistema de visión por computadora. Una vez identificadas las bases de datos apropiadas, las imágenes se descargan y se organizan en categorías según si los sujetos tienen anemia o no.

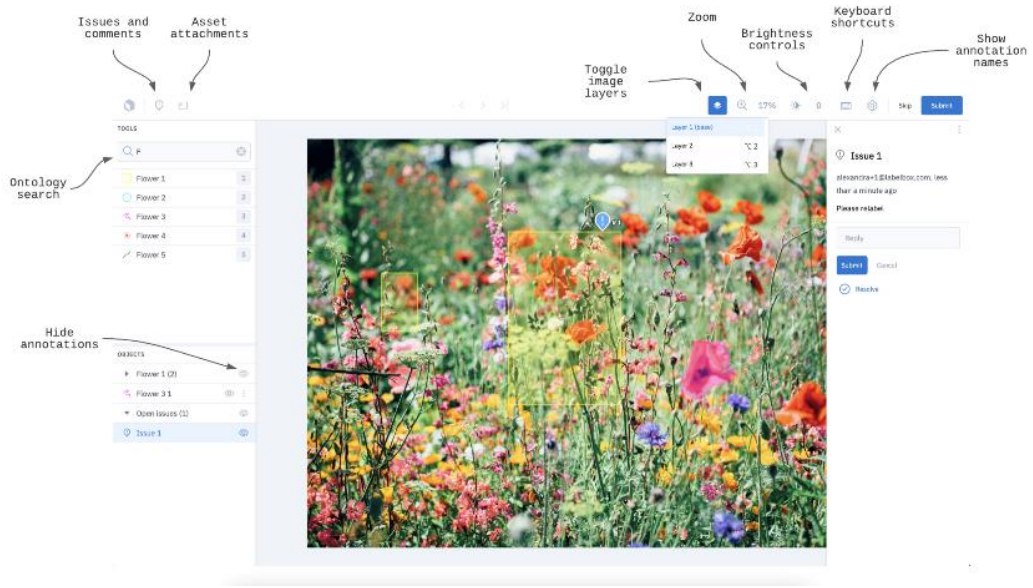
Entregable: Base de datos con la información del paciente y una imagen del ojo del paciente mostrando el interior del parpado inferior.

- **Actividad 2: Anotación de las Imágenes:**

La anotación de imágenes es un procedimiento vital en la creación de un conjunto de datos de entrenamiento fiable para los sistemas de visión por computadora. En este estudio, el proceso de anotación se centra en la conjuntiva ocular, la región que puede mostrar variaciones visibles en color y textura en casos de anemia.

Para llevar a cabo la anotación de las imágenes, se utilizan herramientas de anotación de imágenes, como Labelbox, que proporcionan una interfaz de usuario interactiva y fácil de usar para marcar áreas de interés en las imágenes. Específicamente, en este proyecto, se utiliza la herramienta de polígono disponible en Labelbox.

Figura 33 *Imagen del entorno de labelbox*

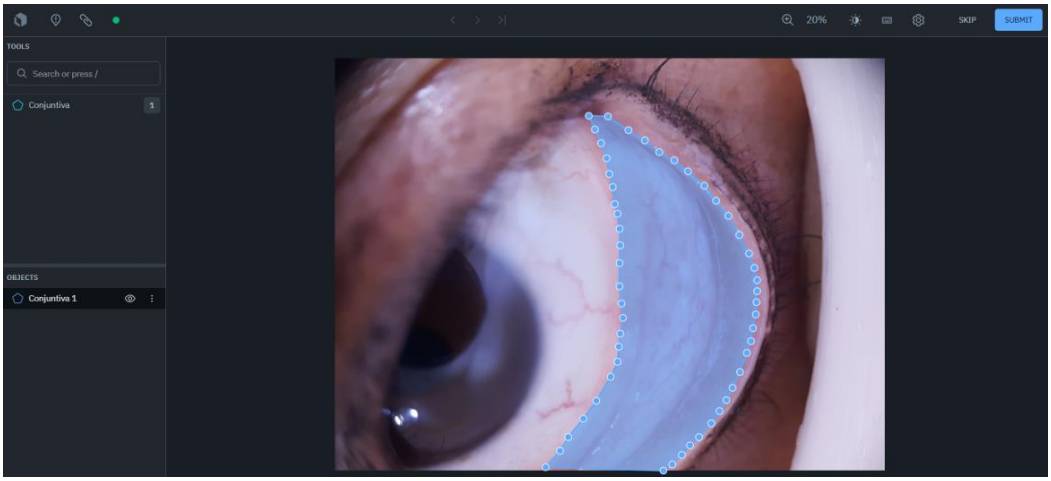


Nota. Extraído de https://docs.labelbox.com/docs/label-data

La herramienta de polígono permite al usuario dibujar formas poligonales personalizadas alrededor de la región de interés en una imagen. Cada vértice del polígono se coloca con un clic del mouse, y el polígono se completa al unir el último vértice con el primero. Esta característica permite una gran flexibilidad, ya que los

polígonos pueden adaptarse a diversas formas y tamaños, lo que es especialmente útil cuando la región de interés (en este caso, la conjuntiva ocular) no tiene una forma uniforme en todas las imágenes.

Figura 34 *Funcionamiento de la herramienta polygon*



Nota. Elaboración propia

Una vez completada la anotación, las coordenadas de los vértices de cada polígono se guardan y se pueden descargar en un formato compatible, como ndjson, para su posterior análisis y procesamiento. Para garantizar la precisión y la fiabilidad de las anotaciones, es aconsejable revisar y validar las anotaciones realizadas. Este paso podría involucrar la colaboración con expertos en el campo, como hematologistas, que puedan revisar y confirmar la precisión de las anotaciones.

Entregable: Archivo ndjson con las imágenes y la anotación sobre el área del interés.

- **Actividad 3: Pre procesamiento de las Imágenes:**

El pre procesamiento de las imágenes es una etapa crucial en la visión por computadora, que tiene como objetivo preparar las imágenes para su posterior análisis y procesamiento. En este estudio, el pre procesamiento se enfoca en la transformación de las imágenes anotadas en un formato listo para la segmentación de la conjuntiva ocular. Las actividades en esta etapa incluyen:

1. Transferencia de las imágenes a un código Python: El conjunto de anotaciones descargadas en formato ndjson se cargan en un entorno de Jupyter utilizando bibliotecas de Python como NumPy y Matplotlib. Esto permite la manipulación de las anotaciones como arreglos de NumPy y su representación visual en el entorno de Jupyter.

Pseudocódigo 1: *Procesamiento de datos etiquetados con anotaciones y objetos en JSON*

```
1. Importar la biblioteca para manejar JSON
2. Abrir el archivo con los datos etiquetados y cargar cada línea en la lista ';datos';
3. Crear un diccionario vacío ';anotaciones';
4. Para cada ítem en ';datos';:
        a. Si el ítem contiene información relevante:
        i. Si el ítem contiene etiquetas:
                1. Para cada etiqueta en las etiquetas del ítem:
                        a. Si la etiqueta contiene anotaciones y objetos:
                                i. Añadir las anotaciones de los objetos al diccionario ';anotaciones'; con un identificador único como clave
5. Para cada imagen y anotación en las primeras entradas de ';anotaciones';:
        a. Imprimir ';Imagen: {imagen}';
        b. Para cada objeto en ';anotación';:
                i. Imprimir ';Objeto: {nombre del objeto}';
                        ii. Para cada punto en el polígono del objeto:
                                1. Imprimir ';Punto: {punto}';
```

Nota. Elaboración propia

2. Conversión de las anotaciones en coordenadas: Las anotaciones, que se encuentran en formato ndjson, se convierten en coordenadas. Las coordenadas indican la posición de la conjuntiva ocular en las imágenes, y se usarán para generar una máscara en la fase de segmentación de la imagen.
3. Dibujo del polígono en la imagen: Utilizando Matplotlib, se dibuja el polígono definido por las coordenadas en las imágenes. Esto proporciona una visualización gráfica de la región de interés y permite verificar la precisión de las anotaciones.

Pseudocódigo 2*: Visualización de anotaciones de objetos en imágenes descargadas*

1. Importar las bibliotecas necesarias
2. Para cada anotación en la lista de anotaciones:
    a. Obtener la URL de la imagen de la anotación
    b. Descargar la imagen desde la URL
    c. Convertir la imagen descargada en un array numérico
    d. Crear una nueva figura y eje con la herramienta de visualización
    e. Mostrar la imagen en el eje
    f. Obtener las etiquetas de la anotación
    g. Para cada etiqueta en las etiquetas:
        i. Si el nombre del objeto en la etiqueta es el de interés:
            1. Obtener las coordenadas del polígono del objeto
            2. Convertir las coordenadas en un array numérico
            3. Dibujar el polígono en la imagen y esperar durante un intervalo de tiempo
    h. Mostrar la figura

Nota. Elaboración propia

El pre procesamiento de las imágenes asegura que las anotaciones sean correctas y estén listas para la segmentación, permitiendo un procesamiento más eficiente y efectivo de las imágenes durante la fase de segmentación.

Entregable: Grupo de imágenes con las coordenadas del área de interés resaltadas

- **Actividad 4: Segmentación de la Imagen:**

La segmentación de imágenes es un paso vital en la visión por computadora, que tiene como objetivo distinguir la región de interés (ROI) de las imágenes para su posterior análisis. En este estudio, la ROI es la conjuntiva ocular, que es de particular relevancia para el pre-diagnóstico de la anemia. Las actividades principales de esta fase son:

a) Creación de una máscara a partir de las coordenadas del polígono: Basándose en las coordenadas del polígono obtenidas en la fase de pre procesamiento, se crea una "máscara" que identifica y aísla la región de interés. Esta máscara se superpone a la imagen original, resaltando sólo la conjuntiva ocular.

b) Aplicación de la máscara y recorte de la imagen: La máscara se aplica a la imagen original, y se recorta la imagen alrededor de la conjuntiva ocular, eliminando efectivamente el resto de la imagen. Esto resulta en una imagen que contiene exclusivamente la región de interés.

c) Guardado de la imagen recortada: La imagen recortada, que muestra sólo la conjuntiva ocular, se guarda para su posterior análisis. Este paso asegura que las futuras etapas de procesamiento se centren únicamente en la conjuntiva ocular, contribuyendo a la precisión del diagnóstico.

d) Eliminación del fondo de la imagen: Se elimina el fondo negro creado por la máscara para obtener únicamente la imagen de la conjuntiva ocular. Este paso garantiza que se analicen solo las características de la conjuntiva, eliminando cualquier posibilidad de interferencia del fondo de la imagen.

Este proceso garantiza que solo las características relevantes de la conjuntiva ocular se evalúen en las etapas posteriores de este estudio, lo que contribuye a la precisión y efectividad del diagnóstico de anemia basado en la visión por computadora (Gonzalez & Woods, 2007).

Entregable: Imágenes de la base de datos segmentadas y sin el fondo

Pseudocódigo 3: *Procesamiento de imágenes para convertir pixeles casi negros en transparentes*

1. Importar la biblioteca para manejar imágenes
2. Asignar a 'folder_path'; la ruta de la carpeta donde se encuentran las imágenes
3. Para cada archivo en la carpeta:
    a. Si el archivo es una imagen (es decir, su nombre termina en 'png';):
        i. Combinar 'folder_path'; y el nombre del archivo para obtener la ruta completa a la imagen
        ii. Abrir la imagen
        iii. Convertir la imagen a RGBA (Red, Green, Blue, Alpha)
        iv. Obtener los datos de la imagen
        v. Crear una lista vacía 'newData';
        vi. Para cada pixel en los datos de la imagen:
            1. Si el pixel es (casi) negro, añadir un pixel transparente a 'newData';
            2. Si no, añadir el pixel a 'newData';
        vii. Reemplazar los datos de la imagen con ';newData';
        viii. Guardar la imagen

Nota. Elaboración propia

- **Actividad 5: Validación de la base de datos:**

La validación de los datos constituye una etapa imprescindible en cualquier proyecto de investigación para garantizar que la información utilizada es precisa, relevante y fiable. En el contexto de este estudio, la validación involucra la verificación de la exactitud y la adecuación de las imágenes de la conjuntiva ocular que se han recopilado y procesado.

La etapa de validación en este estudio se lleva a cabo en colaboración con un especialista en hematología. Un hematólogo es un médico que se especializa en

enfermedades de la sangre, como la anemia, y por lo tanto posee un amplio conocimiento y experiencia en la identificación y diagnóstico de esta condición. Su experticia es invaluable para asegurar que las imágenes de la conjuntiva ocular que se están utilizando para este estudio son representativas y adecuadas para el propósito del diagnóstico de la anemia. Las actividades principales de esta fase son:

a) Contacto con un Hematólogo: Se establece una comunicación con un hematólogo, quien será el encargado de revisar y validar la base de datos de imágenes de la conjuntiva ocular. La colaboración con un hematólogo es vital para asegurar que las imágenes que se utilizan en el estudio son adecuadas y representativas de las variaciones que pueden estar presentes en las conjuntivas de pacientes con anemia.
b) Revisión de las Imágenes: El hematólogo revisa las imágenes de la conjuntiva ocular y las anotaciones asociadas. Este proceso puede implicar la inspección de las imágenes y la confirmación de que las anotaciones corresponden correctamente a la conjuntiva.
c) Validación de la Base de Datos: Tras la revisión de las imágenes, el hematólogo proporciona retroalimentación y valida la base de datos. Esto puede resultar en la aprobación de la base de datos tal como está, o puede requerir ajustes y modificaciones basados en la experiencia y el conocimiento del hematólogo.
d) Realización de Ajustes Necesarios: Si la retroalimentación del hematólogo indica que se requieren cambios, estos se implementan en la base de datos. Esto puede

incluir la eliminación de imágenes no representativas, la modificación de anotaciones, o la incorporación de más imágenes para mejorar la representatividad de la base de datos.

Este proceso de validación garantiza la precisión y la representatividad de la base de datos, lo que es fundamental para el éxito de cualquier sistema de diagnóstico basado en la visión por computadora (Kumar & Kapoor, 2016).

Entregable: Imágenes verificadas por un profesional del área.

3.9.2. Actividades de la fase de Pre procesamiento (Base de datos)

Tabla 17 *Actividades de la fase de Pre procesamiento (Base de datos)*

Actividades	Descripción	Tareas
Evaluación de la clasificación de imágenes	Las imágenes adquiridas de la base de datos deben ser asignadas a categorías relevantes basadas en el estado del paciente, específicamente si el paciente padece de anemia o no.	Desarrollo de una tabla comparativa para la clasificación de pacientes con anemia
		Determinación del grupo al que pertenece cada imagen en base a la información adicional del paciente
		Almacenamiento de la información clasificada en una hoja de cálculo Excel con detalles adicionales correspondientes a cada imagen.
Etiquetado de imágenes	Las imágenes son clasificadas en los grupos " Resultado Positivo" y "Resultado Negativo " según los resultados de la evaluación inicial.	Asignación de las etiquetas "Resultado Positivo" y "Resultado Negativo" a las imágenes basándose en los datos obtenidos de la evaluación anterior.
		Limpieza de los datos para corregir errores tipográficos y gestionar datos faltantes.
		Substitución de estas etiquetas textuales por representaciones numéricas, es decir, 0 y 1.
Evaluación de las variables predictoras	Se utilizan los datos adicionales para confirmar la clasificación de las imágenes y garantizar su correcta catalogación.	Análisis de correlación entre las variables predictoras y la variable a predecir.
		Evaluación e interpretación de los resultados del Análisis de correlación.
Análisis de la Base de Datos	Se desarrolla un modelo que permite evaluar la eficacia de la base de datos.	Partición de datos.
		Entrenamiento del modelo
		Análisis de los resultados obtenidos tras el entrenamiento.

Nota. Elaboración propia

- **Actividad 1: Evaluación de la clasificación de imágenes**

La evaluación de la clasificación de imágenes es un proceso crítico en el pre procesamiento de los datos de imágenes médicas para el diagnóstico basado en aprendizaje automático. Esta actividad asegura la precisión en la asociación de cada imagen con su respectivo resultado de diagnóstico - "Resultado Positivo" o "Resultado Negativo". Este proceso comprende una serie de tareas esenciales para garantizar la integridad de los datos y, finalmente, la efectividad del modelo de aprendizaje automático. Las actividades principales de esta fase son:

a) Con base en la información disponible en el recurso en línea "Hemoglobin Levels Chart" (etoolsage.com, 2023), se extrae una tabla que vincula cada imagen con su respectivo resultado de diagnóstico. Esta etapa inicial proporciona una visión general de los datos y facilita la asignación precisa de las etiquetas a las imágenes (Bishop, 2006).

Tabla 18 *Tabla de Niveles Normales de Hemoglobina*

Age	Normal hemoglobin Level (g/dl)
Newborns	13.5 - 24
<1 month	10 - 20
1-2 months	10-18
0.5 to 2 years	10.5-13.5
2 to 6 years	11.5-13.5
6-12 years	11.5-15.5
Female: 12-18 years	12.0-16.0
Male:12-18 years	13.0-16.0
Female: > 18 years	12.1-15.1
Male: >18 years	13.6-17.7
Men after middle age	12.4-14.9
Woman after middle age	11.7-13.8

Nota: Extraído de Etoolsage (2012). https://www.etoolsage.com/Chart/Hemoglobin_Levels_Chart.asp

b) Verificación de la clasificación de imágenes: Con base en los registros médicos y otra información adicional del paciente, se verifica el resultado de diagnóstico correspondiente a cada imagen. Esta tarea es crucial para garantizar la precisión en la clasificación de las imágenes, lo cual es fundamental para el rendimiento del aprendizaje supervisado (Hastie, Tibshirani & Friedman, 2009).

c) Almacenamiento de la clasificación: La información de clasificación de cada imagen se registra en una hoja de cálculo de Excel junto con detalles adicionales de la imagen. Este paso final asegura que los datos clasificados están accesibles y organizados para su posterior análisis (Brownlee, 2019).

Entregable: Base de datos con la clasificion Anemia o No Anemia

- **Actividad 2: Etiquetado de imágenes**

El etiquetado de imágenes es un procedimiento esencial en el manejo de datos de imágenes médicas, especialmente en el contexto del aprendizaje supervisado, donde cada observación o entrada (en este caso, imagen) requiere una etiqueta o clase asociada. En este proyecto, las imágenes son etiquetadas como "Resultado Positivo" o "Resultado Negativo" en función del diagnóstico. Este proceso asegura la identificación precisa de las categorías de diagnóstico en las imágenes, lo cual es crítico para el entrenamiento efectivo del modelo de aprendizaje automático. Las tareas principales de esta fase incluyen:

a) Asignación de etiquetas: Basándose en la tabla comparativa desarrollada previamente, cada imagen es asignada a la etiqueta correspondiente - "Resultado Positivo" o "Resultado Negativo". Esta actividad asegura la correspondencia correcta entre las imágenes y su clasificación de diagnóstico.

b) Depuración de datos: Durante este proceso, los datos se depuran para eliminar posibles inconsistencias y errores. Esta actividad puede implicar la eliminación de datos faltantes, la corrección de errores tipográficos, y la gestión de datos duplicados. La depuración de los datos es fundamental para mantener la integridad y la calidad de los datos, lo cual es crucial para el rendimiento del modelo de aprendizaje automático (Hastie, Tibshirani, & Friedman, 2009).

c) Codificación de etiquetas: En esta etapa, las etiquetas asignadas a las imágenes se codifican en un formato adecuado para el aprendizaje automático. En el caso de este proyecto, las etiquetas "Resultado Positivo" y "Resultado Negativo" se tradu-

cen a 1 y 0, respectivamente. La codificación de las etiquetas facilita el procesamiento de los datos por parte de los algoritmos de aprendizaje automático y permite la evaluación cuantitativa del rendimiento del modelo.

Pseudocódigo 4: *Manipulación de datos*

```
Inicio
        Reemplazar ValoresEnLaData 'ValorErroneo' por 'ValorCorrecto'
        Crear NuevaColumna en LaData
        Para cada Valor en NuevaColumna hacer
                Si el tipo de Valor es Cadena entonces
                        Convertir Valor a NumeroFlotante
                        Agregar NumeroFlotante a NuevaColumna
                Sino
                        Agregar Valor a NuevaColumna
                Fin Si
        Fin Para
Fin
```

Nota. Elaboración propia

Entregable: Base de datos con los valores de predicción codificados.

- **Actividad 3: Evaluación de las variables predictoras:**

La fase de evaluación de las variables predictoras se compone de dos tareas principales: el análisis de correlación y el análisis de resultados.

a) Análisis de Correlación: El análisis de correlación es un procedimiento estadístico que se utiliza para determinar si existe una relación entre dos o más variables de interés. A través de este análisis, es posible cuantificar el grado de asociación entre las variables, lo cual se indica mediante un coeficiente de correlación (Dormann et al., 2013).

Este coeficiente de correlación varía de -1 a +1. Un valor de +1 denota una correlación positiva perfecta, lo que implica que cuando una variable aumenta, la otra también lo hace en proporción constante. Un valor de -1, por

otro lado, representa una correlación negativa perfecta, lo que significa que cuando una variable aumenta, la otra disminuye en proporción constante. Un valor de 0 denota que no existe ninguna relación lineal entre las variables.

El análisis de correlación es esencial en diversas aplicaciones de investigación y análisis de datos. Permite identificar las variables que tienen una asociación significativa, proporcionando una valiosa información que puede orientar el desarrollo de modelos predictivos y la toma de decisiones basada en datos.

b) Evaluación e Interpretación de los Resultados del Análisis de Correlación: La etapa de evaluación e interpretación de los resultados del análisis de correlación es crítica para discernir la relevancia y aplicabilidad de los hallazgos obtenidos. Este proceso se compone de varias tareas fundamentales:

    1. Interpretación de Coeficientes de Correlación: Los coeficientes de correlación obtenidos en el análisis deben ser interpretados para entender la naturaleza de las relaciones entre las variables. Coeficientes cercanos a +1 indican una fuerte correlación positiva, mientras que los cercanos a -1 representan una fuerte correlación negativa. Un coeficiente cerca de cero sugiere que no existe una relación lineal significativa entre las variables (Dormann et al., 2013).
    2. Evaluación de Significancia Estadística: Se debe determinar si las correlaciones observadas son estadísticamente significativas. Esto generalmente implica la realización de una prueba de hipótesis para determinar

si el coeficiente de correlación difiere significativamente de cero. Un resultado estadísticamente significativo proporciona evidencia de que la relación observada no se debe al azar.

3. Determinación de la Relevancia Práctica: Más allá de la significancia estadística, también es importante evaluar la relevancia práctica de las correlaciones observadas. Por ejemplo, una correlación puede ser estadísticamente significativa, pero tan pequeña que su impacto en el mundo real es mínimo. La relevancia práctica se considera en el contexto del área de estudio y las preguntas de investigación específicas.

En conjunto, estas tareas proporcionan una comprensión completa de los resultados del análisis de correlación, permitiendo su aplicación efectiva en el estudio y en la toma de decisiones.

Entregable: Informe sobre las variables predictoras.

- **Actividad 4: Análisis de la Base de Datos**

La realización de un análisis exhaustivo de la base de datos es de suma importancia para asegurar la calidad y relevancia de los datos utilizados en la investigación. Este análisis permite entender la estructura de los datos, las relaciones entre las variables y su idoneidad para responder las preguntas de investigación planteadas.

a) Partición de Datos: Para garantizar una evaluación imparcial y efectiva del rendimiento del modelo, los datos se dividen en conjuntos de entrenamiento y prueba (o validación). La partición comúnmente utilizada es la de 80/20, donde

el 80% de los datos se utiliza para el entrenamiento del modelo y el 20% se reserva para la prueba (Hastie, Tibshirani, & Friedman, 2009). La segmentación de los datos permite ajustar el modelo con una porción de los datos y evaluar su rendimiento con datos no vistos anteriormente.

Pseudocódigo 5: *Partición de datos*

Inicio

i. Dividir LosDatos en ConjuntosDeEntrenamiento y ConjuntosDe-Prueba

ii. Imprimir el número de muestras en ConjuntosDeEntrenamiento y ConjuntosDePrueba

Fin

Nota. Elaboración propia

b) Entrenamiento del Modelo: El entrenamiento del modelo es una etapa crucial en cualquier análisis de datos. En esta fase, se implementa un algoritmo de aprendizaje automático que utiliza los datos de entrenamiento para aprender a predecir la variable de interés. En el contexto de nuestro estudio sobre la anemia, el entrenamiento del modelo implicará la selección de un algoritmo de aprendizaje automático adecuado, el ajuste de sus parámetros utilizando los datos de entrenamiento y la validación del modelo para garantizar que está aprendiendo correctamente.

La elección del algoritmo de aprendizaje automático depende de la naturaleza de los datos y del problema que se está tratando de resolver. Los algoritmos de clasificación como la regresión logística, el árbol de decisiones, el bosque aleatorio, el vecino más cercano, la máquina de vectores de soporte, entre otros, son opciones comunes para los problemas de clasificación binaria como el nuestro.

El entrenamiento del modelo también implica la selección de los hiperparámetros del modelo, que son los parámetros que el modelo no puede aprender de los datos y que deben ser configurados manualmente

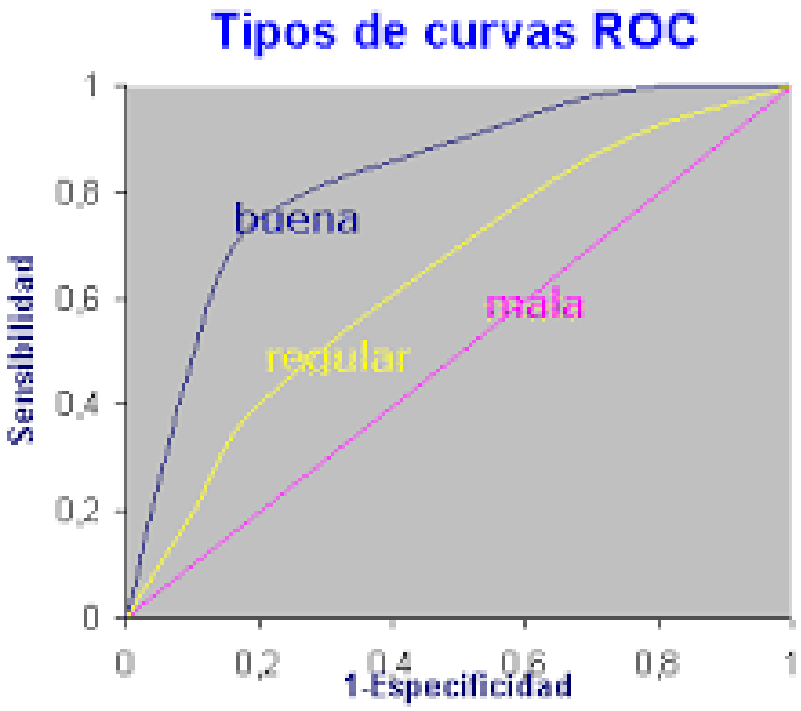
Pseudocódigo 6: *Búsqueda en cuadrícula para ajustar modelo SVM*

```
Inicio
    Definir parámetros para la búsqueda en cuadrícula
    Crear una instancia de BusquedaCuadricula con el estimador y los parámetros definidos
    Ajustar el modelo utilizando los conjuntos de entrenamiento
    Imprimir el puntaje de entrenamiento y prueba
    Imprimir los mejores parámetros encontrados por BusquedaCuadricula
Fin
```

Nota. Elaboración propia

c) Análisis de Resultados: Después de entrenar el modelo, se evalúa su rendimiento con el conjunto de prueba. Se utilizan métricas de evaluación relevantes como la precisión, el área bajo la curva ROC, la sensibilidad, la especificidad, entre otras, para cuantificar la capacidad del modelo para predecir correctamente los casos de anemia. También es esencial examinar la matriz de confusión para entender los tipos de errores cometidos por el modelo.
Además, se examinan los coeficientes del modelo (si es un modelo de regresión) o la importancia de las características (si es un modelo de árbol de decisión o bosque aleatorio) para entender qué variables son las más importantes para predecir la anemia. Esta información puede ser útil para mejorar el modelo y para entender mejor los factores que contribuyen a la anemia.

Figura 35 *Curva ROC*



Nota. Extraído de http://www.hrc.es/bioest/roc_1.html

Entregable: Porcentaje de eficiacia de la base de datos, viablidad para el proyecto

3.9.3. Pre procesamiento de la imagen

Tabla 19 *Pre procesamiento de la imagen*

Pre procesamiento		Imagen
Actividades	Descripción	Tareas
Organización y exploración de datos	Se prepara el entorno para la manipulación de los datos y se realiza un reconocimiento inicial de la información disponible.	Organización inicial: Creación de espacios para guardar la información. Exploración de la información: Revisión de la lista de imágenes disponibles y la categoría a la que pertenecen. Selección y categorización: Para cada imagen, se toma su ruta o dirección específica y se obtiene su categoría. Apertura de la imagen: Se abre la imagen en la dirección especificada.
Pre procesamiento de imágenes	Las imágenes se ajustan a un formato estándar y se simplifican al convertirlas a blanco y negro.	Redimensionamiento: Se cambia el tamaño de la imagen a un formato estándar. Conversión a blanco y negro: Se convierte la imagen a blanco y negro, eliminando la información de color.
Optimización de imágenes	Se mejoran las imágenes para resaltar las características importantes y facilitar el análisis posterior.	Aumento de los detalles: Se utiliza una herramienta para incrementar los detalles de la imagen y resaltar los bordes. Suavizado: Se aplica un filtro que suaviza la imagen, disminuyendo el ruido visual. Reorganización de la información de la imagen: Se reajusta la estructura de la información de la imagen para el análisis posterior.
Etiquetado y almacenamiento:	Se asignan categorías a las imágenes procesadas y se almacenan para su posterior uso en análisis o entrenamiento de modelos.	Etiquetado: Se toma la etiqueta o clasificación correspondiente a la imagen del conjunto de datos. Almacenamiento: Finalmente, se guarda la imagen procesada y su etiqueta en los espacios creados al principio.

Partición de datos	Se repartirá los datos en un grupo para entrenar el modelo y otro para probar su eficiencia	Separación de datos: División de los datos en conjuntos de entrenamiento, prueba.
Aumento de data	Se incrementa el número de datos de entrenamiento mediante técnicas de aumento de datos para mejorar la capacidad del modelo para generalizar.	Técnicas de aumento: Implementación de técnicas de aumento de datos como la rotación de imagen, el volteo, el zoom, etc. Aplicación de aumento: Aplicación del aumento de datos al conjunto de entrenamiento.
División de Datos	Se realiza una distribución estratégica de los datos para asegurar un equilibrio entre la cantidad de datos de entrenamiento, validación y prueba.	Divide los datos y etiquetas en conjuntos de entrenamiento y validación Codificación one-hot de las etiquetas de entrenamiento y validación Muestra la longitud del conjunto de entrenamiento y el conjunto de validación.

Nota. Elaboración propia

- **Actividad 1: Organización y exploración de datos**

La organización y exploración de los datos son las etapas preliminares en cualquier flujo de trabajo de ciencia de datos, incluido el procesamiento de imágenes. A continuación, se detallan las tareas que se llevan a cabo en estas etapas.

d) Organización Inicial: La organización inicial implica la creación de directorios o espacios para almacenar la información. Esta etapa requiere un diseño estructurado de directorios para facilitar el almacenamiento y recuperación eficientes de las imágenes. Los directorios pueden organizarse por categorías de imágenes, por ejemplo, pacientes con anemia y pacientes sin anemia.

e) Exploración de la Información: La exploración de la información implica revisar las imágenes disponibles y su categorización correspondiente. En esta

etapa, es importante comprender la naturaleza y características de las imágenes, como su tamaño, resolución, formato y cualquier atributo específico que pueda tener un impacto en el diagnóstico de la anemia. También es esencial confirmar que todas las imágenes están correctamente etiquetadas con su categoría correspondiente.

f) Selección y Categorización: Esta tarea se refiere a la obtención de la ruta específica de cada imagen y su respectiva categoría. Las rutas de las imágenes y sus categorías correspondientes se almacenan generalmente en una estructura de datos como un DataFrame de Pandas, lo que permite un manejo eficiente de los datos durante las etapas de pre procesamiento y análisis.

La organización y exploración de datos son cruciales para asegurar que los datos sean manejables y estén listos para el pre procesamiento. Estas etapas sientan las bases para todas las operaciones de análisis de datos que se realizarán posteriormente.

Pseudocódigo 7*: Extracción de datos de un diccionario y llenado de listas*

```
Iniciar listas vacías X e Y
    Para cada par Clave-Elementos en Diccionario:
        Para cada Elemento en Elementos:
                Extraer RutaImagen y Clase de Elemento
                Extraer NombreClase de Clase
```

Nota. Elaboración propia

Entregable: Creacion de directorios y espacios de categorización para las imágenes.

- **Actividad 2: Pre procesamiento de imágenes**

En el contexto de análisis de datos visuales, el pre procesamiento de imágenes es una fase crítica que implica la preparación y transformación de los datos de

imagen para facilitar su análisis posterior (Gonzalez & Woods, 2017). Las tareas involucradas en este proceso incluyen:

g) Apertura de la Imagen: La primera etapa del pre procesamiento de imágenes implica abrir cada imagen en su ubicación específica. Se emplean bibliotecas especializadas en el manejo de imágenes, como OpenCV o PIL en Python, para realizar esta tarea. Durante esta fase, se cargan los datos brutos de la imagen en la memoria, permitiendo su manipulación directa a través de las funciones de procesamiento de imágenes.

h) Redimensionamiento de la Imagen: El redimensionamiento implica ajustar todas las imágenes a un tamaño estándar. Dado que las imágenes en un conjunto de datos pueden tener tamaños variados, es esencial uniformizarlas para facilitar las comparaciones y análisis posteriores. Además, el redimensionamiento puede ayudar a reducir la carga computacional al minimizar la cantidad de píxeles que deben procesarse. Sin embargo, la elección del tamaño de la imagen debe considerar el equilibrio entre la eficiencia computacional y la preservación de los detalles críticos en la imagen.

i) Conversión a Blanco y Negro: Esta tarea involucra la conversión de imágenes a escala de grises. Al hacer esto, se eliminan los canales de color, reduciendo la imagen a una matriz de intensidades de píxeles. Esta simplificación puede ser útil para reducir la complejidad computacional y para centrarse en las características estructurales de la imagen que son independientes del color.

Pseudocódigo 8*: Tareas de procesamiento de imagen*

```
1. Leer imagen de RutaImagen con HerramientaDeLectura
2. Redimensionar imagen a tamaño específico
```

3. Convertir imagen a escala de grises

Nota. Elaboración propia

Estas tareas de pre procesamiento de imágenes preparan los datos para análisis posteriores, asegurando que las imágenes estén en un formato adecuado y sean coherentes en todo el conjunto de datos. Esto es crucial para obtener resultados precisos y confiables durante la etapa de análisis.

Entregable: Imagen redimencionada y convertida a blanco y negro

- **Actividad 3: Optimización de Imágenes**

La optimización de imágenes es una etapa importante en el pre procesamiento de imágenes que tiene como objetivo mejorar la calidad de las imágenes para realzar características clave y facilitar el análisis posterior (Szeliski, 2010). Esta etapa puede ser crítica para mejorar la precisión de los modelos analíticos. Las tareas involucradas en la optimización de imágenes incluyen:

a) Incremento de los Detalles: Se utilizan técnicas de mejora de la imagen para aumentar los detalles de la imagen y resaltar los bordes. Este proceso puede implicar la aplicación de filtros de mejora, como el filtro de realce de bordes, que pueden ayudar a mejorar los detalles de la imagen y a distinguir las características relevantes.

b) Suavizado de la Imagen: El suavizado de la imagen implica la aplicación de técnicas de filtrado para reducir el ruido en las imágenes. El ruido es una variación aleatoria en la intensidad de la imagen y puede interferir con la identificación de características relevantes en la imagen. Los filtros de suavizado, como el filtro Gaussiano, pueden ser útiles para reducir el ruido y mejorar la calidad de la imagen.

c) Reorganización de la Información de la Imagen: Esta tarea implica reestructurar la información de la imagen para facilitar su análisis. Esto puede implicar el cambio de la orientación de la imagen, la transformación de la escala de la imagen, o la reorganización de los píxeles en la imagen.

Pseudocódigo 9*: Procesamiento de imagen*

```
1. Crear un núcleo de filtro de agudización
2. Aplicar el filtro de agudización a la imagen
3. Aplicar un desenfoque gaussiano a la imagen
4. Cambiar la forma de la imagen a una sola capa de color
```

Nota. Elaboración propia

Entregable: Imagen optimizada luego del preprocesamiento.

- **Actividad 4: Etiquetado y Almacenamiento de Imágenes**

El etiquetado y almacenamiento de imágenes son actividades esenciales en el pre procesamiento de imágenes. Estos pasos garantizan que las imágenes se categoricen adecuadamente y se almacenen de manera organizada para su análisis posterior (Everingham et al., 2010). Las tareas específicas incluyen:

d) Etiquetado de Imágenes: Este paso implica asignar etiquetas o categorías a cada imagen basándose en su contenido. Las etiquetas se extraen de los metadatos de las imágenes o se asignan manualmente mediante técnicas de anotación. El etiquetado preciso es esencial para el entrenamiento supervisado de modelos de aprendizaje automático y para la validación de resultados en el análisis de imágenes.

e) Almacenamiento de Imágenes: Una vez etiquetadas, las imágenes y sus correspondientes etiquetas se almacenan en un formato y ubicación adecuados para su uso posterior. Este paso puede involucrar la escritura de los datos de

la imagen y las etiquetas en archivos locales, bases de datos o sistemas de almacenamiento en la nube. Es crucial mantener una organización consistente y un acceso fácil a los datos para su uso en etapas posteriores del análisis de imágenes.

Pseudocódigo 10: *Extracción de etiqueta y agregado de imagen procesada a listas X e Y*

```
1. Extraer la etiqueta correspondiente a la clase de la imagen
2. Agregar la imagen procesada a la lista X
3. Agregar la etiqueta a la lista Y
```

Nota. Elaboración propia

Entregable: Imágenes etiquetadas y almacendas

- **Actividad 5: Partición de datos**

La partición de los datos es una actividad esencial en el proceso de análisis y modelado de datos, particularmente en el contexto del aprendizaje supervisado (Train, test, validation split, 2021). Este proceso implica dividir el conjunto de datos en subconjuntos distintos, comúnmente conocidos como conjunto de entrenamiento, conjunto de validación y conjunto de prueba. Esta división se realiza con el fin de proporcionar datos distintos para diferentes etapas del proceso de modelado y evaluación.

El conjunto de entrenamiento se utiliza para enseñar al modelo a hacer predicciones, permitiendo que ajuste sus parámetros para minimizar su error de predicción. Esta es la fase donde el modelo "aprende" de los datos. Por otro lado, el conjunto de validación se utiliza para ajustar los hiperparámetros del modelo y seleccionar la mejor versión del mismo. Esto implica, por ejemplo, la selección de la complejidad del modelo o los parámetros de regularización que eviten el sobreajuste.

Finalmente, el conjunto de prueba es un conjunto de datos que se mantiene separado y se utiliza para evaluar la capacidad del modelo para generalizar a datos nuevos e invisibles. Este subconjunto proporciona una estimación imparcial del rendimiento del modelo final (Kohavi, 1995). El tamaño y la proporción de estos subconjuntos pueden variar según el problema y el conjunto de datos específicos, pero un reparto comúnmente utilizado es 70% para entrenamiento, 15% para validación y 15% para prueba. Sin embargo, es fundamental que estos subconjuntos sean representativos de los datos en general, para evitar sesgos en el entrenamiento y la evaluación del modelo.

Pseudocódigo 11: *Tarea de división de datos en conjuntos de entrenamiento y prueba*

1. Utilizar función 'DividirEntrenamientoPrueba' para dividir listas X e Y
2. Asignar 80% de los datos para entrenamiento y 20% para prueba
3. Establecer 'EstadoAleatorio' en 0 para garantizar reproducibilidad

Nota. Elaboración propia

Entregable: Base de datos segmentado en valores de entrenamiento y prueba

- **Actividad 6: Aumento de datos**

El aumento de datos en el aprendizaje automático es un método estratégico para mejorar la robustez y la generalización de los modelos, particularmente en contextos donde los datos pueden ser escasos o costosos de adquirir. En el procesamiento de imágenes, dos tareas críticas en esta actividad son la implementación de técnicas de aumento y la aplicación del aumento de datos al conjunto de entrenamiento.

f) Técnicas de aumento:

Las técnicas de aumento de datos generan variaciones controladas de los datos de entrenamiento existentes para expandir el conjunto de datos sin la necesidad de adquirir nuevos ejemplos. Estas técnicas pueden ser tan simples como el volteo o la rotación de una imagen, o pueden ser más complejas como el cambio de escala, el recorte, las transformaciones elásticas o la alteración de la intensidad del brillo o el contraste. Cada una de estas técnicas introduce variaciones en los datos que permiten al modelo aprender una representación más robusta y resistente a las variaciones irrelevantes, lo que resulta en una mejor capacidad de generalización (Shorten & Khoshgoftaar, 2019).

Pseudocódigo 12: *Aumento de datos hasta alcanzar número objetivo por etiqueta*

```
1. Establecer etiquetas a aumentar y número objetivo de muestras
2. Iniciar diccionario para contar muestras por etiqueta
3. Iniciar listas para almacenar datos aumentados
4. Mientras sea verdadero:
5.     Obtener lote de datos aumentados
6.     Para cada imagen y etiqueta en lote:
7.         Si etiqueta a aumentar y no se alcanza número objetivo:
8.             Añadir imagen y etiqueta a datos aumentados
9.             Incrementar conteo de etiqueta
10.    Si se alcanza número objetivo para todas etiquetas, salir del bucle
```

Nota. Elaboración propia

g) Aplicación del aumento: La aplicación de las técnicas de aumento de datos al conjunto de entrenamiento es un proceso esencial que debe llevarse a cabo cuidadosamente. El objetivo de este proceso es asegurar que la variabilidad introducida por el aumento de datos esté bien distribuida y sea representativa de las variaciones que el modelo puede encontrar en los datos no vistos. Al mismo tiempo, es crucial asegurarse de que el aumento de datos no introduzca ruido o distorsiones que puedan dificultar el aprendizaje del modelo.

Pseudocódigo 13: *Verificación y combinación de datos aumentados con datos originales*

1. Verificar recuentos de etiquetas en datos aumentados
2. Comprobar dimensiones de datos aumentados
3. Combinar datos originales y datos aumentados
4. Actualizar recuentos de etiquetas

Nota. Elaboración propia

Entregable: Nueva base de datos con las imágenes aumentadas.

- **Actividad 7: División de datos o data spliting**

En la fase de preparación de datos, se lleva a cabo una división estratégica de los datos con el propósito de garantizar un equilibrio adecuado entre la cantidad de datos destinados al entrenamiento, validación y prueba del modelo. Esta distribución estratégica es fundamental para evaluar y ajustar el rendimiento del modelo de manera confiable.

Para lograr esto, se emplea la función train_test_split que divide los datos y las etiquetas en conjuntos de entrenamiento y validación. El conjunto de entrenamiento se utiliza para ajustar los parámetros del modelo, mientras que el conjunto de validación se utiliza para ajustar los hiperparámetros y evaluar el rendimiento general del modelo. Esta división ayuda a evitar el sobreajuste (overfitting) y permite una evaluación más precisa del modelo en datos no vistos.

Además de la división de datos, se realiza la codificación one-hot de las etiquetas de entrenamiento y validación mediante el uso de la función to_categorical. Esta codificación transforma las etiquetas en un formato binario multidimensional, lo que facilita el entrenamiento de modelos de clasificación.

Finalmente, se proporciona información relevante sobre la longitud de los conjuntos de entrenamiento y validación. Esta información es útil para comprender

la distribución de los datos y verificar que la división haya sido realizada correctamente.

Pseudocódigo 14: *Preparación de datos y división en conjuntos de entrenamiento y validación*

1. Establecer X como 'characters' y Y como 'labels'
2. Determinar número de clases únicas en Y
3. Dividir datos (X, Y) en conjuntos de entrenamiento y validación (80% y 20%)
4. Importar módulo 'np_utils' de keras.utils
5. Convertir etiquetas de conjuntos en representaciones categóricas
6. Imprimir longitud de conjuntos de entrenamiento, validación y prueba

Nota. Elaboración propia

Entregable: Base de datos segmentado en valores de entrenamiento, prueba y validacion

3.9.4. Extracción de características

Tabla 20 *Extracción de características*

Actividades	Descripción	Tareas
Extracción de características con LBPH	Extracción de las características de cada registro utilizando el método Local Binary Patterns Histograms (LBPH).	Preprocesar la imagen. Aplicar LBPH Almacenar y validar los resultados.
Extracción de características con HOG	Extracción de las características de cada registro utilizando el método Histogram of Oriented Gradients (HOG).	Preprocesar la imagen. Aplicar HOG Almacenar y validar los resultados.
Extracción de características con SIFT	Extracción de las características de cada registro utilizando el método Scale-Invariant Feature Transform (SIFT).	Preprocesar la imagen. Aplicar SIFT Almacenar y validar los resultados.
Extracción de características con SIFT Padding	Como una variacion al modelo se utiliza al extractor de caracteristicas SIFT se le agrega un proceso de Padding.	Aplicar Sift padding Almacenar y validar los resultados.
Extracción de características con ORB	Extracción de las características de cada registro utilizando el método Oriented FAST and Rotated BRIEF (ORB).	Preprocesar la imagen. Aplicar ORB Almacenar y validar los resultados.

Nota. Elaboración propia

- **Actividad 1:** Extracción de características con LBPH

  En la extracción de características utilizando el método Local Binary Patterns Histograms (LBPH), se aplica un conjunto de procedimientos detallados y precisos para asegurar una caracterización efectiva de la imagen.

  Inicialmente, se realiza una conversión de la imagen original a su equivalente en escala de grises. Este paso simplifica la complejidad de la imagen y proporciona una representación adecuada para el análisis textural que sigue.

  Después de esta conversión, se emplea el algoritmo LBPH, que analiza cada pixel de la imagen en relación con sus vecinos circundantes. Este proceso de evaluación genera un patrón binario local para cada pixel central, basándose en si la intensidad de cada pixel vecino es mayor o menor que la del pixel central. Este procedimiento resulta en una matriz de patrones binarios locales para la imagen completa.

  Para comprimir la dimensionalidad del espacio de características y eliminar la variabilidad no uniforme, se emplea una estrategia de patrones binarios locales uniformes, la cual cuenta solo los patrones que contienen un número máximo de transiciones de 0 a 1 o viceversa (Pietikäinen, Hadid, Zhao, & Ahonen, 2011).

  Posteriormente, se genera un histograma basado en estos patrones binarios locales. Cada bin del histograma representa un patrón binario local específico, y la altura de cada bin corresponde a la frecuencia de dicho patrón en la imagen.

Para garantizar la comparabilidad de los histogramas de diferentes imágenes, se normaliza el histograma de cada imagen. Esta normalización se realiza dividiendo cada bin del histograma por la suma total de los bins, lo que da como resultado un histograma que suma 1.

Los histogramas normalizados se almacenan entonces para su posterior utilización. Estos histogramas, que representan las características texturales de las imágenes, se pueden utilizar en tareas de clasificación y reconocimiento de imágenes.

Pseudocódigo 15: *Extracción de características utilizando Local Binary Patterns (LBP)*

1. Clase LocalBinaryPatterns
   1.1. Inicializar con numPoints y radius
   1.2. Método preprocess para convertir imagen a escala de grises
   1.3. Método describe para:
      1.3.1. Preprocesar imagen a escala de grises
      1.3.2. Aplicar Local Binary Pattern a imagen preprocesada
      1.3.3. Crear histograma de patrones
      1.3.4. Normalizar histograma
      1.3.5. Retornar histograma y Local Binary Pattern

Nota. Elaboración propia

Entregable: Caracteristicas extraidas con el metodo LBPH

- **Actividad 2**: Extracción de características con HOG

El proceso de extracción de características con el método Histogram of Oriented Gradients (HOG) se desglosa en una serie de pasos meticulosos que permiten captar efectivamente la forma y la apariencia de los objetos en una imagen.

El procedimiento comienza con la transformación de la imagen original a una versión en escala de grises. Esta conversión conduce a una representación simplificada de la imagen, la cual es apropiada para el análisis detallado de los gradientes que HOG lleva a cabo.

El algoritmo HOG, en su núcleo, evalúa la dirección y la magnitud del gradiente para cada pixel en la imagen. Este cálculo del gradiente identifica cambios rápidos en la intensidad del pixel y provee una información crítica acerca de los límites y texturas de los objetos contenidos en la imagen.

Una vez determinados los gradientes, se construyen histogramas de los gradientes orientados. Esto se lleva a cabo dentro de regiones específicas de la imagen denominadas celdas. Estos histogramas sirven para contabilizar la frecuencia de los gradientes en distintas orientaciones dentro de cada celda. Cada bin en el histograma representa una orientación específica del gradiente y su valor indica la suma ponderada de las magnitudes de los gradientes que caen dentro de dicha orientación.

La etapa siguiente del procedimiento HOG es la normalización de bloques. Este paso mejora la robustez del descriptor HOG al ajustar los histogramas de cada celda con la información de las celdas adyacentes, incrementando así la invariancia a cambios en la iluminación y las sombras (Dalal & Triggs, 2005).

Una vez realizada la normalización, se concatenan los histogramas de todas las celdas para formar un vector de características de alto nivel. Este vector ofrece una descripción detallada de la apariencia de los objetos en la imagen, en términos de la distribución y orientación de sus gradientes.

Los vectores de características finales, luego de su construcción y verificación, se almacenan para su posterior uso en las tareas de clasificación e identificación. Estos vectores de características constituyen una descripción útil y potente de las imágenes, y su eficacia se valida a través de su uso en el reconocimiento y clasificación de objetos.

Pseudocódigo 16: Extracción de características utilizando Histogram of Oriented Gradients (HOG)

```
1. Clase HistogramOrientedGradient
  1.1. Inicializar con parámetros para HOG
  1.2. Método preprocess para convertir imagen a escala de grises
  1.3. Método describe para:
      1.3.1. Preprocesar imagen a escala de grises
      1.3.2. Calcular descriptor HOG de imagen preprocesada
      1.3.3. Retornar descriptor HOG
2. Inicializar listas vacías para características (X) y etiquetas (y)
3. Crear instancia de descriptor HOG
4. Para cada clave y elementos en diccionario img_files:
  4.1. Para cada elemento en elementos:
      4.1.1. Leer imagen de img_path
      4.1.2. Redimensionar imagen a (256, 256)
      4.1.3. Calcular descriptor HOG de imagen redimensionada
      4.1.4. Obtener etiqueta correspondiente
      4.1.5. Agregar descriptor HOG a lista X
      4.1.6. Agregar etiqueta a lista y
```

Nota. Elaboración propia

Entregable: Caracteristicas extraidas con el metodo HOG

- **Actividad 3:** Extracción de características con SIFT

El procedimiento para la extracción de características utilizando el método Scale-Invariant Feature Transform (SIFT) se desglosa en una serie de pasos meticulosos que aseguran la eficacia de la identificación y descripción de los puntos clave en una imagen.

Inicialmente, la imagen se transforma a una versión en escala de grises, reduciendo así su complejidad y preparándola para el análisis de texturas y estructuras que realiza SIFT.

Posteriormente, se lleva a cabo la construcción de la pirámide de escala gaussiana. Este proceso genera diferentes niveles de "borrosidad" de la imagen original, cada uno de los cuales captura detalles de la estructura de la imagen en una escala específica.

El espacio de escala de la diferencia gaussiana se obtiene mediante la restación de imágenes gaussianas adyacentes, capturando así los detalles clave en una amplia gama de escalas. A partir de este espacio de escala de la diferencia gaussiana, se detectan los puntos clave mediante la identificación de máximos y mínimos locales en el espacio de escala y la orientación (Lowe, 2004).

Una vez identificados los puntos clave, se les asigna una orientación principal basada en los gradientes locales. Esto proporciona a los puntos clave una característica de invarianza a la rotación, lo que significa que las mismas características pueden ser identificadas en la imagen incluso si esta ha sido girada.

Finalmente, se calculan los descriptores SIFT para cada punto clave. Estos descriptores se generan tomando un conjunto de gradientes orientados en torno al punto clave y los mismos se acumulan en un histograma. El resultado es un vector de 128 dimensiones que es altamente distintivo para cada punto clave y es invariante a la escala, orientación y cambios de iluminación.

La salida del proceso SIFT es un conjunto de puntos clave junto con sus descriptores asociados que se almacenan para su posterior uso en tareas de correspondencia y reconocimiento de imágenes.

Pseudocódigo 17: *Extracción de características utilizando SIFT (Scale-Invariant Feature Transform)*

```
1. Clase SIFT
  1.1. Inicializar con descriptor SIFT de OpenCV
  1.2. Método preprocess para convertir imagen a escala de grises
  1.3. Método describe para:
      1.3.1. Preprocesar imagen a escala de grises
      1.3.2. Detectar y computar características y descriptores SIFT en imagen pre-
procesada
      1.3.3. Retornar keypoints y descriptores SIFT
2. Inicializar listas vacías para características (X) y etiquetas (y)
3. Crear instancia de descriptor SIFT
4. Para cada clave y elementos en diccionario img_files:
  4.1. Para cada elemento en elementos:
      4.1.1. Leer imagen de img_path
      4.1.2. Obtener descriptores SIFT de imagen con método describe
      4.1.3. Obtener etiqueta correspondiente
      4.1.4. Agregar descriptores SIFT a lista X
      4.1.5. Agregar etiqueta a lista y
```

Nota. Elaboración propia

Entregable: Caracteristicas extraidas con el metodo SIFT

- **Actividad 4:** Extracción de características con SIFT Padding

En la extracción de características utilizando el método Scale-Invariant Feature Transform con Padding (SIFT Padding), se introduce una etapa adicional al procedimiento original de SIFT que se utiliza para garantizar que los puntos clave ubicados cerca del borde de la imagen sean manejados adecuadamente.

El primer paso es convertir la imagen a escala de grises, luego, se aplica el procedimiento SIFT estándar, que incluye la construcción de la pirámide de escala

gaussiana, la detección de puntos clave y la asignación de orientaciones y descriptores a estos puntos clave.

La diferencia principal en SIFT Padding es la incorporación de un paso de relleno (padding) antes de la construcción de la pirámide de escala gaussiana. El relleno se aplica para extender la imagen original, añadiendo píxeles adicionales en los bordes. Estos píxeles extra se generan comúnmente por duplicación de los píxeles del borde o por espejo, aunque existen otros métodos más sofisticados. El objetivo de este paso es garantizar que los puntos clave situados cerca de los bordes de la imagen puedan tener vecindarios suficientemente grandes para calcular sus descriptores (Lowe, 2004).

Así, el proceso de padding permite una mejor descripción y comparación de los puntos clave en los bordes de la imagen, mejorando la precisión y robustez del método SIFT en estas áreas.

Pseudocódigo 18: *Preprocesamiento de etiquetas y características para ajuste de dimensiones*

1. Transformar etiquetas a representación binaria.
2. Determinar dimensión máxima de características.
3. Si dimensión de características es menor a la máxima, ampliar replicando valores.
4. Determinar longitud mínima de características.
5. Redimensionar características a longitud mínima.

Nota. Elaboración propia

Entregable: Caracteristicas extraidas con el metodo SIFT(Paddin)

- **Actividad 5:** Extracción de características con ORB

La extracción de características mediante el método Oriented FAST and Rotated BRIEF (ORB) comienza con la conversión de la imagen original a escala de

grises. Este paso inicial simplifica el análisis de la imagen al concentrarse únicamente en la intensidad lumínica de cada píxel, lo que resulta en un estudio más efectivo de los patrones texturales y estructurales.

La detección de puntos clave es el siguiente componente del proceso ORB. Este procedimiento recurre al algoritmo Features from Accelerated Segment Test (FAST). Como detector de esquinas, FAST examina los patrones circulares de píxeles alrededor del píxel objetivo, identificando los puntos clave a través de un proceso de comparación de intensidades de píxeles (Rosten & Drummond, 2006).

Para el cálculo de la orientación de los puntos clave detectados, ORB aplica una medida de intensidad centróide, proporcionando a estos una invariancia a la rotación. Este aspecto es crítico para permitir que las mismas características sean reconocidas en la imagen incluso cuando esta ha sido rotada.

En la fase final de la extracción de características ORB, se generan descriptores para los puntos clave detectados utilizando el método Binary Robust Independent Elementary Features (BRIEF) con la orientación asignada. BRIEF crea un vector de características binario de alta eficiencia, que permite una correspondencia rápida y eficiente en memoria (Calonder, Lepetit, Strecha, & Fua, 2010). Para este propósito, BRIEF realiza pruebas de intensidad comparativa de píxeles y compila los resultados en un vector de características.

Al concluir el proceso ORB, los descriptores generados se almacenan junto con sus puntos clave correspondientes para su uso en etapas posteriores de reconocimiento y correspondencia de imágenes.

Pseudocódigo 19*: Extracción de características utilizando ORB*

```
1. Clase ORB:
```

- Inicializar: Crear descriptor ORB
- Método preprocess: Convertir imagen a gris
- Método describe: Preprocesar, detectar y calcular keypoints y descriptores ORB

2. Listas vacías para características (X) y etiquetas (y)

3. Crear descriptor ORB

4. Para cada imagen en img_files:
- Leer imagen
- Obtener descriptores ORB con método describe
- Obtener etiqueta
- Agregar descriptores a X
- Agregar etiqueta a y

Nota. Elaboración propia

Entregable: Caracteristicas extraidas con el metodo ORB

3.9.5. Creación del modelo

Tabla 21 *Creación de modelo*

Actividades	Descripción	Tareas
Codificación de Etiquetas de Clase	En este paso, las etiquetas de las clases tanto para los conjuntos de entrenamiento como de prueba son transformadas a una representación binaria, lo que se conoce como codificación one-hot.	Realizar la codificación one-hot de las etiquetas de clases para los conjuntos de datos de entrenamiento y prueba.
Creacion del modelo con Tensorflow	Implementación de una red neuronal a través de la librería TensorFlow, la cual permitirá llevar a cabo el proceso de aprendizaje profundo.	Establecer la estructura de la red neuronal.
		Determinar los parámetros para el proceso de aprendizaje.
		Compilar el modelo para prepararlo para el entrenamiento
Creacion del modelo con SVM	Implementación de un modelo de clasificación utilizando el algoritmo de Máquina de Vectores de Soporte (SVM), el cual construye un hiperplano óptimo para la separación de clases.	Definir los hiperparámetros del modelo SVM.
		Entrenar el modelo SVM utilizando el conjunto de datos de entrenamiento.
		Evaluar la precisión del modelo SVM utilizando el conjunto de datos de prueba.
Creacion del modelo con Random Forest	Desarrollo de un modelo de clasificación mediante el algoritmo de Random Forest, que opera generando un conjunto de árboles de decisión durante el entrenamiento.	Configurar los hiperparámetros del modelo Random Forest.
		Entrenar el modelo Random Forest utilizando el conjunto de datos de entrenamiento.
		Evaluar la precisión del modelo Random Forest utilizando el conjunto de datos de prueba.
		Seleccionar el valor óptimo de 'k' para el modelo KNN.

Creacion del modelo con KNN	Implementación de un modelo de clasificación utilizando el algoritmo de K-Nearest Neighbors (KNN), el cual clasifica una observación basándose en las etiquetas de sus 'k' vecinos más cercanos.	Entrenar el modelo KNN utilizando el conjunto de datos de entrenamiento.
		Evaluar la precisión del modelo KNN utilizando el conjunto de datos de prueba.
Creacion del modelo con MLP	Configuración de un modelo de red neuronal, más específicamente un Perceptrón Multicapa (MLP), que consiste en al menos tres capas de nodos.	Definir la arquitectura del MLP.
		Entrenar el MLP utilizando el conjunto de datos de entrenamiento.
		Evaluar la precisión del MLP utilizando el conjunto de datos de prueba.

Nota. Elaboración propia

- **Actividad 1:** Codificación de Etiquetas de Clase

  El proceso esencial en el preprocesamiento de los datos para modelos de aprendizaje automático y profundo. En este procedimiento, se transforman las etiquetas de clase originales en un formato que puede ser interpretado y procesado por los modelos de forma eficiente. Un enfoque comúnmente utilizado para la codificación de etiquetas es la "codificación one-hot".

  En la codificación one-hot, cada etiqueta de clase se convierte en un vector binario de longitud igual al número total de clases. Este vector está compuesto principalmente por ceros, con un uno en la posición correspondiente a la clase a la que pertenece la observación. Por ejemplo, en un problema de clasificación binaria, la clase 0 puede ser codificada como [1, 0] y la clase 1 como [0, 1].

Este proceso se realiza tanto para los conjuntos de entrenamiento como de prueba, garantizando que las etiquetas de clase en todos los conjuntos de datos sean representadas de manera uniforme y adecuada para el entrenamiento del modelo (Chollet & Allaire, 2018)

Pseudocódigo 20: *Asignación de etiquetas numéricas a clases de imágenes*

1. Crear listas vacías para las etiquetas.
2. Para cada imagen:
   - Identificar clase de la imagen
   - Asignar etiqueta numérica
   - Agregar etiqueta a la lista

Nota. Elaboración propia

Entregable: Etiquetas codificas en 0 y 1

- **Actividad 2:** Creación del Modelo con Tensorflow

La creación de un modelo de aprendizaje profundo en TensorFlow implica varias etapas claves, las cuales se describen a continuación.

Primero, es esencial estructurar la arquitectura del modelo. En este contexto, un modelo Sequential de TensorFlow ha sido utilizado, que permite el apilamiento lineal de capas para construir una red neuronal. Este modelo es apropiado cuando hay exactamente una entrada y una salida para cada capa de la red. Se definen varias capas densas (también conocidas como completamente conectadas), cada una con una función de activación ReLU, seguida de una capa de salida.

En segundo lugar, se define una función de pérdida para cuantificar el error entre las predicciones del modelo y los valores verdaderos durante la fase de entrenamiento. En este caso, se utiliza la función de pérdida de entropía cruzada categórica, que es una elección común para problemas de clasificación multiclase.

En tercer lugar, se selecciona un optimizador, que es responsable de actualizar los pesos del modelo en respuesta a la salida de la función de pérdida. Se usa 'adam', un algoritmo de optimización popular debido a su eficiencia computacional y su buen rendimiento en una variedad de problemas.

Finalmente, el modelo se entrena llamando al método 'fit' en el objeto del modelo, pasándole los datos de entrenamiento (entradas y salidas) así como el número de épocas, que es el número de veces que el modelo pasa por todo el conjunto de entrenamiento.

A lo largo de todo este proceso, es posible evaluar el rendimiento del modelo utilizando métricas adicionales (como la precisión, en este caso) y técnicas de regularización o ajuste de hiperparámetros para mejorar la robustez y la capacidad de generalización del modelo.

Pseudocódigo 21:Construcción y entrenamiento del modelo TensorFlow para clasificación

1. Importar TensorFlow
2. Convertir etiquetas a formato categórico
3. Construir modelo:
   - Inicializar modelo secuencial
   - Agregar capa de entrada y capas intermedias
   - Agregar capa de salida
4. Compilar modelo con función de pérdida y optimizador
5. Entrenar modelo con los datos de entrenamiento
6. Evaluar modelo con los datos de prueba e imprimir resultados

Nota. Elaboración propia

Entregable: Modelo Tensorflow

- **Actividad 3:** Creación del Modelo con SVM

La creación de un modelo con Máquina de Vectores de Soporte (SVM, por sus siglas en inglés) en esta implementación específica se lleva a cabo siguiendo varios pasos esenciales.

Inicialmente, se define una cuadrícula de parámetros que se desea explorar. En el caso de las SVM, dos parámetros críticos son 'C', que controla el compromiso entre lograr un margen amplio y minimizar las violaciones del margen (es decir, clasificar correctamente todos los puntos de entrenamiento); y 'gamma', que define el alcance de la influencia de un solo ejemplo de entrenamiento, con valores bajos que significan 'lejos' y valores altos que significan 'cerca' (Cortes & Vapnik, 1995).

Posteriormente, se utiliza el método GridSearchCV para realizar una búsqueda exhaustiva sobre la cuadrícula de parámetros especificada. Este método lleva a cabo la validación cruzada, que implica dividir el conjunto de datos de entrenamiento en 'k' subconjuntos y luego entrenar el modelo en 'k-1' de estos, mientras que el restante se utiliza para la validación. Esta estrategia garantiza una evaluación robusta del rendimiento del modelo.

Una vez que se ha completado la búsqueda en la cuadrícula, el modelo con la mejor combinación de parámetros (es decir, aquel que logra el rendimiento más alto en la validación cruzada) se selecciona y se vuelve a entrenar con estos parámetros óptimos en todo el conjunto de datos de entrenamiento.

Finalmente, se evalúa el rendimiento del modelo seleccionado en el conjunto de datos de prueba, proporcionando una evaluación imparcial del rendimiento esperado del modelo en datos no vistos.

Pseudocódigo 22: *Optimización de hiperparámetros utilizando GridSearchCV*

1. Importar bibliotecas requeridas.

2. Definir diccionario de hiperparámetros.
3. Instanciar GridSearchCV con modelo y hiperparámetros.
4. Ajustar GridSearchCV con datos de entrenamiento.
5. Evaluar y mostrar rendimiento en entrenamiento y prueba.
6. Mostrar hiperparámetros óptimos.

Nota. Elaboración propia

Entregable: Modelo SVM

- **Actividad 4:** Creación del Modelo con Random Forest

La construcción de un modelo con el método Random Forest sigue un conjunto particular de pasos. Este método, que es un tipo de algoritmo de ensamble de aprendizaje automático, se basa en la creación de numerosos árboles de decisión durante la fase de entrenamiento y en la producción de una predicción de clasificación como la moda de las predicciones de los árboles individuales (Breiman, 2001).

Inicialmente, se definen los parámetros para la creación del bosque. Entre estos parámetros se incluyen el número de árboles a crear (n_estimators), la profundidad máxima de los árboles (max_depth), el número mínimo de muestras requeridas para dividir un nodo interno (min_samples_split), entre otros.

En el siguiente paso, se instancia un objeto de Random Forest del módulo sklearn.ensemble, pasándole los parámetros previamente definidos.

A continuación, se utiliza el método 'fit' en el objeto del modelo de Random Forest para entrenar el modelo en el conjunto de datos de entrenamiento. Este método ajusta los árboles de decisión en el modelo utilizando los datos de entrada y salida proporcionados.

Finalmente, se evalúa la precisión del modelo en los conjuntos de entrenamiento y prueba utilizando el método 'score'. Esta métrica proporciona una evaluación cuantitativa de la habilidad del modelo para predecir correctamente las etiquetas de clase en datos no vistos.

Pseudocódigo 23: *Entrenamiento y evaluación de Random Forest*

1. Definir hiperparámetros para Random Forest.
2. Instanciar Random Forest con hiperparámetros.
3. Ajustar modelo con datos de entrenamiento.
4. Evaluar rendimiento con datos de entrenamiento.
5. Evaluar rendimiento con datos de prueba
6. Mostrar hiperparámetros óptimos.

Nota. Elaboración propia

Entregable: Modelo Random Forest

- **Actividad 5:** Creación del Modelo con KNN

La creación de un modelo de clasificación utilizando el método de los k-Vecinos Más Cercanos (k-Nearest Neighbors, KNN) se lleva a cabo siguiendo un proceso definido. Este método, que es un tipo de algoritmo de aprendizaje supervisado, basa sus predicciones en un parámetro de proximidad, es decir, un ejemplo no etiquetado se clasifica según las etiquetas de los 'k' ejemplos más cercanos en el conjunto de entrenamiento (Cover & Hart, 1967).

El primer paso consiste en determinar el valor óptimo para 'k', el número de vecinos más cercanos a considerar. Esto se puede hacer mediante una serie de experimentos o por medio de validación cruzada.

Una vez establecido el valor de 'k', se instancia un objeto de KNN del módulo sklearn.neighbors, proporcionándole el parámetro 'k' previamente definido.

En la siguiente fase, se entrena el modelo utilizando el método 'fit', que recibe los datos de entrenamiento y las respectivas etiquetas como entrada.

Finalmente, se realiza la evaluación del modelo. Para esto, se puede utilizar el método 'score' para obtener la precisión del modelo en el conjunto de datos de prueba, lo que proporciona una medida cuantitativa de la eficacia del modelo para predecir correctamente las etiquetas de los datos no vistos.

Pseudocódigo 24: *Entrenamiento, evaluación y optimización de KNN*

1. Entrenar modelo KNN con datos de entrenamiento.
2. Evaluar con datos de prueba.
3. Optimizar hiperparámetros mediante búsqueda en cuadrícula.
4. Evaluar rendimiento con datos de prueba.
5. Obtener mejor puntuación y hiperparámetros óptimos.

Nota. Elaboración propia

Entregable: Modelo KNN

- **Actividad 6:** Creación del Modelo con MLP

La elaboración de un modelo de clasificación utilizando una Red Neuronal de Perceptrones Multicapas (Multilayer Perceptron, MLP) implica un conjunto de pasos sistemáticos. MLP es una clase de red neuronal artificial feedforward (propagación hacia adelante) que consiste en al menos tres capas de nodos: una capa de entrada, una o más capas ocultas y una capa de salida. Cada nodo en una capa está conectado con todos los nodos en la capa siguiente, lo que permite el aprendizaje de representaciones de datos no lineales y complejas (Rumelhart, Hinton & Williams, 1986).

El primer paso en la creación del modelo MLP implica la configuración de la arquitectura de la red, lo que incluye determinar el número de capas ocultas, la

cantidad de nodos en cada una de estas capas y la función de activación que se utilizará.

Una vez establecida la arquitectura, se procede a instanciar un objeto MLP desde el módulo sklearn.neural_network. Este objeto es configurado con los parámetros definidos para la arquitectura de la red.

A continuación, el modelo se entrena utilizando el método 'fit', que toma los datos de entrenamiento y sus correspondientes etiquetas como entrada. Por último, se evalúa la capacidad predictiva del modelo. Esto puede realizarse utilizando el método 'score' en el conjunto de datos de prueba, lo que proporciona una medida de la precisión del modelo en la predicción correcta de las etiquetas en los datos no vistos anteriormente.

Pseudocódigo 25:*Construcción, entrenamiento y evaluación de un modelo de redes neuronales*

1. Importar bibliotecas necesarias.
2. Definir modelo con capas densas y función de activación.
3. Compilar modelo con optimizador, función de pérdida y métricas.
4. Entrenar modelo con datos de entrenamiento.
5. Evaluar rendimiento en datos de prueba.
6. Hacer predicciones en nuevos datos.

Nota. Elaboración propia

Entregable: Modelo MLP

3.9.6. Desarrollo y evaluación de modelos CNN

Tabla 22 *Desarrollo y evaluación de modelos CNN*

Actividades	Descripción	Tareas
Creación del Modelo CNN Personalizado	Diseño de una arquitectura CNN personalizada con capas convolucionales, aplanamiento y densas.	Inicialización de la arquitectura CNN.
		Incorporación de capas convolucionales.
		Aplicación de técnicas de regularización

		Aplanamiento de la salida convolucional
		Adición de capas densas.
CNN con EfficientNet	Uso del modelo preentrenado EfficientNetB0, personalizándolo con capas adicionales.	Carga del modelo base EfficientNetB0
		Personalización del modelo base
CNN con ResNet	Aplicación del modelo preentrenado ResNet50, adaptado con capas adicionales.	Carga del modelo base ResNet50
		Personalización del modelo base
CNN con Inception-ResNet	Implementación del modelo preentrenado InceptionResNetV2, personalizado con capas adicionales.	Carga del modelo base InceptionResNetV2
		Personalización del modelo base
Entrenamiento de los Modelos CNN	Entrenamiento de cada modelo utilizando los datos de entrada y etiquetas.	Entrenamiento del modelo en varias épocas
		Guardado de los pesos del modelo después de cada época
Evaluación del Modelo Entrenado	Evaluación del rendimiento del modelo en un conjunto de pruebas.	Carga de los pesos del modelo guardados
		Realización de predicciones en el conjunto de pruebas
		Cálculo de las métricas de rendimiento, como precisión, recuperación y puntaje F1
		Documentación de las métricas de rendimiento para cada época.
Guardar del Modelo Entrenado	Conservación del estado del modelo después del entrenamiento.	Guardado del modelo final para uso futuro

Nota. Elaboración propia

- **Actividad 1:** Creación del Modelo CNN Personalizado

  La creación de un Modelo CNN (Redes Neuronales Convolucionales) personalizado se basa en la implementación de una serie de capas específicas que corresponden a la estructura fundamental de una CNN. La arquitectura de la red es

inicializada utilizando la clase Sequential de la biblioteca de aprendizaje profundo Keras, lo que permite apilar capas linealmente en el modelo (Chollet, 2015).

El primer paso es la adición de capas convolucionales al modelo mediante la clase Conv2D. Estas capas tienen la función de aplicar una serie de filtros a la entrada para extraer características de la misma. En este caso, se implementan tres capas convolucionales con diferentes números de filtros y tamaños de kernel, cada una seguida por una capa de normalización por lotes (BatchNormalization) que ayuda a acelerar el entrenamiento y aporta una cierta regularización al modelo (Ioffe & Szegedy, 2015).

Posteriormente, se aplica la capa de MaxPooling (MaxPooling2D) para reducir la dimensionalidad de las matrices resultantes sin perder las características más importantes. Las capas de MaxPooling toman el valor máximo de cada ventana deslizante, lo que permite conservar las características más destacadas y reducir el riesgo de sobreajuste (Scherer, Müller, & Behnke, 2010).

Se implementan las capas Dropout como una forma de regularización, que tiene el objetivo de prevenir el sobreajuste durante el entrenamiento. Esta técnica consiste en "desactivar" aleatoriamente una proporción de las neuronas durante el entrenamiento, lo que hace que el modelo sea menos dependiente de las neuronas individuales y mejora su capacidad de generalización (Srivastava, Hinton, Krizhevsky, Sutskever, & Salakhutdinov, 2014).

Por último, los datos de salida de las capas convolucionales son aplanados utilizando la capa Flatten y procesados a través de las capas Dense. La capa Flatten convierte la matriz multidimensional en un vector que puede ser procesado por las

capas densas, que son las responsables del procesamiento final de los datos y la obtención de la salida de la red.

Pseudocódigo 26: *Construcción de modelo de redes neuronales convolucionales*

1. Importar bibliotecas necesarias.
2. Definir forma de entrada, tasa de abandono y dimensiones de filtros convolucionales.
3. Construir modelo secuencial:
   - Capa convolucional con filtros, tamaño de kernel y función de activación.
   - Normalización por lotes, MaxPooling y Dropout.
   - Repetir capas anteriores según necesidad.
   - Aplanar características para capa densa.
   - Capa densa con unidades y función de activación.
   - Normalización por lotes y Dropout.
   - Capa de salida con unidades y función de activación softmax.
4. Compilar modelo con optimizador, función de pérdida y métricas.
5. Crear directorio para guardar modelos entrenados en futuras etapas.

Nota. Elaboración propia

Entregable: Modelo CNN Personalizado

- **Actividad 2:** CNN con EfficientNet

En este caso, se utiliza EfficientNet, una arquitectura de red preentrenada. EfficientNet es una familia de modelos CNN que se ha demostrado que son más eficientes en términos de rendimiento y consumo de recursos en comparación con las arquitecturas tradicionales, gracias a su estrategia de escalado compuesto que ajusta la profundidad, la resolución y el ancho de la red de manera coordinada (Tan & Le, 2019).

La arquitectura de EfficientNet se carga inicialmente sin su última capa de clasificación, lo que permite que las características extraídas por la red se adapten a la tarea específica que se está abordando. Para este propósito, se utiliza la función EfficientNetB0 de la biblioteca TensorFlow.

Una vez cargada la arquitectura, se procede a añadir una serie de capas personalizadas para adaptar el modelo a la tarea específica. En este caso, se añade una capa GlobalAveragePooling2D que realiza una operación de pooling promediando los valores en cada canal de la entrada, reduciendo así su dimensionalidad. Esta capa es seguida por una capa densa Dense que actúa como un clasificador y es responsable de generar la salida final del modelo.

Para mejorar el rendimiento y la estabilidad del modelo, se añade una capa de normalización por lotes BatchNormalization y una capa de regularización Dropout. Esta combinación ayuda a prevenir el sobreajuste y acelera el entrenamiento del modelo, resultando en un modelo más robusto y eficiente (Chollet, 2015; Srivastava et al., 2014).

Pseudocódigo 27:*Construcción de modelo basado en EfficientNetB0 para clasificación*

1. Importar bibliotecas necesarias.
2. Cargar modelo EfficientNetB0 sin la última capa y adaptarlo a los datos de entrada.
3. Crear nuevo modelo secuencial.
4. Agregar modelo EfficientNetB0 al nuevo modelo.
5. Añadir capas adicionales:
   - GlobalAveragePooling2D para reducir la dimensionalidad.
   - Capa densa con unidades y función de activación ReLU.
   - Capa de BatchNormalization.
   - Capa de Dropout para prevenir sobreajuste.
   - Capa de salida densa con unidades y función de activación softmax.
6. Compilar modelo con optimizador, función de pérdida y métricas.
7. Preparar directorio para guardar modelos entrenados.

Nota. Elaboración propia

Entregable: Modelo CNN EfficientNet

- **Actividad 3:** CNN con ResNet

Para esta implementación se emplea la arquitectura de Red Neuronal Convolucional (CNN) conocida como ResNet50, un modelo destacado en el ámbito de la visión por computadora por su capacidad para entrenar redes muy profundas sin caer en el problema del desvanecimiento del gradiente.

El procedimiento inicia cargando el modelo ResNet50 preentrenado utilizando la biblioteca TensorFlow, excluyendo su última capa de clasificación. Esta técnica, conocida como transferencia de aprendizaje, permite beneficiarse del conocimiento que ResNet50 ya ha adquirido de enormes conjuntos de datos y adaptarlo a nuestra tarea específica.

Seguidamente, se añaden al modelo capas personalizadas que permiten la adaptación precisa a la tarea en cuestión. Primero se incorpora una capa de agrupamiento promedio global (GlobalAveragePooling2D), que se encarga de reducir la dimensionalidad espacial de los mapas de características. Posteriormente, se añade una capa densa (Dense) que actúa como un clasificador que aprende patrones más complejos en los datos.

Por último, se añaden capas de normalización por lotes (BatchNormalization) y regularización (Dropout) para mejorar la estabilidad y el rendimiento del modelo. La normalización por lotes proporciona un medio para reducir el cambio interno de covarianza en las redes neuronales, acelerando su entrenamiento. Por otro lado, la capa de Dropout se utiliza para prevenir el sobreajuste al ignorar aleatoriamente algunas unidades de la red durante el entrenamiento (He et al., 2016).

Pseudocódigo 28: *Construcción de modelo ResNet50 para clasificación*

1. Importar bibliotecas necesarias.
2. Cargar modelo ResNet50 sin última capa y adaptarlo a los datos de entrada.

3. Crear nuevo modelo secuencial.
4. Agregar modelo ResNet50 al nuevo modelo.
5. Añadir capas adicionales:
   - GlobalAveragePooling2D para reducir la dimensionalidad.
   - Capa densa con unidades y función de activación ReLU.
   - Capa de BatchNormalization.
   - Capa de Dropout para prevenir el sobreajuste.
   - Capa de salida densa con unidades y función de activación softmax.
6. Compilar modelo con optimizador, función de pérdida y métricas.
7. Preparar directorio para guardar modelos entrenados.

Nota. Elaboración propia

Entregable: Modelo CNN ResNet

- **Actividad 4:** CNN con Inception-ResNetV2

La implementación de una Red Neuronal Convolucional (CNN) mediante la arquitectura Inception-ResNetV2 constituye un enfoque avanzado y altamente eficaz para el procesamiento y análisis de imágenes. Inception-ResNetV2 es una arquitectura robusta que combina las características eficientes de la arquitectura Inception con las conexiones residuales de ResNet, diseñadas para aliviar el problema del desvanecimiento del gradiente en las redes profundas.

En el primer paso, se carga el modelo Inception-ResNetV2 preentrenado a través de la biblioteca TensorFlow, excluyendo la última capa de clasificación. Esta acción permite la adaptación del modelo a los requerimientos específicos de la tarea en cuestión.

Posteriormente, se procede a agregar capas personalizadas al modelo con el fin de ajustarlo de manera precisa al problema. Una capa GlobalAveragePooling2D es agregada para realizar una operación de pooling con el propósito de reducir la

dimensionalidad de la entrada. A continuación, se incorpora una capa densa Dense con la función de aprendizaje de características y patrones complejos de los datos.

Para promover la eficiencia y estabilidad del modelo durante el proceso de entrenamiento, se incorporan capas de normalización por lotes BatchNormalization y regularización Dropout. Estas capas cumplen roles cruciales en el entrenamiento y generalización del modelo, ayudando a acelerar el aprendizaje y prevenir el sobreajuste (Ioffe & Szegedy, 2015).

Construcción de modelo InceptionResNetV2 para clasificación de imágenes

1. Importar bibliotecas necesarias.
2. Cargar modelo InceptionResNetV2 sin última capa y adaptarlo a los datos de entrada.
3. Crear nuevo modelo secuencial.
4. Agregar modelo InceptionResNetV2 al nuevo modelo.
5. Añadir capas adicionales:
   - GlobalAvgPooling2D para reducir dimensionalidad.
   - Capa densa con unidades y activación ReLU.
   - Capa BatchNormalization.
   - Capa Dropout para prevenir sobreajuste.
   - Capa de salida densa con unidades y activación softmax.
6. Compilar modelo con optimizador, función de pérdida y métricas.
7. Preparar directorio para guardar modelos entrenados.

Nota. Elaboración propia

Entregable: Modelo CNN Inception-ResNetV2

- **Actividad 5**: Entrenamiento de los Modelos CNN

Una vez que las arquitecturas de las Redes Neuronales Convolucionales (CNN) están configuradas y personalizadas para el problema en cuestión, el siguiente paso es el entrenamiento de estos modelos. Este proceso consiste en alimentar a las redes con los datos de entrenamiento para que puedan aprender los patrones subyacentes en dichos datos.

Durante el entrenamiento, se hace uso de un conjunto de datos de entrenamiento (X_train, Y_train), que son presentados al modelo en lotes. Estos datos consisten en imágenes etiquetadas, donde X_train contiene las imágenes y Y_train las etiquetas correspondientes.

La estrategia de entrenamiento empleada se basa en la ejecución de múltiples épocas (iteraciones a través de todo el conjunto de datos de entrenamiento), especificadas por la variable epochs. Durante cada época, el modelo se ajusta para minimizar una función de pérdida, que es una medida de cuán bien el modelo está realizando la tarea de clasificación. Este ajuste se logra mediante la actualización de los pesos del modelo en función del gradiente de la función de pérdida con respecto a dichos pesos.

Además del conjunto de datos de entrenamiento, se usa un conjunto de datos de validación (X_val, Y_val). Este conjunto es empleado para evaluar el rendimiento del modelo después de cada época y ajustar los hiperparámetros si es necesario. De esta manera, se puede monitorizar y prevenir el sobreajuste, que ocurre cuando el modelo aprende demasiado bien las peculiaridades del conjunto de entrenamiento y pierde capacidad para generalizar a datos no vistos.

Finalmente, los pesos del modelo se guardan después de cada época para permitir la reutilización del modelo en futuras tareas y para posibilitar la interrupción y reanudación del proceso de entrenamiento. El rendimiento del modelo se mide en términos de precisión de clasificación y pérdida en el conjunto de prueba, y estas métricas son registradas para su posterior análisis (Goodfellow et al., 2016).

Pseudocódigo 29:*Entrenamiento y validación iterativa del modelo con guardado de pesos*

```
Inicializar contador 'i' en 0
```

Inicializar listas para pérdida y precisión

Para cada iteración hasta el número máximo de épocas:
  Imprimir número de iteración
  Ajustar modelo con datos de entrenamiento y validar con datos de validación
  Guardar pesos del modelo en ruta especificada con número de iteración en nombre del archivo
  Incrementar contador 'i' en uno

Nota. Elaboración propia

Entregable: Modelos CNN Inception-ResNetV2, ResNet, EfficientNet y personalizado entrenado.

3.9.7. Evaluación del Modelo Entrenado

Tabla 23 *Evaluación del Modelo Entrenado*

Actividades	Descripción	Tareas
Análisis Métrico Comparativo de los Modelos	Examinar cada modelo en base a las métricas de accuracy, precision, recall y f1-score.	Determinar el modelo de mejor rendimiento en cada métrica.
		Elaborar representaciones visuales para contrastar las métricas de los distintos modelos.
		Guardado del Modelo Entrenado y resultados

Nota. Elaboración propia

- **Actividad 1:** Análisis Metrico Comparativo de los modelos

En esta actividad, todos los modelos son comparados en términos de accuracy, precision, recall y f1-score. Aunque es posible que algunos modelos destaquen en ciertas métricas individuales, nuestra elección del modelo óptimo se basará principalmente en los resultados de accuracy y f1-score.

El f1-score es particularmente significativo en esta evaluación ya que proporciona un equilibrio entre la precision y el recall, proporcionando una medida más holística del rendimiento del modelo. En casos donde varios modelos muestren resultados comparables en términos de accuracy y f1-score, se tomarán en cuenta las métricas adicionales para desempatar y determinar cuál modelo es superior.

El resultado de esta actividad será la selección del modelo de clasificación que más efectivamente logra nuestros objetivos de eficiencia y eficacia.

Pseudocódigo 30: *Entrenamiento iterativo y evaluación periódica del modelo*

1. Inicializar contador 'i' en 0 y listas para precisión, recall, F1 score, pérdida y precisión de prueba.
2. Para cada iteración hasta el número máximo de épocas:
   - Si la iteración es múltiplo de un valor especificado:
     - Cargar pesos del modelo desde el archivo correspondiente.
     - Realizar predicciones en los datos de prueba.
     - Convertir las predicciones en etiquetas de clase.
     - Evaluar el modelo en los datos de prueba.
     - Calcular precisión, recall y F1 score.
     - Agregar resultados a las listas correspondientes.
   - Incrementar contador 'i' en uno.

Nota. Elaboración propia

h) Guardado del Modelo Entrenado: Tras la fase de entrenamiento, es crucial guardar el estado del modelo, que incluye la arquitectura del modelo y los pesos neuronales que se han aprendido durante el entrenamiento. Este proceso

de guardado permite reutilizar el modelo sin necesidad de reentrenarlo, ahorrando tiempo y recursos valiosos. Además, facilita la replicación de los resultados y la implementación del modelo en nuevas tareas de predicción.

Pseudocódigo 31:*Guardar modelo entrenado y mostrar métricas destacadas*

1. Guardar el modelo entrenado en un archivo específico y escribir los recursos asociados.
2. Imprimir la máxima precisión de prueba y la época correspondiente.
3. Imprimir la mínima pérdida de prueba y la época correspondiente.

Nota. Elaboración propia

Entregable: Mejor modelo de clasificación en términos de accuracy, precision, recall y f1-score.

CAPÍTULO IV: DESARROLLO DEL EXPERIMENTO

En este capítulo, se expone el proceso integral del experimento que abarca desde la adquisición del conjunto de datos, su adecuado preprocesamiento, la extracción sistemática de las características relevantes, hasta la implementación de los modelos predictivos que se emplearon para analizar los resultados obtenidos.

4.1. Adquisición del dataset

4.1.1. Identificación de las Fuentes de Información

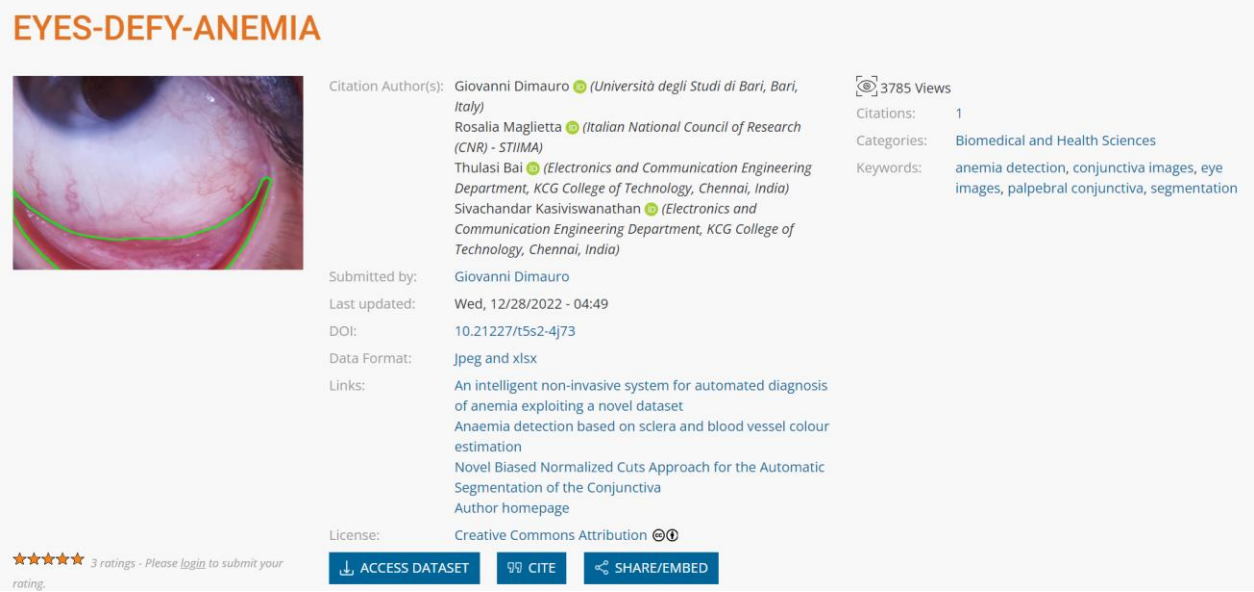
Para la presente investigación, la identificación y selección de la base de datos correcta es crucial. La base de datos seleccionada para este estudio proviene del repositorio EYES-DEFI-ANEMIA disponible en IEEE-DATAPORT. Este recurso fue elegido debido a las características específicas de las imágenes que proporciona, centradas principalmente en la conjuntiva ocular del paciente.

En la literatura científica previa, se enfatiza la importancia de la representación visual de la conjuntiva palpebral y la fórnix en la detección de la anemia, debido a la correlación entre el color de estas áreas y la presencia de la enfermedad. Por lo tanto, un criterio clave en la elección de la base de datos fue la clara visualización de estas áreas en las imágenes. Además, otro factor determinante para la elección del conjunto de datos fue la necesidad de información demográfica y médica relevante del paciente, incluyendo la edad, el sexo y el nivel de hemoglobina. Estos datos son esenciales para la correcta etiquetación de las imágenes, permitiendo un análisis más profundo y contextual de las mismas.

La base de datos EYES DEFI ANEMIA satisface todos estos criterios. Consta de 217 imágenes, todas obtenidas bajo circunstancias similares y centradas en la conjuntiva ocular.

A su vez, se incluye información relevante de cada paciente, lo que refuerza su adecuación para el objetivo del presente estudio.

Figura 36 *Página IEEE Explore*



Nota. Extraído de https:// org/documents/eyes-defy-anemia

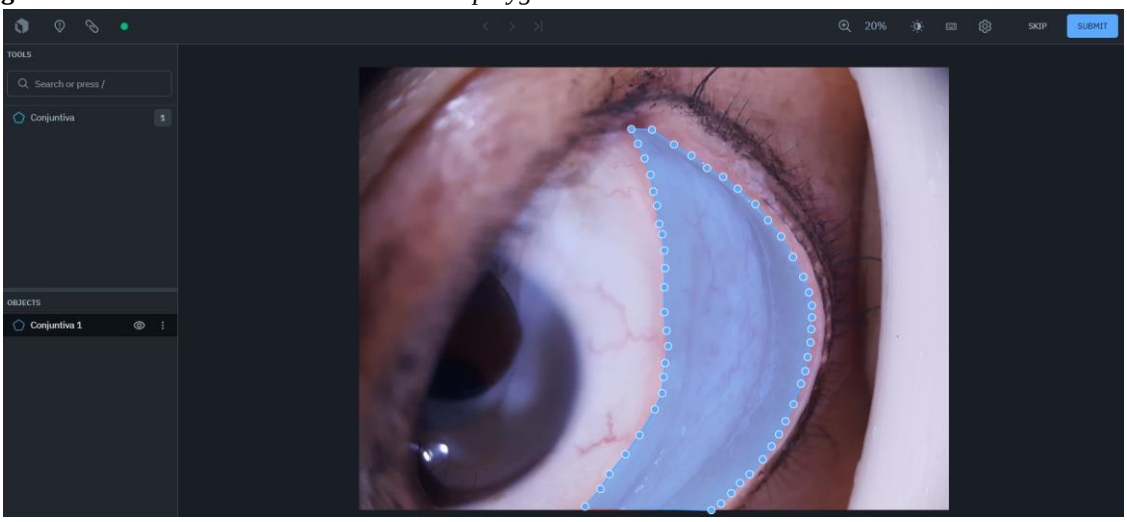
4.1.2. Anotación de las Imágenes

Tras identificar el área de interés para la investigación, los archivos se transfirieron a la plataforma Labelbox, preservando las etiquetas individuales de cada archivo para mantener clara la asociación con cada paciente. Mediante la selección manual en cada imagen, se determinaron únicamente las áreas de la conjuntiva palpebral y la conjuntiva fórnix como regiones de interés.

La herramienta elegida para esta tarea es la función de polígono incorporada en Labelbox. Esta herramienta permite al usuario trazar formas poligonales personalizadas alrededor de la región de interés en una imagen. Cada vértice del polígono se establece con un clic del ratón, y la figura poligonal se completa al conectar el último vértice con el primero. Esta funcionalidad aporta una considerable versatilidad, ya que los polígonos pueden ajustarse a diversas formas y tamaños. Esta característica es especialmente valiosa cuando la

región de interés, en este caso, la conjuntiva ocular, no posee una forma uniforme en todas las imágenes.

Figura 37 *Funcionamiento de la herramienta polygon*



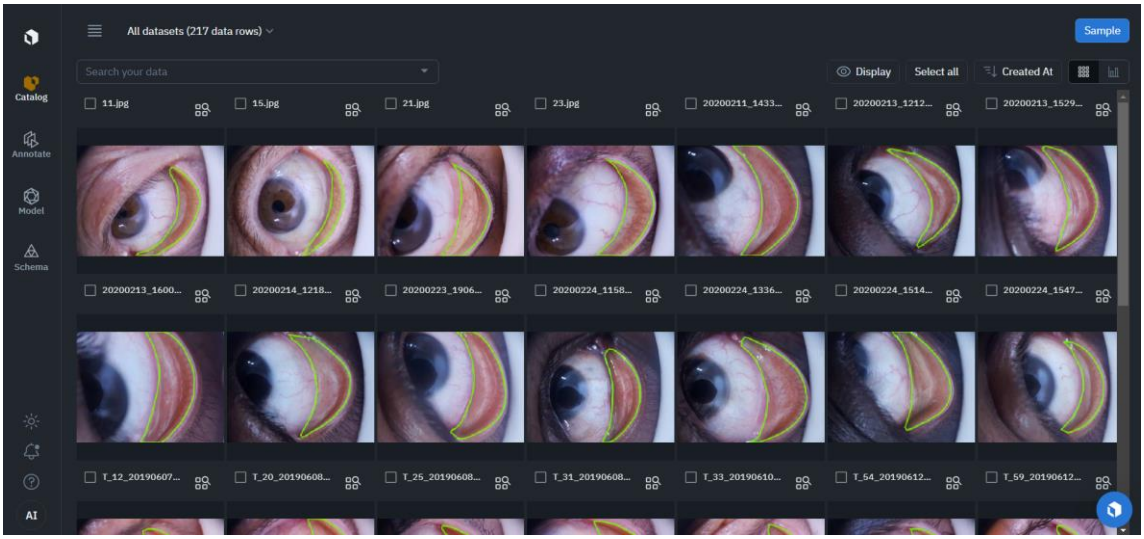
Nota. Elaboración propia

Una vez completada la selección, los archivos se descargan en formato ndjson para su posterior análisis y preprocesamiento. Este paso es esencial para facilitar la posterior fase de análisis de las imágenes.

4.1.3. Preprocesamiento de las Imágenes

Para facilitar el procesamiento y análisis de las imágenes, estas se transfieren a Python en formato ndjson. Este formato permite realizar la lectura de las imágenes con el fin de identificar las áreas seleccionadas como de interés, que posteriormente se utilizarán como áreas de recorte.

Figura 38 *Imágenes luego de la selección de la área de interés*

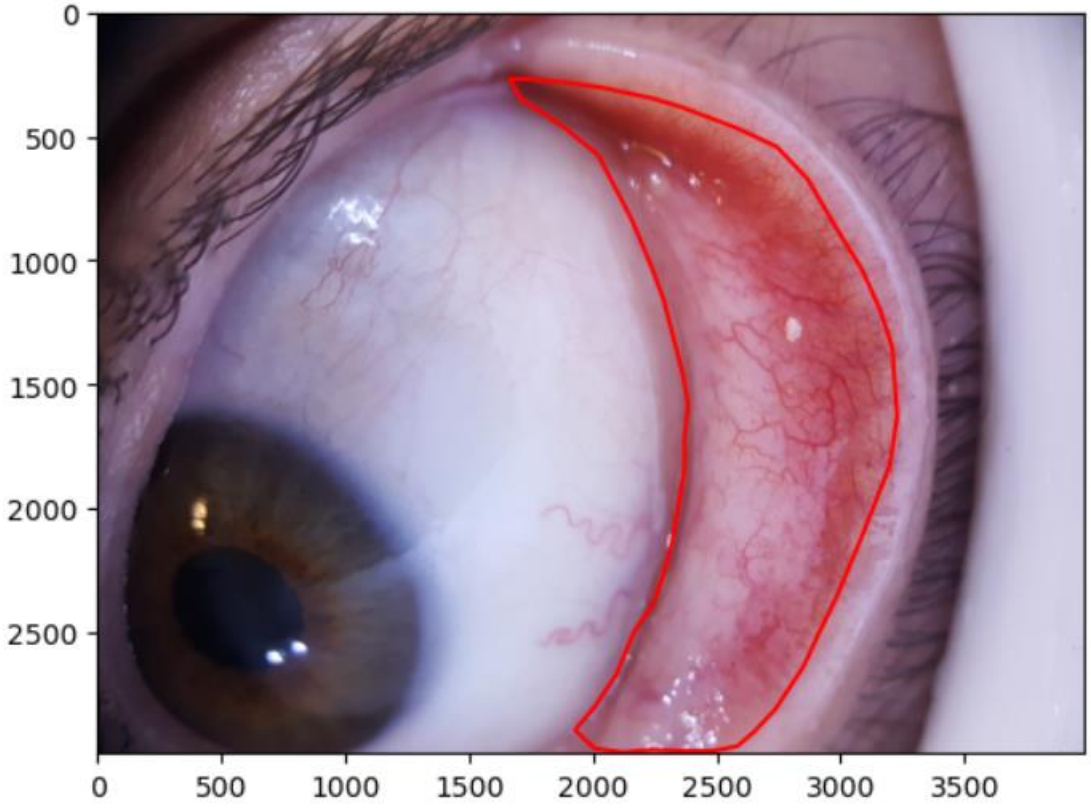


Nota. Elaboración propia

Las anotaciones generadas en Labelbox se convertirán en coordenadas y se emplearán para generar una máscara. Esta máscara facilitará el proceso de segmentación de la imagen, esencial para el análisis de la región de interés.

Utilizando la función de polígono, se traza el contorno de la región seleccionada, que determina el área de recorte de la imagen. Este proceso asegura que solo las áreas de interés sean sometidas a análisis posterior, optimizando así la eficacia y precisión del estudio.

Figura 39 *Área de recorte*



Nota. Elaboración propia

4.1.4. Segmentación de la Imagen

Se genera la máscara a partir de los parámetros establecidos por el polígono. Al aplicar esta máscara a la imagen original, se consigue un recorte centrado en el área de interés, resultando en una imagen que muestra únicamente la conjuntiva ocular. Cualquier otro componente de la imagen es eliminado, para garantizar un análisis centrado y detallado.

A continuación, se almacena la imagen recortada. Sin embargo, es necesario efectuar una etapa adicional de procesamiento para eliminar el fondo de las imágenes. Este proceso es crucial ya que las imágenes cuentan con un fondo negro que podría interferir con la precisión del resultado final durante la evaluación del modelo.

Figura 40 *Imagen recortada*



Nota. Elaboración propia

Finalmente, se ha generado una nueva base de datos compuesta por las imágenes de la conjuntiva ocular de diversos pacientes. Estos pacientes pueden presentar signos de anemia o no, proporcionando un conjunto diverso y representativo para el estudio de esta condición médica.

4.2. Pre procesamiento Base de datos:

4.2.1. Evaluación de la clasificación de imágenes

A partir de la información recolectada de las imágenes, se emprende la tarea de asignar cada una a su respectivo grupo. Para ello, se utiliza la tabla extraída de Etoolsage. Mediante variables como el nivel de sangre, la edad y el sexo del paciente, se determina a qué grupo de salud corresponde el sujeto.

Tabla 24 *Evaluación de la clasificación de imágenes*

Imagen Original	Palpebral	Fornix y Palpebral	Fornix
			
No tiene anemia	Género: Femenino	Hgb: 11.7	Edad: 73

Nota. Elaboración propia

Para organizar de manera eficiente esta clasificación, se crearon dos carpetas distintas. En una se almacenan las imágenes correspondientes a los pacientes diagnosticados con anemia, mientras que en la otra se guardan las imágenes de los pacientes que no presentan esta condición. De esta manera, se facilita el acceso y el análisis de los datos conforme a las necesidades del estudio.

4.2.2. Etiquetado de imágenes

Una vez creadas las dos carpetas y clasificados los pacientes en función de su condición de anemia, se asignaron las etiquetas correspondientes: "Anemia" para los pacientes diagnosticados con la condición y "NoAnemia" para aquellos que no la presentan.

Se procedió a la eliminación de los datos incompletos o dañados. En este caso, de las 218 imágenes proporcionadas por el repositorio, una estaba dañada y, por tanto, se excluyó del conjunto de datos. Además, se corrigieron los errores topográficos para optimizar la calidad de los datos.

Posteriormente, las etiquetas previamente asignadas se transformaron en valores numéricos, con el diagnóstico positivo de anemia representado por el número 1 y el diagnóstico negativo por el 0. Este cambio se realizó con el objetivo de facilitar la representación de estos datos en el código y su posterior procesamiento en el análisis.

4.2.3. Evaluación de las variables predictoras

El análisis de correlación es una técnica estadística que examina y cuantifica la asociación entre dos o más variables (Dormann et al., 2013). Aplicada al diagnóstico de anemia, se realiza una correlación entre las variables predictoras (p.ej., edad, género, historial médico) y la variable objetivo (presencia o ausencia de anemia). De este modo, se pueden identificar características del paciente que pueden estar correlacionadas con un mayor riesgo de anemia, lo que potencialmente aumenta la precisión y la fiabilidad de la clasificación de las imágenes.

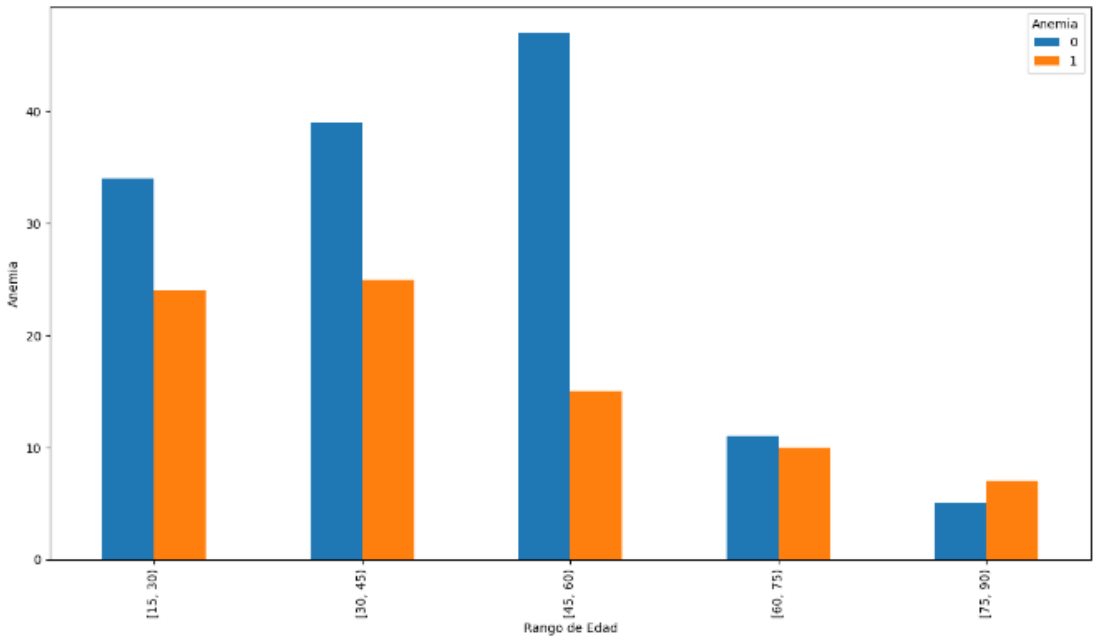
- Distribución de la prevalencia de anemia por rangos de edad

  Este gráfico de barras muestra de manera detallada la relación observada entre la edad y la incidencia de la anemia dentro de la población estudiada. Se puede apreciar que, en los grupos de menor edad, la prevalencia de la anemia es significativamente menor en comparación con los casos sin anemia.

  Sin embargo, un hallazgo notable se presenta en el grupo de edad de 60 años en adelante, donde la proporción de pacientes con anemia se iguala a la de los pacientes sin anemia, a pesar de la disminución en el número total de pacientes. Este cambio en la proporción sugiere la presencia de una correlación entre el avance de la edad y la prevalencia de la anemia.

En este sentido, los datos presentados en este gráfico sugieren una relación entre la edad y la anemia, especialmente a partir de los 60 años de edad, aunque también se observan casos de anemia en todos los rangos de edad estudiados

Figura 41 *Rango de edad y anemia*



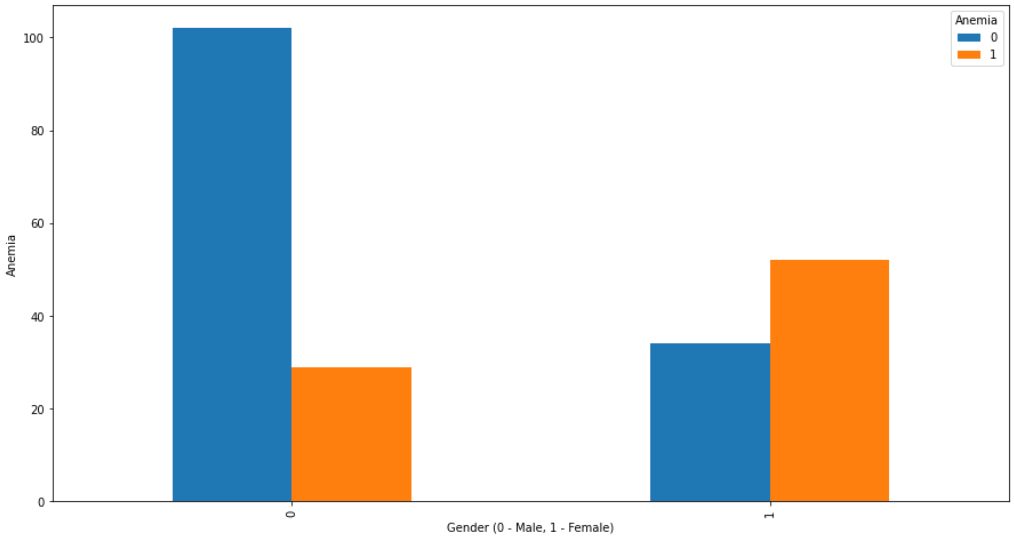
Nota. Elaboración propia

- Distribución de anemia en función del género.

Este gráfico de barras ilustra la relación entre el género y la prevalencia de la anemia en la población estudiada. A pesar de que el número total de pacientes masculinos es superior, se observa una prevalencia más alta de anemia en las pacientes femeninas. En contraste, los pacientes masculinos muestran un predominio de casos sin anemia.

Este patrón sugiere una correlación entre el género y la prevalencia de la anemia, siendo las mujeres más propensas a presentar anemia según los datos recolectados.

Figura 42 *Sexo y anemia*



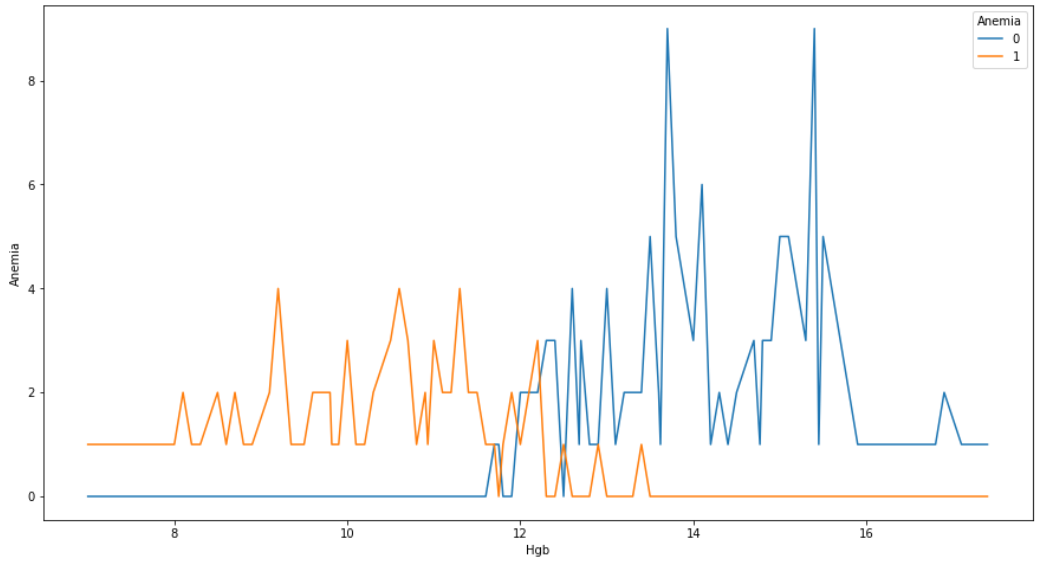
Nota. Elaboración propia

- Correlación entre el nivel de hemoglobina y la anemia.

Este gráfico de línea muestra la relación entre el nivel de hemoglobina (Hgb) en sangre y la prevalencia de la anemia. Como es de esperar, se observa que los individuos con niveles de hemoglobina superiores al estándar básico tienden a no tener anemia, mientras que aquellos con niveles de hemoglobina inferiores al estándar suelen presentar anemia.

Este gráfico proporciona una representación visual clara de la definición clínica de anemia y refuerza la validez de los niveles de hemoglobina como indicador fiable de esta condición. La correlación negativa entre los niveles de hemoglobina y la presencia de anemia, evidente en el gráfico, es consistente con el conocimiento médico y científico actual sobre la fisiopatología de la anemia.

Figura 43 *Hemoblohina en sangre y anemia*

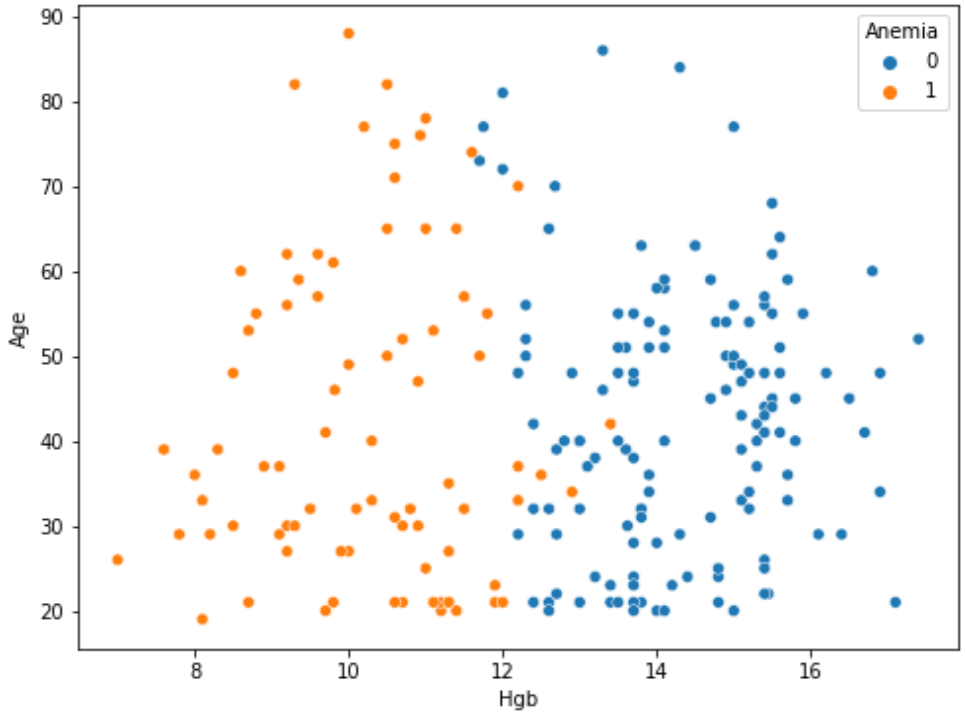


Nota. Elaboración propia

- Correlación entre edad, nivel de hemoglobina y anemia.

El gráfico de dispersión muestra una clara distinción entre los individuos con anemia (puntos naranjas) y sin anemia (puntos azules) en términos de niveles de hemoglobina, con los primeros agrupados hacia los valores más bajos de hemoglobina. La distribución de la edad entre los individuos, por otro lado, es más uniforme, aunque se nota una disminución en la densidad de los puntos en los rangos de edad más avanzados. El gráfico proporciona una visión inicial de las correlaciones entre estas variables, aunque se requieren análisis estadísticos más rigurosos para su confirmación.

Figura 44 *Correlación entre edad, nivel de hemoglobina y anemia*



Nota. Elaboración propia

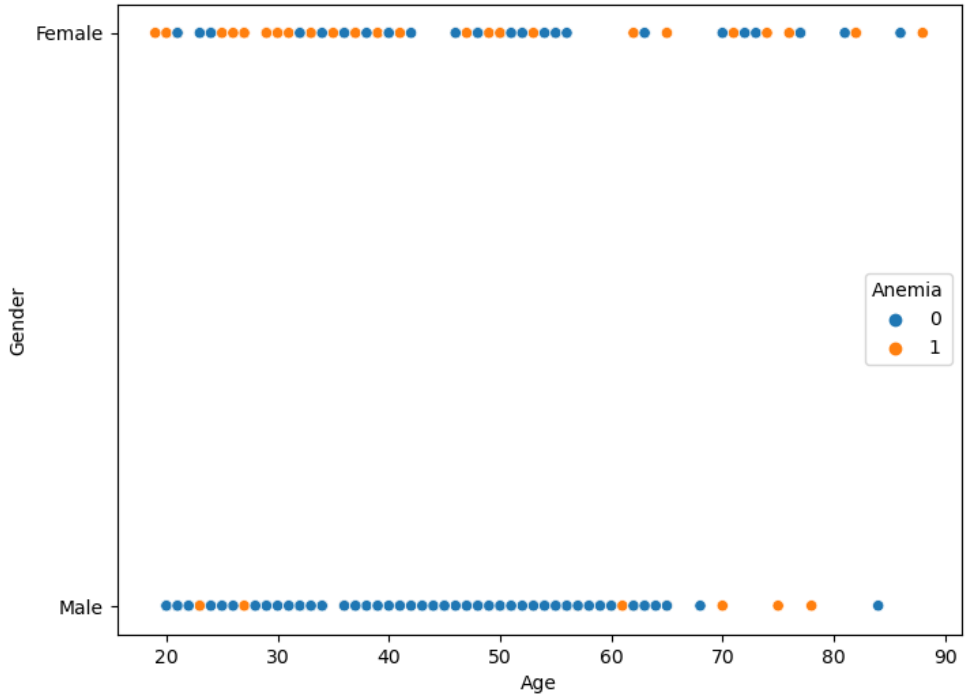
- Distribución de anemia en función de género y edad.

El gráfico de dispersión representa a los individuos en base a su género (masculino o femenino) y edad, con colores distintos para aquellos con y sin anemia. Los puntos azules representan a los individuos sin anemia y los puntos naranjas representan a los que sí la tienen.

Para el género masculino, se aprecia una prevalencia de puntos azules, especialmente entre las edades de 20 a 60 años, lo que sugiere una menor incidencia de anemia en este grupo demográfico. Este hallazgo es de interés, ya que podría indicar que los hombres en este rango de edad están menos afectados por la anemia. En contraste, en el género femenino, se nota una presencia significativa de puntos naranjas en los rangos de edad de 20 a 40 y 50 a 60 años. Esto señala que la anemia es

más prevalente en las mujeres de estas edades. Este patrón podría sugerir la necesidad de prestar mayor atención a la anemia en las mujeres en estos rangos de edad.

Figura 45 *Distribución de anemia en función de género y edad.*



Nota. Elaboración propia

- Análisis de Resultados

Nuestro estudio evidencia correlaciones entre anemia, edad, género y niveles de hemoglobina. Se observó una mayor prevalencia de anemia en los rangos de edad de 20 a 40 y 50 a 60 años, especialmente en mujeres, lo que sugiere una mayor susceptibilidad en estos grupos. En contraste, los hombres de 20 a 60 años presentaron menor prevalencia.

4.2.4. Análisis de la Base de Datos

Se desarrolla y evalúa un modelo destinado a examinar la efectividad de una base de datos, particularmente en lo que respecta a la identificación de la anemia.

Figura 46 *Concatenación de datos y selección de variables*

```
d1 = pd.read_excel(file1)
d1["Anemia"] = d1["Note"]
d1 = d1[["Hgb", "Gender", "Age", "Anemia" ]]
d2 = pd.read_excel(file2)[["Hgb", "Gender", "Age", "Anemia" ]]
data = pd.concat([d1, d2])
data.head(3)
```

	Hgb	Gender	Age	Anemia
0	12.2	M	29	No Anemia
1	8.0	F	36	Anemia
2	10.7	F	30	Anemia

```
len(data)
```

```
218
```

Nota. Elaboración propia

Como se menciono anteriomente la base de datos se reparte en grupos de India e Italia. El documento Excel abarca información sobre niveles de hemoglobina, género, edad, y diagnóstico de anemia de cada individuo, los por un proceso de limpieza inicial para tratar valores perdidos y para transformar todas las variables categóricas en numéricas.

Inicialmente, se ejecutaron procesos de depuración de datos para tratar la incidencia de valores ausentes y para convertir todas las variables de naturaleza categórica en su correspondiente formato numérico, facilitando así el análisis subsecuente. A continuación, todas las variables fueron normalizadas mediante un escalador MinMax para garantizar la homogeneidad y la comparabilidad de los datos. Como paso final, los datos se estratificaron en un conjunto de entrenamiento, representando el 70% del total, y un conjunto de prueba, que contenía el 30% restante.

Para la fase analítica, se construyó un modelo secuencial con el apoyo de la biblioteca TensorFlow. Dicho modelo incluía una capa de entrada con 128 nodos, una capa oculta con 56 nodos y una capa de salida con 2 nodos. La entropía cruzada categórica fue

la función de pérdida seleccionada, mientras que el algoritmo de optimización Adam se empleó para el entrenamiento del modelo. Este proceso de entrenamiento se llevó a cabo durante un total de 50 épocas con el conjunto de entrenamiento.

Figura 47 *Código de Implementación del Modelo Secuencial con TensorFlow*

```
model =   tf.keras.models.Sequential([
                            tf.keras.layers.Dense(128, input_shape=(None, X_train.shape[1]), activation='relu'),
                            tf.keras.layers.Dense(56,  activation='relu'),
                            tf.keras.layers.Dense(2)
                            ])
```

```
loss_fn = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)
model.compile(optimizer='adam',loss=loss_fn, metrics=['accuracy'])
```

```
model.fit(X_train, Y_train, epochs=50)
```

```
res = model.evaluate(X_test, Y_test)
```

```
WARNING:tensorflow:Model was constructed with shape (None, None, 3) for input KerasTensor(type_spec=TensorSpec(shape=(None, Non
e, 3), dtype=tf.float32, name='dense_9_input'), name='dense_9_input', description="created by layer 'dense_9_input'"), but it w
as called on an input with incompatible shape (None, 3).
3/3 [==============================] - 0s 2ms/step - loss: 0.1185 - accuracy: 0.9697
```

```
res
```

```
[0.42827439308166504, 0.8131868243217468]
```

Nota. Elaboración propia

Una vez concluida la fase de entrenamiento, se emplearon diversas técnicas avanzadas de aprendizaje automático con el objetivo de realizar un análisis comparativo de su rendimiento en la clasificación. Entre los algoritmos utilizados se incluyen la Red Neuronal Artificial (ANN), la Máquina de Vectores de Soporte (SVM), los Vecinos más Cercanos (KNN), el Bosque Aleatorio (Random Forest) y el Perceptrón Multicapa (MLP). Estos métodos se aplicaron con la intención de examinar su efectividad en la predicción de diagnósticos de anemia a partir del conjunto de datos analizado.

Tabla 25 *Precisión de los Modelos de Aprendizaje Automático en el Conjunto de Prueba*

Algoritmo de Aprendizaje Automático	Precisión (%)
Red Neuronal Artificial (ANN)	97%
Máquina de Vectores de Soporte (SVM)	98%
Vecinos más Cercanos (KNN)	98%
Bosque Aleatorio (Random Forest)	98%
Perceptrón Multicapa (MLP)	97%

Nota. Elaboración propia

Estos resultados indican un alto rendimiento de los algoritmos de aprendizaje automático empleados en la clasificación. Las cifras revelan una notable capacidad predictiva, destacando la efectividad de las técnicas de procesamiento y análisis de datos implementadas en este estudio. La elección de hiperparámetros óptimos mediante la búsqueda en cuadrícula, junto con el uso de técnicas de aprendizaje automático avanzadas, ha demostrado ser una estrategia eficaz para el análisis de este tipo de datos.

4.3. Pre Procesamiento de la imagen:

4.3.1. Organización y exploración de datos

La preparación inicial de los datos implicó la lectura de dos archivos de Excel, que contenían el identificador de cada imagen y la respectiva clasificación de la anemia, para ser almacenados en dos marcos de datos distintos. Como medida de coherencia, todas las referencias de "No anemia" se estandarizaron a "No Anemia".

Para facilitar el manejo eficiente de las categorías, se identificaron los valores únicos de la clasificación de anemia y se les asignó un mapeo numérico único. Este proceso de

enumeración simplifica el análisis posterior y la manipulación de datos al reemplazar los valores originales en la columna 'Anemia' con estos números enteros únicos en ambos conjuntos de datos.

Finalmente, se crearon directorios para cada conjunto de datos, que mapean el identificador de cada imagen a su clasificación de anemia. Este esfuerzo de organización de datos es fundamental para las etapas de procesamiento y análisis posteriores, y está en consonancia con la creación de directorios y espacios dedicados para el almacenamiento de datos, etiquetas e imágenes.

Tabla 26 *Anemia y no anemia*

Anemia	No Anemia
	
Hgb 9.1	Hgb 15.3

Nota : Elaboración propia

4.3.2. Pre procesamiento de imágenes

En el ámbito del preprocesamiento de las imágenes, se implementó un enfoque en etapas para garantizar la coherencia y la optimización de los datos de las imágenes.

La primera etapa consistió en la apertura de la imagen, a través de la lectura de la imagen desde la ubicación especificada en el sistema de archivos. Este procedimiento garantizó el acceso efectivo a los datos de la imagen para las operaciones de procesamiento posteriores. A continuación, se realizó un redimensionamiento de la imagen, donde cada

imagen se ajustó a un tamaño estándar de 64x64 píxeles. Este paso facilitó la uniformidad de los datos de las imágenes, un factor esencial para garantizar un análisis preciso y coherente en las etapas posteriores.

Posteriormente, las imágenes fueron convertidas a escala de grises. Esta transformación eliminó la información de color de las imágenes, lo que permitió enfocar el análisis en características más prominentes, como la textura y la forma. Finalmente, las imágenes procesadas se añadieron a la lista X, y su etiqueta correspondiente se agregó a la lista y. Estas listas acumulativas sirven como entradas para los modelos de aprendizaje automático que se emplearán en las etapas siguientes de este estudio.

4.3.3. Optimización de imágenes

La optimización de la imagen se realizó mediante la aplicación de dos filtros específicos. Primero, se utilizó un filtro de enfoque para mejorar los detalles de la imagen y resaltar los bordes, esencial para la identificación precisa de características.

Posteriormente, se aplicó un suavizado con un filtro Gaussiano. Este paso ayuda a minimizar el ruido visual y los detalles innecesarios de la imagen, mejorando así la claridad de las características de interés. Finalmente, se reorganizó la información de la imagen, ajustándola a un único canal de color. Este paso prepara la imagen para su posterior análisis, adaptando la estructura de la información a los requerimientos de los algoritmos de aprendizaje automático.

Figura 48 *Optimización de imágenes*

```
X = []
y = []


for key, items in img_files.items():
    for item in items:
        img_path, cls = item
        cls           = cls.split("/")[-1]
        img           = cv2.imread(img_path)
        img           = cv2.resize(img, (64, 64))
        img           = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        kernel        = np.array([[-1,-1,-1],
                                  [-1, 9,-1],
                                  [-1,-1,-1]])
        sharpened     = cv2.filter2D(img, -1, kernel)
        img           = cv2.GaussianBlur(sharpened, (5, 5), 0)
        img           = img.reshape(64, 64, 1)
        lbl           = labels[key][int(cls)]

        X.append(img)
        y.append(lbl)
```

Nota. Elaboración propia

4.3.4. Etiquetado y almacenamiento:

En la actividad de etiquetado y almacenamiento, se llevó a cabo un proceso binario en las etiquetas de las imágenes. Las imágenes asociadas con la etiqueta "0" representan una categoría, mientras que las imágenes asociadas con cualquier otro número representan una categoría distinta.

Tras el etiquetado, se almacenaron las imágenes en un array de NumPy. Esta estructura de datos es particularmente útil para la manipulación y el análisis eficiente de los datos en etapas posteriores de la investigación. Posteriormente, se validó la forma del array de las imágenes para asegurar la consistencia en el tamaño y la estructura de los datos. Después, se identificó la cantidad de clases únicas en las etiquetas, resultando ser dos. Este resultado implica que las imágenes se clasifican binariamente.

4.3.5. Partición de datos

El paso de la partición de datos es esencial en el proceso de modelado de datos. En este estudio, se ha empleado empleado la función train_test_split de la biblioteca de aprendizaje automático de Python, sklearn, para dividir nuestros datos de imagen en conjuntos separados para el entrenamiento y la prueba.

Esta función divide los datos de entrada y sus etiquetas correspondientes en dos segmentos de acuerdo con una proporción especificada. En este caso, se eligió un tamaño de entrenamiento del 80%. Esto significa que el 80% de las imágenes y sus etiquetas correspondientes se utilizaron para entrenar los modelos de aprendizaje automático, mientras que el restante 20% se reservó para evaluar la eficacia y la precisión de los modelos.

Figura 49 *Particion de datos*

```
1 X_train, X_test, Y_train, Y_test = train_test_split(X, y, train_size=0.8, random_state=0)
```

Nota. Elaboración propia

4.3.6. Aumento de data

El aumento de datos es una técnica efectiva para mejorar la capacidad de generalización de los modelos de aprendizaje automático, especialmente cuando se tienen conjuntos de datos de tamaño limitado. Con esta técnica se puede crear una versión más grande y más diversa del conjunto de datos original, lo que permite una mejora en el rendimiento del modelo.

En este estudio, se utilizó la biblioteca ImageDataGenerator de keras para implementar diversas técnicas de aumento de datos, incluyendo rotación de imagen, desplazamiento de ancho y altura, zoom, cizallamiento y volteo horizontal. Estas transformaciones permiten que el modelo se entrene con diversas versiones de la misma imagen, aumentando su capacidad para generalizar a partir de patrones más variados.

Figura 50 *Generación de la imagen aumentada*

```
X_train.shape
```

```
(174, 64, 64, 1)
```

```
characters  = X_train
labels      = np.array(Y_train.copy())
```

```
datagen = ImageDataGenerator( width_shift_range=[-200,200],
    rotation_range=30,
    zoom_range=0.15,
    height_shift_range=0.2,
    shear_range=0.15,
    horizontal_flip=True,
    fill_mode="nearest")
```

```
iterator = datagen.flow(characters, labels)
```

```
set(labels)
```

```
{0, 1}
```

Nota. Elaboración propia

El aumento de datos se aplicó solamente al conjunto de datos de entrenamiento. La finalidad de esto es garantizar que el conjunto de datos de prueba permanezca independiente y representativo de los datos sin manipulaciones, permitiendo así una evaluación más precisa del rendimiento del modelo. Para garantizar una distribución uniforme de las clases tras el aumento de datos, se implementó un bucle que asegura que cada clase tenga al menos 1500 ejemplos. Se añadieron estas nuevas imágenes al conjunto de datos de entrenamiento original, aumentando el tamaño total del conjunto de datos y equilibrando la distribución de las clases.

Figura 51 *Etiquetado de las imágenes aumentadas*

```
augment_lbls = [0, 1]
NUM = 1500
sample_count = {item:0 for item in augment_lbls}
x_aug = []
y_aug = []
while True:
    x_tmp, y_tmp = iterator.next()
    for x, y in zip(x_tmp, y_tmp):
    if y in augment_lbls:
        if sample_count[y] < NUM:
            x_aug.append(x)
            y_aug.append(y)
            sample_count[y] += 1
    if sum([sample_count[key] >= NUM for key in sample_count.keys()]) == len(sample_count.keys()):
    break
```

```
set(y_aug)
```

```
{0, 1}
```

```
label_counts = [(lbl, y_aug.count(lbl) ) for lbl in set(y_aug)]
label_counts
```

```
[(0, 1000), (1, 1000)]
```

```
np.array(x_aug).shape
```

```
(2000, 64, 64, 1)
```

Nota. Elaboración propia

Este proceso resultó en un conjunto de datos de entrenamiento considerablemente más grande y balanceado, que ayudará a prevenir el sobreajuste y mejorará la capacidad de nuestro modelo para hacer predicciones precisas.

Figura 52 *Distribución de etiquetas en la clasificación de imágenes*

```
type(characters)
```

```
numpy.ndarray
```

```
characters.shape
```

```
(174, 64, 64, 1)
```

```
characters = list(characters)
characters.extend(x_aug)
characters = np.array(characters)
characters.shape
```

```
(2174, 64, 64, 1)
```

```
labels = list(labels)
labels.extend(y_aug)
len(labels)
```

```
2174
```

```
label_counts = [(lbl, labels.count(lbl) ) for lbl in set(labels)]
label_counts
```

```
[(0, 1102), (1, 1072)]
```

Nota. Elaboración propia

4.3.7. División de Datos

Después de la etapa de aumento de datos, el conjunto de datos aumentado fue dividido estratégicamente para crear conjuntos de entrenamiento y validación. La función train_test_split de sklearn fue utilizada para realizar esta división, asignando el 80% de los datos para el entrenamiento y el 20% restante para la validación. Esta división equilibrada asegura que el modelo tenga suficientes datos para aprender, al mismo tiempo que se reservan suficientes datos para evaluar su desempeño y evitar el sobreajuste.

Posteriormente, las etiquetas de los conjuntos de entrenamiento, prueba y validación se convirtieron a codificación "one-hot". Esta transformación es necesaria para la fase de entrenamiento del modelo, ya que esta estructura de datos es más compatible con muchos algoritmos de aprendizaje automático.

Finalmente, se mostró el tamaño de los conjuntos de datos de entrenamiento, validación y prueba para verificar que la división de datos se realizó correctamente. Los tamaños resultantes confirmaron que el proceso de división de datos se llevó a cabo de manera efectiva, asegurando que haya suficientes datos en cada conjunto para las etapas subsiguientes de entrenamiento y evaluación del modelo.

Figura 53 *Partición del dataset*

```
characters.shape
```

```
(2174, 64, 64, 1)
```

```
X = characters
Y = labels
number_classes = len(set(labels))
```

```

X_train, X_val, Y_train, Y_val= train_test_split(X, Y, train_size=0.8, random_state=0)
```

```
from keras.utils import np_utils

Y_train = keras.utils.np_utils.to_categorical(Y_train, number_classes)
Y_test = keras.utils.np_utils.to_categorical(Y_test, number_classes)
Y_val = keras.utils.np_utils.to_categorical(Y_val, number_classes)

#Y = keras.utils.to_categorical(Y, number_classes)
```

```
print(len(X_train))
print(len(X_val))
print(len(X_test))
```

```
1739
435
44
```

Nota. Elaboración propia

4.4. Extracción de características

4.4.1. Extracción de características con LBPH

Para la extracción de características de las imágenes se utilizó el método Local Binary Patterns Histograms (LBPH). Este método es efectivo para describir las texturas locales de una imagen, lo que es particularmente útil en tareas de clasificación de imágenes.

Primero, se definió una clase LocalBinaryPatterns que incluye dos funciones principales: preprocess y describe. La función preprocess convierte la imagen en escala de grises, lo que es necesario para aplicar el método LBPH. La función describe aplica el método LBPH a la imagen en escala de grises. Con este método, se calcula un patrón binario local para cada píxel de la imagen, considerando un número de puntos y un radio específicos en torno al píxel. Este proceso resulta en un histograma de los patrones binarios locales, que se normaliza para evitar cualquier sesgo debido a la diferencia en los tamaños de las imágenes.

El resultado de la función describe es un histograma normalizado que representa las características texturales locales de la imagen, que se pueden usar para entrenar un modelo de aprendizaje automático. Además, se devuelve también la imagen con los patrones binarios locales aplicados, que puede ser útil para la visualización y el análisis posterior.

Figura 54 *Extracción de características con LBPH*

```
class LocalBinaryPatterns:
    def __init__(self, numPoints, radius):
        self.numPoints = numPoints
        self.radius = radius

    def preprocess(self, image):
        gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
        return gray

    def describe(self, image, eps = 1e-7):
        image = self.preprocess(image)
        lbp   = feature.local_binary_pattern(image, self.numPoints, self.radius, method="uniform")
        (hist, _) = np.histogram(lbp.ravel(), bins=np.arange(0, self.numPoints+3), range=(0, self.numPoints + 2))

        # Normalize the histogram
        hist = hist.astype('float')
        hist /= (hist.sum() + eps)

        return hist, lbp
```

Nota. Elaboración propia

4.4.2. Extracción de características con HOG

Además del método Local Binary Patterns Histograms (LBPH), se utilizó también el método Histogram of Oriented Gradients (HOG) para la extracción de características de las

imágenes. Este método es popular para la detección de objetos en imágenes debido a su capacidad para capturar la estructura y la forma de los objetos.

Figura 55 *Extracción de características con HOG*

```
class HistogramOrientedGradient:
    def __init__(self, winSize = (60,60), blockSize = (12,12), blockStride = (12,12), cellSize = (12,12), nbins = 9,
                 derivAperture = 1, winSigma = -1., histogramNormType = 0, L2HysThreshold = 0.2, gammaCorrection = 1,
                 nlevels = 64, signedGradient = True):
        self._descriptor = cv2.HOGDescriptor(winSize,blockSize,blockStride,cellSize,nbins,derivAperture,winSigma,
                                             histogramNormType,
                                    L2HysThreshold,gammaCorrection,nlevels, signedGradient)

    def preprocess(self, image):
        gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
        return gray

    def describe(self, image):
        image = self.preprocess(image)
        hog   = self._descriptor.compute(image)
        return hog
```

Nota. Elaboración propia

Similarmente al LBPH, se definió una clase HistogramOrientedGradient que consta de las funciones preprocess y describe. La función preprocess convierte la imagen en escala de grises, preparándola para la aplicación del método HOG.

La función describe utiliza el descriptor HOG de OpenCV para calcular el histograma de gradientes orientados. Esta función toma una serie de parámetros que definen cómo se calcula el descriptor. Entre ellos, se incluyen el tamaño de la ventana, el tamaño del bloque, el tamaño de la celda y el número de contenedores del histograma. Estos parámetros se pueden ajustar para optimizar la eficacia del método en función del problema específico de clasificación de imágenes.

El resultado de la función describe es el descriptor HOG de la imagen, que representa las características de forma y estructura de la imagen. Este descriptor se puede usar para entrenar un modelo de aprendizaje automático para la clasificación de imágenes.

Figura 56 *Extracción de características HOG y generación de histogramas*

```
# HOG Hist
X = []
y = []

desc = HistogramOrientedGradient()

for key, items in img_files.items():
    for item in items:
        img_path, cls = item
        img           = cv2.imread(img_path)
        img           = cv2.resize(img, (256, 256))
        hist          = desc.describe(img)
        lbl           = labels[key][int(cls)]

        X.append(hist)
        y.append(lbl)

```

```
y = [0 if i == 0 else 1 for i in y]
```

Nota. Elaboración propia

4.4.3. Extracción de características con SIFT

En este proceso, también se implementó el método Scale-Invariant Feature Transform (SIFT) para la extracción de características de las imágenes. Este método es conocido por su eficacia en la extracción de características invariantes a la escala y la orientación, lo que lo hace adecuado para problemas de reconocimiento de imágenes en los que las mismas pueden tener diferentes tamaños y orientaciones. En términos de implementación, se definió una clase SIFT que incluye las funciones preprocess y describe. En la función preprocess, se convierte la imagen a escala de grises como paso inicial.

En la función describe, se utiliza el método SIFT provisto por OpenCV para calcular las características de la imagen. Este método detecta y calcula los puntos clave de SIFT y sus descriptores correspondientes. Los puntos clave de SIFT son regiones de la imagen que

se consideran distintivas y los descriptores son vectores de características que describen estas regiones distintivas. El resultado de la función describe es una lista de puntos clave y sus descriptores correspondientes. Estos pueden ser utilizados para entrenar un modelo de aprendizaje automático que pueda distinguir entre diferentes clases de imágenes basándose en sus características SIFT.

Figura 57 *Extracción de características con SIFT*

```
class SIFT:
    def __init__(self):
        self._descriptor = cv2.xfeatures2d.SIFT_create()

    def preprocess(self, image):
        gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
        return gray

    def describe(self, image):
        image                       = self.preprocess(image)
        keypoints_sift, descriptors   = self._descriptor.detectAndCompute(image, None)
        return keypoints_sift, descriptors


#hog =  np.array(hog_descriptors)
#hog = hog.reshape(hog.shape[0], hog.shape[1]*hog.shape[2])
```

Nota. Elaboración propia

4.4.4. Extracción de características con SIFT Padding

En esta variante del método SIFT, se añade un proceso de "padding" para abordar el problema de las características de diferentes dimensiones. El proceso de "padding" es una técnica común en el procesamiento de datos y de imágenes que permite uniformizar la longitud o las dimensiones de los datos añadiendo valores adicionales, normalmente ceros. Sin embargo, en este caso, se utilizó el método de replicación del borde en lugar de ceros.

El "padding" se realiza utilizando la función copyMakeBorder de OpenCV, que copia los píxeles de los bordes de la imagen en el espacio de "padding". Para cada imagen, se calcula la cantidad de "padding" necesario para que todas las imágenes tengan la misma dimensión, que es la dimensión máxima de todas las imágenes.

Este proceso garantiza que todas las características extraídas a través del método SIFT tengan la misma dimensión, lo que es crucial para los algoritmos de aprendizaje automático que requieren entradas de tamaño fijo. Al replicar los píxeles de borde en lugar de añadir ceros, se preserva mejor la información en las características de la imagen, lo que podría mejorar la eficacia del modelo de aprendizaje automático.

Figura 58 *Ajuste de dimensiones mediante relleno (padding)*

```
# We have edifferent dimensions in our features,
#Strategy 1: Bring at same scale, we need to pad the features

max_dimension = max([len(i) for i in X] )
X = [cv2.copyMakeBorder(i,max_dimension-len(i), 0, 0, 0,cv2.BORDER_REPLICATE) for i in X]
```

```
# Strategy 2: Use minimum number of features from our features
min_feat_len = min([len(i) for i in X] )
min_feat_len
```

```
27
```

```
X = [i[:min_feat_len] for i in X]
```

```
X[0].shape
```

```
(27, 128)
```

```
X = [i.reshape(27*128) for i in X]
```

Nota. Elaboración propia

4.4.5. Extracción de características con ORB

La extracción de características mediante el método Oriented FAST and Rotated BRIEF (ORB) proporciona una solución eficiente y robusta para esta tarea esencial en el procesamiento de imágenes. ORB combina dos algoritmos muy reconocidos en el campo de la visión por computadora: FAST, un detector de esquinas de alta velocidad, y BRIEF, un descriptor binario eficiente. La implementación de ORB en la biblioteca OpenCV se ha utilizado para esta tarea.

La imagen de entrada se convierte primero a escala de grises, un paso comúnmente requerido en el procesamiento de imágenes antes de la extracción de características. Posteriormente, el algoritmo ORB se aplica a la imagen preprocesada para detectar keypoints y calcular sus correspondientes descriptores.

El descriptor ORB se inicializa con un parámetro nfeatures, que define el número máximo de características que se detectarán en la imagen. En este caso, se ha establecido que se detecten hasta 1500 características. Sin embargo, este valor puede ser ajustado de acuerdo a las necesidades específicas del conjunto de datos y las restricciones de recursos.

En contraste con otros métodos como SIFT y SURF, que generan descriptores de punto flotante, ORB produce descriptores binarios. Este enfoque reduce significativamente los requerimientos de almacenamiento y los tiempos de cálculo, lo cual contribuye a la eficiencia del proceso de aprendizaje automático.

Figura 59 *Extracción de características con ORB*

```python
class ORB:
    def __init__(self, nfeatures=1500):
        self._descriptor = cv2.ORB_create(nfeatures=nfeatures)

    def preprocess(self, image):
        gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
        return gray

    def describe(self, image):
        image                        = self.preprocess(image)
        keypoints_orb, descriptors   = self._descriptor.detectAndCompute(image, None)
        return keypoints_orb, descriptors


#hog =  np.array(hog_descriptors)
#hog = hog.reshape(hog.shape[0], hog.shape[1]*hog.shape[2])
```

Nota. Elaboración propia

4.5. Creación del modelo:

4.5.1. Codificación de Etiquetas de Clase

En esta fase de la implementación, se realiza una codificación one-hot de las etiquetas de clase para los conjuntos de datos de entrenamiento y prueba. Esta técnica transforma las categorías nominales en una forma binaria que facilita su procesamiento por los algoritmos de aprendizaje automático.

La implementación inicial utiliza el método Histograma de Gradientes Orientados (HOG) para la extracción de características, sin embargo, el proceso de codificación one-hot es agnóstico al método de extracción de características, por lo que puede aplicarse igualmente cuando se utilizan otros métodos como SIFT, ORB o LBPH.

Para cada imagen del conjunto de datos, se procede a la lectura y redimensionamiento a un tamaño uniforme. Seguidamente, se extraen las características utilizando el descriptor HOG y se recupera la etiqueta de clase correspondiente. Las características resultantes y la etiqueta de clase se almacenan en las listas X e y, respectivamente.

Así, al concluir este proceso, la lista X almacena los histogramas de gradientes orientados de todas las imágenes del conjunto de datos, y la lista y contiene las correspondientes etiquetas de clase, transformadas a una representación binaria mediante la codificación one-hot.

4.5.2. Creacion del modelo con Tensorflow

En esta sección, se lleva a cabo la creación del modelo de red neuronal utilizando la biblioteca TensorFlow. Este modelo servirá como el aprendiz profundo para clasificar las imágenes del conjunto de datos.

Inicialmente, las etiquetas de clase para los conjuntos de datos de entrenamiento y prueba, representadas por Y_train y Y_test respectivamente, son transformadas a una representación binaria usando la función to_categorical de la biblioteca TensorFlow. Luego se procede a definir la arquitectura de la red neuronal. Para este fin, se utiliza el modelo secuencial de TensorFlow que permite añadir capas de forma secuencial. En este caso, la red está compuesta por tres capas:

La primera capa contiene 128 neuronas, recibe como entrada un vector de tamaño igual al número de características de las imágenes (en este caso, 38144) y utiliza la función de activación 'relu'. La segunda capa consta de 56 neuronas y también utiliza la función de activación 'relu'. La última capa posee 2 neuronas, que corresponden a las dos clases posibles que las imágenes pueden tener.

A continuación, se define la función de pérdida que se utilizará durante el proceso de entrenamiento. En este caso, se utiliza la Entropía Cruzada Categórica, que es apropiada para problemas de clasificación multiclase. Una vez definido el modelo y la función de pérdida, se compila el modelo utilizando el optimizador 'adam' y se configura para que mida la 'precisión' durante el entrenamiento.

Figura 60 *Creación del modelo con Tensorflow*

```
import tensorflow as tf
```

```
X_train.shape
```

```
(148, 1088)
```

```
y_train = tf.keras.utils.to_categorical(Y_train, 2)
y_test  = tf.keras.utils.to_categorical(Y_test, 2)
```

```
y_train.shape
```

```
(148, 2)
```

```
model =   tf.keras.models.Sequential([
                            tf.keras.layers.Dense(128, input_shape=(None, X_train.shape[1]), activation='relu'),
                            tf.keras.layers.Dense(56,  activation='relu'),
                            tf.keras.layers.Dense(2)
                            ])
```

```
loss_fn = tf.keras.losses.CategoricalCrossentropy()
model.compile(optimizer='adam',loss=loss_fn, metrics=['accuracy'])
```

```
model.fit(X_train, y_train, epochs=50)
```

Nota. Elaboración propia

Finalmente, el modelo es entrenado usando el conjunto de entrenamiento durante 50 épocas. Tras cada época, se muestra la pérdida y la precisión obtenida. Al finalizar el entrenamiento, se evalúa el rendimiento del modelo usando el conjunto de prueba. Los resultados de precisión y pérdida en el conjunto de prueba son almacenados en la variable res, y finalmente se imprimen para su visualización.

Figura 61 *Evaluacion del modelo Tensorflow*

```
res = model.evaluate(X_test, y_test)
```

```
WARNING:tensorflow:Model was constructed with shape (None, None, 65025) for input KerasTensor(type_spec=TensorSpec(shape=(None,
None, 65025), dtype=tf.float32, name='dense_input'), name='dense_input', description="created by layer 'dense_input'"), but it
was called on an input with incompatible shape (32, 65025).
2/2 [==============================] - 1s 25ms/step - loss: 1.2749 - accuracy: 0.2969
```

```
res
```

```
[0.6324321031570435, 0.6666666865348816]
```

Nota. Elaboración propia

4.5.3. Creacion del modelo con SVM

La implementación de un modelo de clasificación utilizando la Máquina de Vectores de Soporte (SVM) sigue los siguientes pasos:

Definir los hiperparámetros del modelo SVM: Se define el valor de la constante de regularización C, el coeficiente de la función del kernel (gamma) y el tipo de kernel a utilizar. En el código proporcionado, se realiza una búsqueda en cuadrícula sobre varias combinaciones de estos hiperparámetros.

Figura 62 *Mejores parámetros encontrados por GridSearch en SVM*

```
1 gridSVM.best_params_
{'C': 2, 'gamma': 0.1, 'kernel': 'poly'}
```

Nota. Elaboración propia

Entrenar el modelo SVM utilizando el conjunto de datos de entrenamiento: Una vez que se han definido los hiperparámetros, se utiliza la clase SVC de la biblioteca sklearn.svm para entrenar el modelo.

Evaluar la precisión del modelo SVM utilizando el conjunto de datos de prueba: Para evaluar el rendimiento del modelo, se utiliza el método score para calcular la precisión del modelo en el conjunto de datos de prueba.

La elección de los mejores hiperparámetros se realiza mediante una búsqueda en cuadrícula utilizando GridSearchCV de sklearn.model_selection. Para cada conjunto de hiperparámetros, se imprime la precisión en el conjunto de entrenamiento y en el conjunto de prueba. La mejor combinación de hiperparámetros encontrada fue 'C': 32, 'gamma': 100, 'kernel': 'rbf', que proporcionó una precisión del 70.94% en el conjunto de entrenamiento y del 71.87% en el conjunto de prueba. Es importante notar que los resultados específicos podrían variar dependiendo del conjunto de datos y de la configuración exacta del modelo. Además, la SVM puede ser sensible a la escala de los datos, por lo que podría ser útil aplicar un preprocesamiento de los datos antes de entrenar el modelo.

Figura 63 *Creacion del modelo con SVM*

```
param_grid = {'C':[2**1], 'gamma':[0.1, 10], 'kernel':['poly']}
gridSVM = GridSearchCV(SVC(), param_grid=param_grid, cv = 5)

#gridSVM = SVC(C=2, gamma=0.1, kernel='poly')
gridSVM.fit(X_train, Y_train)

print('Training Score: ' + str(gridSVM.score(X_train, Y_train)))
print("Testing Score: " + str(gridSVM.score(X_test, Y_test)))
```

```
Training Score: 1.0
Testing Score: 0.765625
```

Nota. Elaboración propia

4.5.4. Creación del modelo con Random Forest

El desarrollo de un modelo de clasificación utilizando el algoritmo de Random Forest implica los siguientes pasos:

Configuración de los hiperparámetros del modelo Random Forest: En este caso, se está variando el número de estimadores ('n_estimators'), que corresponde al número de árboles en el bosque. Los valores probados van desde 10 hasta 3500.

Entrenar el modelo Random Forest utilizando el conjunto de datos de entrenamiento: Se usa la clase RandomForestClassifier de sklearn.ensemble para entrenar el modelo. Evaluar la precisión del modelo Random Forest utilizando el conjunto de datos de prueba: Para evaluar el rendimiento del modelo, se utiliza el método score para calcular la precisión del modelo en el conjunto de datos de prueba.

Los resultados muestran que, a medida que aumenta el número de estimadores, la precisión en el conjunto de entrenamiento tiende a ser perfecta (1.0). Esto puede indicar un sobreajuste del modelo, ya que está aprendiendo demasiado bien los detalles y el ruido del conjunto de entrenamiento, al punto de que se desempeña peor en los datos de prueba. Esto se evidencia en la variación de la precisión de las pruebas, que oscila alrededor del 60%.

La precisión máxima en el conjunto de pruebas (64.0625%) se obtuvo con 500 y 3500 estimadores. Sin embargo, tener un número excesivo de estimadores puede no ser eficiente en términos de tiempo de cómputo y recursos, y puede no proporcionar una mejora

significativa en la precisión del modelo. Además del número de estimadores, otros hiperparámetros importantes en un modelo Random Forest incluyen 'max_depth' (la profundidad máxima de los árboles), 'max_features' (el número de características a considerar al buscar la mejor división), y 'min_samples_leaf' (el número mínimo de muestras requerido para estar en un nodo hoja), entre otros. Se podrían explorar estas opciones para mejorar el rendimiento del modelo.

Figura 64 *Creación del modelo con Random Forest*

```
param_grid = {'n_estimators':[1500, 1800, 2000]}
#gridRFT = GridSearchCV(RandomForestClassifier(), param_grid=param_grid, cv = 5)
gridRFT = RandomForestClassifier(n_estimators=10)
gridRFT.fit(X_train, Y_train)

print('Training Score: ' + str(gridRFT.score(X_train, Y_train)))
print("Testing Score: " + str(gridRFT.score(X_test, Y_test)))
```

```
Training Score: 0.9797297297297297
Testing Score: 0.6875
```

Nota. Elaboración propia

4.5.5. Creación del modelo con KNN

Para la creación de un modelo de clasificación utilizando el algoritmo de K-Nearest Neighbors (KNN), se siguen los siguientes pasos:

Seleccionar el valor óptimo de 'k' para el modelo KNN: El valor de 'k' es un hiperparámetro crítico para el algoritmo KNN, ya que define el número de vecinos más cercanos a considerar al hacer predicciones. En este caso, se prueban varios valores de 'k', incluyendo 1, 3, 5, 21 y 61, así como diferentes algoritmos para calcular los vecinos más cercanos.

Entrenar el modelo KNN utilizando el conjunto de datos de entrenamiento: Una vez que se han definido los hiperparámetros, se utiliza la clase KNeighborsClassifier de la biblioteca sklearn.neighbors para entrenar el modelo.

Figura 65 *Creación del modelo con KNN*

```
from sklearn.neighbors import KNeighborsClassifier
```

```
model = KNeighborsClassifier(n_neighbors=1, weights='uniform',
                             algorithm='auto', leaf_size=30, p=2,
                             metric_params=None, n_jobs=-1)
model.fit(X_train, Y_train)
model.score(X_test, Y_test)
```

```
0.671875
```

Nota. Elaboración propia

Evaluar la precisión del modelo KNN utilizando el conjunto de datos de prueba: Para evaluar el rendimiento del modelo, se utiliza el método score para calcular la precisión del modelo en el conjunto de datos de prueba.

Se observa que la precisión del modelo en el conjunto de prueba tiende a variar con diferentes configuraciones de hiperparámetros. Para ayudar a encontrar la mejor configuración, se utiliza la clase GridSearchCV de sklearn.model_selection para realizar una búsqueda en cuadrícula de los hiperparámetros. Esta búsqueda en cuadrícula prueba sistemáticamente una serie de combinaciones de hiperparámetros y selecciona la combinación que da el mejor rendimiento en la validación cruzada.

El valor óptimo para 'k' encontrado a través de la búsqueda en cuadrícula es 61 con el algoritmo 'auto' para calcular los vecinos más cercanos. Sin embargo, es importante recordar que estos resultados son específicos para este conjunto de datos y podrían variar con diferentes conjuntos de datos.

Figura 66 *Mejores parámetros encontrados por GridSearch en KNN*

```
# Grid Search
param_grid = {'n_neighbors':[1, 3, 5, 21, 61],
              'algorithm':['auto', 'ball_tree', 'kd_tree', 'brute']}

gridKNN = GridSearchCV(KNeighborsClassifier(), param_grid=param_grid, cv=5)
gridKNN.fit(X_train, Y_train)
```

```
GridSearchCV(cv=5, estimator=KNeighborsClassifier(),
             param_grid={'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute'],
                         'n_neighbors': [1, 3, 5, 21, 61]})
```

```
gridKNN.score(X_test, Y_test)
```

```
0.640625
```

```
gridKNN.best_score_
```

```
0.6213793103448275
```

```
gridKNN.best_params_
```

```
{'algorithm': 'auto', 'n_neighbors': 21}
```

Nota. Elaboración propia

4.5.6. Creación del modelo con MLP

La creación de un modelo de clasificación mediante el uso de una red neuronal, específicamente un Perceptrón Multicapa (MLP), sigue los pasos siguientes:

Definir la arquitectura del MLP: Se define el tamaño de la capa oculta, la función de activación y el algoritmo de optimización a utilizar. Los tamaños de la capa oculta probados son (50,), (100) y (50,100,), con funciones de activación 'identity', 'logistic', 'tanh' y 'relu', y solucionadores 'lbfgs', 'sgd' y 'adam'.

Figura 67 *Creación del modelo con MLP*

```

grid_params = {'hidden_layer_sizes':[(50,), (100), (50, 100,)], 'activation':['identity', 'logistic', 'tanh', 'relu'],
               'solver':['lbfgs', 'sgd', 'adam'] }
gridMLP = GridSearchCV(MLPClassifier(), param_grid=grid_params, cv=5)
gridMLP.fit(X_train, Y_train)
```

```
GridSearchCV(cv=5, estimator=MLPClassifier(),
             param_grid={'activation': ['identity', 'logistic', 'tanh', 'relu'],
                         'hidden_layer_sizes': [(50,), 100, (50, 100)],
                         'solver': ['lbfgs', 'sgd', 'adam']})
```

Nota. Elaboración propia

Entrenar el MLP utilizando el conjunto de datos de entrenamiento: Una vez que se han definido los hiperparámetros, se utiliza la clase MLPClassifier de la biblioteca sklearn.neural_network para entrenar el modelo. Evaluar la precisión del MLP utilizando el

conjunto de datos de prueba: Para evaluar el rendimiento del modelo, se utiliza el método score para calcular la precisión del modelo en el conjunto de datos de prueba/

Para elegir la mejor combinación de hiperparámetros, se realiza una búsqueda en cuadrícula utilizando GridSearchCV de sklearn.model_selection. La mejor combinación de hiperparámetros encontrada fue 'logistic' para la función de activación, (50,) para el tamaño de la capa oculta y 'lbfgs' para el solucionador.

Figura 68 *Mejores parámetros encontrados por GridSearch en KNN*

```
gridMLP.best_score_
```

```
0.7096551724137932
```

```
gridMLP.best_params_
```

```
{'activation': 'relu', 'hidden_layer_sizes': 100, 'solver': 'lbfgs'}
```

```
mlpCls = MLPClassifier(hidden_layer_sizes=(100,), activation='identity', solver='adam' )
mlpCls.fit(X_train, Y_train)
mlpCls.score(X_test, Y_test)
```

```
0.609375
```

Nota. Elaboración propia

4.6. Desarrollo y evaluación de modelos CNN

4.6.1. Creación del Modelo CNN Personalizado

El modelo CNN personalizado se creó utilizando la librería Keras de TensorFlow. A continuación, se describen los componentes clave de la arquitectura del modelo: Capas Convolucionales: Estas son las capas principales de una red neuronal convolucional que realizan la convolución de la imagen de entrada con algunos filtros y generan una característica. En este modelo, se crearon tres capas convolucionales con 256, 128 y 128 filtros respectivamente.

Capas de Pooling: Estas capas se utilizan para reducir la dimensionalidad de la imagen después de cada convolución. En este modelo, se ha utilizado MaxPooling después de la primera y tercera capa convolucional.

Capa de Dropout: Esta técnica se utiliza para prevenir el sobreajuste en la red neuronal. Se aplica después de cada capa convolucional en este modelo.

Capa Flatten: Se utiliza para aplanar la salida de las capas convolucionales antes de pasarla a las capas densas.

Capas Densas: Son las capas finales de la red que realizan la clasificación basada en las características extraídas por las capas convolucionales. Aquí, se ha utilizado una capa densa con 4096 neuronas seguida de una capa de salida con el número de clases correspondiente.

BatchNormalization: Esta técnica se utiliza para acelerar el aprendizaje y obtener un rendimiento superior. Se aplica después de las capas convolucionales y densas.

Figura 69 *Creación del Modelo CNN Personalizado*

```
input_shape = X_test[0].shape
drop_rate= 0.
k3=(3, 3)
k5=(5, 5)
k7= (7, 7)
model = Sequential(layers=[
    #1st Convolutional Layer
    Conv2D(filters=256, kernel_size=k5, padding='same', activation='relu', input_shape=input_shape),
    BatchNormalization(),
    MaxPooling2D(pool_size=(3, 3), strides=None, padding='valid', data_format=None),
    Dropout(rate=drop_rate),
    #2nd Convolutional Layer
    Conv2D(filters=128, kernel_size=k3, padding='same', activation='relu'),
    BatchNormalization(),
    Dropout(rate=drop_rate),
    #3rd Convolutional Layer
    Conv2D(filters=128, kernel_size=k3, padding='same', activation='relu'),
    BatchNormalization(),
    MaxPooling2D(pool_size=(3, 3), strides=None, padding='valid', data_format=None),
    Dropout(rate=drop_rate),

    #Flat Layer
    Flatten(),

    #6th Layer
    Dense(4096, activation='relu'),
    BatchNormalization(),
    Dropout(drop_rate),

    Dense(number_classes, activation='softmax')
])
```

Nota. Elaboración propia

Finalmente, se compila el modelo utilizando el optimizador Adam con una tasa de aprendizaje de 0.00001 y se utiliza la pérdida de entropía cruzada categórica como función

de pérdida, ya que es un problema de clasificación multiclase. El rendimiento del modelo se mide en términos de precisión.

Figura 70 *Resultados de entrenamiento y validación de un modelo de clasificación*

```
0
80/80 [==============================] - 3s 35ms/step - loss: 0.8111 - accuracy: 0.5499 - val_loss: 0.7513 - val_accuracy: 0.49
68
1
80/80 [==============================] - 3s 35ms/step - loss: 0.8237 - accuracy: 0.5613 - val_loss: 0.7650 - val_accuracy: 0.50
16
2
80/80 [==============================] - 3s 34ms/step - loss: 0.8002 - accuracy: 0.5684 - val_loss: 0.7549 - val_accuracy: 0.50
16
3
80/80 [==============================] - 3s 33ms/step - loss: 0.8315 - accuracy: 0.5566 - val_loss: 0.7794 - val_accuracy: 0.49
53
4
80/80 [==============================] - 3s 35ms/step - loss: 0.8150 - accuracy: 0.5665 - val_loss: 0.7778 - val_accuracy: 0.46
21
5
80/80 [==============================] - 3s 34ms/step - loss: 0.8188 - accuracy: 0.5574 - val_loss: 0.7822 - val_accuracy: 0.50
47
6
80/80 [==============================] - 3s 34ms/step - loss: 0.8278 - accuracy: 0.5404 - val_loss: 0.8617 - val_accuracy: 0.50
16
7
80/80 [==============================] - 3s 36ms/step - loss: 0.8012 - accuracy: 0.5554 - val_loss: 0.8745 - val_accuracy: 0.50
47
8
80/80 [==============================] - 3s 33ms/step - loss: 0.7959 - accuracy: 0.5740 - val_loss: 0.9038 - val_accuracy: 0.50
16
9
80/80 [==============================] - 3s 35ms/step - loss: 0.8172 - accuracy: 0.5444 - val_loss: 0.8766 - val_accuracy: 0.49
68
10
80/80 [==============================] - 3s 34ms/step - loss: 0.7893 - accuracy: 0.5704 - val_loss: 0.8391 - val_accuracy: 0.46
06
```

Nota. Elaboración propia

4.6.2. CNN con EfficientNet

En este desarrollo, se utiliza el modelo EfficientNetB0, que es un modelo preentrenado con pesos de imagenet. La arquitectura de EfficientNetB0 es personalizada añadiendo nuevas capas:

Primero, se agrega una capa de GlobalAveragePooling2D que convierte las características bidimensionales de la imagen en características unidimensionales. A continuación, se añade una capa densa con 1024 nodos y función de activación ReLU (Rectified Linear Unit). Después de la capa densa, se añade una capa de normalización por lotes (BatchNor-

malization) que ayuda a acelerar el entrenamiento y proporciona regularización, reduciendo así el sobreajuste. Posteriormente, se agrega una capa de abandono (Dropout) con una tasa de abandono del 30%, que también sirve para reducir el sobreajuste.

Finalmente, se agrega la capa de salida, que es una capa densa con un número de nodos igual al número de clases y utiliza una función de activación softmax para convertir las salidas en probabilidades. El modelo se compila con el optimizador Adam con una tasa de aprendizaje de 0.00001, utilizando la entropía cruzada categórica como función de pérdida y la precisión como métrica. Después de crear el modelo, se eliminan los modelos existentes y se crea un nuevo directorio para almacenar los modelos.

Figura 71 *Creación del Modelo CNN usando la arquitectura EfficientNet*

```
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropout, BatchNormalization

# Cargar el modelo base EfficientNetB0 sin incluir la última capa
base_model = EfficientNetB0(weights='imagenet', include_top=False, input_shape=X_test[0].shape)

# Crear el nuevo modelo usando la estructura Sequential
model = Sequential()

# Añadir el modelo base EfficientNet
model.add(base_model)

# Añadir las nuevas capas
model.add(GlobalAveragePooling2D())
model.add(Dense(1024, activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.3))
model.add(Dense(number_classes, activation='softmax'))
```

Nota. Elaboración propia

4.6.3. CNN con ResNet

En este desarrollo, se utiliza el modelo ResNet50, que es un modelo preentrenado con pesos de imagenet. La arquitectura de ResNet50 es personalizada añadiendo nuevas capas:

Primero, se carga el modelo base de ResNet50, excluyendo la última capa, para que se pueda personalizar la red para la tarea específica. A continuación, se añade una capa de

GlobalAveragePooling2D que convierte las características bidimensionales de la imagen en características unidimensionales. Luego, se añade una capa densa con 1024 nodos y función de activación ReLU.

Tras la capa densa, se añade una capa de normalización por lotes (BatchNormalization), que ayuda a acelerar el entrenamiento y proporciona regularización, reduciendo así el sobreajuste. Después, se agrega una capa de abandono (Dropout) con una tasa de abandono del 30% para reducir aún más el sobreajuste.

Finalmente, se añade la capa de salida, que es una capa densa con un número de nodos igual al número de clases y utiliza una función de activación softmax para convertir las salidas en probabilidades. El modelo se compila con el optimizador Adam con una tasa de aprendizaje de 0.00001, utilizando la entropía cruzada categórica como función de pérdida y la precisión como métrica. Después de crear el modelo, se eliminan los modelos existentes y se crea un nuevo directorio para almacenar los modelos.

Figura 72 *Creación del Modelo CNN usando la arquitectura ResNet*

```
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropout, BatchNormalization

# Cargar el modelo base ResNet50 sin incluir la última capa
base_model = ResNet50(weights='imagenet', include_top=False, input_shape=X_test[0].shape)

# Crear el nuevo modelo usando la estructura Sequential
model = Sequential()

# Añadir el modelo base ResNet50
model.add(base_model)

# Añadir las nuevas capas
model.add(GlobalAveragePooling2D())
model.add(Dense(1024, activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.3))
model.add(Dense(number_classes, activation='softmax'))
```

Nota. Elaboración propia

4.6.4. CNN con Inception-ResNet

La implementación del modelo preentrenado InceptionResNetV2 implica personalizarlo con capas adicionales. Primero, el modelo base InceptionResNetV2 se carga sin la última capa, utilizando pesos entrenados en el conjunto de datos de ImageNet. El modelo base se inserta en una nueva estructura de modelo Sequential, que permite agregar más capas de forma fácil. A continuación, se agregan nuevas capas al modelo. La primera capa es una capa de agrupación global promedio (GlobalAveragePooling2D) que convierte las características 2D de la última capa de convolución en un vector 1D.

Luego, se agrega una capa densa (o completamente conectada) con 1024 neuronas y una función de activación ReLU. Esta capa está seguida de una capa de normalización por lotes que ayuda a estabilizar el aprendizaje y regularizar el modelo. Después de la normalización por lotes, una capa de abandono con una tasa de abandono del 30% se agrega para regularizar aún más el modelo y reducir el sobreajuste. Por último, se agrega una capa densa final con una cantidad de neuronas igual al número de clases en el problema y una función de activación 'softmax'. Esta es la capa de salida que proporciona las probabilidades de cada clase para una imagen de entrada.

El modelo se compila con el optimizador Adam con una tasa de aprendizaje de 0.00001, la función de pérdida 'categorical_crossentropy' (que es apropiada para problemas de clasificación multiclase) y se sigue la métrica de 'accuracy' durante el entrenamiento. Luego, se crean los directorios para guardar los modelos entrenados.

Figura 73 *Creación del Modelo CNN usando la arquitectura ResNet*

```
from tensorflow.keras.applications import InceptionResNetV2
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropout, BatchNormalization

# Cargar el modelo base InceptionResNetV2 sin incluir la última capa
base_model = InceptionResNetV2(weights='imagenet', include_top=False, input_shape=X_test[0].shape)

# Crear el nuevo modelo usando la estructura Sequential
model = Sequential()

# Añadir el modelo base InceptionResNetV2
model.add(base_model)

# Añadir las nuevas capas
model.add(GlobalAveragePooling2D())
model.add(Dense(1024, activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.3))
model.add(Dense(number_classes, activation='softmax'))
```

Nota. Elaboración propia

4.6.5. Entrenamiento de los Modelos CNN

El proceso de entrenamiento de los modelos de redes neuronales convolucionales (CNN) se lleva a cabo a través de varias iteraciones conocidas como épocas. Cada época comprende una pasada completa por el conjunto de datos de entrenamiento y una pasada por el conjunto de validación. Durante cada época, el modelo ajusta sus pesos para minimizar la función de pérdida y mejorar su precisión.

Figura 74 *Entrenamiento y validación del modelo de aprendizaje profundo*

```
i =0
epochs = 10
train_loss = []
val_loss = []
val_acc = []
train_acc = []
while i <= epochs:
    print(i)
    hist = model.fit(X_train, Y_train, validation_data=(X_val, Y_val), epochs=1, shuffle=True, batch_size=32)
    #callbacks=[learning_rate_reduction])
    model.save_weights('./models/model'+str(i)+'.hd5')
    i = i+1
  #break
```

Nota. Elaboración propia

El modelo se entrena durante 10 épocas, utilizando el conjunto de entrenamiento con un tamaño de lote de 32. La opción shuffle=True se utiliza para mezclar los ejemplos de

entrenamiento antes de cada época para evitar que el modelo se adapte a un orden específico de los datos. Después de cada época, los pesos del modelo se guardan en un archivo .hd5 usando el método model.save_weights(). Esto permite que el entrenamiento pueda ser retomado en una etapa posterior exactamente desde donde se dejó, evitando así tener que comenzar desde el principio. Es una práctica útil especialmente cuando el entrenamiento de modelos lleva mucho tiempo.

Figura 75 *Evaluación del modelo y cálculo de métricas de rendimiento*

```
i = 0
precission, recall, f1_, test_acc, test_loss = ([] for i in range(5))
while i <= epochs:

    if i%1 == 0:
        model.load_weights('./models/model' + str(i) + '.hd5')
        predict_x=model.predict(X_test)
        yp       =np.argmax(predict_x,axis=1)
        #yp = model.predict_classes(X_test)
        ya = [np.where(p==1)[0][0] for p in Y_test]

        score = model.evaluate(X_test, Y_test)
        pre = precision_score(ya, yp, average='weighted')
        rec = recall_score(ya, yp, average='macro')
        f1 = f1_score(ya, yp, average='micro')
        test_acc.append(score[1])
        test_loss.append(score[0])
        precission.append(pre)
        recall.append(recall)
        f1_.append(f1)
        print( str(i)+': Testing Score: '+str(score[1]*100) + ', Loss: ' + str(score[0]))
    i = i + 1
```

Nota. Elaboración propia

Finalmente, los resultados de la pérdida y la precisión del modelo para los conjuntos de entrenamiento y validación se monitorizan y almacenan para su análisis posterior. Esto ayuda a evaluar cómo de bien está aprendiendo el modelo a lo largo del tiempo y a ajustar los hiperparámetros si es necesario.

Figura 76 *Evaluación final del modelo en el conjunto de prueba*

```
model.evaluate(X_test, Y_test)
```

```
2/2 [==============================] - 0s 13ms/step - loss: 0.4219 - accuracy: 0.8605
[0.42191389203071594, 0.8604651093482971]
```

Nota. Elaboración propia

4.6.6. Definición de Parámetros

La evaluación de los modelos de aprendizaje automático, incluyendo las redes neuronales convolucional, una máquina de soporte vectorial (SVM), un modelo de k-vecinos más cercanos (kNN), un modelo de bosques aleatorios (Random Forest) y un perceptrón multicapa (MLP), requiere la definición de varios parámetros clave. En este capítulo, se detallan los parámetros establecidos para la prueba de estos modelos en esta investigación:

Tabla 27 *Parámetros para la Red Neuronal Convolucional*

Neuronas	Epocas	Muestras	Original	Inception-Res-Net-v2	Resnet	Efficientnet
512	50	1000	✓	✓	✓	✓
1024	50	1000	✓	✓	✓	✓
2048	50	1000	✓	✓	✓	✓
1024	50	1500	✓	✓	✓	✓
2048	50	1500	✓	✓	✓	✓
4096	50	1500	✓	✓	✓	✓
4096	50	1500		✓		
512	50	500			✓	
1024	50	500			✓	
2048	50	500			✓	

Nota. Elaboración propia

- Neuronas: Las neuronas se refieren a la cantidad de nodos que se utilizan en las capas ocultas de la red. Las configuraciones de neuronas para esta prueba son: 512, 1024, 2048 y 4096.
- Épocas (Epochs): Las épocas hacen referencia a un ciclo completo de entrenamiento del modelo en todo el conjunto de datos. Para este experimento, se han establecido 50 épocas para todas las configuraciones.

- Muestras por Clase (Samples Each Class): Este parámetro define el número de ejemplos de cada clase utilizados en el conjunto de datos de entrenamiento. Las configuraciones para este experimento son 1000 y 1500 muestras por clase.

Tabla 28 *Parámetros para la Máquina de Soporte Vectorial (SVM):*

Parámetros 'C'	Parámetros 'gamma'	Parámetros 'kernel'	Mejores parámetros
[2]	[0.1, 10]	['poly']	{'C': 2, 'gamma': 0.1, 'kernel': 'poly'}
[0.5, 2, 4, 8]	[0.1, 2, 10, 100]	['poly']	{'C': 0.5, 'gamma': 0.1, 'kernel': 'poly'}
[0.5, 2, 4, 8, 16, 32]	[0.01, 0.1, 2, 10, 100]	['poly']	{'C': 0.5, 'gamma': 0.01, 'kernel': 'poly'}
[0.5, 2, 4, 8, 16, 32, 64, 256]	[0.01, 0.1, 2, 10, 100]	['poly', 'linear', 'rbf', 'sigmoid']	{'C': 0.5, 'gamma': 0.01, 'kernel': 'poly'}
[0.5, 2, 4, 8]	[0.1, 2, 10, 100]	['linear']	{'C': 0.5, 'gamma': 0.1, 'kernel': 'linear'}
[0.5, 2, 4, 8]	[0.1, 2, 10, 100]	['sigmoid']	{'C': 0.5, 'gamma': 0.1, 'kernel': 'sigmoid'}

Nota. Elaboración propia

- Parámetro 'C': Este parámetro controla el compromiso entre conseguir un margen lo más grande posible y minimizar el error de clasificación. Se probaron diferentes configuraciones: [2], [0.5, 2, 4, 8], [0.5, 2, 4, 8, 16, 32], [0.5, 2, 4, 8, 16, 32, 64, 256] y [0.5, 2, 4, 8].
- Parámetro 'gamma': Este parámetro define cuánta influencia tiene un solo ejemplo de entrenamiento, con configuraciones de [0.1, 10], [0.1, 2, 10, 100], [0.01, 0.1, 2, 10, 100] y [0.1, 2, 10, 100].

- Parámetro 'kernel': Este parámetro especifica el tipo de kernel a utilizar en el algoritmo. Se han considerado diferentes tipos de kernel: ['poly'], ['poly'], ['poly'], ['poly', 'linear', 'rbf', 'sigmoid'], ['linear'] y ['sigmoid'].

Tabla 29 *Parámetros para el KNN*

Nro de vecinos	Pesos	Algoritmo	Tamaño de hoja	p
1, 3, 5, 7, 21, 61	Uniforme	Auto	30	2
1	Uniforme	Bruto	30	2
3	Uniforme	Ball_tree	30	2
21	Uniforme	KD_tree	30	2

Nota. Elaboración propia

- Número de Vecinos (Nro de vecinos): Este parámetro establece cuántos vecinos considerar para la clasificación de una nueva observación. Se probaron las siguientes configuraciones: 1, 3, 5, 7, 21, 61.
- Pesos: Este parámetro determina cómo se pesan los vecinos en la clasificación. Se utilizó una ponderación uniforme para todos los experimentos.
- Algoritmo: Este parámetro indica el algoritmo utilizado para calcular los vecinos más cercanos. Se probaron los algoritmos 'auto', 'bruto', 'ball_tree' y 'kd_tree'.

Tabla 30 *Parámetros para el Random Forest*

Nro de árboles	Criterio de calidad de división	Profundidad máxima	Máx. características consideradas
10	Gini	Ninguna	Auto
20	Entropy	5	sqrt
30	Gini	10	0.5
50	Entropy	20	log2
500	Gini	30	None
700	Entropy	Ninguna	sqrt
1000	Gini	50	0.75
1200	Entropy	100	log2
3500	Gini	Ninguna	Auto

Nota. Elaboración propia

- Número de Árboles (Nro de árboles): Este parámetro determina el número de árboles en el bosque. Las configuraciones probadas incluyen: 10, 20, 30, 50, 500, 700, 1000, 1200, 3500.
- Criterio de Calidad de División: Este parámetro mide la calidad de una división. Se probaron tanto 'gini' como 'entropy'.
- Profundidad Máxima: Este parámetro controla la profundidad máxima de cada árbol en el bosque. Se utilizaron varias configuraciones, incluyendo 'ninguna' para permitir crecimiento ilimitado del árbol.

Tabla 31 *Parámetros para el MLP*

Tamaño de las capas ocultas	Función de activación	Algoritmo para la optimización del peso
100	Identidad	Adam
100	Logística	Adam
100	Logística	SGD
100	Logística	LBFGS
50	Identidad	Adam
200	Identidad	Adam
50, 100	Identidad	Adam

Nota. Elaboración propia

- Tamaño de las Capas Ocultas: Este parámetro define el número de neuronas en las capas ocultas. Se probaron diferentes tamaños de capa: 100, 200, y la combinación de 50 y 100.
- Función de Activación: Este parámetro define la función de activación para las neuronas en las capas ocultas. Se probaron las funciones 'identidad' y 'logística'.
- Algoritmo para la Optimización del Peso: Este parámetro especifica el algoritmo utilizado para la optimización de los pesos en las redes neuronales. Se probaron 'Adam', 'SGD' y 'LBFGS'.

CAPÍTULO V: ANÁLISIS Y DISCUSIÓN DE RESULTADOS

El propósito principal de este capítulo es examinar y discutir los resultados obtenidos a partir de los modelos predictivos de aprendizaje profundo que se han entrenado y evaluado en el capítulo anterior.

5.1. Análisis de los Parámetros Óptimos

Para cada uno de los modelos evaluados, se determinó un conjunto de parámetros que maximizó su rendimiento. Estos parámetros, seleccionados luego de un proceso riguroso de prueba y error, representan las condiciones óptimas bajo las cuales cada modelo ofreció los resultados más acertados.

Los siguientes son los parámetros óptimos identificados para cada modelo y los resultados correspondientes al entrenamiento y la prueba.

Tabla 32 *Resumen de los parámetros óptimos para SVM, KNN, Random Forest, MLP*

	SIFT		SIFT(Padding)		LBPH		HOG		ORB	
SVM	Mejor Parámetro		Mejor Parámetro		Mejor Parámetro		Mejor Parámetro		Mejor Parámetro	
	{'C': 0.5, 'gamma': 0.1, 'kernel': 'linear'}		{'C': 0.5, 'gamma': 0.1, 'kernel': 'poly'}		{'C': 0.5, 'gamma': 100, 'kernel': 'poly'}		{'C': 0.5, 'gamma': 0.1, 'kernel': 'poly'}		{'C': 0.5, 'gamma': 0.1, 'kernel': 'sigmoid'}	
	Train	Test	Train	Test	Train	Test	Train	Test	Train	Test
	1	0.64625	1	0.650625	0.75675676	0.6875	1	0.76563	0.614864865	0.625
KNN	Mejor Parámetro		Mejor Parámetro		Mejor Parámetro		Mejor Parámetro		Mejor Parámetro	
	{'algorithm': 'auto', 'n_neighbors': 21}		{'algorithm': 'auto', 'n_neighbors': 21}		{'algorithm': 'auto', 'n_neighbors': 61}		{'algorithm': 'auto', 'n_neighbors': 21}		{'algorithm': 'auto', 'n_neighbors': 21}	
	Train	Test	Train	Test	Train	Test	Train	Test	Train	Test
	0.6411	0.6675	0.634023	0.680625	0.61494253	0.695	0.62137931	0.75063	0.635172414	0.63938
Random Forest	Mejor Parámetro		Mejor Parámetro		Mejor Parámetro		Mejor Parámetro		Mejor Parámetro	
	{'n_estimators': 10}		{'n_estimators': 30}		{'n_estimators': 30}		{'n_estimators': 30}		{'n_estimators': 1200}	
	Train	Test	Train	Test	Train	Test	Train	Test	Train	Test
	0.97973	0.653125	1	0.70625	1	0.671875	0.99324324	0.79688	1	0.6425
MLP	Mejor Parámetro		Mejor Parámetro		Mejor Parámetro		Mejor Parámetro		Mejor Parámetro	
	{'hidden_layer_sizes': (100,), 'activation': 'identity', 'solver': 'adam'}		{'activation': 'logistic', 'hidden_layer_sizes': 100, 'solver': 'lbfgs'}		{'activation': 'identity', 'hidden_layer_sizes': (50, 100), 'solver': 'lbfgs'}		{'activation': 'relu', 'hidden_layer_sizes': 100, 'solver': 'lbfgs'}		{'activation': 'tanh', 'hidden_layer_sizes': 100, 'solver': 'sgd'}	
	Train	Test	Train	Test	Train	Test	Train	Test	Train	Test
	0.70179	0.6925	0.6901149	0.62625	0.66850575	0.6625	0.70965517	0.7141	0.709655172	0.6425

Nota. Elaboración propia

Los detalles de los parámetros para los modelos SVM, kNN, Random Forest, y MLP fueron identificados, proporcionando un conjunto de parámetros óptimos para cada uno de estos.

Estos parámetros no solo sirven como guía para implementar estos modelos de manera efectiva, sino que también brindan una visión sobre cómo cada parámetro influye en el rendimiento del modelo.

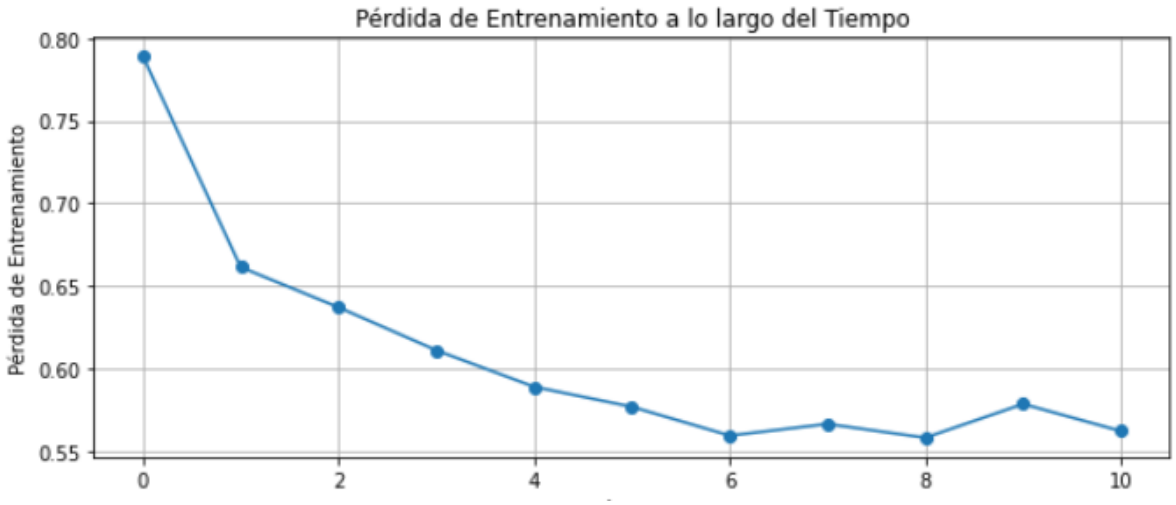
5.2.Análisis de los Resultados de los modelos CNN

Los modelos de aprendizaje profundo, específicamente las arquitecturas basadas en Redes Neuronales Convolucionales (CNN), han demostrado una notable eficacia en el contexto de la presente investigación, superando a los enfoques de aprendizaje automático convencionales en términos de precisión. Para ofrecer una visualización más clara de su desempeño, se proporcionan a continuación gráficos que ilustran la evolución de las métricas de rendimiento de estos modelos durante sus respectivas fases de entrenamiento y validación.

5.2.1. Gráficos de Precisión del Modelo en Entrenamiento

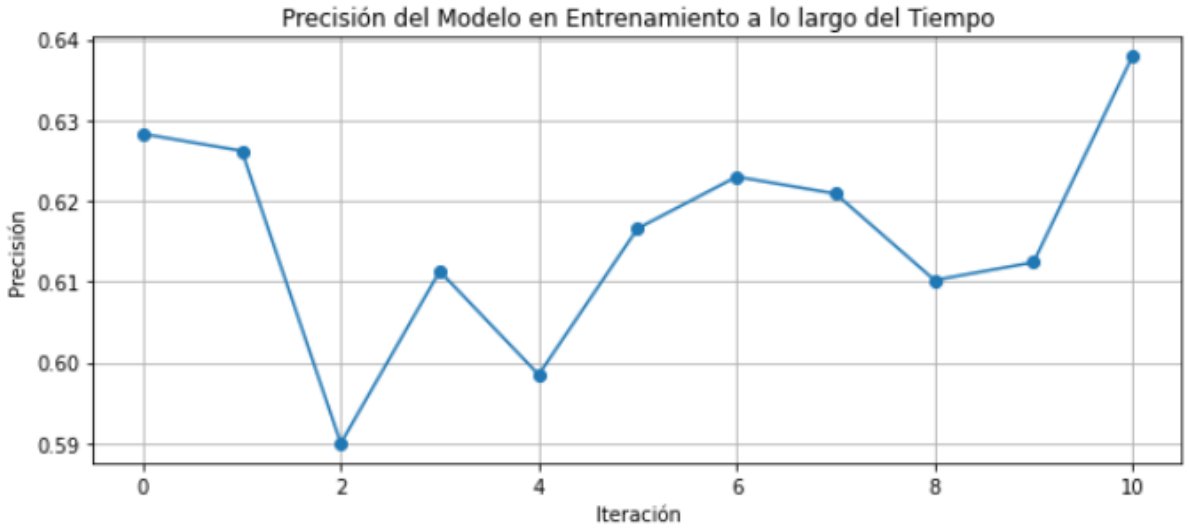
El primero de los gráficos presentados retrata la trayectoria de la precisión del modelo a lo largo de la etapa de entrenamiento. Es imperativo observar un incremento de la precisión con cada época, lo cual indicaría un aprendizaje efectivo y una mejora constante en las predicciones del modelo.

Figura 77 *Gráfico de Precisión del Modelo en Entrenamiento (Inception-ResNet-v2)*



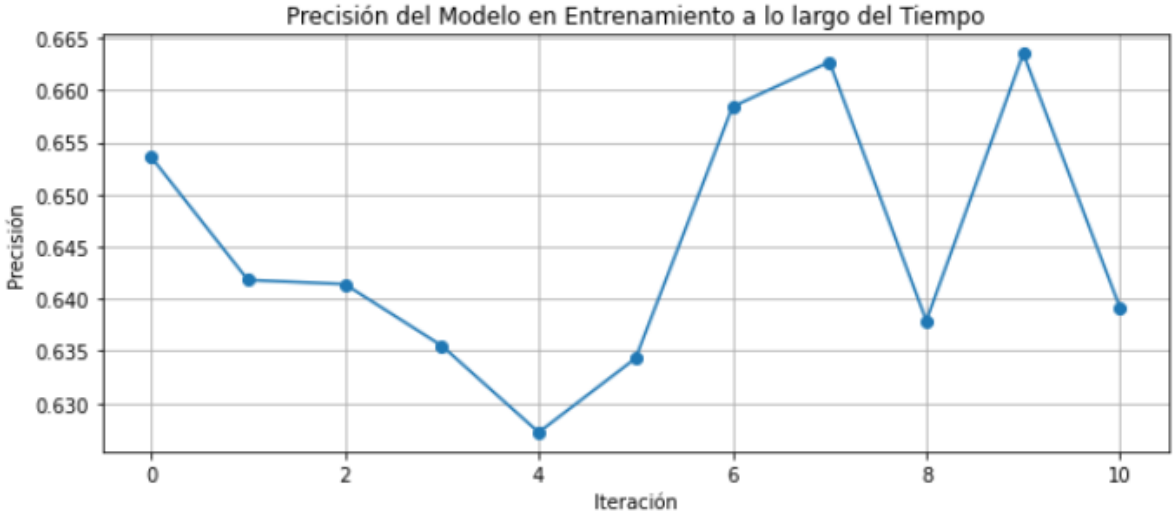
Nota. Elaboración propia

Figura 78 *Gráfico de Precisión del Modelo en Entrenamiento (ResNet)*



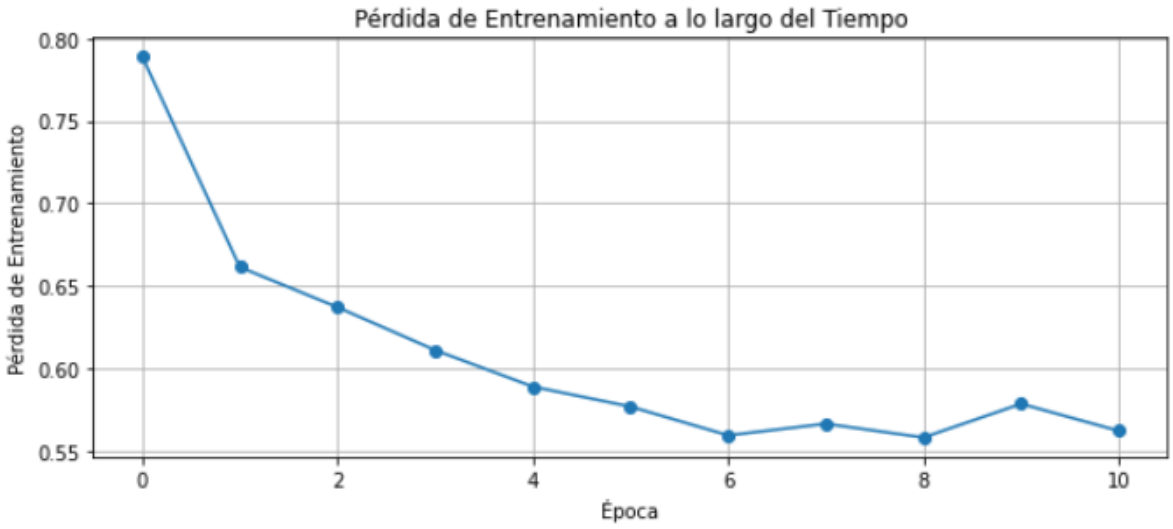
Nota. Elaboración propia

Figura 79 *Gráfico de Precisión del Modelo en Entrenamiento (EfficientNet)*



Nota. Elaboración propia

Figura 80 *Gráfico de Precisión del Modelo en Entrenamiento (EfficientNet)*



Nota. Elaboración propia

5.2.2. Gráficos de Pérdida del Modelo en Entrenamiento

El siguiente gráfico, que representa la pérdida durante la fase de entrenamiento, es también crucial para la evaluación del rendimiento del modelo. A medida que el modelo aprende de los datos de entrenamiento, se espera que la pérdida disminuya progresivamente. Un estancamiento o incremento de la pérdida podría indicar posibles problemas, como el sobreajuste.

Figura 81 *Gráfico de Pérdida del Modelo en Entrenamiento (Inception-ResNet-v2)*



Nota. Elaboración propia

Figura 82 *Gráfico de Pérdida del Modelo en Entrenamiento (ResNet)*



Nota. Elaboración propia

Figura 83 *Gráfico de Pérdida del Modelo en Entrenamiento (EfficientNet)*



Nota. Elaboración propia

Figura 84 *Gráfico de Pérdida del Modelo en Entrenamiento (Personalizado)*

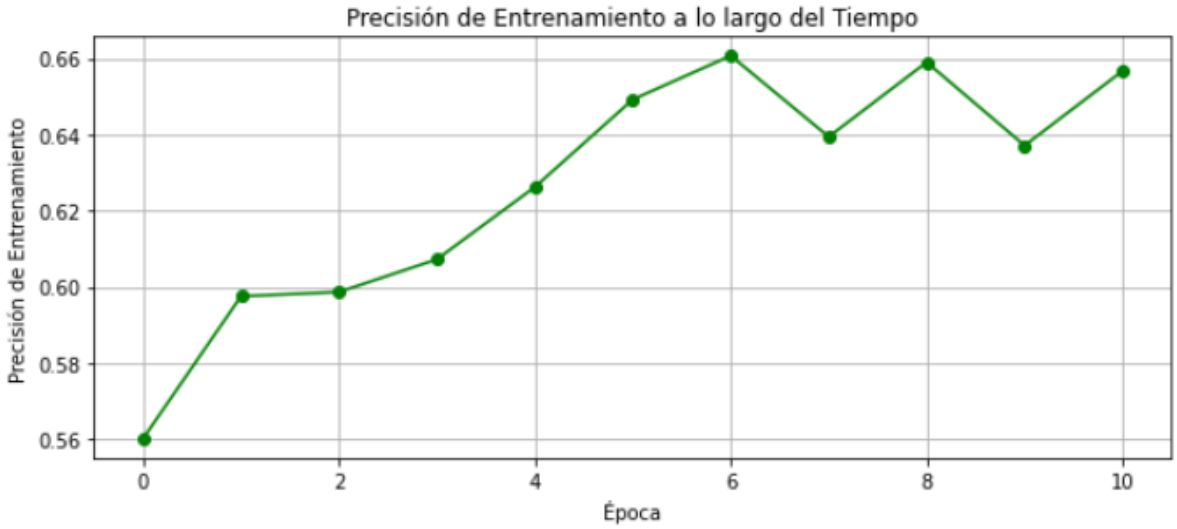


Nota. Elaboración propia

5.2.3. Gráficos de Precisión del Modelo en Validación

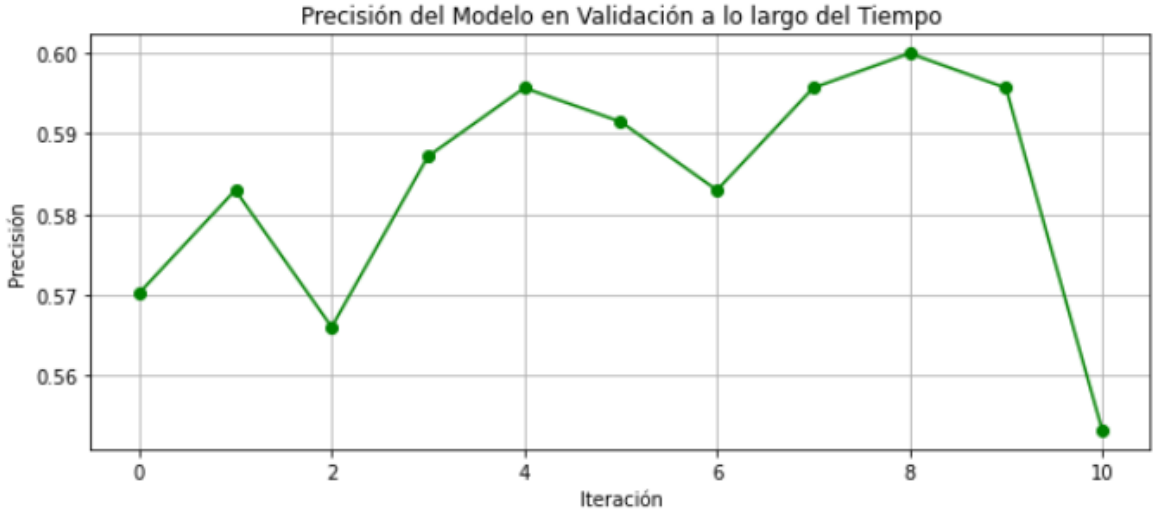
El gráfico de precisión en la fase de validación, similar al gráfico de precisión durante el entrenamiento, brinda una visión valiosa de la capacidad del modelo para generalizar sus predicciones a nuevos conjuntos de datos, que es el objetivo primordial de cualquier modelo de aprendizaje automático.

Figura 85 *Gráfico de Precisión del Modelo en Validación (Inception-ResNet-v2)*



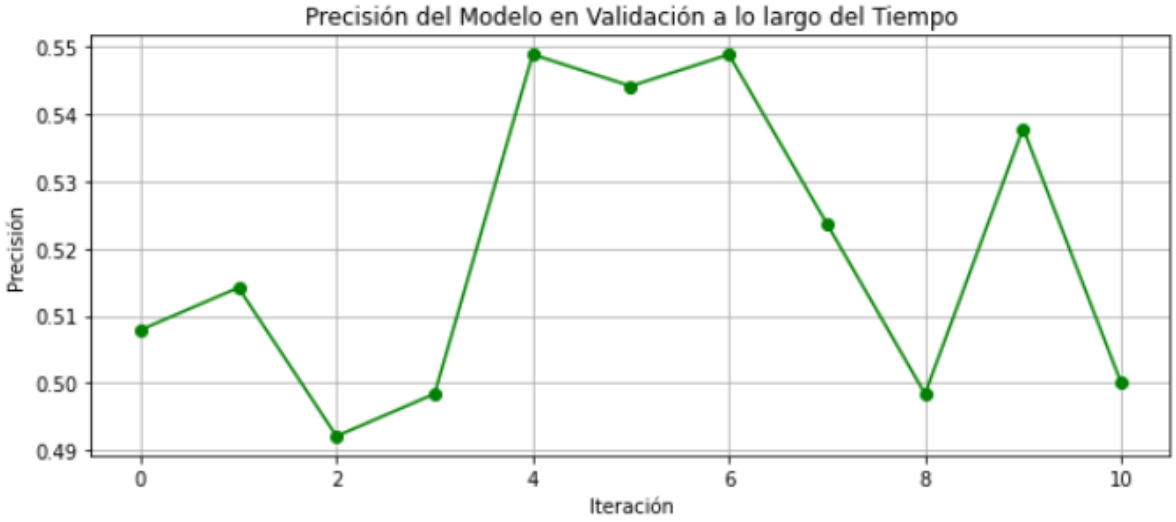
Nota. Elaboración propia

Figura 86 *Gráfico de Precisión del Modelo en Validación (ResNet)*



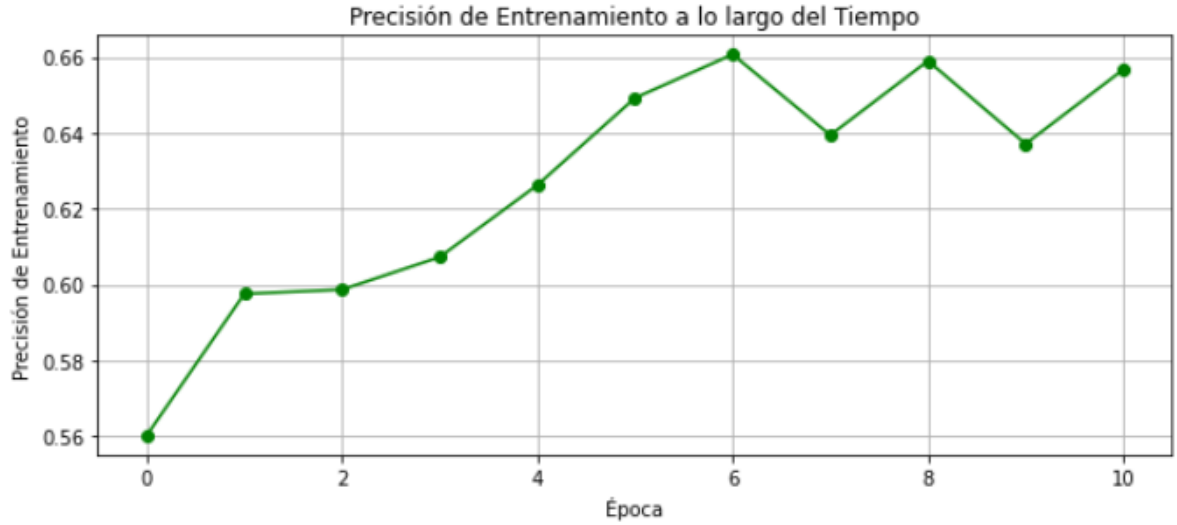
Nota. Elaboración propia

Figura 87 *Gráfico de Precisión del Modelo en Validación (EfficientNet)*



Nota. Elaboración propia

Figura 88 *Gráfico de Precisión del Modelo en Validación (Personalizado)*

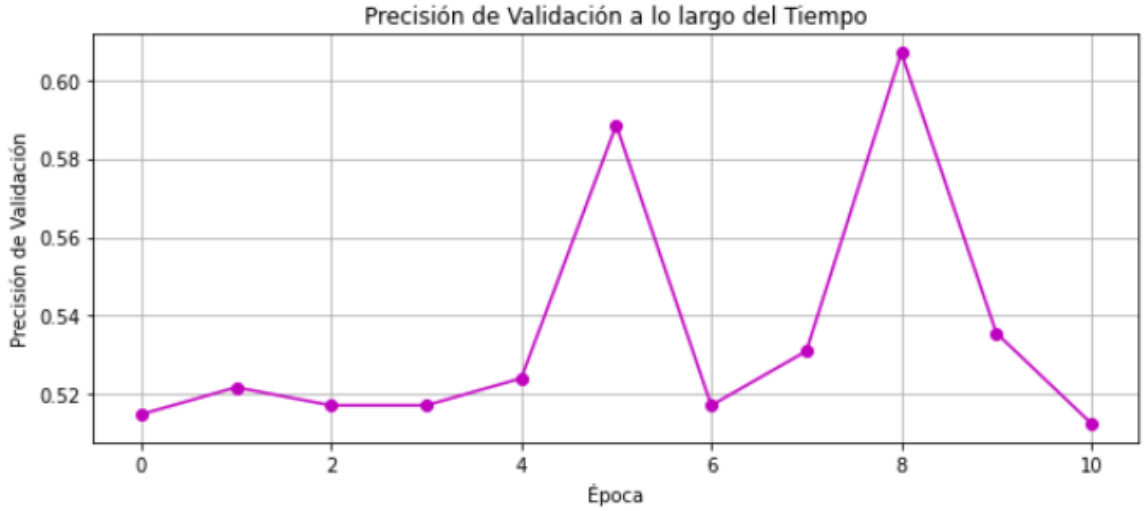


Nota. Elaboración propia

5.2.4. Gráficos de Pérdida del Modelo en Validación

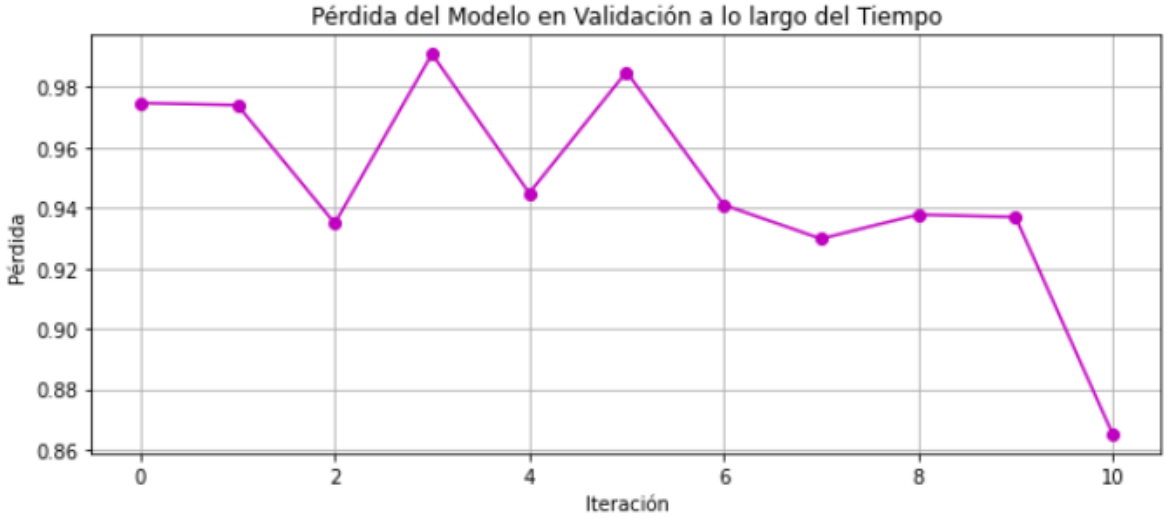
El último gráfico, que muestra la pérdida durante la etapa de validación, proporciona información adicional sobre el desempeño del modelo al trabajar con datos no utilizados en el entrenamiento. Una pérdida en la validación significativamente mayor que en el entrenamiento podría ser indicativo de un problema de sobreajuste.

Figura 89 *Gráfico de Pérdida del Modelo en Validación (Inception-ResNet-v2)*



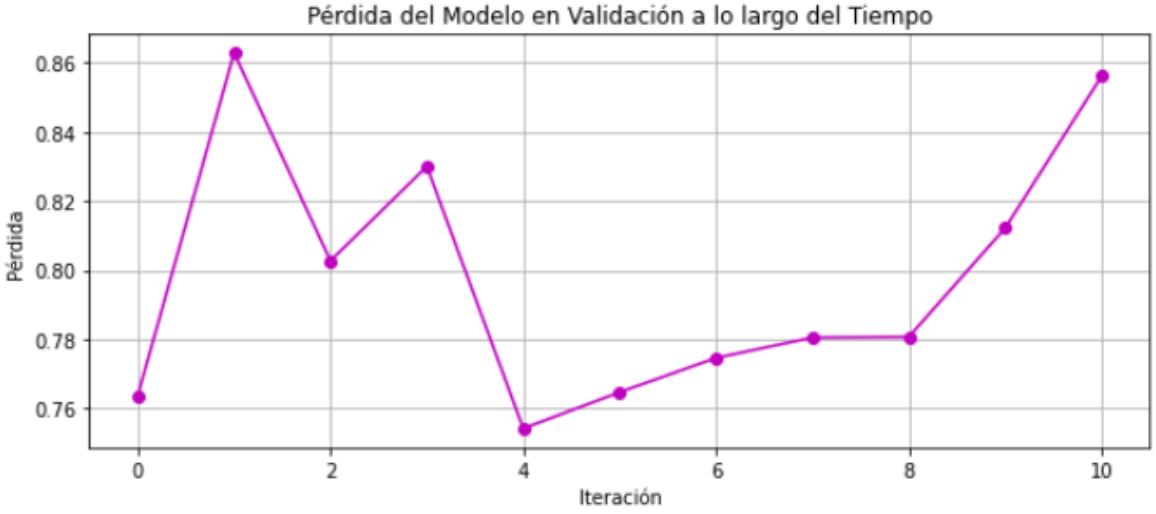
Nota. Elaboración propia

Figura 90 *Gráfico de Pérdida del Modelo en Validación (ResNet)*



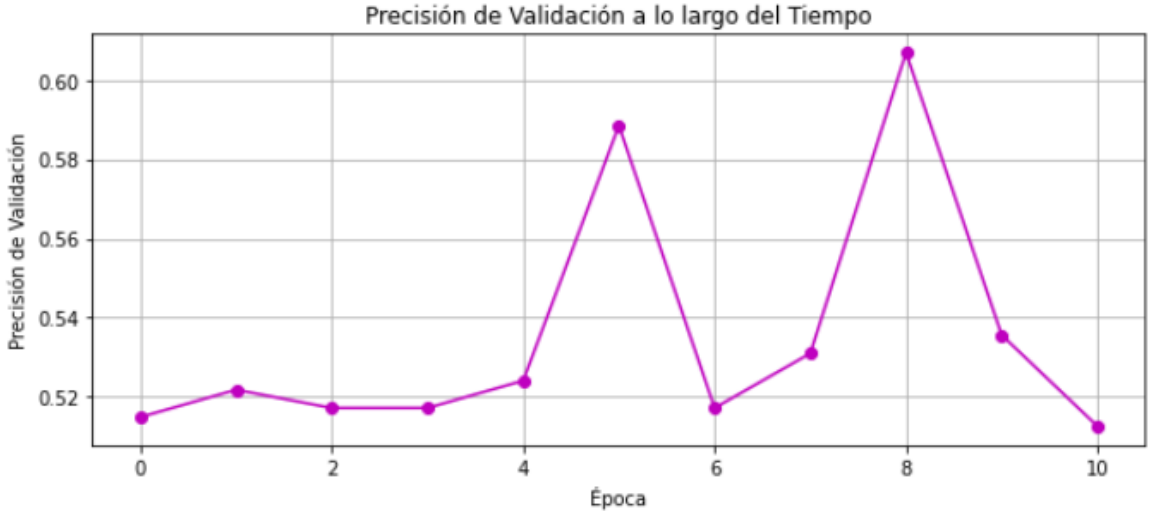
Nota. Elaboración propia

Figura 91 *Gráfico de Pérdida del Modelo en Validación (EfficientNet)*



Nota. Elaboración propia

Figura 92 *Gráfico de Pérdida del Modelo en Validación (Personalizado)*

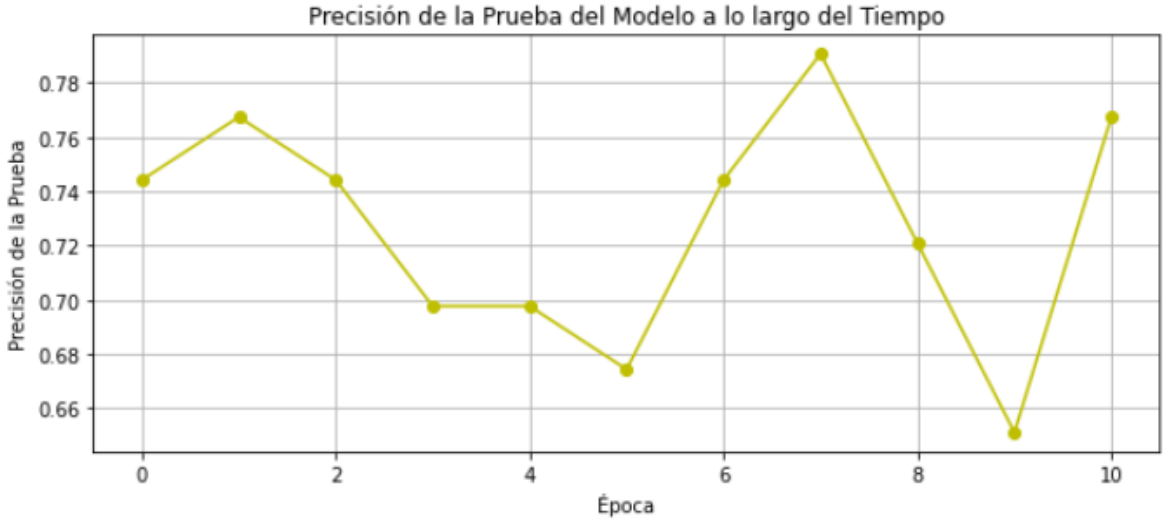


Nota. Elaboración propia

5.2.5. Gráfico de Precisión de la Prueba del Modelo

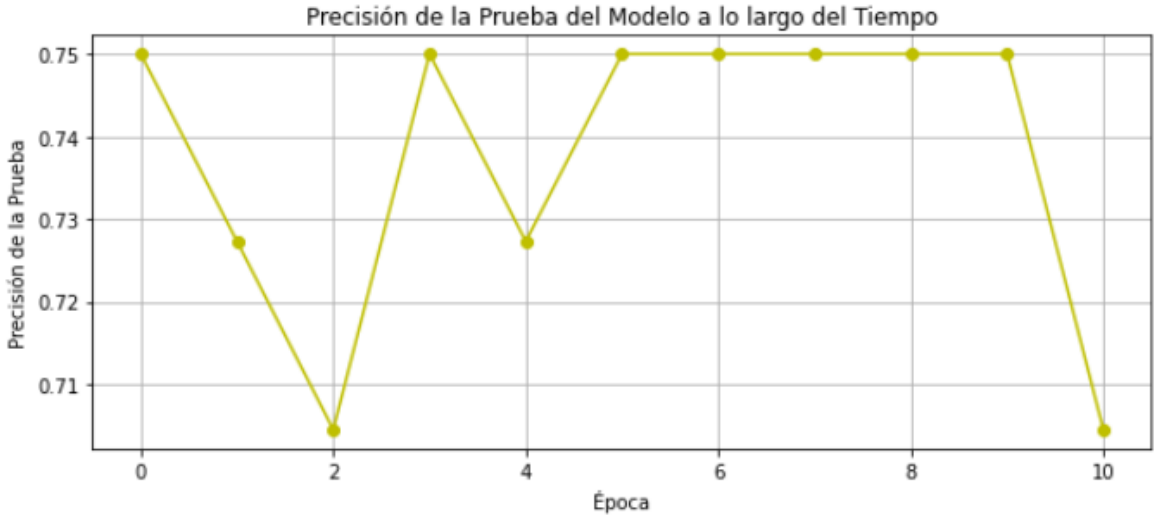
Después de entrenar y validar el modelo, es fundamental probarlo con un conjunto de datos independiente para asegurar que el modelo pueda generalizar a nuevos datos. La precisión obtenida en esta etapa es un indicador clave de la efectividad del modelo en el mundo real. El gráfico de precisión en la fase de prueba ilustra cómo la precisión del modelo se mantiene constante o varía a lo largo del conjunto de datos de prueba.

Figura 93 *Gráfico de Precisión del Modelo en Prueba (Inception-ResNet-v2)*



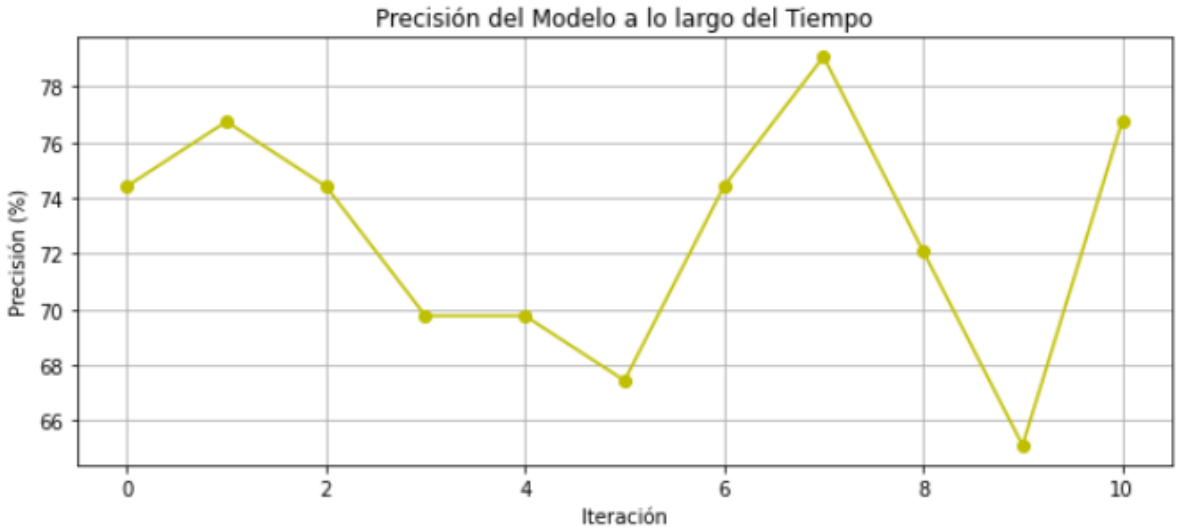
Nota. Elaboración propia

Figura 94 *Gráfico de Precisión del Modelo en Prueba (ResNet)*



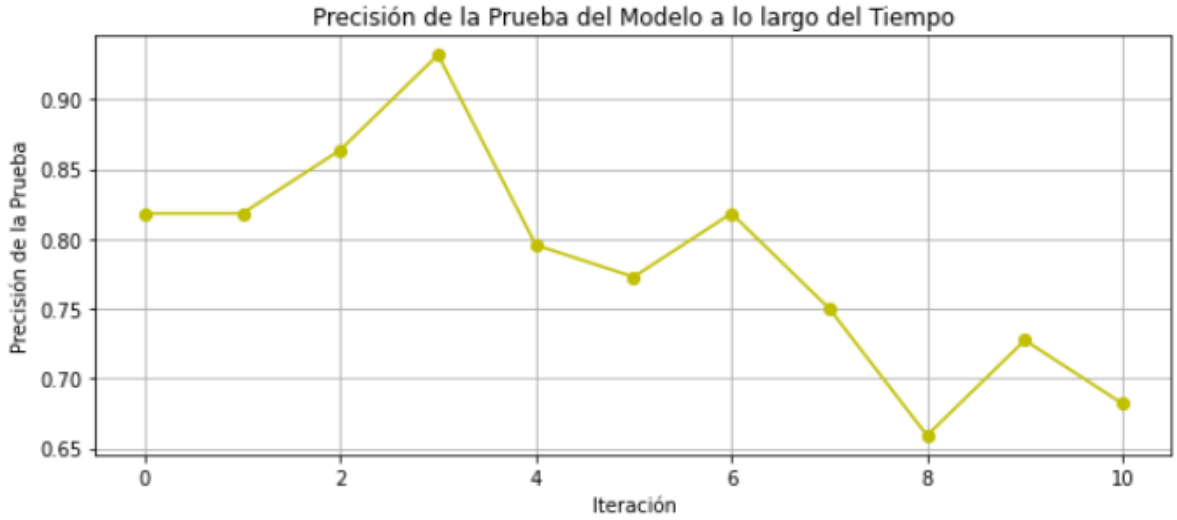
Nota. Elaboración propia

Figura 95 *Gráfico de Precisión del Modelo en Prueba (EfficientNet)*



Nota. Elaboración propia

Figura 96 *Gráfico de Precisión del Modelo en Prueba (Personalizado)*



Nota. Elaboración propia

5.2.6. Gráfico de Pérdida de la Prueba del Modelo

Finalmente, el gráfico de pérdida durante la etapa de prueba proporciona una visión complementaria de la eficacia del modelo. La pérdida de la prueba revela cuán lejos están las predicciones del modelo de los valores reales, lo que indica la capacidad del modelo para hacer predicciones precisas en datos no vistos durante la formación. Al igual que con

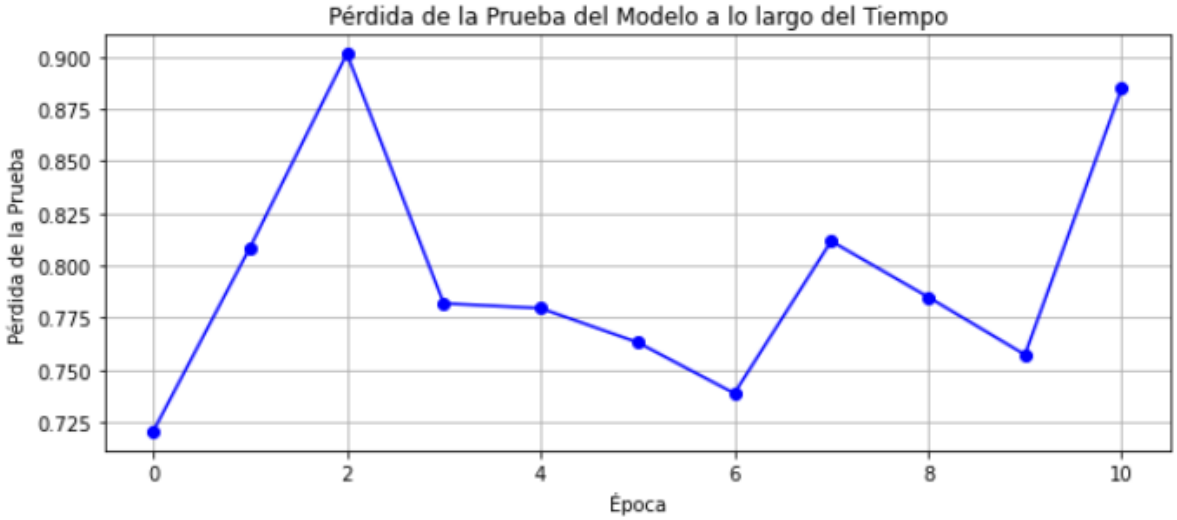
los gráficos anteriores, es esencial evaluar esta métrica para confirmar la robustez y la aplicabilidad del modelo.

Figura 97 *Gráfico de Pérdida del Modelo en Modelo (Inception-ResNet-v2)*



Nota. Elaboración propia

Figura 98 *Gráfico de Pérdida del Modelo en Modelo (ResNet)*



Nota. Elaboración propia

Figura 99 *Gráfico de Pérdida del Modelo en Modelo (EfficientNet)*



Nota. Elaboración propia

Figura 100 *Gráfico de Pérdida del Modelo en Modelo (Personalizado)*



Nota. Elaboración propia

5.3. Presentación de los Resultados del Modelo

La eficacia de los modelos, evaluada en términos de precisión (accuracy), se presenta en las siguientes tablas. Estos resultados reflejan la capacidad de cada modelo para realizar predicciones precisas cuando se entrenan con los parámetros óptimos identificados en la sección anterior.

Tabla 33 *Precisión de los modelos usando varios extractores de características*

Modelo	Extractor				
	SIFT	SIFT(Padding)	LBPH	HOG	ORB
SVM	64%	65%	68%	76%	62%
KNN	67%	68%	65%	75%	63%
Random Forest	65%	70%	67%	79%	64%
MLP	69%	62%	66%	71%	64%

Nota. Elaboración propia

Esta tabla proporciona una comparación clara de cómo cada modelo (SVM, KNN, Random Forest y MLP) se comporta en función del extractor de características utilizado (SIFT, SIFT(Padding), LBPH, HOG, ORB). Cabe destacar que el mejor resultado se obtuvo al usar el modelo Random Forest con el extractor de características HOG, alcanzando una precisión del 79%.

Tabla 34 *Precisión de los modelos de aprendizaje profundo usando diferentes parámetros*

Modelo	**Neurons**	**Epochs**	**Samples**	**Accuracy**	**Loss**
Inception-ResNet-v2	2048	50	1500	90.69%	0.2788
Resnet	512	50	500	84.09%	0.6131
Efficientnet	2048	50	1500	86.04%	0.4077
Personalizado	1024	50	1000	93.18%	0.3563

Nota. Elaboración propia

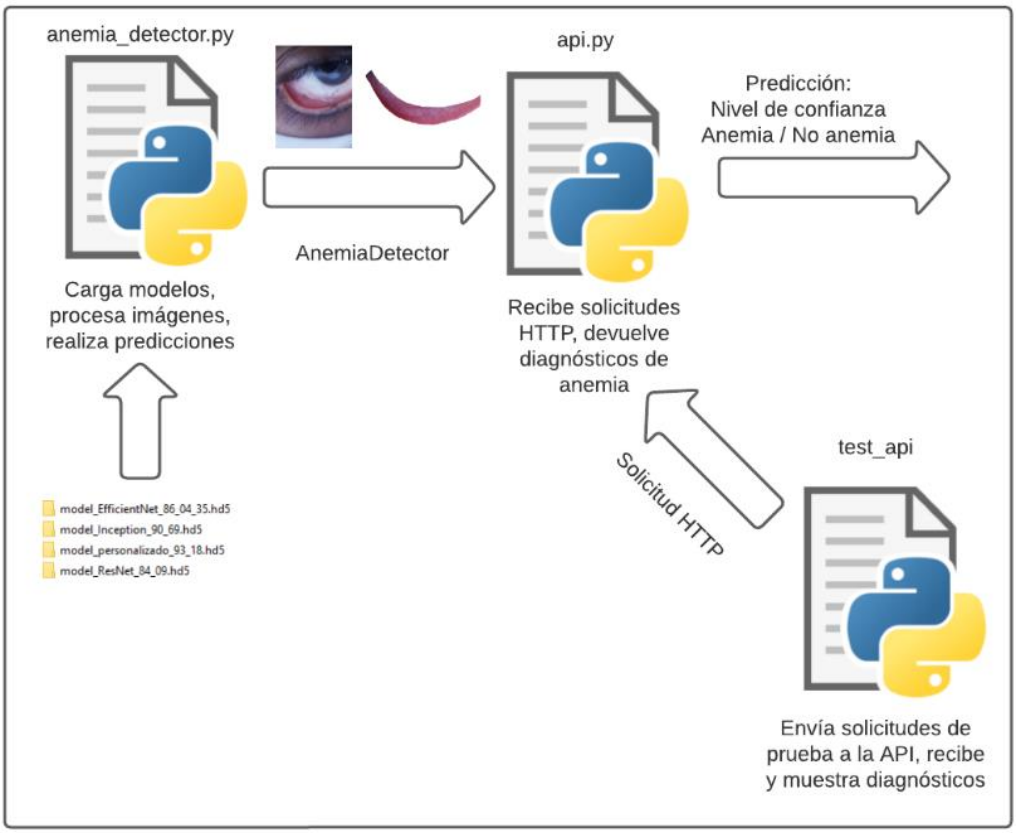
La tabla resume los resultados de varios modelos de aprendizaje profundo con parámetros ajustados. Aunque el modelo personalizado alcanzó la mayor precisión (93.18%) utilizando 1024 neuronas, 50 épocas y 1000 muestras, no solo la precisión es crucial. También se debe considerar la pérdi-da y la solidez de la arquitectura utilizada. A pesar de su alta precisión, el modelo personalizado carece de la validación empírica que proporcionan las arquitecturas reconocidas, como la de In-ception-ResNet-v2, que mostró la menor pérdida, indicando una posible mejor adaptabilidad a nuevos datos. La selección del modelo óptimo requerirá un balance entre estos factores, tema que se explorará más a fondo en la siguiente sección.

El modelo Inception-ResNet-v2, con 2048 neuronas, 50 épocas y 1500 muestras, mostró la pérdida más baja (0.2788) a pesar de una precisión ligeramente inferior (90.69%). Esta baja pérdida sugiere una posible mejor generalización a nuevos datos, un factor crucial para la aplicación práctica de los modelos. La elección del mejor modelo, por lo tanto, requerirá un equilibrio entre precisión y pérdida, así como las necesidades específicas del proyecto.

5.4. Implementación de una API con los mejores modelos predictivos

El presente subtema se dedica a profundizar en la transición desde un modelo teórico de detección de anemia hasta su aplicación práctica a través de una API. Esta API, diseñada utilizando librerías de vanguardia como TensorFlow, Keras y Flask, materializa los hallazgos de nuestra investigación en una interfaz funcional.

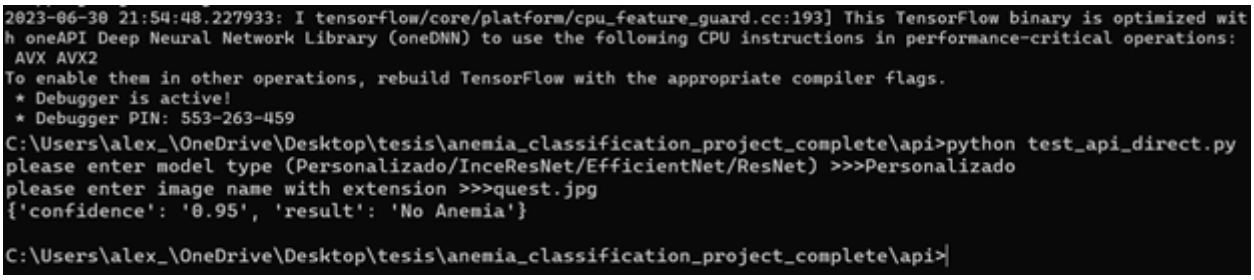
Figura 101 *Diagrama de Flujo de la Interacción de los Scripts y Procesamiento de la API*



Nota. Elaboración propia

- Código **anemia_detector.py**: Este script define la clase AnemiaDetector que carga los modelos de aprendizaje automático preentrenados, procesa imágenes para el análisis y realiza predicciones sobre la presencia de anemia.
- Código **api.py:** Este script establece una API de Flask que utiliza la clase AnemiaDetector para procesar solicitudes de detección de anemia. La ruta '/detect' acepta solicitudes POST, que contienen una imagen y un tipo de modelo, y devuelve un resultado de predicción.
- Código **test_api.py:** Este script interactúa con la API enviando una solicitud POST con una imagen de prueba y un tipo de modelo seleccionado. Imprime la respuesta de la API, que contiene el resultado de la detección de anemia y la confianza de la predicción.

Figura 102 *Ejecución de la API de Detección de Anemia*



Nota. Elaboración propia

La funcionalidad primordial de la API consiste en la utilización de un modelo previamente entrenado, elegido entre alternativas como Inception-Resnetv2, una CNN personalizada, ResNet y EfficientNet. Esta API recibe una imagen como entrada y, basándose en el modelo de predicción, identifica la presencia o ausencia de anemia en la muestra. Además, proporciona un porcentaje de confianza asociado a dicha detección, demostrando de manera explícita la fiabilidad del modelo predictivo.

CAPÍTULO VI: CONCLUSIONES Y RECOMENDACIONES

6.1. Conclusiones

La anemia es un problema persistente de salud pública en Perú, con repercusiones significativas en la calidad de vida individual y el progreso socioeconómico a nivel nacional. A pesar de que el hemograma se considera el método de referencia para diagnosticar la anemia, su necesidad de tiempo y un entorno de laboratorio para ser realizado puede representar una limitación considerable, especialmente en áreas remotas o menos desarrolladas. Como una práctica común, los profesionales de la salud a menudo recurren a exámenes visuales para identificar posibles indicios de anemia. Sin embargo, este enfoque puede no ser completamente preciso y depende en gran medi-da de la experiencia subjetiva del médico. Este estudio aspira a superar tales limitaciones ofre-ciendo una solución de prediagnóstico innovadora, accesible y confiable, diseñada para ser utilizada por cualquier persona, aumentando así la detección temprana y el tratamiento oportuno de la anemia.

La metodología de este trabajo comenzó con la recopilación de imágenes de la conjuntiva ocular de pacientes con y sin anemia, cumpliendo así el primer objetivo. Posteriormente, estas imágenes se preprocesaron para optimizar su análisis. El preprocesamiento incluyó la segmentación de la imagen en cuatro regiones diferentes de interés (conjuntiva palpebral, conjuntiva fornix, la suma de conjuntiva palpebral y fornix, y la imagen original), la transformación a escala de grises y el reajuste de tamaño. Además, se implementaron técnicas de aumento de datos para compensar la limitación de una pequeña base de datos inicial.

En cuanto al análisis de datos, se utilizaron diversos extractores de características como SIFT, SURF, ORB y HOG para obtener los vectores característicos de cada imagen. Con la aplicación de técnicas de aprendizaje automático como SVM, RF, MLP, y RNN, se clasificaron estas

carac-terísticas. El modelo de Random Forest con extractor HOG obtuvo la mayor precisión, con un 79%.

Además, se exploraron modelos de aprendizaje profundo, ajustando parámetros como el número de neuronas, las épocas y las muestras en cada modelo. Aunque inicialmente el modelo personalizado obtuvo la mayor precisión 93.18%, se optó finalmente por el modelo Inception-ResNet-v2, respaldado por estudios existentes, el cual demostró una sólida precisión del 90.69% y perdida de 0.2788, siento esta mejor que la perdida 0.3563 que se obtuvo con el modelo en comparación.

Para validar y facilitar la implementación de los modelos, se desarrolló una API integrando los modelos con mayor rendimiento. Es fundamental enfatizar que, a pesar de la alta precisión obte-nida con el modelo Inception-ResNet-v2, este estudio proporciona un método de prediagnóstico. Cualquier diagnóstico definitivo de anemia debe estar respaldado por un profesional médico y pruebas complementarias.

En resumen, este estudio ha demostrado que las técnicas de aprendizaje automático y aprendizaje profundo pueden proporcionar una predicción confiable de la anemia a partir de imágenes de la conjuntiva ocular. La implementación del modelo Inception-ResNet-v2, con su precisión del 90.69%, ofrece una herramienta prometedora para el diagnóstico preliminar de la anemia. Se espera que estos hallazgos contribuyan efectivamente a la lucha contra la anemia en Perú, y que, al hacerlo, mejoren la salud pública y el bienestar general de la población.

6.2. Recomendaciones

Este estudio ha establecido la factibilidad de desarrollar un modelo para la predicción de la anemia a partir de imágenes de partes del cuerpo que muestren características indicativas de anemia. No obstante, se debe considerar que la calidad y la forma en que se toma la imagen son

factores cruciales para garantizar la precisión del diagnóstico. Específicamente, debido a la di-ver-sidad y singularidad de las imágenes de personas, puede ser desafiante encontrar una base de datos extensa y representativa. Por ende, es esencial asegurar que la imagen sea de alta calidad y que se conozca su procedencia, para así facilitar una predicción de pre-diagnóstico más precisa.

Asimismo, este trabajo sugiere una línea de investigación para futuros estudios, que es explorar y profundizar en el uso de la inteligencia artificial para la generación de datos de imáge-nes. El avance en esta dirección podría proporcionar una solución al problema de la limitada dis-ponibili-dad de bases de datos de imágenes adecuadas.

Este estudio ha explorado la aplicación de tres arquetipos de modelos, los cuales han de-mostrado ser eficientes para este tipo de trabajos. Sin embargo, para optimizar la precisión de la predicción, es recomendable considerar la experimentación con otros arquetipos en trabajos futu-ros.

La visión de este trabajo es que la herramienta desarrollada sea accesible al público en general. Por lo tanto, se recomienda que los esfuerzos futuros se centren en la creación de un en-torno de usuario más interactivo que permita pruebas sencillas y eficientes.

Finalmente, este trabajo proporciona una nueva perspectiva para abordar un problema que ha sido tratado de diversas maneras en el pasado. Sin embargo, como con cualquier herra-mienta o mode-lo, el impacto más significativo se logra a través de la aplicación práctica de estos modelos. En este sentido, el campo de la salud en Perú presenta numerosas oportunidades para la implementa-ción y el desarrollo de tales modelos. Se recomienda, por tanto, tomar la iniciativa en la aplicación de estos hallazgos y modelos en el sector de la salud.

FUENTES DE INFORMACIÓN

- **Referencias Bibliográficas:**

A. Kalantri, M. Karambelkar, R. Joshi, S. Kalantri, and U. Jajoo, “Accuracy and reliability of

Aggarwal, C. C. (2018). An Introduction to Data Classification. En Data Classification: Algorithms and Applications (pp. 1-40). CRC Press.

Aha, D. W., Kibler, D., & Albert, M. K. (1991). Instance-based learning algorithms. Machine Learning, 6(1), 37-66.

Alpaydin, E. (2020). Introduction to machine learning. MIT press.

American Society of Hematology. (2021). Aplastic Anemia. Recuperado de https://www.hematology.org/education/patients/aplastic-anemia

Anastas, J.W. (1999). Research Design for Social Work and the Human Services. (2da ed.). Columbia University Press

Arkalab. (17 de abril de 2018). Hablamos con el SAT de Akralab sobre Analizadores hematológicos. Arkalab. https://www.akralab.es/sat-akralab-analizadores-hematologicos/#:~:text=Un%20contador%20Hematológico%20tiene%20varios,la%20lectura%20de%20las%20células

Auerbach, M., & Adamson, J. W. (2016). How we diagnose and treat iron deficiency anemia. American Journal of Hematology, 91(1), 31-38. https://doi.org/10.1002/ajh.24201

Bain, B. J. (2015). Blood Cells: A Practical Guide. Wiley-Blackwell.

Ballard, D. H., & Brown, C. M. (1982). Computer Vision. Prentice Hall.

Bauskar, S., Jain, P., & Gyanchandani, M. (2019). A Noninvasive Computerized Technique to Detect Anemia Using Images of Eye Conjunctiva. Pattern Recognition and Image Analysis, 29(3), 438–446. doi:10.1134/s1054661819030027

Bauskar, S., Jain, P., & Gyanchandani, M. (2019). A Noninvasive Computerized Technique to Detect Anemia Using Images of Eye Conjunctiva. Pattern Recognition and Image Analysis, 29(3), 438–446. https://doi.org/10.1134/S1054661819030027

Berrar, D. (2019). Cross-Validation. In D. Berrar, W. Dubitzky, & M. Granzow (Eds.), Fundamentals of Data Mining in Genomics and Proteomics (pp. 61–72). Springer. DOI: 10.1007/978-0-387-69937-8_4

Beucher, S., & Meyer, F. (1993). The morphological approach to segmentation: the watershed transformation. Mathematical morphology in image processing, 34(3), 433-481.

Biau, G. (2012). Analysis of a Random Forests Model. Journal of Machine Learning Research, 13, 1063-1095.

Bishop, C. M. (2006). Pattern recognition and machine learning. Springer.

Bojarski, M., Del Testa, D., Dworakowski, D., Firner, B., Flepp, B., Goyal, P., ... & Zhang, X. (2016). End to end learning for self-driving cars. arXiv preprint arXiv:1604.07316.

Bottou, L. (2010). Large-scale machine learning with stochastic gradient descent. In Proceedings of COMPSTAT'2010 (pp. 177-186). Springer, Heidelberg.

Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.

Burney, S., & Saleem, H. (06 de marzo de 2008). Inductive & Deductive Research Approach. [Presentación]. Auditorium of Faculty of Arts and Science, University of Karachi, Karachi, Pakistan.

Calonder, M., Lepetit, V., Strecha, C., & Fua, P. (2010). BRIEF: Binary Robust Independent Elementary Features. In European Conference on Computer Vision (pp. 778-792). Springer, Ber-lin, Heidelberg.

Camaschella, C. (2015). Iron-deficiency anemia. New England Journal of Medicine, 372(19), 1832-1843. https://doi.org/10.1056/NEJMra1401038

Canny, J. (1986). A computational approach to edge detection. IEEE Transactions on Pattern Analysis and Machine Intelligence, (6), 679-698.

Centers for Disease Control and Prevention. (2020). Folic Acid. Recuperado de

Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R.(2000). CRISP-DM 1.0: Step-by-step data mining guide. SPSS.

Chen, P., Li, Y., Ribeiro, A., Schmidt, L., & Liang, X. (2018). Robust Neural Networks Inspired by Strong Stability Preserving Runge–Kutta Methods. In Proceedings of the 32nd International Conference on Neural Information Processing Systems (NIPS'18), 3912-3922.

Chollet, F. (2015). Keras. https://keras.io.

Chollet, F., & Allaire, J. J. (2018). Deep Learning with R. Manning Publications Co.

Cortes, C., & Vapnik, V. (1995). Support-vector networks. Machine Learning, 20(3), 273-297.

Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. IEEE Transactions on Information Theory, 13(1), 21-27.

Culjak, I., Abram, D., Pribanić, T., Džapo, H., & Cifrek, M. (2012). A brief introduction to OpenCV. Proceedings of the 35th International Convention MIPRO, 1725-1730.

Dalal, N., & Triggs, B. (2005). Histograms of oriented gradients for human detection. In Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recogni-tion (Vol. 1, pp. 886-893). IEEE.

De Benoist, B., Cogswell, M., Egli, I., y McLean, E. (2008). Worldwide Prevalence of Anaemia 1993-2005. WHO Global Database of Anaemia.

Dimauro, G., & Simone, L. (2020). Novel Biased Normalized Cuts Approach for the Automatic Segmentation of the Conjunctiva. Electronics, 9(6), 997. DOI: 10.3390/electronics9060997

Dimauro, G., Camporeale, M. G., Dipalma, A., Guarini, A., & Maglietta, R. (2023). Anaemia detection based on sclera and blood vessel colour estimation. Biomedical Signal Processing and Control, 81, 104489. https://doi.org/10.1016/j.bspc.2022.104489

Dimauro, G., De Ruvo, S., Di Terlizzi, F., Ruggieri, A., Volpe, V., Colizzi, L., & Girardi, F. (2020). Estimate of Anemia with New Non-Invasive Systems—A Moment of Reflection. Electronics, 9(5), 780. doi:10.3390/electronics9050780

Dimauro, G., Griseta, M.E., Camporeale, M. G., Clemente, F., Guarini, A., & Maglietta, R. (2022). An intelligent non-invasive system for automated diagnosis of anemia exploiting a novel dataset. Artificial Intelligence in Medicine, 102477. https://doi.org/10.1016/j.artmed.2022.102477

Dormann, C. F., Elith, J., Bacher, S., Buchmann, C., Carl, G., Carré, G., ... & Lautenbach, S. (2013). Collinearity: a review of methods to deal with it and a simulation study evaluating their performance. Ecography, 36(1), 27-46. DOI: 10.1111/j.1600-0587.2012.07348.x

Duda, R. O., Hart, P. E., & Stork, D. G. (2001). Pattern classification. John Wiley & Sons.

Efron, B., & Tibshirani, R. J. (1993). An Introduction to the Bootstrap. Chapman and Hall/CRC. ISBN: 978-0-412-04231-7.

Etools. (s/f). Hemoglobin Levels Chart. Etools. https://www.etoolsage.com/Chart/Hemoglobin_Levels_Chart.asp

etoolsage.com. (2023). Hemoglobin Levels Chart. Recuperado de https://www.etoolsage.com/Chart/Hemoglobin_Levels_Chart.asp

Everingham, M., Van Gool, L., Williams, C. K. I., Winn, J., & Zisserman, A. (2010). The Pascal Visual Object Classes (VOC) Challenge. International Journal of Computer Vision, 88(2), 303-338.

Falkingham, M., Abdelhamid, A., Curtis, P., Fairweather-Tait, S., Dye, L., & Hooper, L. (2010). The effects of oral iron supplementation on cognition in older children and adults: a systematic review and meta-analysis. Nutrition journal, 9(1), 4.

Fawcett, T. (2006). An introduction to ROC analysis. Pattern Recognition Letters, 27(8), 861-874.

Gijsenij, A., Gevers, T., & Van De Weijer, J. (2011). Computational color constancy: Survey and experiments. IEEE Transactions on Image Processing, 20(9), 2475-2489.

Gonzalez, R. C., & Woods, R. E. (2017). Digital image processing. Pearson Education India.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). Generative adversarial nets. In Advances in neural information processing systems (pp. 2672-2680).

Gordon, N. (2003). Iron deficiency and the intellect. Brain and Development, 25(1), 3–8. https://doi.org/10.1016/S0387-7604(02)00148-1

GradesFixer. (28 de enero de 2019). Research Methodology: Exploratory, Descriptive and Explanatory. GradesFixer. https://gradesfixer.com/free-essay-examples/research-methodology-exploratory-descriptive-and-explanatory/

Grinias, J., & Madec, S. (2018). Deep learning for agricultural applications. arXiv preprint arXiv:1809.05675.

Guo, Z., Zhang, L., & Zhang, D. (2017). A completed modeling of local binary pattern operator for texture classification. IEEE Transactions on Image Processing, 26(6), 2667-2681.

Hagan, M. T., Demuth, H. B., & Beale, M. H. (1996). Neural Network Design. PWS Publishing. ISBN-10: 0534943322.

Haralick, R. M., & Shapiro, L. G. (1985). Image segmentation techniques. Computer Vision, Graphics, and Image Processing, 29(1), 100-132.

Harrington, P. (2012). Machine learning in action. Manning.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning: Data Min-ing, Inference, and Prediction. Springer.

Haykin, S. (2009). Neural Networks and Learning Machines. Pearson.

He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In Proceedings of the IEEE international confer-ence on computer vision (pp. 1026-1034).

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recog-nition, 770-778.

Heikkilä, M., Pietikäinen, M., & Schmid, C. (2006, May). Description of interest regions with local binary patterns. In Pattern Recognition, 2006. ICPR 2006. 18th International Con-ference on (Vol. 3, pp. 42-45). IEEE.

Hernández , R., & Mendoza , C. (2018). Metodología de la investigación:Las rutas cuantitativa, cualitativa y mixta. Mexico: Mc Graw Hill.

Hernández, R., Fernández, C., & Baptista, P. (2010). Metodología de la investigación (5a ed.). McGraw-Hill Interamericana.

Hinojosa, J. (2017). El arte de hacer una tesis. Lima, Perú: José Adolfo Hinojosa Pérez.

Hoffman, R., Benz, E.J., Silberstein, L.E., Heslop, H., Weitz, J., Anastasi, J. (2018). Hematology: Basic Principles and Practice, 7th edition. Elsevier Health Sciences.

Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. Neural Networks, 2(5), 359-366.

Howard, A., Sandler, M., Chu, G., Chen, L. C., Chen, B., Tan, M., ... & Adam, H. (2017). MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. arXiv pre-print arXiv:1704.04861.

Hsu, C. W., Chang, C. C., & Lin, C. J. (2003). A practical guide to support vector classification. Department of Computer Science, National Taiwan University.

Huber, S., Wiemer, H., Schneider, D., & Ihlenfeldt, S. (2019). DMME: Data mining methodology for engineering applications – a holistic extension to the CRISP-DM model. Procedia CIRP, 79, 403–408. https://doi.org/10.1016/J.PROCIR.2019.02.106

Hurrell, R., & Egli, I. (2010). Iron bioavailability and dietary reference values. The American Journal of Clinical Nutrition, 91(5), 1461S-1467S. https://doi.org/10.3945/ajcn.2010.28674D

IEEEDataPort. (2022). EYES-DEFY-ANEMIA. Recuperado el 31 de enero de 2022, de https://ieee-dataport.org/.

Instituto Nacional de Estadística e Informática (2020). La prevalencia de anemia en la población peruana. Lima: INEI.

Instituto Nacional de Estadística e Informática. Encuesta Demográfica y de Salud Familiar 2018 Nacional y Regional (ENDES 2018). Lima: INEI; 2019 [acceso 10/12/2019] Disponible

en: https://www.inei.gob.pe/media/MenuRecursivo/publicaciones_digitales/Est/Lib1656/index1.html

Instituto Nacional de Salud (2019). Informe Gerencial Estado nutricional de niños y gestantes que acceden a establecimientos de salud. Lima: MINSA.

Ioffe, S., & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. In Proceedings of the 32nd International Conference on Ma-chine Learning. https://arxiv.org/abs/1502.03167.

Jain, A. K., Kasturi, R., & Schunck, B. G. (1995). Machine vision. McGraw-Hill, Inc.

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). An Introduction to Statistical Learning: with Applications in R. Springer.

Jane Thompson, Beverley-Ann Biggs, Sant-Rayn Pasricha; Effects of Daily Iron Supplementation in 2- to 5-Year-Old Children: Systematic Review and Meta-analysis. Pediatrics April 2013; 131 (4): 739–753. 10.1542/peds.2012-2256

José Carlos Jaime Pérez, "Hematología la sangre y sus enfermedades /", -: McGraw-Hill Interamericana, 2015. Consultado en línea en la Biblioteca Digital de Bogotá (https://www.bibliotecadigitaldebogota.gov.co/resources/2215500/), el día 2022-04-25.

Kalantri, A., Karambelkar, M., Joshi, R., Kalantri, S., & Jajoo, U. (2010). Accuracy and reliability of pallor for detecting anaemia: a hospital-based diagnostic accuracy study. PloS One, 5(1). https://doi.org/10.1371/JOURNAL.PONE.0008545

Kerlinger, F. N. (1986). Foundations of Behavioral Research (3rd ed.). Holt, Rinehart and Winston.

Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. Proceedings of the 14th international joint conference on Artificial intelligence-Volume 2. Citeseer.

Kothari, C. R. (2004). Research methodology: Methods and techniques. New Age International.

Kotsiantis, S. B., Kanellopoulos, D., & Pintelas, P. E. (2006). Data preprocessing for supervised leaning. International Journal of Computer Science, 1(2), 111-117.

Kourou, K., Exarchos, T. P., Exarchos, K. P., Karamouzis, M. V., & Fotiadis, D. I. (2015). Machine learning applications in cancer prognosis and prediction. Computational and structural bio-technology journal, 13, 8-17.

Kriesel, D. (2007). A Brief Introduction to Neural Networks. Retrieved from http://www.dkriesel.com.

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. In Advances in Neural Information Processing Systems.

Kumar, A., Cohen, P. R., & LeCun, Y. (2016). Explanation-Based Learning in Computer Vision: A Case Study in Handwritten Digit Recognition. In CVPR (Vol. 2, p. 4).

Kumar, V., Abbas, A.K., Aster, J.C. (2018). Robbins Basic Pathology, 10th Edition. Elsevier.

Labelbox. (n.d.). Training data, reimagined. https://www.labelbox.com/

Lane, D. J. R., & Richardson, D. R. (2014). The active role of vitamin C in mammalian iron metabolism: much more than just enhanced iron absorption! Free Radical Biology and Medicine, 75, 69-83. https://doi.org/10.1016/j.freeradbiomed.2014.07.007

Laposata, M. (2011). Diagnostic strategies for common medical problems. ACP Press.

Las Máquinas de Vectores de Soporte son una herramienta poderosa y versátil en el campo del aprendizaje automático, capaces de manejar conjuntos de datos de alta dimensión y separar con precisión diferentes categorías.

LeCun, Y. A., Bottou, L., Orr, G. B., & Müller, K. R. (2012). Efficient backprop. In Neural networks: Tricks of the trade (pp. 9-48). Springer.

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. Nature, 521(7553), 436-444.

Lecun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), 2278-2324.

Liaw, A., & Wiener, M. (2002). Classification and Regression by randomForest. R News, 2(3), 18-22.

Litjens, G., Kooi, T., Bejnordi, B. E., Setio, A. A. A., Ciompi, F., Ghafoorian, M., ... & Sánchez, C. I. (2017). A survey on deep learning in medical image analysis. Medical Image Analysis, 42, 60-88.

Lowe, D. G. (2004). Distinctive image features from scale-invariant keypoints. International journal of computer vision, 60(2), 91-110.

Maenpaa, T., & Pietikainen, M. (2004). Classification with color and texture: jointly or separately? Pattern Recognition, 37(8), 1629-1640.

Mannino, R. G., Myers, D. R., Tyburski, E. A., Caruso, C., & Boudreaux, J. (2018). Smartphone app for non-invasive detection of anemia using only patient-sourced photos. Nature Communications, 9(1), 4924. https://doi.org/10.1038/s41467-018-07262-2

Mannino, R. G., Myers, D. R., Tyburski, E. A., Caruso, C., Boudreaux, J., Leong, T., Clifford, G. D., y Lam, W. A. (2018). Smartphone app for non-invasive detection of anemia using only patient-sourced photos. Nature Communications, 9(1), 4924.

Marbán, Ó., Morales, D. R., & Menasalvas, E. (2019). A data mining & knowledge discovery process model. In Data Mining and Knowledge Discovery Handbook (pp. 89-112). Springer.

Mayo Clinic. (2020). Anemia. Recuperado de https://www.mayoclinic.org/diseases-conditions/anemia/symptoms-causes/syc-20351360

McNulty, H., & Pentieva, K. (2011). Folate bioavailability. In Proceedings of the Nutrition Society (Vol. 70, No. 2, pp. 245-249). Cambridge University Press. https://doi.org/10.1017/S0029665111000049

Ministro de Salud destaca necesidad de una mejor distribución del recurso humano. (2021, 18 diciembre). Andina. https://andina.pe/agencia/noticia-ministro-salud-destaca-necesidad-una-mejor-distribucion-del-recurso-humano-873910.aspx

Mirmehdi, M., Xie, X., & Suri, J. (2011). Handbook of texture analysis. Imperial College Press.

Montavon, G., Samek, W., & Müller, K. R. (2018). Methods for interpreting and understanding deep neural networks. Digital Signal Processing, 73, 1-15. doi:10.1016/j.dsp.2017.10.011.

Nathan, D. G., Orkin, S. H., Ginsburg, D., & Look, A. T. (2003). Nathan and Oski's hematology of infancy and childhood (6th ed.). Philadelphia, PA: Saunders.

National Heart, Lung, and Blood Institute. (2021). Iron-Deficiency Anemia. Recuperado de https://www.nhlbi.nih.gov/health-topics/iron-deficiency-anemia

National Heart, Lung, and Blood Institute. (2021). Iron-Deficiency Anemia. Recuperado de https://www.nhlbi.nih.gov/health-topics/iron-deficiency-anemia

National Heart, Lung, and Blood Institute. (2021). Iron-Deficiency Anemia. Recuperado de https://www.nhlbi.nih.gov/health-topics/iron-deficiency-anemia

National Institute of Diabetes and Digestive and Kidney Diseases. (2021). Pernicious Anemia. Recuperado de https://www.niddk.nih.gov/health-information/blood-diseases/pernicious-anemia

National Library of Medicine. (2021). Anemia of Chronic Disease. MedlinePlus. Recuperado de https://medlineplus.gov/ency/article/000565.htm

National Library of National Library of Medicine. (2021). Hemolytic Anemia. MedlinePlus. Recuperado de https://medlineplus.gov/hemolyticanemia.html

Nature Communications. (2023). Smartphone app for non-invasive detection of anemia using only patient-sourced photos. Recuperado el 23 de junio de 2023, de https://www.nature.com/articles/s41467-023-21435-4.

Ñaupas, H., Valdivia, J., Palacios, W., & Romero, P. (2018). Metodología de la investigación científica e informática. Universidad Nacional Mayor de San Marcos.

Ñaupas, H., Valdivia, M., Palacios, J., & Romero, E. (2018). Metodología de la investigación cuantitativa - cualitativa y redacción de tesis. Bogotá, Colombia : Ediciones de la U.

NumPy. (n.d.). NumPy: The fundamental package for scientific computing with Python. https://numpy.org/

Ojala, T., Pietikainen, M., & Harwood, D. (1996). A comparative study of texture measures with classification based on feature distributions. Pattern Recognition, 29(1), 51-59.

Ojala, T., Pietikainen, M., & Maenpaa, T. (2002). Multiresolution gray-scale and rotation invariant texture classification with local binary patterns. IEEE Transactions on Pattern Analysis and Machine Intelligence, 24(7), 971-987. doi:10.1109/TPAMI.2002.1017623.

Ong, Y. L., Heng, B. H., Cheung, Y. B., & Lee, Y. S. (2019). Prevalence and risk factors of anemia among elderly inpatients of an acute care hospital. Aging clinical and experimental re-search, 31(11), 1659-1665.

Organización Mundial de la Salud. Carencia de micronutrientes [Internet]. Ginebra: OMS; 2020 [citado el 12 de agosto de 2020]. Disponible en: https://www.who.int/nutrition/topics/ida/es/ [Links]

Organización Mundial de la Salud. Metas mundiales de nutrición 2025. Documento normativo sobre anemia [Internet]. Ginebra: OMS; 2017 [citado el 12 de agosto de 2020]. Disponible en: https://apps.who.int/iris/bitstream/handle/10665/255734/WHO_NMH_NHD_14.4_spa.pdf?ua=1pallor for detecting anaemia: A hospital-based diagnostic accuracy study," PLoS One 5 (1), e8545 (2010).

Pal, N. R., & Pal, S. K. (1993). A review on image segmentation techniques. Pattern Recognition, 26(9), 1277-1294.

Parkhi, O. M., Vedaldi, A., & Zisserman, A. (2015). Deep face recognition. In British Machine Vision Conference.

Paulsen, F., Thale, A., Maune, S., & Tillmann, B. (1998). New insights into the pathophysiology of primary acquired dacryostenosis. Ophthalmology, 105(11), 2095-2102.

Perez, L., & Wang, J. (2017). The effectiveness of data augmentation in image classification using deep learning. arXiv preprint arXiv:1712.04621.

Pham, T. D., Salvetti, F., & Wang, B. (2018). Computational intelligence in digital and network designs and applications. Springer.

Pietikäinen, M., Hadid, A., Zhao, G., & Ahonen, T. (2011). Computer Vision Using Local Binary Patterns. Springer, London.

Pizer, S. M., Amburn, E. P., Austin, J. D., Cromartie, R., Geselowitz, A., Greer, T., ... & Zuiderveld, K. (1987). Adaptive histogram equalization and its variations. Computer vision, graphics, and image processing, 39(3), 355-368.

Powers, D. M. (2020). Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness and Correlation. arXiv preprint arXiv:2010.16061.

Project Jupyter. (n.d.). Project Jupyter: Open source software, open standards, and services. https://jupyter.org/

Radiological Society of North America. (2019). Chest X-ray (Chest Radiography). Recuperado de https://www.radiologyinfo.org/en/info.cfm?pg=chestrad

Radiological Society of North America. (2020). Magnetic Resonance Imaging (MRI). Recuperado de https://www.radiologyinfo.org/en/info.cfm?pg=bodymr

Rafael, V., Badugu, R., Lakowicz, J. R., & Geddes, C. D. (2020). Ocular angiography for the in vivo detection of human anemia. Ophthalmic research, 63(1), 86-95.

Rajagopal, L., & Thakur, N. (2017). Contrast enhancement algorithms for images–a comparative study. In Information and Communication Technology for Intelligent Systems (pp. 279-287). Springer, Cham.

Reyes-Aldasoro, C. C. (2016). The proportion congruent effect: An image processing approach. Journal of Microscopy, 262(1), 43-53.

Roberts, L. G. (1963). Machine perception of three-dimensional solids. In Optical and Electro-optical Information Processing (pp. 159-197). MIT Press.

Rojas, R. (1996). Neural Networks: A Systematic Introduction. Berlin: Springer-Verlag.

Rosenblatt, F. (1958). The Perceptron: A Probabilistic Model for Information Storage and Organ-ization in the Brain. Psychological Review, 65(6), 386-408.

Rosten, E., & Drummond, T. (2006). Machine learning for high-speed corner detection. In European Conference on Computer Vision (pp. 430-443). Springer, Berlin, Heidelberg.

Rublee, E., Rabaud, V., Konolige, K., & Bradski, G. (2011). ORB: An efficient alternative to SIFT or SURF. In Proceedings of the 2011 International Conference on Computer Vision (pp. 2564-2571). IEEE.

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. Nature, 323, 533-536.

Russell, S., & Norvig, P. (2016). Artificial Intelligence: A Modern Approach. Pearson.

Samuel, A. L. (1959). Some studies in machine learning using the game of checkers. IBM Journal of research and development, 3(3), 210-229.

Scherer, D., Müller, A., & Behnke, S. (2010). Evaluation of pooling operations in convolutional architectures for object recognition. In Artificial neural networks–ICANN 2010 (pp. 92-101). Springer, Berlin, Heidelberg.

Scholkopf, B., & Smola, A. J. (2001). Learning with kernels: support vector machines, regulariza-tion, optimization, and beyond. MIT press.

Sengur, A., Akbulut, Y., Guo, Y., & Bajaj, V. (2017). An investigation on the effect of dataset size in software defect prediction. IET Software, 11(2), 49-57.

Shearer, C. (2000). The CRISP-DM model: The new blueprint for data mining. JDM, 4(2), 13-22.

Shorten, C., & Khoshgoftaar, T. M. (2019). A survey on Image Data Augmentation for Deep Learning. Journal of Big Data, 6(1), 60.

Silva, C., Bouwmans, T., & Frélicot, C. (2018). An eXtended Center-Symmetric Local Binary Pattern for Background Modeling and Subtraction in Videos. 13th International Joint Confer-ence on Computer Vision, Imaging and Computer Graphics Theory and Applications, 5, 26-37. doi: 10.5220/0006647803540363.

Silvestre, I., & Huamán, C. (2019). Pasos para elaborar la investigación y redacción de la tesis universitaria. Lima, Perú: San Marcos.

Silvestre, J. & Huamán, F. (2019). Metodología de la investigación científica y tecnológica. Universidad Nacional Mayor de San Marcos.

Simel, D. L., & Rennie, D. (2009). The Rational Clinical Examination: Evidence-Based Clinical Diagnosis. McGraw-Hill Professional Publishing.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. Journal of Machine Learning Re-search, 15, 1929-1958. http://jmlr.org/papers/v

Stabler, S. P. (2013). Clinical practice. Vitamin B12 deficiency. New England Journal of Medicine, 368(2), 149-160

Swamynathan, M. (2017). Mastering Machine Learning with Python in Six Steps. Mastering Machine Learning with Python in Six Steps. https://doi.org/10.1007/978-1-4842-2866-1

Szegedy, C., Ioffe, S., Vanhoucke, V., & Alemi, A. A. (2017). Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning. In Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence (AAAI), 4278-4284.

Szeliski, R. (2010). Computer vision: algorithms and applications. Springer Science & Business Media.

Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. In Proceedings of the 36th International Conference on Machine Learning. http://proceedings.mlr.press/v97/tan19a.html.

Thaver, I. H., y Baig, L. (1994). Anaemia in Children: ¿Part I. Can Simple Observations by Primary Care Provider Help in Diagnosis? Journal of Pakistan Medical Association 44, 282-284.

Thoft, R. A., & Friend, J. (1983). The X, Y, Z hypothesis of corneal epithelial maintenance. Investigative Ophthalmology & Visual Science, 24(10), 1442-1443.

Thrun, S., Montemerlo, M., Dahlkamp, H., Stavens, D., Aron, A., Diebel, J., ... & Haehnel, D. (2006). Stanley: The robot that won the DARPA Grand Challenge. Journal of Field Robotics, 23(9), 661-692.

Train, test, validation split. (2021). In Practical Deep Learning for Coders. Fast.ai. https://www.fast.ai/2017/11/13/validation-sets/

Tseng, S. C. (1985). Anatomy and physiology of the conjunctiva. Journal of cataract and refractive surgery, 11(Supplement 1), 61-64.

Tvedten, H. (2022). Classification and Laboratory Evaluation of Anemia. Schalm's Veterinary Hematology, 198–208. https://doi.org/10.1002/9781119500537.CH25

Ubillos, S., Del líbano, M., & Ambrona, T. (2016). Manual práctico de análisis estadístico en educación social: Análisis transversales y longitudinales. Burgos, España: Servicio de publicaciones e imagen institucional.

Valderrama, S. (2019). Pasos para Elaborar Proyectos de investigación Cientifica. Lima: Editorial San Marcos.

VanderPlas, J. (2016). Python data science handbook: Essential tools for working with data. "O'Reilly Media, Inc.".

Vapnik, V. (1995). The nature of statistical learning theory. Springer science & business media.

Wallander, J. L., & Koot, H. M. (2016). Quality of life in children: A critical examination of concepts, approaches, issues, and future directions. Clinical Psychology Review, 45, 131–143. https://doi.org/10.1016/J.CPR.2015.11.007

Watanabe, F., & Bito, T. (2018). Vitamin B12 sources and microbial interaction. Experimental Biology and Medicine, 243(2), 148-158. https://doi.org/10.1177/1535370217746614

Weiss, G., & Goodnough, L. T. (2005). Anemia of chronic disease. New England journal of medicine, 352(10), 1011-1023.

Wirth, R., & Hipp, J. (2000). CRISP-DM: Towards a standard process model for data mining. In Proceedings of the 4th international conference on the practical applications of knowledge discovery and data mining (pp. 29-39).

World Health Organization (2015). The global prevalence of anaemia in 2011. Geneva: World Health Organization.

World Health Organization. (2011). Haemoglobin concentrations for the diagnosis of anaemia and assessment of severity. Vitamin and Mineral Nutrition Information System (VMNIS). Recuperado de https://www.who.int/vmnis/indicators/haemoglobin.pdf

World Health Organization. (2021). Anaemia. Recuperado de https://www.who.int/health-topics/anaemia#tab=tab_1

Worldwide Prevalence of Anemia 1993–2005: WHO Global Database on Anemia, Ed. by B. de Benoist, E. McLean, I. Egli, and M. Cogswell (World Health Organization, Geneva, 2008). Available at: http:// apps.who.int/iris/bitstream/10665/43894/1/9789

Wright, J. T., & Fine, D. H. (2016). Conjunctiva. In Oral pathology (pp. 65-68). Springer.

Zhang, D., Wu, X., Shen, D., & Meng, M. Q. H. (2020). Automated anemia detection via appearance and shape analysis of human conjunctiva. IEEE Transactions on Automation Science and Engineering, 18(2), 790-800.

Zhang, H., & Lu, J. (2004). Review of shape representation and description techniques. Pattern Recognition, 37(1), 1-19.

Zhang, H., Suen, C. Y., & Zhang, L. (2017). A review of recent advances in optical character recognition. International Journal on Document Analysis and Recognition, 1-7.

Zhou, B., Xu, Y., Guo, Y., Song, Y., Liu, W., Liu, Z., Chen, X., Pan, Q., Shi, L., Jiang, T., & Zhao, Y. (2020). The clinical blood parameters white blood cell and red blood cell distribution width predict survival in nasopharyngeal carcinoma: a large cohort study (N = 2,533). BMC Can-cer, 20(1), 240.

Zijp, I. M., Korver, O., & Tijburg, L. B. M. (2000). Effect of tea and other dietary factors on iron absorption. Critical Reviews in Food Science and Nutrition, 40(5), 371-398. https://doi.org/10.1080/10408690091189194

ANEXOS

Anexo 1: Tabla de papers

Antecedente	Autor	Año	Resultado
A Noninvasive Computerized Technique to Detect Anemia Using Images of Eye Conjunctiva	Shubham Bauskar, Prakhar Jain, Manasi Gyanchandani	2019	93%
Detection of anemia disease using DPSO Algorithm	S. Dhanasekaran and N. R. Shanker	2019	-
Comparación de técnicas basadas en visión computacional y machine learning para la detección temprana de anemia a partir del análisis de uñas	Heredia Menor, Keico Anavela Mamani Ticona, Wilfredo	2021	98.9%
Deep Convolutional Neural Network Model for Detection of Sickle Cell Anemia in Peripheral Blood Images	Emmanuel Gbenga Dada, David Opeoluwa Oyewola and Stephen Bassi Joseph	2022	99-100%
Smartphone app for non-invasive detection of anemia using only patient-sourced photos	Robert G. Mannino, David R. Myers, Erika A. Tyburski, Christina Caruso, Jeanne Boudreaux, Traci Leong, G. D. Clifford & Wilbur A. Lam	2018	95%

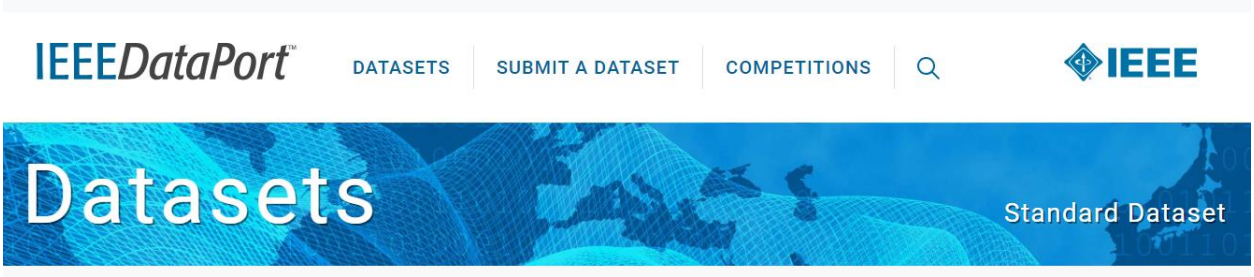
Anexo 2: Matriz de consitencia.

MATRIZ DE CONSISTENCIA				
Propuesta de **sistema de visión computacional** para el **pre-diagnóstico de la Enfermedad de anemia** a partir de la imagen de la conjuntiva ocular				
PROBLEMA GENERAL	**OBJETIVO GENERAL**	**HIPOTESIS GENERAL**	**VARIABLES**	**DIMENSIONES**
¿Es posible desarrollar un **sistema de visión por computadora** que pueda determinar el **pre-diagnóstico de anemia** a partir de imágenes de la conjuntiva del ojo?	Desarrollar e instalar un **sistema de procesamiento de imágenes computarizado** para **categorizar de manera precisa los síntomas preliminares de la anemia** utilizando imágenes de la conjuntiva ocular.	Mediante la implementación de un **modelo de visión computacional** se logra determinar el **pre-diagnóstico de la enfermedad de anemia** a partir de la conjuntiva ocular.	**Variable Independiente**: **Sistema de visión computacional** Según Szeliski (2010) en su libro 'Computer Vision: Algorithms and Applications', "La visión por computadora busca automatizar y mejorar la capacidad humana de entender imágenes digitales. Los sistemas de visión por computadora pueden extraer características de las imágenes, reconocer objetos y realizar muchas otras tareas que normalmente requieren la intervención humana."	**Recopilación y Preparación de Datos**
PROBLEMA ESPECIFICO	**OBJETIVOS ESPECIFICOS**	**HIPOTESIS ESPECIFICA**		
¿Se dispone de **bases de datos con imágenes** de la conjuntiva ocular de individuos con anemia?	**Recopilar imágenes de la conjuntiva ocular de pacientes**, tanto de aquellos que padecen anemia como de aquellos que no la padecen.	Existen datos suficientes para ser modelados usando los **algoritmos de machine learning.**		**Procesamiento de Imágenes**
¿Qué **técnicas de procesamiento de imágenes** pueden mejorar la calidad o la cantidad de imágenes en la base de datos?	**Preparar los datos para su análisis**, incluyendo el pre procesamiento, la limpieza y el análisis de datos faltantes y la transformación de variables en estudio.	El conjunto de datos es apto para ser **modelado y entrenado usando algoritmo de aprendizaje no supervisado.**	**Variable Dependiente: Pre-diagnóstico de la Enfermedad de anemia** Según el estudio de Nissenson, Goodnough, & Dubois (2003) titulado 'Anemia: No longer an 'innocent bystander', but rather a risk factor for chronic disease', "La anemia es un indicador clave de varias enfermedades crónicas y puede ser identificada a través de varios métodos de diagnóstico. Su identificación temprana puede ser crucial para prevenir o tratar eficazmente la enfermedad subyacente."	**Extracción y Clasificación de Características**
¿Qué técnicas de **visión por computadora** permiten convertir los descriptores de imagen como el color o la forma en vectores de características?	**Aplicar técnicas de aprendizaje automático, como SVM, RF y RNN,** para clasificar las características obtenidas de las imágenes.	**Los algoritmos de aprendizaje Random Forest, SVM, RNN**, son modelados y entrenados satisfactoriamente.		**Evaluación de Modelos**
¿Qué medidas se pueden utilizar para **evaluar la efectividad** de la clasificación de imágenes en la **detección de anemia en pacientes**?	**Evaluar los algoritmos propuestos** en términos de su ajuste a los datos, su capacidad para estimar el error de manera precisa.	El algoritmo de Random Forest presenta mayor bondad de ajuste a los datos, menor error de estimación y tiempo de procesamiento en comparación a los **modelos de aprendizaje no superviado SVM, RNN.**		**Aplicación Clínica y Competitividad**

Anexo 3: Afiche anemia en el Perú



Anexo 4: Interfaz IEE DataPort



Anexo 5: Subscripción IEEDataPort

Congratulations! You are an IEEE DataPort Subscriber! Your benefits as a subscriber include full access to all IEEE DataPort datasets. You may also upload your own datasets to IEEE DataPort (up to 2TB) and/or consider initiating a Competition on IEEE DataPort. Thank you!

First Name: Alejandro
Last Name: Ildefonso

Get New AWS Access Key

Open Access Datasets

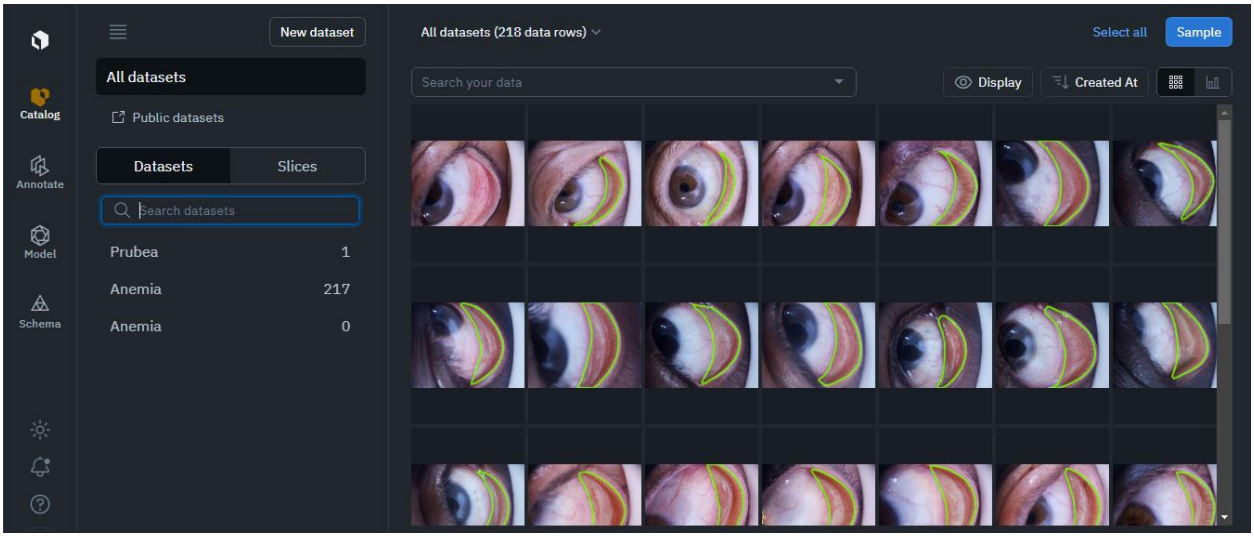
Purchase Open Access Upload Credits

Data Management Plan

A data management plan or DMP is a formal document that outlines how data are to be handled both during a research project, and after the project is completed. All IEEE DataPort logged in users can currently create a Data Management Plan (DMP) on IEEE DataPort during the pilot phase of this new feature. Learn more.

Create a Data Management Plan

Anexo 6: Interfaz LabelBox



Anexo 7: Codigo evaluación del dataset

```
import pandas as pd
import numpy as np

#for plotting
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
```

```
#for data splitting
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV

#for the model prediction
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from xgboost import XGBClassifier
from sklearn.linear_model import LogisticRegression

data_dir = './dataset anemia/'
file1    = data_dir+'India/IndiaRd.xlsx'
file2    = data_dir+'Italy/Italyrd.xlsx'
d1 = pd.read_excel(file1)
d1["Anemia"] = d1["Note"]
d1 = d1[["Hgb", "Gender", "Age", "Anemia" ]]
d2 = pd.read_excel(file2)[["Hgb", "Gender", "Age", "Anemia" ]]
data = pd.concat([d1, d2])
data.head(3)

# Importamos la librería pandas para manipulación de datos

import pandas as pd

# Suponemos que tienes un DataFrame llamado "data" con las columnas 'Hgb',

# data = pd.DataFrame(...)

# Verificamos el tamaño del dataframe

print(len(data))

# Reemplazamos todas las ocurrencias de "No anemia" por "No Anemia"

data.replace("No anemia", "No Anemia", inplace=True)

# Convertimos todas las entradas de la columna 'Hgb' a float, reemplazando las comas por

puntos

data['Hgb'] = [float(i.replace(",", ".")) if type(i) is str else float(i) for i in list(data['Hgb'])]
```

```
# Eliminamos las filas que contienen valores NaN
data.dropna(inplace=True)
# Imprimimos la lista de columnas
print(list(data.columns))  # ['Hgb', 'Gender', 'Age', 'Anemia']
# Verificamos la forma del dataframe después de limpiar los datos
print(data.shape)  # (217, 4)


# Codificamos la columna 'Anemia' como una variable categórica
unique_values_anemia = list(data['Anemia'].unique())
enum_anemia = {val: i for i, val in enumerate(unique_values_anemia)}
print(enum_anemia)  # {'No Anemia': 0, 'Anemia': 1}
data['Anemia'].replace(enum_anemia, inplace=True)
# Mostramos las primeras filas de la tabla
print(data.head())
# Codificamos la columna 'Gender' como una variable categórica
unique_values_gender = list(data['Gender'].unique())
enum_gender = {val: i for i, val in enumerate(unique_values_gender)}
print(enum_gender)  # {'M': 0, 'F': 1}
data['Gender'].replace(enum_gender, inplace=True)
# Mostramos las primeras filas de la tabla después de codificar 'Gender'
print(data.head())
```

```
# Imprimimos los tipos de datos de las columnas para verificar que todo está como queremos

print(data.dtypes)

# Análisis de datos

# Relación entre la edad y la anemia

pd.crosstab(data.Age,data.Anemia).plot(kind="bar", figsize=(15,8))

plt.xlabel('Age')

plt.ylabel('Anemia')

plt.show()

# Relación entre el género y la anemia

pd.crosstab(data.Gender, data.Anemia).plot(kind="bar", figsize=(15,8))

plt.xlabel('Gender (0 - Male, 1 - Female)')

plt.ylabel('Anemia')

plt.show()

# Relación entre el Hgb y la anemia

pd.crosstab(data.Hgb, data.Anemia).plot(kind="line", figsize=(15,8))

plt.xlabel('Hgb')

plt.ylabel('Anemia')

plt.show()

# Relación entre Hgb y Edad con posibilidades de Anemia (0 o 1)

sns.scatterplot(x='Hgb', y='Age', data=data, hue='Anemia')

plt.show()
```

```
# Relación entre Edad y Género con posibilidades de Anemia (0 o 1)

sns.scatterplot(x='Age', y='Gender', data=data, hue='Anemia')

plt.show()

# Preparación de los datos para el entrenamiento

# Comprobamos el número de valores únicos en 'Anemia'

print(len(data['Anemia'].unique()))  # 2

# Separamos la columna 'Anemia' como el objetivo de la predicción

Y = data['Anemia']

data.drop(columns=['Anemia'], inplace=True)

# Convertimos el resto de datos a una matriz de numpy para el procesamiento

X = np.array(data)

# Normalizamos las características utilizando MinMaxScaler

scaler = MinMaxScaler()

scaler.fit(X)

X = scaler.transform(X)

# Dividimos los datos en conjuntos de entrenamiento y prueba

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, train_size=0.7, random_state=123)

print('Training Samples: ' + str(len(X_train)) + '\nTest Samples: ' + str(len(X_test)))  # Training

Samples: 151, Test Samples: 66

Tf Model

model =   tf.keras.models.Sequential([
```

```
                    tf.keras.layers.Dense(128, input_shape=(None, X_train.shape[1]), activa-
tion='relu'),
                    tf.keras.layers.Dense(56,  activation='relu'),
                    tf.keras.layers.Dense(2)
                    ])
loss_fn = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)
model.compile(optimizer='adam',loss=loss_fn, metrics=['accuracy'])
model.fit(X_train, Y_train, epochs=50)
res = model.evaluate(X_test, Y_test)
WARNING:tensorflow:Model was constructed with shape (None, None, 3) for input KerasTen-
sor(type_spec=TensorSpec(shape=(None, None, 3), dtype=tf.float32, name='dense_9_input'),
name='dense_9_input', description="created by layer 'dense_9_input'"), but it was called on an
input with incompatible shape (None, 3).
3/3 [==============================] - 0s 2ms/step - loss: 0.1185 - accu-
racy: 0.9697
res
[0.42827439308166504, 0.8131868243217468]
SVM
param_grid = {'C':[2**1], 'gamma':[0.1, 10], 'kernel':['poly']}
gridSVM = GridSearchCV(SVC(), param_grid=param_grid, cv = 5)
#gridSVM = SVC(C=2, gamma=0.1, kernel='poly')
```

```
gridSVM.fit(X_train, Y_train)

print('Training Score: ' + str(gridSVM.score(X_train, Y_train)))

print("Testing Score: " + str(gridSVM.score(X_test, Y_test)))

Training Score: 0.9735099337748344

Testing Score: 0.9393939393939394

gridSVM.best_params_

{'C': 2, 'gamma': 10, 'kernel': 'poly'}

param_grid = {'C':[0.5, 2**1, 2**2, 2**3 ], 'gamma':[0.1, 2, 10, 100], 'kernel':['poly']}

gridSVM = GridSearchCV(SVC(), param_grid=param_grid, cv = 5)

#gridSVM = SVC(C=2, gamma=0.1, kernel='poly')

gridSVM.fit(X_train, Y_train)

print('Training Score: ' + str(gridSVM.score(X_train, Y_train)))

print("Testing Score: " + str(gridSVM.score(X_test, Y_test)))

Training Score: 0.9602649006622517

Testing Score: 0.9848484848484849

gridSVM.best_params_

{'C': 0.5, 'gamma': 2, 'kernel': 'poly'}

param_grid = {'C':[0.5, 2**1, 2**2, 2**3, 2**4, 2**5 ], 'gamma':[0.01, 0.1, 2, 10, 100], 'ker-

nel':['poly']}

gridSVM = GridSearchCV(SVC(), param_grid=param_grid, cv = 5)

#gridSVM = SVC(C=2, gamma=0.1, kernel='poly')
```

```
gridSVM.fit(X_train, Y_train)

print('Training Score: ' + str(gridSVM.score(X_train, Y_train)))

print("Testing Score: " + str(gridSVM.score(X_test, Y_test)))

Training Score: 0.9602649006622517

Testing Score: 0.9848484848484849

gridSVM.best_params_

{'C': 0.5, 'gamma': 2, 'kernel': 'poly'}

param_grid = {'C':[0.5, 2**1, 2**2, 2**3, 2**4, 2**5, 2**6, 2**8 ], 'gamma':[0.01, 0.1, 2, 10,

100], 'kernel':['poly', 'linear', 'rbf', 'sigmoid']}

gridSVM = GridSearchCV(SVC(), param_grid=param_grid, cv = 5)

#gridSVM = SVC(C=2, gamma=0.1, kernel='poly')

gridSVM.fit(X_train, Y_train)


print('Training Score: ' + str(gridSVM.score(X_train, Y_train)))

print("Testing Score: " + str(gridSVM.score(X_test, Y_test)))

Training Score: 0.9668874172185431

Testing Score: 0.9696969696969697

gridSVM.best_params_

{'C': 2, 'gamma': 10, 'kernel': 'rbf'}

param_grid = {'C':[0.5, 2**1, 2**2, 2**3 ], 'gamma':[0.1, 2, 10, 100], 'kernel':['linear']}

gridSVM = GridSearchCV(SVC(), param_grid=param_grid, cv = 5)
```

```
#gridSVM = SVC(C=2, gamma=0.1, kernel='poly')

gridSVM.fit(X_train, Y_train)

print('Training Score: ' + str(gridSVM.score(X_train, Y_train)))

print("Testing Score: " + str(gridSVM.score(X_test, Y_test)))
```

Training Score: 0.9536423841059603

Testing Score: 0.9696969696969697

```
gridSVM.best_params_
```

{'C': 4, 'gamma': 0.1, 'kernel': 'linear'}

```
param_grid = {'C':[0.5, 2**1, 2**2, 2**3 ], 'gamma':[0.1, 2, 10, 100], 'kernel':['sigmoid']}

gridSVM = GridSearchCV(SVC(), param_grid=param_grid, cv = 5)

#gridSVM = SVC(C=2, gamma=0.1, kernel='poly')

gridSVM.fit(X_train, Y_train)



print('Training Score: ' + str(gridSVM.score(X_train, Y_train)))

print("Testing Score: " + str(gridSVM.score(X_test, Y_test)))
```

Training Score: 0.9403973509933775

Testing Score: 0.9393939393939394

```
gridSVM.best_params_
```

{'C': 8, 'gamma': 0.1, 'kernel': 'sigmoid'}

KNN

```
from sklearn.neighbors import KNeighborsClassifier
```

```
model = KNeighborsClassifier(n_neighbors=1, weights='uniform', algorithm='auto',
leaf_size=30, p=2, metric_params=None, n_jobs=-1)
model.fit(X_train, Y_train)
model.score(X_test, Y_test)
0.9242424242424242
knnAlgos = ['auto', 'ball_tree', 'kd_tree', 'brute']
model = KNeighborsClassifier(n_neighbors=1, weights='uniform', algorithm=knnAlgos[3],
leaf_size=30, p=2, metric_params=None, n_jobs=-1)
model.fit(X_train, Y_train)
model.score(X_test, Y_test)
0.9242424242424242
model = KNeighborsClassifier(n_neighbors=3, weights='uniform', algorithm='ball_tree',
leaf_size=30, p=2, metric_params=None, n_jobs=-1)
model.fit(X_train, Y_train)
model.score(X_test, Y_test)
0.9545454545454546
model = KNeighborsClassifier(n_neighbors=5, weights='uniform', algorithm='auto',
leaf_size=30, p=2, metric_params=None, n_jobs=-1)
model.fit(X_train, Y_train)
model.score(X_test, Y_test)
0.9696969696969697
```

```
model = KNeighborsClassifier(n_neighbors=7, weights='uniform', algorithm='auto',
leaf_size=30, p=2, metric_params=None, n_jobs=-1)
model.fit(X_train, Y_train)
model.score(X_test, Y_test)
0.9848484848484849
model = KNeighborsClassifier(n_neighbors=21, weights='uniform', algorithm='auto',
leaf_size=30, p=2, metric_params=None, n_jobs=-1)
model.fit(X_train, Y_train)
model.score(X_test, Y_test)
0.8787878787878788
model = KNeighborsClassifier(n_neighbors=21, weights='uniform', algorithm='kd_tree',
leaf_size=30, p=2, metric_params=None, n_jobs=-1)
model.fit(X_train, Y_train)
model.score(X_test, Y_test)
0.8787878787878788
model = KNeighborsClassifier(n_neighbors=61, weights='uniform', algorithm='auto',
leaf_size=30, p=2, metric_params=None, n_jobs=-1)
model.fit(X_train, Y_train)
model.score(X_test, Y_test)
0.6666666666666666
# Grid Search
```

```
param_grid = {'n_neighbors':[1, 3, 5, 21, 61], 'algorithm':['auto', 'ball_tree', 'kd_tree', 'brute']}

gridKNN = GridSearchCV(KNeighborsClassifier(), param_grid=param_grid, cv=5)

gridKNN.fit(X_train, Y_train)

GridSearchCV(cv=5, estimator=KNeighborsClassifier(),

             param_grid={'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute'],

                         'n_neighbors': [1, 3, 5, 21, 61]})

gridKNN.score(X_test, Y_test)

0.9242424242424242

gridKNN.best_score_

0.9466666666666667

gridKNN.best_params_

{'algorithm': 'auto', 'n_neighbors': 1}
```

Random Forest

```
param_grid = {'n_estimators':[1500, 1800, 2000]}

#gridRFT = GridSearchCV(RandomForestClassifier(), param_grid=param_grid, cv = 5)

gridRFT = RandomForestClassifier(n_estimators=10)

gridRFT.fit(X_train, Y_train)

print('Training Score: ' + str(gridRFT.score(X_train, Y_train)))

print("Testing Score: " + str(gridRFT.score(X_test, Y_test)))

Training Score: 1.0

Testing Score: 0.9696969696969697
```

```
param_grid = {'n_estimators':[1500, 1800, 2000]}

#gridRFT = GridSearchCV(RandomForestClassifier(), param_grid=param_grid, cv = 5)

gridRFT = RandomForestClassifier(n_estimators=20)

gridRFT.fit(X_train, Y_train)

print('Training Score: ' + str(gridRFT.score(X_train, Y_train)))

print("Testing Score: " + str(gridRFT.score(X_test, Y_test)))

Training Score: 1.0

Testing Score: 0.9545454545454546

param_grid = {'n_estimators':[1500, 1800, 2000]}

#gridRFT = GridSearchCV(RandomForestClassifier(), param_grid=param_grid, cv = 5)

gridRFT = RandomForestClassifier(n_estimators=30)

gridRFT.fit(X_train, Y_train)

print('Training Score: ' + str(gridRFT.score(X_train, Y_train)))

print("Testing Score: " + str(gridRFT.score(X_test, Y_test)))

Training Score: 1.0

Testing Score: 0.9696969696969697

param_grid = {'n_estimators':[1500, 1800, 2000]}

#gridRFT = GridSearchCV(RandomForestClassifier(), param_grid=param_grid, cv = 5)

gridRFT = RandomForestClassifier(n_estimators=50)

gridRFT.fit(X_train, Y_train)

print('Training Score: ' + str(gridRFT.score(X_train, Y_train)))
```

```
print("Testing Score: " + str(gridRFT.score(X_test, Y_test)))
```

```
Training Score: 1.0
```

```
Testing Score: 0.9696969696969697
```

```
param_grid = {'n_estimators':[1500, 1800, 2000]}
```

```
#gridRFT = GridSearchCV(RandomForestClassifier(), param_grid=param_grid, cv = 5)
```

```
gridRFT = RandomForestClassifier(n_estimators=500)
```

```
gridRFT.fit(X_train, Y_train)
```

```
print('Training Score: ' + str(gridRFT.score(X_train, Y_train)))
```

```
print("Testing Score: " + str(gridRFT.score(X_test, Y_test)))
```

```
Training Score: 1.0
```

```
Testing Score: 0.9848484848484849
```

```
param_grid = {'n_estimators':[1500, 1800, 2000]}
```

```
#gridRFT = GridSearchCV(RandomForestClassifier(), param_grid=param_grid, cv = 5)
```

```
gridRFT = RandomForestClassifier(n_estimators=700)
```

```
gridRFT.fit(X_train, Y_train)
```

```
print('Training Score: ' + str(gridRFT.score(X_train, Y_train)))
```

```
print("Testing Score: " + str(gridRFT.score(X_test, Y_test)))
```

```
Training Score: 1.0
```

```
Testing Score: 0.9696969696969697
```

```
param_grid = {'n_estimators':[1500, 1800, 2000]}
```

```
#gridRFT = GridSearchCV(RandomForestClassifier(), param_grid=param_grid, cv = 5)

gridRFT = RandomForestClassifier(n_estimators=1000)

gridRFT.fit(X_train, Y_train)

print('Training Score: ' + str(gridRFT.score(X_train, Y_train)))

print("Testing Score: " + str(gridRFT.score(X_test, Y_test)))

Training Score: 1.0

Testing Score: 0.9696969696969697

param_grid = {'n_estimators':[1500, 1800, 2000]}

#gridRFT = GridSearchCV(RandomForestClassifier(), param_grid=param_grid, cv = 5)

gridRFT = RandomForestClassifier(n_estimators=1200)

gridRFT.fit(X_train, Y_train)



print('Training Score: ' + str(gridRFT.score(X_train, Y_train)))

print("Testing Score: " + str(gridRFT.score(X_test, Y_test)))

Training Score: 1.0

Testing Score: 0.9848484848484849

param_grid = {'n_estimators':[1500, 1800, 2000]}

#gridRFT = GridSearchCV(RandomForestClassifier(), param_grid=param_grid, cv = 5)

gridRFT = RandomForestClassifier(n_estimators=3500)

gridRFT.fit(X_train, Y_train)
```

```
print('Training Score: ' + str(gridRFT.score(X_train, Y_train)))

print("Testing Score: " + str(gridRFT.score(X_test, Y_test)))
```

Training Score: 1.0

Testing Score: 0.9696969696969697

MLP

```
from sklearn.neural_network import MLPClassifier

grid_params = {'hidden_layer_sizes':[(50,), (100), (50, 100,)], 'activation':['identity', 'logistic',

'tanh', 'relu'], 'solver':['lbfgs', 'sgd', 'adam'] }

gridMLP = GridSearchCV(MLPClassifier(), param_grid=grid_params, cv=5)

gridMLP.fit(X_train, Y_train)

GridSearchCV(cv=5, estimator=MLPClassifier(),

            param_grid={'activation': ['identity', 'logistic', 'tanh', 'relu'],

                        'hidden_layer_sizes': [(50,), 100, (50, 100)],

                        'solver': ['lbfgs', 'sgd', 'adam']})

gridMLP.best_score_

0.96

gridMLP.best_params_

{'activation': 'relu', 'hidden_layer_sizes': (50, 100), 'solver': 'adam'}

mlpCls = MLPClassifier(hidden_layer_sizes=(100,), activation='identity', solver='adam' )

mlpCls.fit(X_train, Y_train)
```

```
mlpCls.score(X_test, Y_test)
```

```
0.8333333333333334
```

```
# grid_params = {'hidden_layer_sizes':[(50,), (100), (50, 100,)], 'activation':['identity', 'logistic',
'tanh', 'relu'], 'solver':['lbfgs', 'sgd', 'adam'] }
mlpCls = MLPClassifier(hidden_layer_sizes=(100,), activation='logistic', solver='adam' )
mlpCls.fit(X_train, Y_train)
mlpCls.score(X_test, Y_test)
```

```
0.7878787878787878
```

```
# grid_params = {'hidden_layer_sizes':[(50,), (100), (50, 100,)], 'activation':['identity', 'logistic',
'tanh', 'relu'], 'solver':['lbfgs', 'sgd', 'adam'] }
mlpCls = MLPClassifier(hidden_layer_sizes=(100,), activation='logistic', solver='sgd' )
mlpCls.fit(X_train, Y_train)
mlpCls.score(X_test, Y_test)
```

```
0.7272727272727273
```

```
# grid_params = {'hidden_layer_sizes':[(50,), (100), (50, 100,)], 'activation':['identity', 'logistic',
'tanh', 'relu'], 'solver':['lbfgs', 'sgd', 'adam'] }
mlpCls = MLPClassifier(hidden_layer_sizes=(100,), activation='logistic', solver='lbfgs' )
mlpCls.fit(X_train, Y_train)
mlpCls.score(X_test, Y_test)
  self.n_iter_ = _check_optimize_result("lbfgs", opt_res, self.max_iter)
```

```
0.9696969696969697
```

```
# grid_params = {'hidden_layer_sizes':[(50,), (100), (50, 100,)], 'activation':['identity', 'logistic',
'tanh', 'relu'], 'solver':['lbfgs', 'sgd', 'adam'] }
mlpCls = MLPClassifier(hidden_layer_sizes=(50,), activation='identity', solver='adam' )
mlpCls.fit(X_train, Y_train)
mlpCls.score(X_test, Y_test)
0.7424242424242424
# grid_params = {'hidden_layer_sizes':[(50,), (100), (50, 100,)], 'activation':['identity', 'logistic',
'tanh', 'relu'], 'solver':['lbfgs', 'sgd', 'adam'] }
mlpCls = MLPClassifier(hidden_layer_sizes=(200,), activation='identity', solver='adam' )
mlpCls.fit(X_train, Y_train)
mlpCls.score(X_test, Y_test)
0.9242424242424242
# grid_params = {'hidden_layer_sizes':[(50,), (100), (50, 100,)], 'activation':['identity', 'logistic',
'tanh', 'relu'], 'solver':['lbfgs', 'sgd', 'adam'] }
mlpCls = MLPClassifier(hidden_layer_sizes=(50, 100,), activation='identity', solver='adam' )
mlpCls.fit(X_train, Y_train)
mlpCls.score(X_test, Y_test)
0.9696969696969697
```

Anexo 8: Codigo anemia_detector.py

```
import cv2
import numpy as np
```

```
from tensorflow.keras.models import load_model


class AnemiaDetector():
    def __init__(self) -> None:
        self._detectors = {}
        self._load_models()
        self._labels = {0:"No Anemia", 1:"Anemia"}


    def _load_models(self):
        """ load pretrained model"""
        try:
            for key, mpath in { "palpebral"     : "model_palpebral_93_18",
                                "forniceal"     : "model_forniceal_90_69",
                                "forniceal_palp": "model_forniceal_palp_86_04_35",
                                "org"           : "model_org_84_09"}.items():
                self._detectors[key] = load_model(f"models/{mpath}.hd5")


        except Exception as e:
            print(f"[EXCEPTION] ", e)
            self._detector = None
```

```
def _preprocess(self, image):
    img         = cv2.resize(image, (64, 64))
    img         = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    kernel      = np.array([[-1,-1,-1],
                            [-1, 9,-1],
                            [-1,-1,-1]])
    sharpened   = cv2.filter2D(img, -1, kernel)
    img         = cv2.GaussianBlur(sharpened, (5, 5), 0)
    img         = img.reshape(64, 64, 1)
    return img


def detect(self, image, model_type="palpebral"):
    def result_json(lbl, conf):
        return {"result":lbl, "confidence":f"{conf}"}

    try:
        image   = self._preprocess(image)
        probs   = self._detectors[model_type].predict( np.array([image]) )
        lbl     = np.argmax(probs)
        return result_json(self._labels[lbl], "{:.2f}".format(probs[0][lbl]) )
```

```
        except Exception as e:
            print("[EXCEPTION] ", e)
            return result_json(None, None)


if __name__ == "__main__":
    detector= AnemiaDetector()
    img     = cv2.imread("test.png")
    print   ( detector.detect(img) )
```

Anexo 9: api.py

```
import cv2
import base64
import numpy as np
from anemia_detector import AnemiaDetector
from flask import Flask, request, abort, jsonify


detector          = AnemiaDetector()
app               = Flask(__name__)
app.config["DEBUG"] = True


@app.route('/detect', methods=['POST'])
def detect_anemia():
```

```
    if not request.json or 'image' not in request.json:
        abort(400)

    print       (request.json['model_type'])
    im_b64      = request.json['image']
    img_bytes   = base64.b64decode(im_b64.encode('utf-8'))
    jpg_as_np   = np.frombuffer(img_bytes, dtype=np.uint8)
    img         = cv2.imdecode(jpg_as_np, flags=1)
    resp        = detector.detect(img)
    return resp

def run_server_api():
    app.run(host='0.0.0.0', port=8080)

if __name__ == "__main__":
    run_server_api()
```

Anexo 10:

```
import requests
import base64
import json
```

```
api_url    = "http://localhost:8080/detect"


model_type  = input("please enter model type (palpebral/forniceal/forniceal_palp/org) >>>")
image_file = f'test_imgs/test_{model_type}.png'


with open(image_file, "rb") as f:
    im_bytes = f.read()
im_b64 = base64.b64encode(im_bytes).decode("utf8")


headers = {'Content-type': 'application/json', 'Accept': 'text/plain'}
payload = json.dumps({"image": im_b64, "model_type": model_type})


response = requests.post(api_url, data=payload, headers=headers)
try:
    data = response.json()
    print(data)
except requests.exceptions.RequestException:
    print(response.text)
```